

Приложения и предметный указатель

В приложениях представлены грамматика языка C++, обсуждение вопросов совместимости C и C++ и вопросов совместимости между стандартом C++ и предстандартными версиями, а также множество технических деталей. Для стандартной библиотеки представлены средства интернационализации, а также концепции, гарантии и методы реализации безопасности при генерации исключений. Предметный указатель достаточно обширен и является неотъемлемой частью данной книги.

Главы

- A.** Грамматика
- B.** Совместимость
- C.** Технические подробности
- D.** Локализация
- E.** Исключения и безопасность
стандартной библиотеки

Предметный указатель

Приложение А

Грамматика

*Самая большая опасность для преподавателя —
учить словам, а не вещам.
— Марк Блок*

Введение — ключевые слова — лексические соглашения — программы — выражения — операторы — объявления — классы — производные классы — особые функции-члены — перегрузка — шаблоны — обработка исключений — директивы препроцессора.

А.1. Введение

Итоговое концентрированное изложение синтаксиса C++ призвано быть формальным дополнением и должно способствовать лучшему его пониманию. Оно не является абсолютно точным и строгим описанием языка C++. В частности, описанная здесь грамматика содержит некоторые расширения допустимых для C++ конструкций. Требуется применять правила разрешения неоднозначностей для различения выражений от объявлений (§A.5, §A.7). Кроме того, для выправления синтаксически корректных, но бессмысленных конструкций нужно применять правило доступа, правило типов и правило разрешения неоднозначностей.

Стандартные грамматики C и C++ различаются с синтаксической точки зрения минимально, в большей степени через ограничения. Это способствует точности, но не читаемости кода.

А.2. Ключевые слова

Новые контекстнозависимые ключевые слова вводятся в программу с помощью *typedef* (§4.9.7), пространств имен (§8.2), с помощью объявлений классов (глава 10), перечислений (§4.8) и шаблонов (глава 13)¹:

typedef-name :
 identifier

namespace-name :
 original-namespace-name
 namespace-alias

original-namespace-name :
 identifier

namespace-alias :
 identifier

class-name :
 identifier
 template-id

enum-name :
 identifier

template-name :
 identifier

Заметим, что *typedef*-имя, именующее класс, есть также и *class*-имя.

Если идентификатор явным образом не именует тип, то считается, что он именует что угодно, но только не тип (см. также §C.13.5).

Ниже перечислены ключевые слова языка C++:

Ключевые слова C++				
<i>and</i>	<i>continue</i>	<i>goto</i>	<i>public</i>	<i>try</i>
<i>and_eq</i>	<i>default</i>	<i>if</i>	<i>register</i>	<i>typedef</i>
<i>asm</i>	<i>delete</i>	<i>inline</i>	<i>reinterpret_cast</i>	<i>typeid</i>
<i>auto</i>	<i>do</i>	<i>int</i>	<i>return</i>	<i>typename</i>
<i>bitand</i>	<i>double</i>	<i>long</i>	<i>short</i>	<i>union</i>
<i>bitor</i>	<i>dynamic_cast</i>	<i>mutable</i>	<i>signed</i>	<i>unsigned</i>
<i>bool</i>	<i>else</i>	<i>namespace</i>	<i>sizeof</i>	<i>using</i>
<i>break</i>	<i>enum</i>	<i>new</i>	<i>static</i>	<i>virtual</i>
<i>case</i>	<i>explicit</i>	<i>not</i>	<i>static_cast</i>	<i>void</i>

¹ Ввиду большой практической ценности и фактической самоочевидности (для разработчиков компиляторов) представляем авторское формализованное изложение грамматики в точном оригинальном виде (без перевода простейших терминов). Здесь *opt* — это сокращение от *optional* (необязательно). — *Прим. ред.*

Ключевые слова C++				
<i>catch</i>	<i>export</i>	<i>not_eq</i>	<i>struct</i>	<i>volatile</i>
<i>char</i>	<i>extern</i>	<i>operator</i>	<i>switch</i>	<i>wchar_t</i>
<i>class</i>	<i>false</i>	<i>or</i>	<i>template</i>	<i>while</i>
<i>compl</i>	<i>float</i>	<i>or_eq</i>	<i>this</i>	<i>xor</i>
<i>const</i>	<i>for</i>	<i>private</i>	<i>throw</i>	<i>xor_eq</i>
<i>const_cast</i>	<i>friend</i>	<i>protected</i>	<i>true</i>	

А.3. Лексические соглашения

Стандартные грамматики языков С и С++ представляют лексические соглашения как грамматические сущности. Это добавляет точности, но не всегда способствует читабельности текстов:

- hex-quad:*
 - hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit*
- universal-character-name:*
 - \u hex-quad*
 - \U hex-quad hex-quad*
- preprocessing-token:*
 - header-name*
 - identifier*
 - pp-number*
 - character-literal*
 - string-literal*
 - preprocessing-op-or-punc*
 - each non-white-space character that cannot be one of the above*
- token:*
 - identifier*
 - keyword*
 - literal*
 - operator*
 - punctuator*
- header-name:*
 - <h-char-sequence>*
 - "q-char-sequence"*
- h-char-sequence:*
 - h-char*
 - h-char-sequence h-char*
- h-char:*
 - any member of the source character set except new-line and >*
- q-char-sequence:*
 - q-char*
 - q-char-sequence q-char*

q-char:

any member of the source character set except new-line and "

pp-number:

digit

. digit

pp-number digit

pp-number nondigit

pp-number e sign

pp-number E sign

pp-number .

identifier:

nondigit

identifier nondigit

identifier digit

nondigit: one of

universal-character-name

_ a b c d e f g h i j k l m n o p q r s t u v w x y z
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

digit: one of

0 1 2 3 4 5 6 7 8 9

preprocessing-op-or-punc: one of

{	}	[	]	#	##	(	)	<:	:>	<%	%>	%:%:
%:	;	:	?	::	.	.*	+	-	*	/	%	^
&		~	!	=	<	>	.,+=	--	*	/=	%=	^=
&=	=	<=	>=	<<	>>	==	=	<=	>=	&&		++
-	,	->	->*	...	new	delete	and	and_eq			bitand	
bitor		comp		not	or	not_eq	xor	or_eq			xor_eq	

literal:

integer-literal

character-literal

floating-literal

string-literal

boolean-literal

integer-literal:

decimal-literal integer-suffix_{opt}

octal-literal integer-suffix_{opt}

hexadecimal-literal integer-suffix_{opt}

decimal-literal:

nonzero-digit

decimal-literal digit

octal-literal:

0

octal-literal octal-digit

hexadecimal-literal:

0x hexadecimal-digit

0X hexadecimal-digit

hexadecimal-literal hexadecimal-digit

nonzero-digit: one of

1 2 3 4 5 6 7 8 9

octal-digit: one of

0 1 2 3 4 5 6 7

hexadecimal-digit: one of

0 1 2 3 4 5 6 7 8 9

a b c d e f

A B C D E F

integer-suffix:

unsigned-suffix long-suffix_{opt}

long-suffix unsigned-suffix_{opt}

unsigned-suffix: one of

u U

long-suffix: one of

l L

character-literal:

'c-char-sequence'

L'c-char-sequence'

c-char-sequence:

c-char

c-char-sequence c-char

c-char:

any member of the source character set except the single-quote, backslash, or new-line character

escape-sequence

universal-character-name

escape-sequence:

simple-escape-sequence

octal-escape-sequence

hexadecimal-escape-sequence

simple-escape-sequence: one of

\ ' \ " \ ? \ \ \ a \ b \ f \ n \ r \ t \ v

octal-escape-sequence:

\ octal-digit

\ octal-digit octal-digit

\ octal-digit octal-digit octal-digit

hexadecimal-escape-sequence:

\ x hexadecimal-digit

hexadecimal-escape-sequence hexadecimal-digit

floating-literal:

fractional-constant exponent-part_{opt} floating-suffix_{opt}

digit-sequence exponent-part floating-suffix_{opt}

fractional-constant:

*digit-sequence*_{opt} . *digit-sequence*
digit-sequence .

exponent-part:

e *sign*_{opt} *digit-sequence*
E *sign*_{opt} *digit-sequence*

sign: one of

+ -

digit-sequence:

digit
digit-sequence digit

floating-suffix: one of

f l F L

string-literal:

"*s-char-sequence*_{opt}"
L"*s-char-sequence*_{opt}"

s-char-sequence:

s-char
s-char-sequence s-char

s-char:

any member of the source character set except double-quote, backslash, or new-line
escape-sequence
universal-character-name

boolean-literal:

false
true

При составлении лексемы выбирается *наиболее длинная возможная последовательность символов*. Например, **double** составляет единую лексему, а не комбинацию из **do** и **uble**; последовательность символов **+++** есть **++** с последующим **+**.

А.4. Программы

Программа ¹ — это набор единиц трансляции, объединяемых в процессе компоновки (linking) (§9.4). *Единица трансляции (translation-unit)*, часто называемая *исходным файлом (source file)* — это последовательность *объявлений (declarations)*:

translation-unit:

*declaration-seq*_{opt}

А.5. Выражения

Выражения рассматриваются в главе 6 с подробным перечислением в §6.2. Определение *списка-выражений (expression-list)* эквивалентно определению *выражения (expression)*. Имеются два правила для отличия запятой как операции (операция следования — sequencing) и как разделителя списка аргументов функции (§6.2.2).

primary-expression:

literal
this
:: identifier
:: operator-function-id
:: qualified-id
 (*expression*)
id-expression

id-expression:

unqualified-id
qualified-id

unqualified-id:

identifier
operator-function-id
conversion-function-id
 ~ *class-name*
template-id

qualified-id:

nested-name-specifier *template*_{opt} *unqualified-id*

nested-name-specifier:

class-or-namespace-name :: *nested-name-specifier*_{opt}
class-or-namespace-name :: *template* *nested-name-specifier*

class-or-namespace-name:

class-name
namespace-name

postfix-expression:

primary-expression
postfix-expression [*expression*]
postfix-expression (*expression-list*_{opt})
simple-type-specifier (*expression-list*_{opt})
typename :: *opt* *nested-name-specifier* *identifier* (*expression-list*_{opt})
typename :: *opt* *nested-name-specifier* *template*_{opt} *template-id* (*expression-list*_{opt})
postfix-expression . *template*_{opt} :: *opt* *id-expression*
postfix-expression -> *template*_{opt} :: *opt* *id-expression*
postfix-expression . *pseudo-destructor-name*
postfix-expression -> *pseudo-destructor-name*
postfix-expression ++
postfix-expression --
dynamic_cast < *type-id* > (*expression*)
static_cast < *type-id* > (*expression*)
reinterpret_cast < *type-id* > (*expression*)
const_cast < *type-id* > (*expression*)
typeid (*expression*)
typeid (*type-id*)

expression-list:

assignment-expression
expression-list , *assignment-expression*

pseudo-destructor-name:

```

::opt nested-name-specifieropt type-name :: ~ type-name
::opt nested-name-specifier template template-id :: ~ type-name
::opt nested-name-specifieropt ~ type-name

```

unary-expression:

```

postfix-expression
++ cast-expression
-- cast-expression
unary-operator cast-expression
sizeof unary-expression
sizeof ( type-id )
new-expression
delete-expression

```

unary-operator: one of

```

*      &      +      -      !      ~

```

new-expression:

```

::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt ( type-id ) new-initializeropt

```

new-placement:

```

( expression-list )

```

new-type-id:

```

type-specifier-seq new-declaratoropt

```

new-declarator:

```

ptr-operator new-declaratoropt
direct-new-declarator

```

direct-new-declarator:

```

[ expression ]
direct-new-declarator [ constant-expression ]

```

new-initializer:

```

( expression-listopt )

```

delete-expression:

```

::opt delete cast-expression
::opt delete [ ] cast-expression

```

cast-expression:

```

unary-expression
( type-id ) cast-expression

```

pm-expression:

```

cast-expression
pm-expression .* cast-expression
pm-expression ->* cast-expression

```

multiplicative-expression:

```

pm-expression
multiplicative-expression * pm-expression
multiplicative-expression / pm-expression
multiplicative-expression % pm-expression

```

additive-expression:

multiplicative-expression
additive-expression + *multiplicative-expression*
additive-expression - *multiplicative-expression*

shift-expression:

additive-expression
shift-expression << *additive-expression*
shift-expression >> *additive-expression*

relational-expression:

shift-expression
relational-expression < *shift-expression*
relational-expression > *shift-expression*
relational-expression <= *shift-expression*
relational-expression >= *shift-expression*

equality-expression:

relational-expression
equality-expression == *relational-expression*
equality-expression != *relational-expression*

and-expression:

equality-expression
and-expression & *equality-expression*

exclusive-or-expression:

and-expression
exclusive-or-expression ^ *and-expression*

inclusive-or-expression:

exclusive-or-expression
inclusive-or-expression | *exclusive-or-expression*

logical-and-expression:

inclusive-or-expression
logical-and-expression && *inclusive-or-expression*

logical-or-expression:

logical-and-expression
logical-or-expression || *logical-and-expression*

conditional-expression:

logical-or-expression
logical-or-expression ? *expression* : *assignment-expression*

assignment-expression:

conditional-expression
logical-or-expression *assignment-operator* *assignment-expression*
throw-expression

assignment-operator: one of

= * = / = % = += -= >> = << = & = ^ = | =

expression:

assignment-expression
expression, *assignment-expression*

constant-expression:

conditional-expression

Грамматические неоднозначности возникают из-за идентичности приведения типа в функциональном стиле и объявлений. Например:

```
int x;
void f()
{
    char (x);           // приведение типа x к char или объявление переменной x типа char?
}
```

Все такие неоднозначности разрешаются в пользу объявлений (если нечто выглядит как объявление, это и есть объявление). Например:

```
T(a) -> m;           // оператор-выражение
T(a) ++;             // оператор-выражение

T(*e) (int(3)) ;     // объявление
T(f) [4] ;           // объявление

T(a) ;               // объявление
T(a) = m;            // объявление
T(*b) () ;           // объявление
T(x) , y, z = 7;      // объявление
```

Единственная информация, связанная с именем, это является ли оно именем типа или шаблона. Если это нельзя однозначно установить, то имя считается принадлежащим чему угодно, но только не типу (и не шаблону).

Конструкция **template** *неквалифицированный-идентификатор* означает, что *неквалифицированный-идентификатор* относится к имени шаблона (если по контексту это не может быть определено) (см. §C.13.6).

A.6. Операторы

См. §6.3.

statement:

labeled-statement
expression-statement
compound-statement
selection-statement
iteration-statement
jump-statement
declaration-statement
try-block

labeled-statement:

identifier : *statement*
case constant-expression : *statement*
default : *statement*

expression-statement:

*expression*_{opt} ;

compound-statement:

{ *statement-seq*_{opt} }

statement-seq:
statement
statement-seq statement

selection-statement:
if (condition) statement
if (condition) statement else statement
switch (condition) statement

condition:
expression
type-specifier-seq declarator = assignment-expression

iteration-statement:
while (condition) statement
do statement while (expression) ;
for (for-init-statement condition_{opt} ; expression_{opt}) statement

for-init-statement:
expression-statement
simple-declaration

jump-statement:
break ;
continue ;
return expression_{opt} ;
goto identifier ;

declaration-statement:
block-declaration

А.7. Объявления

Структура объявлений рассмотрена в главе 4, перечисления — в §4.8, указатели и массивы в главе 5, функции — в главе 7, пространства имен — в §8.2, директивы компоновки — в §9.2.4, классы хранения — в §10.4.

declaration-seq:
declaration
declaration-seq declaration

declaration:
block-declaration
function-definition
template-declaration
explicit-instantiation
explicit-specialization
linkage-specification
namespace-definition

block-declaration:
simple-declaration
asm-definition
namespace-alias-definition

using-declaration

using-directive

simple-declaration:

decl-specifier-seq_{opt} init-declarator-list_{opt} ;

decl-specifier:

storage-class-specifier

type-specifier

function-specifier

friend

typedef

decl-specifier-seq:

decl-specifier-seq_{opt} decl-specifier

storage-class-specifier:

auto

register

static

extern

mutable

function-specifier:

inline

virtual

explicit

typedef-name:

identifier

type-specifier:

simple-type-specifier

class-specifier

enum-specifier

elaborated-type-specifier

cv-qualifier

simple-type-specifier:

::_{opt} nested-name-specifier_{opt} type-name

::_{opt} nested-name-specifier template_{opt} template-id

char

wchar_t

bool

short

int

long

signed

unsigned

float

double

void

type-name:

class-name
enum-name
typedef-name

elaborated-type-specifier:

class-key ::_{opt} *nested-name-specifier*_{opt} *identifier*
enum ::_{opt} *nested-name-specifier*_{opt} *identifier*
typename ::_{opt} *nested-name-specifier* *identifier*
typename ::_{opt} *nested-name-specifier* *template*_{opt} *template-id*

enum-name:

identifier

enum-specifier:

enum *identifier*_{opt} { *enumerator-list*_{opt} }

enumerator-list:

enumerator-definition
enumerator-list, *enumerator-definition*

enumerator-definition:

enumerator
enumerator = *constant-expression*

enumerator:

identifier

namespace-name:

original-namespace-name
namespace-alias

original-namespace-name:

identifier

namespace-definition:

named-namespace-definition
unnamed-namespace-definition

named-namespace-definition:

original-namespace-definition
extension-namespace-definition

original-namespace-definition:

namespace *identifier* { *namespace-body* }

extension-namespace-definition:

namespace *original-namespace-name* { *namespace-body* }

unnamed-namespace-definition:

namespace { *namespace-body* }

namespace-body:

*declaration-seq*_{opt}

namespace-alias:

identifier

namespace-alias-definition:

namespace *identifier* = *qualified-namespace-specifier* ;

qualified-namespace-specifier:

::_{opt} *nested-name-specifier*_{opt} *namespace-name*

using-declaration:

using *typename*_{opt} ::_{opt} *nested-name-specifier* *unqualified-id* ;

using :: *unqualified-id* ;

using-directive:

using namespace ::_{opt} *nested-name-specifier*_{opt} *namespace-name* ;

asm-definition:

asm (*string-literal*) ;

linkage-specification:

extern *string-literal* { *declaration-seq*_{opt} }

extern *string-literal* *declaration*

Грамматика допускает произвольный уровень вложения объявлений. Однако имеются некоторые семантические ограничения. Например, не допускается вложения функций (определения функции в теле другой функции).

Список спецификаторов, начинающих объявление, не может быть пустым (нет умолчательного **int**, §B.2) и включает самую длинную из всех возможных последовательность спецификаторов. Например:

typedef int I;

void f(unsigned I) { /* ... */ }

Здесь **f()** принимает неименованный **unsigned int**.

С помощью **asm()** вводятся фрагменты ассемблерного кода. Более детальный смысл зависит от реализации, но общая идея состоит в том, что строка подается как ассемблерный код, подлежащей внедрению в компилируемый поток в указанном месте.

Модификатор **register** означает намек компилятору на то, чтобы он попытался оптимизировать частый доступ к переменной, но это большинство современных компиляторов понимают и без подсказок.

A.7.1. Деклараторы

См. §4.9.1, главу 5 (указатели и массивы), §7.7 (указатели на функции) и §15.5 (указатели на члены классов).

init-declarator-list:

init-declarator

init-declarator-list, *init-declarator*

init-declarator:

declarator *initializer*_{opt}

declarator:

direct-declarator

ptr-operator declarator

direct-declarator:

declarator-id

direct-declarator (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*exception-specification*_{opt}

direct-declarator [*constant-expression*_{opt}]
 (*declarator*)

ptr-operator:

* *cv-qualifier-seq*_{opt}

&

::_{opt} *nested-name-specifier* * *cv-qualifier-seq*_{opt}

cv-qualifier-seq:

cv-qualifier *cv-qualifier-seq*_{opt}

cv-qualifier:

const

volatile

declarator-id:

::_{opt} *id-expression*

::_{opt} *nested-name-specifier*_{opt} *type-name*

type-id:

type-specifier-seq *abstract-declarator*_{opt}

type-specifier-seq:

type-specifier *type-specifier-seq*_{opt}

abstract-declarator:

ptr-operator *abstract-declarator*_{opt}

direct-abstract-declarator

direct-abstract-declarator:

*direct-abstract-declarator*_{opt} (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*exception-specification*_{opt}

*direct-abstract-declarator*_{opt} [*constant-expression*_{opt}]
 (*abstract-declarator*)

parameter-declaration-clause:

*parameter-declaration-list*_{opt} ... _{opt}

parameter-declaration-list , ... ,

parameter-declaration-list:

parameter-declaration

parameter-declaration-list , *parameter-declaration*

parameter-declaration:

decl-specifier-seq *declarator*

decl-specifier-seq *declarator* = *assignment-expression*

decl-specifier-seq *abstract-declarator*_{opt}

decl-specifier-seq *abstract-declarator*_{opt} = *assignment-expression*

function-definition:

*decl-specifier-seq*_{opt} *declarator* *ctor-initializer*_{opt} *function-body*

*decl-specifier-seq*_{opt} *declarator* *function-try-block*

function-body:
compound-statement

initializer:
 = *initializer-clause*
 (*expression-list*)

initializer-clause:
assignment-expression
 { *initializer-list* , *opt* }
 { }

initializer-list:
initializer-clause
initializer-list , *initializer-clause*

Спецификатор ***volatile*** — это подсказка компилятору, что объект может изменить свое значение способом, не описанным в языке программирования, так что агрессивной оптимизации следует избегать. Например, часы реального времени могут быть объявлены следующим образом:

extern const volatile long clock;

Два последовательных считывания объекта ***clock*** могут порождать разные результаты.

A.8. Классы

См. главу 10.

class-name:
identifier
template-id

class-specifier:
class-head { *member-specification*_{opt} }

class-head:
*class-key identifier*_{opt} *base-clause*_{opt}
*class-key nested-name-specifier identifier base-clause*_{opt}
*class-key nested-name-specifier*_{opt} *template-id base-clause*_{opt}

class-key:
 class
 struct
 union

member-specification:
*member-declaration member-specification*_{opt}
access-specifier : *member-specification*_{opt}

member-declaration:
*decl-specifier-seq*_{opt} *member-declarator-list*_{opt} ;
function-definition ; *opt*
 : *opt* *nested-name-specifier template*_{opt} *unqualified-id* ;

using-declaration
template-declaration

member-declarator-list:
member-declarator
member-declarator-list , *member-declarator*

member-declarator:
declarator pure-specifier_{opt}
declarator constant-initializer_{opt}
identifier_{opt} : constant-expression

pure-specifier:
 = 0

constant-initializer:
 = *constant-expression*

Ради совместимости с языком С допускается в одной и той же области видимости объявлять класс и не-класс с одним и тем же именем (§5.7). Например:

```
struct stat { /* . . . */ ;  
int stat (char* name, struct stat* buf) ;
```

В этом случае, имя *stat* — это не имя класса; на класс нужно ссылаться с помощью классового префикса.

Константные выражения определяются в §С.5.

А.8.1. Производные классы

См. главу 12 и главу 15.

base-clause:
 : *base-specifier-list*

base-specifier-list:
base-specifier
base-specifier-list , *base-specifier*

base-specifier:
 ::_{opt} *nested-name-specifier_{opt}* *class-name*
 virtual *access-specifier_{opt}* ::_{opt} *nested-name-specifier_{opt}* *class-name*
access-specifier virtual_{opt} ::_{opt} *nested-name-specifier_{opt}* *class-name*

access-specifier:
 private
 protected
 public

А.8.2. Особые функции-члены

См. §11.4 (операции преобразования), §10.4.6 (инициализация членов класса) и §12.2.2 (инициализация базового класса).

conversion-function-id:
 operator *conversion-type-id*

conversion-type-id:
type-specifier-seq conversion-declarator_{opt}

conversion-declarator:
ptr-operator conversion-declarator_{opt}

ctor-initializer:
 : *mem-initializer-list*

mem-initializer-list:
mem-initializer
mem-initializer , mem-initializer-list

mem-initializer:
mem-initializer-id (expression-list_{opt})

mem-initializer-id:
 : :_{opt} *nested-name-specifier_{opt} class-name*
identifier

A.8.3. Перегрузка

См. главу 11.

operator-function-id:
operator operator

operator: one of

<i>new</i>	<i>delete</i>	<i>new[]</i>	<i>delete[]</i>												
+	-	*	/	%	^	&		~	!	=	<	>			
+=	-=	*=	/=	%=	^=	&=	=	<<	>>	>>=	<<=	==			
!=	<=	>=	&&		++	-	,	->*	->	()	[]				

A.9. Шаблоны

Шаблоны объясняются в главе 13 и §C.13.

template-declaration:
export_{opt} template < template-parameter-list > declaration

template-parameter-list:
template-parameter
template-parameter-list , template-parameter

template-parameter:
type-parameter
parameter-declaration

type-parameter:
class identifier_{opt}
class identifier_{opt} = type-id
typename identifier_{opt}
typename identifier_{opt} = type-id
template < template-parameter-list > class identifier_{opt}
template < template-parameter-list > class identifier_{opt} = id-expression

```
template-id:
    template-name < template-argument-listopt >
```

```
template-name:
    identifier
```

```
template-argument-list:
    template-argument
    template-argument-list , template-argument
```

```
template-argument:
    assignment-expression
    type-id
    id-expression
```

```
explicit-instantiation:
    template declaration
```

```
explicit-specialization:
    template < > declaration
```

Явная специализация аргументов шаблона открывает возможность запутанной синтаксической неоднозначности. Рассмотрим пример:

```
void h ()
{
    f<1> (0) ; // неоднозначность: ((f)<1>)>(0) или (f<1>)>(0)?
               // разрешение неоднозначности: вызывается f<1> с аргументом 0
}
```

Разрешение неоднозначности просто и эффективно: если *f* — имя шаблона, то *f*< — это начало квалифицированного имени шаблона, так что последующие лексемы должны интерпретироваться отталкиваясь от этого факта; в противном случае < означает знак «меньше, чем». Аналогично, первый невложенный знак > оканчивает список аргументов шаблона. Если требуется знак «больше, чем», то следует применить круглые скобки:

```
f< a>b > (0) ; // синтаксическая ошибка
f< (a>b) > (0) ; // ok
```

Подобная лексическая неоднозначность возникает, когда оканчивающие знаки > расположены вплотную друг к другу. Например:

```
list<vector<int>> lv1; // error: неожиданная операция >> (сдвиг вправо)
list< vector<int> > lv2; // правильно: список векторов
```

Обратите внимание на пробел между двумя знаками >; >> — это операция сдвига. Это может стать источником путаницы.

A.10. Обработка исключений

См. §8.3 и главу 14.

```
try-block:
    try compound-statement handler-seq
```

```

function-try-block:
    try ctor-initializeropt function-body handler-seq

handler-seq:
    handler handler-seqopt

handler:
    catch ( exception-declaration ) compound-statement

exception-declaration:
    type-specifier-seq declarator
    type-specifier-seq abstract-declarator
    type-specifier-seq
    . . .

throw-expression:
    throw assignment-expressionopt

exception-specification:
    throw ( type-id-listopt )

type-id-list:
    type-id
    type-id-list , type-id

```

А.11. Директивы препроцессора

Препроцессор — это относительно простой макрообработчик, работающий в основном с лексемами, а не с отдельными символами. Дополнительно к возможности определять и использовать макросы (§7.8) препроцессор обеспечивает механизмы для включения текстовых файлов и стандартных заголовочных файлов (§9.2.1), а также для условной компиляции, базирующейся на макроопределениях (§9.3.3). Например:

```

#if OPT==4
#include "header4.h"
#elif 0<OPT
#include "someheader.h"
#else
#include <cstdlib>
#endif

```

Все директивы препроцессора начинаются с символа #, который должен быть первым не пробельным символом на своей строке.

```

preprocessing-file:
    groupopt

group:
    group-part
    group group-part

group-part:
    pp-tokensopt new-line
    if-section
    control-line

```

if-section:

if-group elif-groups_{opt} else-group_{opt} endif-line

if group:

if constant-expression new-line group_{opt}

ifdef identifier new-line group_{opt}

ifndef identifier new-line group_{opt}

elif- groups:

elif-group

elif-groups elif-group

elif-group:

elif constant-expression new-line group_{opt}

else-group:

else new-line group_{opt}

endif-line:

endif new-line

control-line:

include pp-tokens new-line

define identifier replacement-list new-line

define identifier lparen identifier-list_{opt}) replacement-list new-line

undef identifier new-line

line pp-tokens new-line

error pp-tokens_{opt} new-line

pragma pp-tokens_{opt} new-line

new-line

lparen:

the left-parenthesis character without preceding white-space

replacement-list:

pp-tokens_{opt}

pp-tokens:

preprocessing-token

pp-tokens preprocessing-token

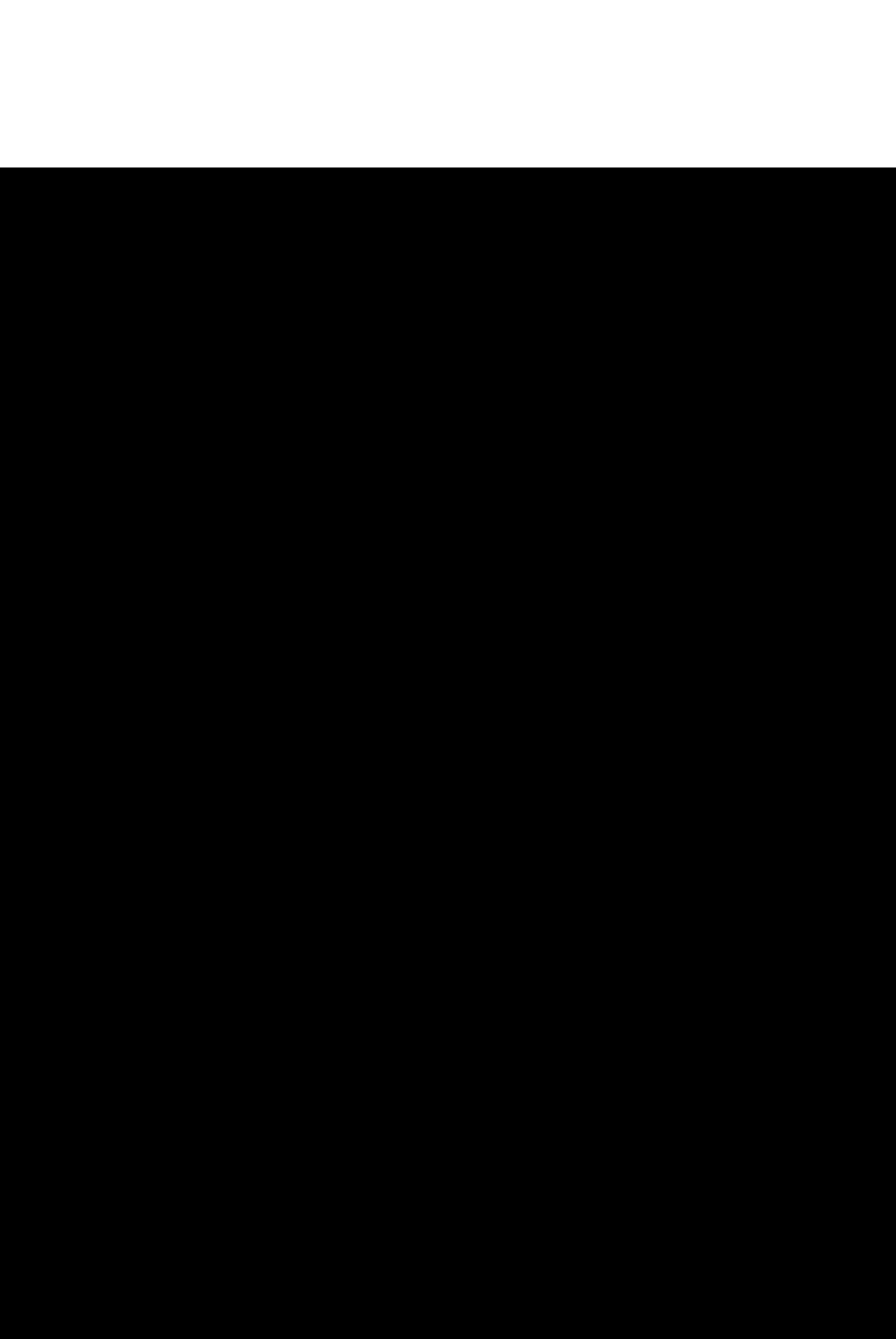
new-line:

the new-line character

identifier-list:

identifier

identifier-list , identifier



Приложение В

Совместимость

Вы следуйте своим обычаям, а я буду следовать своим.
— Ч. Нэпьер

Совместимость С и С++ — «тихие» отличия — код на С, не являющийся С++-кодом — нежелательные особенности — код на С++, не являющийся кодом на С — старые реализации С++ — заголовочные файлы — стандартная библиотека — пространства имен — ошибки выделения памяти — шаблоны — инициализаторы в операторах *for* — советы — упражнения.

В.1. Введение

В данном приложении обсуждаются расхождения между С и С++, а также между стандартом С++ (ISO/IEC 14882) и более ранними реализациями языка С++. Мы хотим документировать эти различия, которые могут создать для программиста некоторые проблемы, а также указать на пути их преодоления. Большинство таких проблем возникают, когда пытаются перевести С-программу на язык С++, когда осуществляют переход с одной из ранних версий С++ на другую или когда компилируют современную С++-программу старым компилятором. Моя цель — не утопить вас в огромной массе всех потенциально возможных проблем совместимости с каждой из существующих версий С++, а, скорее, перечислить наиболее типичные проблемы и представить их стандартные решения.

При рассмотрении вопросов совместимости очень важно учесть диапазон реализаций, при которых программа должна работать. Для изучения С++ целесообразно выбрать наиболее полную и удобную реализацию. Для производства конечного отчуждаемого продукта больше подходит консервативная стратегия, призванная максимизировать число систем, с которыми продукт будет работать. В прошлом это служило оправданием бегства от новшеств С++. Однако теперь реализации сближаются и необходимость в переносимости с одной из них на другую уже не требует такой осторожности, как раньше.

В.2. Совместимость С и С++

За минимальными исключениями язык С (имеется в виду стандарт С89, ISO/IEC 9899:1990) является подмножеством С++. Наиболее значимые различия возникают из-за большей приверженности С++ статической проверке типов. Хорошо написанные на С программы часто оказываются и допустимыми программами на С++. Компилятор выявляет все различия между С и С++.

В.2.1. «Тихие» отличия

За небольшими исключениями программы на С и С++ имеют одинаковый смысл. По счастью, эти исключения («тихие» отличия) не принципиальны.

В языке С размер символьной константы и перечислений равен `sizeof(int)`. В С++ `sizeof('a')` равен `sizeof(char)`, а размер перечислений определяется конкретной реализацией (§4.8).

В языке С++ есть комментарии `//`, а в языке С их официально нет (но многие реализации их допускают). Это отличие можно использовать для написания программы, ведущей себя на разных языках по-разному. Например:

```
int f(int a, int b)
{
    return a /* маловероятно */ / b
    ; /* нереалистично: точка с запятой на отдельной строке во избежание
      синтаксической ошибки */
}
```

Международный стандарт языка С пересматривается с целью введения комментариев `//`.

Имя структуры, объявленной во вложенной области видимости, может скрыть имя объекта, функции, перечисления или типа из объемлющей области видимости. Например:

```
int x[99];
void f()
{
    struct x {int a;};
    sizeof(x); /* в С - размер массива; в С++ - размер структуры */
}
```

В.2.2. Код на С, не являющийся С++-кодом

Разночтения С/С++, которые вызывают большинство проблем, не столь уж и сложны. Большинство из них легко обнаруживаются компилятором. В данном разделе приведены примеры на С, которые не соответствуют языку С++. Многие из них считаются устаревшими для современного С.

В языке С большинство функций можно вызывать без предварительного объявления:

```
main() /* не С++. В С - плохой стиль */
{
    double sq2 = sqrt(2); /* вызов необъявленной функции */
```

```
printf("the square root of 2 is %g\n", sq2); /* вызов необъявленной функции */
}
```

Последовательное использование предварительных объявлений (прототипов функций) рекомендуется и для языка С. Когда этому следуют на практике (или когда компилятор имеет установки следить за этим), то проблем с переходом от С к С++ не возникает. Если же вызываются необъявленные функции, то нужно очень точно знать правила языка С, чтобы понять, где возникла проблема с переносимостью. Например, функция **main()** из предыдущего примера содержит с точки зрения языка С по крайней мере две ошибки.

В языке С функция, объявленная без аргументов, может принимать любое количество аргументов любого типа. Такой стиль стандартом С признан устаревшим, но он не так уж и редко встречается в реальных программах:

```
void f(); /* не упомянуты типы аргументов */
void g()
{
f(2); /* Не С++. Плохой стиль в С */
}
```

В языке С функции можно определять в стиле, когда типы объявляются после перечисления аргументов:

```
void f(a,p,c) char *p; char c; { /* ... */ } /* С. Не С++ */
```

Это лучше переписать в виде

```
void f(int a, char* p, char c) { /* ... */ }
```

В языке С и в старых версиях С++ (предшествовавших принятию стандарта) типом по умолчанию является **int**. Например:

```
const a = 7; /* В С подразумевается int. Не С++ */
```

Поздняя спецификация С99 запрещает "умолчательные **int**", как это имеет место в стандарте языка С++.

Язык С допускает определения структур в объявлениях возврата функций или их аргументов. Например:

```
struct S {int x,y;} f(); /* С. Не С++ */
void g(struct S {int x,y;} y); /* С. Не С++ */
```

С точки зрения правил языка С++ для определения типов такие объявления бесполезны, а потому недопустимы.

В С переменным перечислительного типа можно присваивать целые значения:

```
enum Direction {up,down};
enum Direction d = 1; /* error: int присваивается Direction; ok в С */
```

В С++ гораздо больше ключевых слов, чем в С. Если какое-либо из них присутствует в С-программе как идентификатор, то во имя совместимости это нужно устранить.

Ключевые слова в C++ (но не в C)					
<i>and</i>	<i>and_eq</i>	<i>asm</i>	<i>bitand</i>	<i>bitor</i>	<i>bool</i>
<i>catch</i>	<i>class</i>	<i>compl</i>	<i>const_cast</i>	<i>delete</i>	<i>dynamic_cast</i>
<i>explicit</i>	<i>export</i>	<i>false</i>	<i>friend</i>	<i>inline</i>	<i>mutable</i>
<i>namespace</i>	<i>new</i>	<i>not</i>	<i>not_eq</i>	<i>operator</i>	<i>or</i>
<i>or_eq</i>	<i>private</i>	<i>protected</i>	<i>public</i>	<i>reinterpret_cast</i>	<i>static_cast</i>
<i>template</i>	<i>this</i>	<i>throw</i>	<i>true</i>	<i>try</i>	<i>typeid</i>
<i>typename</i>	<i>using</i>	<i>virtual</i>	<i>wchar_t</i>	<i>xor</i>	<i>xor_eq</i>

В C некоторые из ключевых слов C++ являются макросами, определенными в стандартных заголовочных файлах:

Ключевые слова C++ (макросы в C)							
<i>and</i>	<i>and_eq</i>	<i>bitand</i>	<i>bitor</i>	<i>bool</i>	<i>compl</i>	<i>false</i>	
<i>not</i>	<i>not_eq</i>	<i>or</i>	<i>or_eq</i>	<i>true</i>	<i>wchar_t</i>	<i>xor</i>	<i>xor_eq</i>

То есть в C они могут участвовать в проверках условных препроцессорных директив **#ifdef**, переопределяться и т.д.

В языке C глобальные объекты данных могут несколько раз объявляться в одной и той же единице трансляции без использования спецификатора **extern**. Если инициализатор присутствует только в одном из них, то объект считается определенным один раз. Например:

```
int i; int i; /* определяет или объявляет единственную целую переменную i; не C++ */
```

В C++ любая сущность определяется один раз (§9.2.3).

В языке C++ класс не может иметь имя, совпадающее с именем, определенным оператором **typedef** для ссылки на другой тип в той же самой области видимости (§5.7).

В языке C тип **void*** можно использовать в правой части операции присваивания или при инициализации переменной любого указательного типа; в C++ этого делать нельзя (§5.6). Например:

```
void f(int n)
{
  int* p = malloc (n*sizeof(int) ); /* не C++ */
}
```

Язык C разрешает переходы **goto** к помеченным метками операторам (§A.6) в блок инициализации, что запрещает делать язык C++.

В языке C глобальная константа по определению имеет внешнее связывание; в C++ это не так и ее требуется инициализировать, если только она не объявляется явно с модификатором **extern** (§5.4).

В языке C имена вложенных структур располагаются в той же области видимости, что и объемлющая структура. Например:

```

struct S
{
    struct T { /* ... */ };
    // ...
};

struct T x;           /* ok в C - означает S::T x; Не C++ */

```

В С массив может инициализироваться инициализатором, в котором значений больше, чем элементов у массива. Например:

```
char v[5] = "Oscar"; /* ok в C, терминальный ноль не используется. Не C++ */
```

В.2.3. Нежелательные особенности

Помечая ту или иную особенность языка как *нежелательную* (*deprecated*), комитет по стандартизации выражает желание устранить ее. Однако у комитета нет права удалить из языка широко используемые средства (как бы избыточны или опасны они не были). Эти пометки служат настоятельным советом программисту избегать помеченные особенности.

Ключевое слово **static**, которое в общем случае означает «статическое выделение памяти», может использоваться и для индикации факта, что функция или объект локальны по отношению к единице трансляции. Например:

```

// file1:
static int glob;

// file2:
static int glob;

```

Эта программа имеет две целые переменные с именем **glob**. Каждая из этих **glob** используется исключительно функциями из их единицы трансляции.

Применение **static** для отражения свойства «локально для единицы трансляции» помечено в С++ как нежелательное (*deprecated*). Вместо этого следует применять неименованные пространства имен (§8.2.5.1).

Неявное приведение из строкового литерала в (неконстантный) **char*** также объявлено нежелательным. Используйте именованные массивы элементов типа **char**, или избегайте присваивания строковых литералов переменным типа **char*** (§5.2.2).

Приведение типа в стиле языка С также помечено как нежелательное, ибо в языке С++ введены новые операции приведения типа. Программисты должны отнестись к этому серьезно. Там, где требуются явные приведения типа, операции **static_cast**, **reinterpret_cast**, **const_cast** или их комбинации делают все необходимое. Новые операции приведения (по сравнению с приведением в стиле С) точнее отражают их назначение и более заметны в тексте программы (§6.2.7).

В.2.4. Код на С++, не являющийся кодом на С

Здесь перечисляются средства С++, отсутствующие в языке С. Они отсортированы по назначению. На самом деле, их можно классифицировать по-разному и многие средства служат нескольким целям, так что к данной классификации не стоит относиться слишком серьезно.

- Средства, предназначенные в первую очередь для удобства записи:
 1. Комментарии // (§2.3); добавляются в C99
 2. Поддержка ограниченных наборов символов (§C.3.1); частично добавляется в C99
 3. Поддержка расширенных наборов символов (§C.3.3); частично добавляется в C99
 4. Неконстантные инициализаторы для объектов со статическим хранением (§9.4.1)
 5. *const* в константных выражениях (§5.4, §C.5)
 6. Объявления как операторы (§6.3.1); добавляются в C99
 7. Объявления в инициализаторах операторов *for* (§6.3.3); добавляются в C99
 8. Объявления в условиях (§6.3.2.1)
 9. Имена структур не нужно предварять словом *struct* (§5.7)
- Средства, предназначенные в первую очередь для усиления системы типов:
 1. Проверка типов аргументов функций (§7.1); добавлено в C позже (§B.2.2)
 2. Безопасная по типам компоновка (§9.2, §9.2.3)
 3. Выделение свободной памяти операциями *new* и *delete* (§6.2.6, §10.4.5, §15.6)
 4. *const* (§5.4, §5.4.1); добавлено в C позже
 5. Логический тип *bool* (§4.2); частично добавляется в C99
 6. Новый синтаксис приведения типов (§6.2.7)
- Средства, связанные с пользовательскими типами:
 1. Классы (глава 10)
 2. Функции-члены (§10.2.1) и вложенные классы (§11.12)
 3. Конструкторы и деструкторы (§10.2.3, §10.4.1)
 4. Производные классы (глава 12, глава 15)
 5. Виртуальные функции и абстрактные классы (§12.2.6, §12.3)
 6. Режимы доступа *public/protected/private* (§10.2.2, §15.3, §C.11)
 7. Дружественные функции (§11.5)
 8. Указатели на члены классов (§15.5, §C.12)
 9. Статические члены (§10.2.4)
 10. Ключевое слово *mutable* (§10.2.7.2)
 11. Перегрузка операций (глава 11)
 12. Ссылки (§5.5)
- Средства, предназначенные в первую очередь для организации программ:
 1. Шаблоны (глава 13, §C.13)
 2. Встраиваемые функции (§7.1.1); добавляются в C99
 3. Аргументы по умолчанию (§7.5)
 4. Перегрузка функций (§7.4)

5. Пространства имен (§8.2)
6. Операция разрешения области видимости (операция ::, §4.9.4)
7. Обработка исключений (§8.3; глава 14)
8. Средства RTTI (§15.4)

Ключевые слова, добавленные в C++ (§B.2.2), можно использовать для идентификации большинства средств, специфичных для этого языка. В то же время, некоторые средства C++, такие как перегрузка функций или константы в *константных* выражениях, не идентифицируются по ключевым словам. Кроме перечисленных особенностей стандартная библиотека C++ (§16.1.2) по большей части специфична именно для C++.

Макрос `__cplusplus` применяется для того, чтобы выбрать компилятор C или C++ (§9.2.4).

В.3. Старые реализации C++

Язык C++ интенсивно используется с 1983 года (§1.4). С тех пор сменилось несколько его версий и множество независимо выполненных реализаций. Фундаментальная цель стандартизации языка заключалась в том, чтобы дать разработчикам реализаций и пользователям единое определение C++. В то же время, пока стандартное определение не распространится широко в среде программистов, придется смириться с неприятным фактом, что не каждая реализация поддерживает все средства, описанные в этой книге.

К сожалению, часто люди, начинающие изучать язык C++, сталкиваются с реализациями пятилетней давности. Как правило, это объясняется их широкой распространенностью и бесплатностью. Имея выбор, ни один профессионал не станет работать на таком антиквариате. Для новичков же применение устаревших компиляторов оборачивается серьезными скрытыми потерями. Например, недостаток современных языковых средств и библиотек заставляет их бороться с проблемами, которые уже были успешно решены в новых реализациях. Кроме того, наносится серьезный вред стилю написания программ и порождает неточный взгляд на то, каков же язык C++ на самом деле. Наилучшее подмножество для первичного изучения и использования вовсе не сводится к набору низкоуровневых средств (и не к общему подмножеству языков C и C++; §1.2). Чтобы облегчить изучение и получить правильное представление о языке C++, я рекомендую опереться на стандартную библиотеку и шаблоны.

Первая коммерческая версия C++ появилась в 1985 году. Она соответствовала описанию языка, представленному в первом издании настоящей книги — там не было множественного наследования, шаблонов, средств RTTI, исключений и пространств имен. Сегодня я не вижу смысла пользоваться реализацией, которая не поддерживала бы хотя бы некоторой части из перечисленных свойств. В 1989 году я добавил к языку множественное наследование, шаблоны и исключения. К сожалению, ранняя поддержка шаблонов и исключений оставляла желать лучшего. Если ваша реализация относится к этой фазе развития C++ — срочно выполните ее обновление до современного состояния.

В общем случае разумно применять компиляторы и среды, более-менее соответствующие стандарту и минимально опирающиеся на неопределенные, зависящие

от реализации средства. Выполняйте проектирование в предположении, что вам доступны все современные возможности, и только после этого разыскивайте необходимые компиляторы и инструменты. Это помогает улучшить структуру программы и облегчить ее сопровождение по сравнению со случаем, когда применяется минимальное общее подмножество языка. Также будьте осторожны с применением зависящих от реализации расширений языка и делайте это лишь в случае крайней необходимости.

В.3.1. Заголовочные файлы

Традиционно все заголовочные файлы имели расширение *.h*. То есть реализации C++ предоставляли заголовочные файлы *<map.h>*, *<iostream.h>* и т.д. А для совместимости они так делают и сейчас.

Когда комитету по стандартизации понадобились заголовочные файлы для пересмотренной версии стандартной библиотеки и для новых библиотечных средств, их именование превратилось в небольшую проблему — использование старых *h*-имен вступило бы в конфликт с задачей поддержания совместимости. Кроме того, применение суффиксов *h* все равно избыточно, так как *<>* и без того указывают на стандартные заголовочные файлы.

В итоге, стандартная библиотека предоставляет заголовочные файлы с бессуффиксными именами, такими как *<map>* и *<iostream>*. Объявления в этих файлах помещены в пространство имен *std*, в то время как старые заголовочные файлы (с расширениями *h*) размещают объявления в глобальном пространстве имен. Рассмотрим пример:

```
#include <iostream>

int main ()
{
    std::cout<< "Hello, world!\n";
}
```

Если у вас не получается скомпилировать этот пример, попробуйте более традиционную версию:

```
#include <iostream.h>

int main ()
{
    cout << "Hello, world!\n";
}
```

Некоторые из наиболее серьезных проблем с переносимостью возникают из-за непереносимых заголовочных файлов, причем стандартные заголовочные файлы вносят здесь лишь малую долю. Часто программы зависят от множества заголовочных файлов, присутствующих не на каждой системе, от объявлений, которые на разных системах располагаются в разных заголовочных файлах, и от объявлений, кажущихся стандартными (они располагаются в заголовочных файлах со стандартными именами), но не входящими ни в какие стандарты.

Полностью удовлетворительных решений проблемы несовместимости заголовочных файлов не существует. Самая общая идея заключается в максимальной локализации (компактизации) различий и введения дополнительного уров-

ня косвенности, дабы избежать прямых зависимостей от несовместимых заголовочных файлов. К примеру, пусть необходимые нам объявления на разных системах располагаются в разных заголовочных файлах. Тогда можно включить в приложение единственный промежуточный (специфичный для данного приложения) заголовочный файл, который, в свою очередь, будет включать разные заголовочные файлы для разных систем. Аналогично, если некоторая функциональность немного по-разному предоставляется на разных системах, мы можем обращаться к ней через специфичные для приложения интерфейсные классы и функции.

В.3.2. Стандартная библиотека

Неудивительно, что реализации библиотек C++, предшествовавших стандарту, могли не содержать некоторых частей стандартной библиотеки. В большинстве своем они будут содержать потоки ввода/вывода, нешаблонный класс *complex*, те или иные строковые классы, а также библиотеку языка C. Как правило, будут отсутствовать классы *map*, *list*, *valarray* и т.п. Применяйте в таком случае сторонние библиотеки, допускающие переход к стандарту при обновлении вашей системы до современного состояния. Но в любом случае лучше использовать нестандартные классы строк, списков и ассоциативных массивов, чем возвращаться к низкоуровневому программированию в стиле C. Имейте также в виду, что доступны для бесплатного использования хорошие реализации STL-части стандартной библиотеки (главы 16, 17, 18 и 19).

Ранние реализации стандартной библиотеки также не совсем полны. Например, некоторые контейнеры могут не поддерживать аллокаторов (распределителей памяти), а другие могут требовать явного указания аллокаторов для каждого класса. Похожие проблемы могут касаться аргументов, определяющих политику алгоритма, например критерий сравнения:

```
list<int> li; // ok, но некоторые реализации требуют аллокатора
list<int, allocator<int> > li2; // ok, но некоторые реализации не реализуют аллокаторы

map<string, Record> m1; // ok, но некоторые реализации требуют операцию <
map<string, Record, less<string> > m2;
```

Пользуйтесь той версией, которую допускает ваша реализация.

Ранние реализации C++ предоставляли *istream* и *ostream* в *<istream.h>*, а не *istream* и *ostream* из *<sstream>*, причем работали эти потоки напрямую с *char[]* (см. §21.10[26]).

Достандартные потоки не были параметризуемыми. В частности, шаблоны с префиксом *basic_*, появились только в стандарте, класс *basic_ios* назывался *ios*, а *iosstate* назывался *io_state*.

В.3.3. Пространства имен

Если ваша реализация не поддерживает пространств имен, отражайте логическую структуру программы соответствующей структурой исходных файлов (глава 9). Аналогично, применяйте заголовочные файлы для отражения интерфейсов, предоставляемых разным реализациям, а также для языка C.

В отсутствие пространств имен вообще используйте *static* для компенсации отсутствия неименованного пространства имен. Также для глобальных имен применяйте идентифицирующие их префиксы:

```
class bs_string { /* ... */ };
typedef int bs_bool;
class joe_string;
enum joe_bool {joe_false, joe_true};
```

Внимательно выбирайте префикс. Существующие библиотеки C и C++ переполнены разными префиксами.

В.3.4. Ошибки выделения памяти

До появления в языке C++ обработки исключений операция *new* возвращала нуль, когда не удавалось выделить память. В стандарте C++ операция *new* по умолчанию генерирует в таком случае исключение *bad_alloc*. В любом случае, перехватывая *bad_alloc* или реагируя на возврат нуля, трудно для большинства систем предложить что-то более интересное, чем просто сообщение об ошибке.

Когда по каким-то причинам нужно придерживаться старого способа индикации об ошибке выделения памяти, то в случае, если не задан *_new_handler*, применение аллокатора *nothrow* заставляет возвращать нуль в ошибочных случаях:

```
X* p1 = new X;    // генерируется bad_alloc при отсутствии необходимой памяти
X* p2 = new (nothrow) X; // возвращает 0 при отсутствии необходимой памяти
```

В.3.5. Шаблоны

Стандарт C++ ввел некоторые новые элементы шаблонов и уточнил работу со старыми (ранее имевшимися) средствами.

Если ваша реализация не поддерживает частичную специализацию, используйте для шаблона иное имя (в противном случае он был бы специализацией):

```
template<class T> class plist: private list<void*>
{
    // ...
};
```

Если ваша реализация не поддерживает шаблонных членов шаблонов, некоторые приемы программирования станут недостижимыми. В частности, шаблонные члены шаблонов позволяют программистам выполнять конструирование и преобразование типа с гибкостью, недостижимой иными способами (§13.6.2). Рассмотрим пример:

```
template<class T> class X
{
    // ...
    template<class A> X(const A& a);
};
```

В отсутствие шаблонных членов шаблонов этот пример придется переделать так, чтобы ограничить себя набором определенных типов:

```
template<class T> class X
{
    // ...
    X(const A1& a) ;
    X(const A2& a) ;
    // ...
};
```

В большинстве ранних реализаций для всех функций-членов шаблонов генерировался код в момент конкретизации шаблона (template instantiation). Это могло приводить к ошибкам для неиспользуемых функций-членов (§С.13.9.1). Решение состоит в том, чтобы поместить определение функций-членов позже объявления класса. Например, вместо

```
template<class T> class Container
{
    // ...
public:
    void sort() { /* используем < */ }
};

class Glob { /* для Glob нет операции < */ };
Container<Glob> cg; // некоторые старые реализации пытаются определить
                  // Container<Glob>::sort();
```

используйте

```
template<class T> class Container
{
    // ...
public:
    void sort() ;
};

template<class T> void Container<T>::sort() { /* используем < */ }

class Glob { /* для Glob нет операции < */ };
Container<Glob> cg; // нет проблем до тех пор, пока не вызывается cg.sort()
```

Кроме того, ранние реализации не допускали использования членов классовых шаблонов, объявленных позже точки использования, как в следующем примере:

```
template<class T> class Vector
{
public:
    T& operator [] (size_t i) { return v[i] ; } // v объявляется ниже
    // ...

private:
    T* v; // не найдено!
    size_t sz;
};
```

Тогда нужно либо переупорядочить объявления членов шаблона, либо расположить определения функций-членов после объявления шаблона класса.

Некоторые из ранних реализаций не понимают умолчательных значений для параметров шаблона (§13.4.1). В таком случае каждому формальному параметру типа шаблона нужно в обязательном порядке сопоставить явный аргумент. Например:

```
template<class Key, class T, class LT = less<T> > class map
{
    // ...
};

map<string, int> m;                // Oops: умолчательные аргументы не реализованы
map<string, int, less<string> > m2; // обход проблемы: все указываем явно
```

В.3.6. Инициализаторы в операторах for

Рассмотрим следующий код:

```
void f(vector<char>& v, int m)
{
    for (int i=0; i<v.size() && i<=m; ++i) cout << v[i];
    if (i == m)    // error: i недостижимо после конца for
    {
        // ...
    }
}
```

Такой код обычно работал, поскольку в изначальном определении C++ область видимости переменной цикла совпадала с областью видимости, в которую входил сам оператор **for**. Если вам встретится такой код, просто расположите определение переменной цикла перед оператором **for**:

```
void f2(vector<char>& v, int m)
{
    int i = 0;

    for (; i<v.size() && i<=m; ++i) cout << v[i];
    if (i==m)
    {
        // ...
    }
}
```

В.4. Советы

1. Для изучения C++ применяйте самые современные и самые полные из доступных вам реализаций стандарта C++; §B.3.
2. Общее подмножество языков C и C++ — не лучшее подмножество для первоначального изучения C++; §1.6, §B.3.
3. Помните, что не все реализации C++ поддерживают его современный стандарт. Перед тем, как использовать в производственном коде какую-либо из новейших черт языка, опробуйте ее сначала на тестовых примерах, чтобы убедиться в поддержке стандарта вашей реализацией и уровнем достигаемой при этом производительности; §8.5[6-7], §16.5[10], §B.5[7].

4. Избегайте особенностей языка, помеченных как нежелательные (deprecated), например статических глобальных переменных. Избегайте приведения типа в C-стиле; §6.2.7, §B.2.3.
5. «Умолчательные *int*» запрещены — явно определяйте типы для функций, переменных, констант и т.д.; §B.2.2.
6. Преобразуя программу из языка C в язык C++, последовательно используйте прототипы функций и стандартные заголовочные файлы; §B.2.2.
7. Преобразуя программу из языка C в язык C++, переименуйте переменные, совпадающие с ключевыми словами языка C++; §B.2.2.
8. Преобразуя программу из языка C в язык C++, возвраты *malloc()* приводите к нужному типу, или замените вызовы *malloc()* на операцию *new*; §B.2.2.
9. При переходе от *malloc()* и *free()* к *new* и *delete*, подумайте о применении *vector*, *push_back()* и *reserve()* вместо *realloc()*; §3.8, §16.3.5.
10. Преобразуя программу из языка C в язык C++, помните о том, что нет неявного преобразования из *int* в перечисления; при необходимости используйте явные преобразования; §4.8.
11. Объекты из пространства имен *std* определяются в бессуффиксных заголовочных файлах (например, *std::cout* определяется в файле *<iostream>*). Старые реализации объявляют средства стандартной библиотеки в глобальном пространстве имен и располагают их в заголовочных файлах с суффиксом *.h* (*: :cout* определяется в *<iostream.h>*); §9.2.2, §B.3.1.
12. Если старый код проверяет возврат операции *new* на ноль, то нужно его исправить в сторону перехвата исключения *bad_alloc*, или применить *new (nothrow)*; §B.3.4.
13. Если ваша реализация не поддерживает умолчательных значений параметров шаблона, указывайте аргументы явно. Часто с помощью оператора *typedef* можно избежать повторных указаний аргументов (аналогично тому, как *typedef string* помогает избежать написания *basic_string<char, char_traits<char>, allocator<char>>*); §B.3.5.
14. Используйте *<string>* для доступа к *std::string* (*<string.h>* содержит функции для работы с C-строками); §9.2.2, §B.3.1.
15. Для каждого стандартного заголовочного файла *<X.h>*, помещающего объявления в глобальное пространство имен, заголовочный файл *<eX>* помещает эти имена в пространство имен *std*; §B.3.1.
16. Многие системы имеют заголовочный файл *"String.h"*, определяющий строковый тип. Отметим, что такие строки отличаются от строк *string* стандартной библиотеки.
17. Предпочитайте стандартные средства нестандартным; §20.1, §B.3, §C.2.
18. Используйте *extern "C"* при объявлении C-функций; §9.2.4.

В.5. Упражнения

1. (*2.5) Возьмите С-программу и преобразуйте ее в программу на C++. Составьте список используемых конструкций, не являющихся конструкциями языка C++, и определите, соответствуют ли они стандарту ANSI C. Сначала приведите программу в строгое соответствие с ANSI C (прототипы и т.д.), а затем — с C++. Оцените время, необходимое для подобных преобразований в случае программы, содержащей 100000 строк кода.
2. (*2.5) Напишите программу, помогающую конвертировать С-программы на язык C++ путем переименовывания переменных, совпадающих с ключевыми словами C++, путем замены **malloc**() на операции **new** и т.д. Намек: не пытайтесь достичь полного совершенства.
3. (*2) Замените все вызовы **malloc**() в C++-программах, написанных в стиле языка C, на операции **new**. Подсказка: §B.4[8-9].
4. (*2.5) В C++-программе, написанной в стиле языка C, минимизируйте использование макросов, глобальных переменных, неинициализированных переменных и приведений типа в C-стиле.
5. (*3) Возьмите C++-программу, полученную в результате грубой переделки из С-программы, и покритикуйте ее на предмет локализации информации, абстракции, читабельности, расширяемости и возможности повторного использования ее частей. Выполните хотя бы одно существенное изменение программы, отталкиваясь от вашей критики.
6. (*2) Возьмите маленькую (скажем, строк на 500) C++-программу и преобразуйте ее в программу на C. Сравните оригинал и полученный результат по размеру и возможностям сопровождения.
7. (*3) Напишите небольшой набор тестовых программ для выяснения, поддерживает ли ваша C++-реализация самые последние стандарты. Например, какова область видимости переменной, определенной в инициализирующей секции оператора **for** (§B.3.6)?, поддерживаются ли умолчательные значения параметров шаблонов (§B.3.5)?, поддерживаются ли шаблонные члены шаблонов (§13.6.2)?, поддерживается ли поиск имен по аргументам (§8.2.6)? Подсказка: §B.2.4.
8. (*2.5) Возьмите C++-программу, использующую заголовочный файл **<X.h>** и преобразуйте ее так, чтобы она использовала заголовочные файлы **<X>** и **<cX>**. Минимизируйте применение директив **using**.

Приложение С

Технические подробности

*Глубоко в сердце сознания и Вселенной — есть смысл.
— Слартибартфаст*

Стандарт — символные наборы — целые литералы — константные выражения — продвижения и преобразования — многомерные массивы — битовые поля и объединения — управление памятью — автоматическая сборка мусора — пространства имен — контроль доступа — указатели на члены данных — шаблоны — *статические* члены — друзья — шаблоны в качестве параметров шаблона — выводение аргумента шаблона — шаблоны и ключевое слово *typename* — конкретизация — связывание имен — шаблоны и пространства имен — явная конкретизация — советы.

С.1. Введение и обзор

Данная глава содержит технические детали и примеры, которые не совсем вписываются в то, как я представляю основные свойства языка C++ и их применение. Эти технические детали важны при реальном написании программ и еще более важны при чтении текстов ранее написанных программ, которые могут эти детали содержать. В то же время, я считаю эти детали техническими, поскольку они не должны отвлекать внимание студентов от важнейшей задачи первоначального изучения языка C++, а программистов — от написания программ в стиле, обеспечивающем (насколько это вообще возможно) ясное и непосредственное отражение проектных идей.

С.2. Стандарт

Вопреки устоявшемуся мнению, строгое следование стандарту языка и библиотеки не гарантирует хорошего исходного кода программы и ее переносимости. Стандарт ничего не говорит о том, плох или хорош конкретный фрагмент программы; он просто формулирует, на что программист может рассчитывать в разных реализациях, а на что не может. Вполне можно написать ужасную программу в полном

соответствии со стандартом, в то время как множество успешных программ могут применять особенности языка, не описываемые стандартом.

Много важных вещей стандарт отдает на откуп реализациям. В таких случаях реализации должны обеспечить хотя и специфическое, но четко определенное и хорошо документированное поведение языковых конструкций. Например:

```
unsigned char c1 = 64;    // однозначно: char всегда имеет по крайней мере 8 бит
                          // и всегда может содержать значение 64
unsigned char c2 = 1256; // зависит от реализации: урезание в случае 8-битного char
```

Инициализация **c1** четко определена, поскольку тип **char** обязан содержать по крайней мере 8 бит. В то же время, инициализация **c2** зависит от реализации, так как точное количество бит в **char** определяется не стандартом, а реализацией. Например, если в **char** всего 8 бит, то значение **1256** усекается до **232** (§С.6.2.1). Большая часть зависящих от реализации свойств языка коррелирует с различиями в железе, на котором программе предстоит работать.

При написании реальных программ приходится считаться с их поведением, зависящим от реализации. Такова цена за возможность эффективно работать на разных системах. Например, язык был бы проще, если бы тип **char** был жестко привязан к 8 битам, а тип **int** — к 32 битам. Однако имеются системы с 16- или 32-битовыми символьными наборами, а целые числа в реальных задачах не умещаются в 32 бита — для дисков емкостью более 32 Гбайт более уместны 48-битовые или 64-битовые числа.

Для улучшения переносимости лучше всего четко и явно обозначить места программы, зависящие от реализации, а особо тонкие различия тщательно локализовать и задокументировать. Обычная практика заключается в том, чтобы все зависимости размеров от аппаратуры представить в виде констант и определений типов в некотором заголовочном файле. Стандартная библиотека с этой целью предоставляет **numeric_limits** (§22.2).

Неопределенное поведение более коварно. Конструкция помечается стандартом как *неопределенная* (*undefined*), если невозможно потребовать от реализации какого-либо разумного ее поведения. Как правило, программы, использующие конструкции с неопределенным поведением, ведут себя очень плохо. Например:

```
const int size = 4*1024;
char page[size];
void f()
{
    page[size+size] = 7;    // не определено
}
```

Возможные последствия этого фрагмента могут включать в себя перезаписывание некоторых не имеющих отношения к нему данных и генерирование аппаратных исключений (ошибок). От реализации не требуется в таких случаях осуществлять выбор между возможными последствиями. Там, где используются мощные технологии оптимизации кода, последствия применения в программе конструкций с неопределенным поведением вообще не поддаются предсказанию. Если имеется набор очевидных и легко реализуемых альтернатив, конструкция считается зависящей от реализации, а не как имеющая неопределенное поведение.

В процессе разработки и написания программ стоит уделить серьезное внимание, силы и время, чтобы гарантировать отсутствие в программе конструкций, не

соответствующих стандарту. Часто это можно выполнить с помощью специализированных инструментальных средств.

С.3. Символьные наборы

Примеры в этой книге составлены с учетом американского варианта международного 7-битного символьного набора ISO 646-1983, названного ASCII (ANSI3.4-1968). Это может породить следующие проблемы у людей, работающих с C++ в средах с другими символьными наборами:

- 1. ASCII содержит символы пунктуации и символы операций ([, { и !), которые могут быть недопустимыми в других символьных наборах.
- 2. Требуются обозначения для символов, не имеющих удобного представления (например, «новая строка» или «символ с числовым кодом 17").
- 3. ASCII не содержит символов (ζ, λ и др.), применяемых в языках, отличных от английского.

С.3.1. Ограниченные наборы символов

ASCII символы [,], {, }, | и \ занимают в символьном наборе позиции, трактуемые ISO как алфавитные. В большинстве европейских символьных наборов ISO-646 эти позиции заняты буквами, которых в английском алфавите нет. Например, в датском языке тут присутствуют гласные *Æ, æ, Ø, ø, Å å ä*. Без этих символов по-датски не напишешь сколько-нибудь осмысленного текста.

Чтобы выражать национальные символы переносимым способом при помощи минимальных символьных наборов, вводятся *триграфы (trigraphs)*. Это помогает распространению программ, но не облегчает чтение программ людьми. Долговременным решением проблемы могли бы стать локализованные (под национальные языки) средства разработки программ на C++. Но текущую помощь программистам в этом вопросе при использовании ограниченных символьных наборов предоставляют ключевые слова, *диграфы (digraphs)* и триграфы:

Ключевые слова		Диграфы	Триграфы
<i>and</i>	&&	<% {	??= #
<i>and_eq</i>	&=	%> }	??([
<i>bitand</i>	&	< : [	??< {
<i>bitor</i>		: >]	??/ \
<i>compl</i>	~	~: #	??)]
<i>not</i>	!	~: ~: ##	??> }
<i>or</i>			??* ^
<i>or_eq</i>	=		??!
<i>xor</i>	^		??- ~
<i>xor_eq</i>	^=		
<i>not_eq</i>	!=		

Программы с ключевыми словами и диграфами читаются лучше программ, использующих триграфы. Однако если символы, такие как {, недостижимы, то применение триграфов обязательно, дабы не потерять символы в строках и символьных константах. Например, '{' превращается в '??<'.
Некоторые люди вместо традиционных значков операций предпочитают использовать ключевые слова, например *and*.

C.3.2. Escape-символы

Небольшое количество символов имеют имена, включающие обратную косую черту в качестве escape-символа:

Название	ASCII	C++
newline (перевод строки)	NL (LF)	\n
horizontal tab (горизон-я табуляция)	HT	\t
vertical tab (вертик-я табуляция)	VT	\v
backspace (забой)	BS	\b
carriage return (возврат каретки)	CR	\r
form feed (перевод страницы)	FF	\f
alert (звонок)	BEL	\a
backslash (обратная косая черта)	\	\\
question mark (знак вопроса)	?	\?
single quote (апостроф)	'	\'
double quote (двойная кавычка)	"	\"
octal number (восьмерич. число)	ooo	\ooo
hex number (шестнадцатер. число)	hhh	\xhhh ...

Несмотря на свой внешний вид, это одиночные символы.
Символ можно представить в виде одной, двух или трех восьмеричных цифр (с предшествующим \) или шестнадцатеричным числом (с предшествующим \x). На число шестнадцатеричных цифр в такой последовательности ограничений нет. Последовательность восьмеричных или шестнадцатеричных цифр обрывается на первом отличном от них символе. Например:

Восьмеричная	Шестнадцатеричная	Десятичная	ASCII
'\6'	'\x6'	6	ACK
'\60'	'\x30'	48	'0'
'\137'	'\x05f'	95	'_'

Это позволяет представить любой символ из машинного символьного набора и, в частности, вставить такие символы в символьные строки (см. §5.2.2). Применение

любых цифровых нотаций для символов делает программу непереносимой между машинами с разными символьными наборами.

Допускается записывать символьный литерал с применением нескольких символов, например `'ab'`, но это считается архаичным, зависящим от реализации, так что этого лучше избегать.

Когда числовая константа в восьмеричной нотации внедряется в строку, целесообразно использовать для ее записи именно три цифры, иначе чтение такой строки вызывает затруднение (требуется следить за тем, цифра или нет стоит после числовой константы). Для шестнадцатеричных констант используйте две цифры. Например:

```
char v1[] = "a\xah\129"; //6 char: 'a' '\xa' 'h' '\12' '9' '\0'
char v2[] = "a\xah\127"; //5 char: 'a' '\xa' 'h' '\127' '\0'
char v3[] = "a\xad\127"; //4 char: 'a' '\xad' '\127' '\0'
char v4[] = "a\xad\0127"; //5 char: 'a' '\xad' '\012' '7' '\0'
```

С.3.3. Расширенные символьные наборы

Программа на C++ может писаться и представляться пользователю в рамках символьных наборов, более широких, чем набор из 127 символов ASCII. Там, где реализация поддерживает расширенные наборы символов, идентификаторы, комментарии, символьные константы и строки могут содержать символы вроде α , β и т.д. Для переносимости реализация должна также отображать эти символы в кодировку, содержащую лишь доступные для всех пользователей C++ символы. Так как такой перевод символов происходит до начала компиляции, то он не влияет на семантику программы.

Стандартное кодирование символов из расширенного набора символами менее широкого набора, поддерживаемое в C++, представляется последовательностью четырех или восьми шестнадцатеричных цифр:

```
universal-character-name:
    \u XXXXX XXXX
    \u XXXXX
```

Здесь *X* соответствует одной шестнадцатеричной цифре. Например, `\u1e2b`. Более короткое обозначение `\uXXXX` равносильно `\U0000XXXX`. Количество шестнадцатеричных цифр, не равное четырем или восьми, является лексической ошибкой.

Программист может непосредственно использовать такое кодирование, однако в первую очередь оно предназначено для реализации, чтобы последняя использовала ее для представления видимого пользователю расширенного набора более узким внутренним набором символов.

Если вы полагаетесь на специальные среды для представления идентификаторов символами расширенного набора, то вы, тем самым, снижаете переносимость программы. Такую программу труднее читать и понимать, если нет понимания использованного для идентификаторов и комментариев естественного языка. Следовательно, чтобы программа могла восприниматься программистами разных стран, лучше ограничиться английским и символами ASCII.

С.3.4. Знаковые и беззнаковые символы

В зависимости от реализации тип **char** может быть знаковым или беззнаковым. Это открывает дверь для неприятных сюрпризов. Например:

```
char c = 255;    // 255 - "все единицы", шестнадцатеричное 0xFF
int i = c;
```

Каково будет значение *i*? К сожалению, ответ не определен. Во всех известных мне реализациях он зависит от смысла преобразования к типу **int** «всех единиц» в **char**. На платформе SGI Challenge тип **char** беззнаковый, так что ответ будет **255**. На аппаратных платформах Sun SPARC или IBM PC тип **char** знаковый и ответ равен **-1**. В последнем случае компилятор может предупредить о преобразовании литерала **255** в значение **-1** для типа **char**. Однако C++ не имеет общего механизма для выявления подобного рода проблем. Чтобы избежать этой проблемы, можно вместо **char** указывать точно **unsigned char** или **signed char**. К сожалению, некоторые библиотечные функции, такие как **strcmp()**, принимают только тип **char** (§20.4.1).

Тип **char** должен вести себя идентично **signed char** или **unsigned char**. Тем не менее, все три типа считаются различными, так что нельзя смешивать указатели на разные **char**:

```
void f(char c, signed char sc, unsigned char uc)
{
    char* pc = &uc;           // error: нет преобразования указателей
    signed char* psc = pc;     // error: нет преобразования указателей
    unsigned char* puc = pc;   // error: нет преобразования указателей
    psc = puc;                // error: нет преобразования указателей
}
```

В то же время переменные этих типов можно свободно присваивать друг другу. Однако присваивание слишком больших значений переменной **signed char** (§C.6.2.1) по-прежнему не определено. Например:

```
void f(char c, signed char sc, unsigned char uc)
{
    c = 255;                  // зависит от реализации
    c = sc;                   // ok
    c = uc;                   // зависит от реализации
    sc = uc;                  // зависит от реализации
    uc = sc;                  // ok: преобразование к unsigned
    sc = c;                   // зависит от реализации
    uc = c;                   // ok: преобразование к unsigned
}
```

Ни одна из этих потенциальных проблем не возникает, если использовать просто **char**.

С.4. Типы целых литералов

В общем случае, тип целого литерала зависит от его формы, значения и суффикса:

- Если литерал десятичный и не имеет суффикса, то он относится к первому из следующих типов, достаточных для представления его значения: *int*, *long int*, *unsigned long int*.
- Если литерал восьмеричный и не имеет суффикса, то он относится к первому из следующих типов, достаточных для представления его значения: *int*, *unsigned int*, *long int*, *unsigned long int*.
- Если литерал имеет суффикс *u* или *U*, то его тип — первый из следующих двух типов, достаточных для представления его значения: *unsigned int*, *unsigned long int*.
- Если литерал имеет суффикс *l* или *L*, то его тип — первый из следующих двух типов, достаточных для представления его значения: *long int*, *unsigned long int*.
- Если литерал имеет суффикс *ul*, *lu*, *uL*, *Lu*, *Ul*, *IU*, *UL* или *LU*, то его тип — *unsigned long int*.

Например, число **100000** имеет тип *int* на машине с 32-битовыми *int*, но тип *long int* на машине с 16-битовыми *int* и 32-битовыми *long*. Аналогично, **0XA000** относится к типу *int* на машине с 32-битовыми *int*, но к типу *unsigned int* на машине с 16-битовыми *int*. Таких зависимостей от реализации можно избежать, если явно использовать суффиксы: **100000L** всегда (на всех машинах) имеет тип *long int*, а **0XA000U** — тип *unsigned int* (на всех машинах).

С.5. Константные выражения

Язык С++ требует *константных выражений* (*constant expressions*) для определения границ массивов (§5.2), меток оператора *case* (§6.3.2) и инициализаторов перечислений (§4.8). Вычисление константного выражения приводит к интегральной константе или константе перечисления. Такие выражения состоят из литералов (§4.3.1, §4.4.1, §4.5.1), элементов перечислений (§4.8) и констант, инициализируемых константными выражениями. В шаблонах также можно использовать целый параметр шаблона (§С.13.3). Литералы с плавающей точкой (§4.5.1) можно использовать, преобразовав их явным образом в интегральный тип. Функции, классовые объекты, указатели и ссылки можно использовать в качестве операндов операции *sizeof* (§6.2).

С интуитивной точки зрения, константные выражения — это просто выражения, которые компилятор может вычислить до начала компоновки программы (§9.1) и ее запуска на выполнение.

С.6. Неявное преобразование типов

Интегральные типы и типы с плавающей запятой (§4.1.1) можно свободно смешивать в арифметических выражениях и операциях присваивания. По возможности значения преобразуются таким образом, чтобы не терять информацию. К сожалению, преобразования с потерей значений также могут выполняться неявно. В на-

стоящем разделе приведены правила преобразований, связанные с этим проблемы и их решения.

С.6.1. Продвижения (promotions)

Неявные преобразования, сохраняющие значения, называют *продвижениями* (*promotions*). Перед выполнением арифметической операции выполняется *интегральное продвижение* (*integral promotion*) с целью получения значений типа *int* из более коротких целых типов. Заметьте, что эти продвижения не продолжают в сторону *long* (если только не задействован тип *wchar_t* или элемент перечисления со значением, слишком большим для *int*). Это соответствует исходной цели таких продвижений в языке С: привести операнды к естественным размерам для арифметических операций.

Интегральные продвижения таковы:

- *char*, *signed char*, *unsigned char*, *short int* или *unsigned short int* преобразуются в *int*, если *int* может отобразить все значения исходного типа; в противном случае они преобразуются в *unsigned int*.
- *wchar_t* (§4.3) или перечисления (§4.8) преобразуются к первому из перечисленных ниже типов, способному отобразить все их значения: *int*, *unsigned int*, *long* или *unsigned long*.
- Битовые поля (§С.8.1) преобразуются в *int*, если *int* может представить все значения битового поля; в противном случае осуществляется преобразование в *unsigned int*, если этот тип может представить все значения битового поля. В противном случае никакого интегрального продвижения не выполняется вообще.
- Тип *bool* преобразуется в *int*; *false* становится нулем, а *true* — единицей.

Продвижения используются как часть обычных арифметических преобразований (§С.6.3).

С.6.2. Преобразования

Фундаментальные типы могут преобразовываться друг в друга огромным количеством способов. По моему мнению, позволено слишком много преобразований. Например:

```
void f(double d)
{
    char c = d;    // внимание: double преобразуется к char
}
```

При написании программ вы должны стремиться избегать неопределенного поведения и преобразований, которые незаметным образом «выбрасывают» информацию. Компилятор может предупредить о многих сомнительных преобразованиях. По счастью, многие компиляторы так и делают.

С.6.2.1. Интегральные преобразования

Целое может преобразовываться в иной целый тип. Значение перечислимого типа может преобразовываться к целому типу.

Если целевой тип беззнаковый, результирующее значение есть просто такой набор битов исходного значения, который помещается в целевом типе (старшие биты при необходимости отбрасываются). Точнее, результат будет наименьшим беззнаковым целым, соответствующим остатку от деления исходного целого на 2^n в степени n , где n — это число битов, применяемых для представления беззнакового целого. Например:

```
unsigned char uc = 1023; // 111111111: uc становится 11111111, то есть 255
```

Когда целевой тип знаковый, значение остается неизменным, если оно представимо целевым типом; в противном случае результат зависит от реализации:

```
signed char sc = 1023; // зависит от реализации
```

Возможны следующие результаты: **127** и **-1** (§С.3.4).

Логическое значение или значение перечислимого типа могут неявно преобразовываться в свой целочисленный эквивалент (§4.2, §4.8).

С.6.2.2. Преобразования чисел с плавающей запятой

Значение с плавающей запятой может быть преобразовано в иной тип с плавающей запятой. Если исходное значение точно представимо в целевом типе, результат совпадает с исходным числовым значением. Если исходное значение располагается между двумя соседними значениями целевого типа, результат будет одним из этих значений. В противном случае результат не определен. Например:

```
float f= FLT_MAX; // наибольшее значение типа float
double d=f; // ok:d==f
float f2 = d; // ok:f2==f
double d3 = DBL_MAX; // наибольшее значение типа double
float f3 = d3; // не определено, если FLT_MAX<DBL_MAX
```

С.6.2.3. Преобразования указателей и ссылок

Указатель на любой тип объекта может неявно преобразовываться в тип **void*** (§5.6). Указатель (ссылка) на производный класс может неявно преобразовываться в тип указателя (ссылки) на доступный и недвусмысленно определяемый базовый класс (§12.2). Отметим, что указатель на функцию или указатель на член класса не могут неявно преобразовываться в тип **void***.

Константное выражение (§С.5), значением которого служит **0**, может неявно преобразовываться в любой указательный тип или тип указателя на член класса (§5.1.1). Например:

```
int* p =
    ! ! ! ! !
    !! ! ! !
    ! !! ! !
    ! ! !!!!! !!!!! !!!!!;
```

Тип **T*** может неявно преобразовываться в **const T*** (§5.4.1). Аналогично, **T&** может неявно преобразовываться в **const T&**.

С.6.2.4. Преобразования указателей на члены классов

Указатели и ссылки на члены классов могут неявно преобразовываться так, как описано в §15.5.1.

С.6.2.5. Преобразования в логический тип

Указатели, значения интегральных типов и типов с плавающей запятой могут неявно преобразовываться в тип **bool** (§4.2). Ненулевые значения преобразовываются в **true**, а нулевые — в **false**. Например:

```
void f(int* p, int i)
{
    bool is_not_zero = p;    // true если p!=0
    bool b2 = i;             // true если i!=0
}
```

С.6.2.6. Преобразования «значение интегрального типа — значение с плавающей запятой»

Когда значение с плавающей запятой преобразуется в целое значение, его дробная часть отбрасывается. Другими словами, преобразование из числа с плавающей запятой в целое урезает исходное значение. Например, значение **int**(1.6) равно 1. Если урезанное значение не может быть представлено целевым типом, то результат не определен. Например:

```
int i = 2.7;           // становится 2
char b = 2000.7;       // не определено для 8-битных char
```

Преобразование из целых в типы с плавающей запятой с математической точки зрения корректно настолько, насколько это позволяет аппаратура. Потеря точности происходит в тех случаях, когда целое значение не удастся точно представить в виде значения с плавающей запятой. Например, следующее выражение

```
int i = float(1234567890);
```

оставит **i** со значением **1234567936** на машине с 32-битовыми **int** и **float**.

Ясно, что лучше избегать преобразований, которые потенциально могут исказить исходное значение. Компиляторы могут выявить и предупредить об очевидно наиболее опасных преобразованиях, таких как преобразование числа с плавающей запятой в целое, или **long int** в **char**. Однако в общем случае, выявление всех особенностей преобразований на этапе компиляции недостижимо, так что программисты не должны терять бдительности. Когда одной лишь бдительности недостаточно, можно вставить явные проверки. Например:

```
class check_failed {};

char checked(int i)
{
    char c = i;           // warning: not portable (§C.6.2.1)
    if(i != c) throw check_failed();
    return c;
}

void my_code(int i)
```

```
char c = checked (i) ;
// ...
}
```

Гарантированно контролируемые и переносимые урезания при преобразованиях требуют применения *numeric_limits* (§22.2).

С.6.3. Обычные арифметические преобразования

Эти преобразования выполняются над операндами любой бинарной (двухоперандной) операции, чтобы привести их к одному общему типу, который затем используется для типа результата операции:

1. Если один из операндов относится к типу *long double*, то другой операнд преобразуется к типу *long double*.
 - В противном случае, если один из операндов относится к типу *double*, то другой операнд преобразуется в *double*.
 - В противном случае, если один из операндов имеет тип *float*, то другой преобразуется к типу *float*.
 - В противном случае над обоими операндами выполняются интегральные продвижения (§С.6.1).
2. Затем, если один из операндов относится к типу *unsigned long*, то другой операнд преобразуется к *unsigned long*.
 - В противном случае, если один из операндов относится к типу *long int*, а другой — к типу *unsigned int*, то если *long int* в состоянии представить все значения типа *unsigned int*, операнд типа *unsigned int* преобразуется в *long int*; иначе оба операнда преобразуются к типу *unsigned long int*.
 - В противном случае, если один из операндов относится к типу *long*, то другой операнд преобразуется в *long*.
 - В противном случае, если один из операндов относится к типу *unsigned*, то другой операнд преобразуется в *unsigned*.
 - В противном случае оба операнда *int*.

С.7. Многомерные массивы

Нередко нам требуются векторы векторов, или векторы векторов из векторов и т.д. Как такие многомерные массивы представляются в С++? Для ответа на вопрос я сначала покажу, как для этого применить стандартный класс *vector*. Затем я рассмотрю применение лишь встроенных средств языков С и С++ для построения многомерных массивов.

С.7.1. Векторы

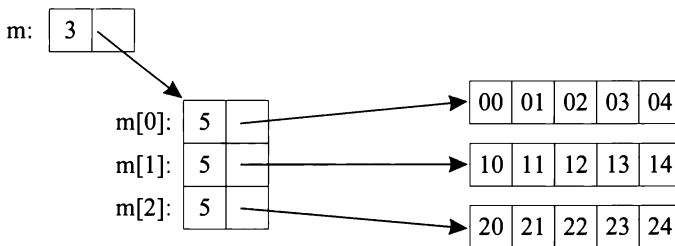
Общее решение можно построить на базе стандартного класса *vector* (§16.3):

```
vector<vector<int> > m (3, vector<int> (5) ) ;
```

Здесь создается вектор из трех векторов, каждый из которых содержит по 5 целочисленных элементов. Все 15 этих элементов получают умолчательное значение 0. Новые значения элементам можно присвоить следующим образом:

```
void print_m ()
{
    for (int i = 0; i < m.size(); i++)
    {
        for (int j = 0; j < m[i].size(); j++) m[i][j] = 10*i+j;
    }
}
```

или графически:



Каждый объект, сгенерированный из шаблона **vector**, содержит число элементов и указатель на элементы, которые в типичном случае содержатся в массиве. Для иллюстрации я задал каждому целому элементу значение, соответствующее его «координатам».

Доступ к элементам осуществляется двойной индексацией. Например, $m[i][j]$ — это j -ый элемент i -го вектора. Мы можем осуществить вывод m следующим образом:

```
void print_m ()
{
    for (int i = 0; i < m.size(); i++)
    {
        for (int j = 0; j < m[i].size(); j++) cout << m[i][j] << '\t';
        cout << '\n';
    }
}
```

что приведет к следующему результату:

```
0    1    2    3    4
10   11   12   13   14
20   21   22   23   24
```

Отметим, что m — это объект типа **vector**, содержащий другие **vector**, а вовсе не простой многомерный массив. Поэтому, в частности, можно осуществить изменение размера (§16.3.8) элемента. Например:

```
void init_ma (int ns)
{
    for (int i = 0; i < m.size(); i++) m[i].resize(ns);
}
```

Не требуется, чтобы вектора типа `vector<int>`, содержащиеся в `vector<vector<int> >`, имели одинаковый размер.

С.7.2. Массивы

Встроенные массивы служат источником многих ошибок в программах — особенно в случае многомерных массивов. А новичков они могут сильно смутить. По-возможности им лучше пользоваться типами `vector`, `valarray`, `string` и т.д.

Многомерные массивы есть массивы массивов; массив 3-на-5 объявляется так:

```
int ma [3] [5];    // 3 массива с 5 int в каждом
```

Инициализировать `ma` можно следующим образом:

```
void init_ma ()
{
    for (int i = 0; i<3; i++)
    {
        for (int j = 0; j<5; j++) ma [i] [j] = 10*i+j;
    }
}
```

или графически:

ma.	00	01	02	03	04	10	11	12	13	14	20	21	22	23	24
-----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Массив `ma` — это просто 15 целых чисел, к которым мы обращаемся так, как будто это 3 массива по 5 чисел в каждом. Кроме того, в памяти машины никакого объекта `ma`, представляющего массив нет — в памяти хранятся лишь элементы. Размеры 3 и 5 существуют лишь в исходном коде. При написании кода мы сами следим за этими размерами. Например, мы можем следующим образом выполнить вывод `ma`:

```
void print_ma ()
{
    for (int i = 0; i<3; i++)
    {
        for (int j = 0; j<5; j++) cout << ma [i] [j] << ' \t';
        cout << '\n';
    }
}
```

В языке С++ нельзя использовать запятую для отделения размеров массивов друг от друга, ибо запятая — это операция следования (§6.2.2). В целом, большинство ошибок с массивами отлавливаются компилятором. Например:

```
int bad [3, 5];           // error: запятая в константных выражениях не допускается
int good [3] [5];        // 3 массива по 5 int в каждом
int ouch = good [1, 4];   // error: good[l,4] это good[4]; int инициализируется типом int*
int nice = good [1] [4];
```

С.7.3. Передача многомерных массивов в функции

Рассмотрим функцию, которая должна манипулировать двумерной матрицей. Если размеры матрицы известны на момент компиляции, то никаких проблем нет:

```
void print_m35 (int m [3] [5] )
{
    for (int i = 0; i<3 ; i++)
    {
        for (int j = 0; j<5; j++) cout << m [i] [j] << '\t';
        cout << '\n';
    }
}
```

Матрица, представленная как многомерный массив, передается с помощью указателя (а не копируется; §5.3). Размер матрицы по первому измерению не имеет отношения к проблеме нахождения местоположения элемента; он просто сообщает, сколько элементов (здесь 3) и какого типа (здесь — типа *int*[5]) присутствует. Например, посмотрев на предыдущее определение *ma*, заметим, что зная лишь размер 5 по второму измерению, мы можем определить местоположение элемента *ma*[*i*][5] для любого *i*. Поэтому размер по первой размерности можно передать в качестве аргумента функции:

```
void print_mi5 (int m [] [5] , int dim1)
{
    for (int i = 0; i<dim1; i++)
    {
        for (int j = 0; j<5; j++) cout << m [i] [j] << '\t';
        cout << '\n';
    }
}
```

Более сложной является ситуация, когда нужно передавать оба размера. Очевидное «решение» не работает:

```
void print_mij (int m [] [] , int dim1 , int dim2)
{
    for (int i = 0; i<dim1; i++)
    {
        for (int j = 0; j<dim2; j++) cout << m [i] [j] << '\t'; // сюрприз!
        cout << '\n';
    }
}
```

Во-первых, объявление аргумента в виде *m* [] [] незаконно, ибо для нахождения элемента массива нужно знать его вторую размерность. Во-вторых, выражение *m*[*i*][*j*] совершенно справедливо интерпретируется как * (* (*m*+*i*) +*j*) , но это не то, что предполагал программист. Правильное решение таково:

```
void print_mij (int* m , int dim1 , int dim2)
{
    for (int i = 0; i<dim1; i++)
```

```
{
  for (int j = 0; j < dim2; j++) cout << m[i*dim2+j] << ' ' << '\t'; // заковыристо
  cout << '\n';
}
}
```

Выражение для доступа к элементам в функции **print_mij()** эквивалентно тому, что генерирует компилятор, когда знает вторую размерность.

При вызове этой функции, мы передает матрицу как обычный указатель:

```
int main ()
{
  int v[3][5] = { { 0, 1, 2, 3, 4 }, { 10, 11, 12, 13, 14 }, { 20, 21, 22, 23, 24 } };
  print_m35(v);
  print_mi5(v, 3);
  print_mij(&v[0][0], 3, 5);
}
```

Обратите внимание на `&v[0][0]` в последнем вызове; сработало бы и `v[0]`, так как это равносильно, но для `v` будет ошибка типа. Такой тонкий и запутанный код лучше спрятать поглубже. Если вам необходимо работать с многомерными массивами напрямую, попробуйте инкапсулировать имеющие к этому отношение участки кода. Тем самым, вы упростите задачу программисту, который потом будет работать с этой программой. Введение пользовательского типа «многомерный массив» с надлежащей операцией индексации избавит пользователей от хлопот с истинным расположением данных в массиве (§22.4.6).

Стандартный тип **vector** (§16.3) такими проблемами не страдает.

С.8. Экономия памяти

При написании нетривиальных программ наступает момент, когда приходится экономить память. Есть два способа «выжимания дополнительной памяти» из имеющейся:

1. Помещать в байт более одного объекта.
2. В разное время использовать одну и ту же память для хранения разных объектов.

Первый способ достигается применением *битовых полей (field)*, а второй способ — применением *объединений (union)*. Мы рассмотрим эти конструкции в последующих разделах. Многие случаи применения битовых полей и объединений относятся к крайним формам оптимизации, основанным на предположениях о конкретных схемах распределения памяти, затрудняющих переносимость программ. Программисту следует дважды подумать перед тем, как применять их. Часто лучше изменить способ управления данными и положиться на динамически выделяемую память (§6.2.6), а не на ее статическое распределение.

С.8.1. Битовые поля

Представляется странным использование целого байта (типы *char* или *bool*) для представления битовых величин (например, переключателя с положениями включено/выключено), но *char* — это наименьший объект C++, под который можно выделить адресуемую память (§5.1). Однако можно связать малюсенькие объекты друг с другом в битовые поля в структурах, поля которых определяются по количеству отведенных под них битов. Допускаются неименованные поля. Они не влияют на именованные поля, но помогают расположить последние более эффективно с аппаратной точки зрения:

```
struct PPN                                // физический номер страницы памяти для R6000
{
    unsigned int PFN : 22;    // номер страничного кадра
    int : 3;                 // не используется
    unsigned int CCA : 3;    // алгоритм согласования с кэшем
    bool nonreachable : 1;
    bool dirty : 1;
    bool valid : 1;
    bool global : 1;
};
```

Этот пример иллюстрирует также другое важное применение битовых полей: именование частей заданной извне раскладки памяти. Поля должны иметь интегральный тип или тип перечисления (§4.1.1). Невозможно получить адрес поля. Если не считать такой особенности, то в остальном поля могут использоваться так же, как другие переменные. Отметим, что поле *bool* реально представимо единственным битом. В ядре операционной системы или в отладчике тип *PPN* может использоваться следующим образом:

```
void part_of_VM_system (PPN*p)
{
    // ...
    if (p->dirty)                // содержимое изменилось
    {
        // копирование на диск
        p->dirty = 0;
    }
    // ...
}
```

Как это ни странно, применение битовых полей для упаковки нескольких переменных в одном байте не всегда экономит память. Это лишь экономит память под данные, но объем кода, манипулирующего такими данными, увеличивается на большинстве машин. Известно, что программы сжимаются существенно, когда битовые поля конвертируются в символы. Еще известно, что обращение к битовым полям выполняется медленнее, чем обращение к переменным типа *int* или *char*. Битовые поля — это просто удобное средство для сокрытия применения логических побитовых операций (§6.2.4) для извлечения информации из части машинного слова и для упаковки в часть машинного слова.

С.8.2. Объединения

Объединение (*union*) — это структура, в которой все члены расположены по одному и тому же адресу, так что в памяти машины объединение занимает ровно столько места, сколько требуется для его наибольшего поля. Естественно, в каждый конкретный момент времени объединение может хранить какое-то одно значение. Например, рассмотрим отдельную запись символьной таблицы, содержащую имя и значение:

```
enum Type {S, I};

struct Entry
{
    char* name;
    Type t;
    char* s;    // используем s если t==S
    int i;      // используем i если t==I
};

void f(Entry* p)
{
    if(p->t == S) cout << p->s;
    // ...
}
```

Поскольку членами *s* и *i* нельзя пользоваться одновременно, то выделяемое под них место в памяти тратится впустую. Этого можно избежать, сделав эти переменные членами объединения (*union*):

```
union Value
{
    char* s;
    int i;
};
```

В языке С++ нет встроенных средств слежения за тем, какого типа данные хранятся в объединении, так что эта задача ложится на плечи программиста:

```
struct Entry
{
    char* name;
    Type t;
    Value v;    // используем v.s если t==S; используем v.i если t==I
};

void f(Entry* p)
{
    if(p->t == S) cout << p->v.s;
    // ...
}
```

К сожалению, переход на объединение заставляет нас также изменить код, использующий *s*, на *v.s*. Этого можно избежать применением анонимного объединения (*anonymous union*), которое не имеет имени и, значит, не является типом. Оно просто гарантирует, что его члены расположены по одному и тому же адресу памяти:

```

struct Entry
{
    char* name;
    Type t;

    union
    {
        char* s; // используем s если t==S
        int i;    // используем i если t==I
    };
};

void f(Entry* p)
{
    if(p->t == S) cout << p->s;
    // ...
}

```

В результате, код, использующий **Entry**, не претерпевает изменений.

К сожалению, невозможно гарантировать, что объединение всегда читается корректно, то есть в соответствии с типом записанного в него значения (что порождает тонкие ошибки). Поэтому целесообразно инкапсулировать объединение, что может дать гарантию соответствия между читаемым значением и его типом (§10.6[20]).

Объединения иногда используются неразумно с целью преобразования типов. Обычно это свойственно программистам с опытом работы на языках, не имеющих операций явного преобразования типов и где приходится из-за этого пускаться на всякие хитрости. Следующий пример показывает преобразование из **int** в **int***, основанное на их битовой эквивалентности:

```

union Fudge
{
    int i;
    int* p;
};

int* cheat(int i)
{
    Fudge a;
    a.i = i;
    return a.p; // плохо
}

```

Это, на самом деле, вовсе и не преобразование: на некоторых машинах **int** и **int*** занимают разный объем памяти, а на других **int** не может иметь нечетный адрес. Такое применение объединений опасно и непереносимо, а для преобразования типов существуют явные и переносимые способы (§6.2.7).

Иногда объединения преднамеренно используются для того, чтобы избежать преобразования типов. Например, **Fudge** можно применить с тем, чтобы узнать числовое представление нулевого указателя:

```

int main ()
{
    Fudge foo;
}

```

```
foo.p = 0;  
cout << "the integer value of the pointer 0 is " << foo.i << '\n';  
}
```

С.8.3. Объединения и классы

Встречаются нетривиальные объединения, у которых имеются поля с размером, много большим размеров других, часто используемых полей. Поскольку размер объединения совпадает с наибольшим размером его полей, то память расходуется напрасно. Этого можно избежать, применив вместо объединений производные классы.

Объединения не могут иметь поля данных, имеющих тип класса с конструктором, деструктором и операцией присваивания, ибо компилятор не знает, какого типа член объединения ему нужно уничтожать.

С.9. Управление памятью

В С++ имеются три фундаментальных способа использования памяти:

Статическая память, в которую компоновщик помещает объект на все время выполнения программы. В статической памяти располагаются глобальные переменные и переменные из пространств имен, статические члены классов (§10.2.4) и статические локальные переменные функций (§7.1.2). Объект, размещаемый в статической памяти, конструируется один раз и сохраняется до окончания работы программы. У него все время один и тот же адрес. Статические объекты могут вызывать проблемы в многопоточных программах (с разделяемым адресным пространством), поскольку они используются совместно и во избежание проблем требуется их блокировка.

Автоматическая память, в которой располагаются аргументы функций и локальные переменные. При каждом входе в функцию (блок) создается их копия. Такая память создается и уничтожается автоматически — отсюда и ее название. Ее также называют *стековой памятью (memory on the stack)*. Если вы хотите явно подчеркнуть характер такой памяти, С++ предоставляет для этого необязательное ключевое слово *auto*.

Свободная память, которую программы запрашивают явным образом для помещения в нее динамически создаваемых объектов, и которую они могут явным образом освободить по исчерпанию нужды в этих объектах (*операции new и delete*). Когда программе нужна дополнительная память, она запрашивает ее у операционной системы операцией *new*. Свободную память называют также *динамической памятью (dynamic memory)* или *кучей (heap)*.

Что касается программиста, то автоматическая и статическая память используются просто, очевидно и неявно. Интерес представляет лишь использование свободной памяти. Хотя выделение памяти (операцией *new*) и просто, при отсутствии единой и согласованной стратегии освобождения памяти и передачи ее назад менеджеру свободной памяти, вся память может быть исчерпана, что особо характерно для долго работающих программ.

Простейшая стратегия заключается в том, чтобы передать объекты в свободной памяти под управление автоматических объектов. Как следствие, многие контейнеры

именно так и реализуются, то есть управляют элементами, расположенными в свободной памяти (§25.7). Например, автоматический объект типа **String** (§11.12) управляет последовательностью символов, расположенных в свободной памяти, и автоматически освобождает память, когда он сам выходит из области видимости. Все стандартные контейнеры (§16.3, глава 17, глава 20, §22.4) могут так реализовываться.

С.9.1. Автоматическая сборка мусора

Когда описанного выше регулярного подхода недостаточно, программист мог бы воспользоваться менеджером памяти, который ищет висячие объекты (на которые никто не ссылается) и уничтожает их (освобождает занятую ими память). Это называют автоматической сборкой мусора или просто *сборкой мусора* (*garbage collection*). Соответствующий менеджер, естественно, называется *сборщиком мусора* (*garbage collector*).

Фундаментальная идея, лежащая в основе сборки мусора, заключается в том, что если в программе нет ссылок на некоторый объект, то этот объект для нее потерян и его самое время уничтожить сборщиком мусора. Например:

```
void f()
{
    int* p = new int;
    p = 0;
    char* q = new char;
}
```

Здесь присваиванием $p=0$ мы обрываем связь программы и объекта типа **int**, так что память, выделенную под этот объект можно безболезненно освободить и предоставить для новых объектов. Поэтому последующую переменную типа **char** можно расположить в той же области памяти, и указатель **q** будет иметь то же самое значение, что было у переменной **p**.

Стандарт не требует обязательной реализации автоматической сборки мусора, но она явочным порядком внедряется там, где ручная сборка мусора обходится слишком дорого. При сравнении альтернатив принимайте во внимание производительность программы, расход памяти, надежность, переносимость, материальные затраты на программирование, стоимость сборщика мусора и предсказуемость поведения программы.

С.9.1.1. Замаскированные указатели

Что означает, что на объект нет ссылок? Рассмотрим пример:

```
void f()
{
    int* p = new int;
    long i1 = reinterpret_cast<long>(p) & 0xFFFF0000
    long i2 = reinterpret_cast<long>(p) & 0x0000FFFF
    p = 0;

    // точка #1: в этой точке нет указателя на int

    p = reinterpret_cast<int*>(i1 | i2);
    // теперь int снова адресуется указателем
}
```

Указатели, хранящиеся в программе в переменных иных типов, называются «замаскированными указателями». В частности, указатель, первоначально хранившийся в переменной *p*, был замаскирован в целых переменных *i1* и *i2*. Но сборщику мусора нет дела до замаскированных указателей; дойдя до точки **#1**, он преспокойно освободит память, выделенную ранее под целое значение. Фактически, подобные программы вряд ли будут корректно работать и без сборщика мусора, поскольку применение *reinterpret_cast* для преобразования целых в указатели в лучшем случае зависит от реализации.

Объединение, которое содержит как указатели, так и не указатели, представляет особую проблему для сборщика мусора, поскольку нет способа узнать, содержит объединение в данный момент указатель или нет. Например:

```
union U
{
    int* p;
    int i;
};

void f(U u, U u2, U u3)
{
    u.p = new int;
    u2.i = 999999;
    u.i = 8;
    // ...
}
```

Здесь будет надежнее полагать, что любое значение в таком объединении представляет собой указатель. Но умный сборщик мусора может сделать кое-что и лучше: он может заметить, что для данной реализации, например, целые не располагаются по нечетным адресам, или что никакой объект не может иметь адрес *8* и т.д. Это позволит сборщику мусора воздержаться от предположения, что объекты со значениями *999999* или *8* используются функцией *f()*.

С.9.1.2. Операция delete

Если в конкретной реализации применяется автоматическая сборка мусора, то операции *delete* и *delete []* больше не нужны, и программист может просто воздержаться от их применения. Однако помимо освобождения памяти эти операции вызывают деструкторы объектов. Поэтому в случае присутствия сборщика мусора,

```
delete p;
```

вызывает деструктор для объекта, на который указывает *p*, а освобождение памяти откладывается до момента ее обработки сборщиком мусора. Одномоментная «переработка» сборщиком мусора множества объектов помогает избежать сильной фрагментации памяти (§С.9.1.4). Кроме того, нивелируются в общем случае серьезные последствия ошибочного повторного удаления объекта, деструктор которого сам освобождает память.

Как всегда, доступ к объекту после его уничтожения не определен.

С.9.1.3. Деструкторы

В момент, когда сборщик мусора собирается обработать висящий объект, возникает альтернатива:

1. Вызвать для объекта деструктор (если он имеется).
2. Отнестись к объекту просто как к «сырой» памяти (то есть не вызывать деструктор).

По умолчанию сборщик мусора должен выбрать второй вариант, ибо объекты, созданные операцией *new*, для которых *delete* не вызывался, не должны уничтожаться. В то же время, можно так спроектировать сборщик мусора, чтобы он вызывал деструкторы для зарегистрированных у него объектов. Заметьте, что важно уничтожать объекты в правильном порядке, чтобы избежать ситуаций, когда деструктор одного объекта ссылается на объект, который был уничтожен ранее. Сборщику мусора трудно реализовать правильный порядок уничтожения объектов без помощи со стороны программиста.

С.9.1.4. Фрагментация памяти

Когда память выделяется для множества разных объектов, а затем освобождается, то она с неизбежностью *фрагментируется* (*fragments*) — большая часть памяти разбивается на кусочки, слишком малые для эффективного использования. Причина заключается в том, что универсальный распределитель памяти не всегда может найти кусок памяти нужного размера. А применить для объекта кусок памяти несколько большего размера, значит оставить часть памяти неиспользуемой. Поработает программа с таким неэффективным распределителем памяти чуть подольше, и уже добрая половина памяти использована под маленькие фрагменты, которые трудно снова пустить в дело.

Существует несколько методик борьбы с фрагментацией памяти. Простейшая методика заключается в запросе у распределителя (аллокатора, *allocator*) большого куска памяти, который затем нарезается на одинаковые доли под объекты одинакового размера (§15.3, §19.4.2). Поскольку большинство актов выделения/освобождения памяти относятся к маленьким объектам вроде узлов дерева, объектов связи (*links*) и т.д., то такая методика оказывается очень эффективной. Иногда аллокаторы могут автоматически применять похожие методики. В любом случае, фрагментация снижается еще сильнее, если все большие «куски» (*chunks*) имеют одинаковый размер (скажем, размер страницы памяти), так что их самих можно динамически размещать в памяти без фрагментации последней.

Существуют сборщики мусора двух стилей:

1. *Копирующий сборщик* (*copying collector*) для дефрагментации памяти перемещает в ней объекты.
2. *Консервативный сборщик* (*conservative collector*) размещает объекты так, чтобы минимизировать фрагментацию.

С точки зрения языка C++ предпочтительнее консервативный сборщик, ибо исключительно трудно (если вообще возможно) переместить объект в памяти и корректно модифицировать все указатели на него. Кроме того, консервативный сборщик позволяет сосуществовать фрагментам программы на C++ с фрагментами на других языках, например, на С. Традиционно копирующие сборщики пользовались популярностью у программистов, имевших дело с языками типа Lisp и Smalltalk, в которых объекты адресуются исключительно косвенно через уникальные указате-

ли или ссылки. Для большинства больших программ, где важны объем копирования и степень взаимодействия аллокаторов со страничными подсистемами операционных систем, современные консервативные сборщики почти также эффективны, как и копирующие. Для программ поменьше часто достигим практически идеальный вариант, когда сборщик мусора вообще никогда не вызывается — особенно в C++, где многие объекты автоматические.

С.10. Пространства имен

Этот раздел посвящен небольшим подробностям, относящимся к пространствам имен, которые выглядят как мелкие технические детали, но часто мелькают в публичных дискуссиях и реальном программном коде.

С.10.1. Удобство против безопасности

Объявление **using** вносит имя в локальную область видимости, а директива **using** — нет; она просто делает доступными все имена из области видимости, в которой они были объявлены. Например:

```
namespace X
{
    int i, j, k;
}

int k;

void f1 ()
{
    int i = 0;
    using namespace X;      // делает доступными имена из X
    i++;                    // локальная i
    j++;                    // X::j
    k++;                    // error: X::k или глобальная k?
    ::k++;                  // глобальная k
    X::k++;                 // X::k
}

void f2 ()
{
    int i = 0;

    using X::i;              // error: i дважды определяется в f2()
    using X::j;
    using X::k;              // скрывает глобальную k

    i++;
    j++;                    // X::j
    k++;                    // X::k
}
```

Локально объявленное имя (обычным образом или **using**-объявлением) скрывает нелокальные объявления того же имени, а все незаконные перегрузки имени выявляются в точке объявления.

Обратите внимание на ошибку неоднозначности для **k++** в **fl()**. Глобальные имена не имеют приоритета перед именами из пространства имен, ставших доступными в глобальной области видимости. Это обеспечивает надежную защиту от случайных конфликтов имен и — что важно — нивелирует какие-либо преимущества «заселения» именами глобального пространства (с его неминуемым замусориванием).

Когда библиотеки, объявляющие много имен, становятся доступными благодаря **using**-директивам, очень важно то, что конфликты неиспользуемых имен не считаются ошибками.

Глобальная область видимости — это просто еще одно пространство имен. Оно отличается лишь тем, что его не нужно явно квалифицировать. То есть **:k** означает, что **k** нужно искать в глобальном пространстве имен и в пространствах имен, упомянутых в **using**-директивах, расположенных в глобальной области видимости, в то время как **X:k** означает «**k**, объявленное в пространстве имен **X** или в пространствах имен, упомянутых в **using**-директивах в **X**» (§8.2.8).

Я надеюсь, что в новых программах с применением пространств имен я увижу радикальное снижение использования глобального пространства имен по сравнению со старыми, традиционными программами на C и C++. Правила работы с пространствами имен нарочито разрабатывались таким образом, чтобы у ленивого почитателя глобального пространства не было никаких преимуществ перед теми, кто не замусоривает глобальную область видимости.

С.10.2. Вложенные пространства имен

Очевидное применение пространства имен заключается в том, что в него помещают весь набор объявлений и определений:

```
namespace X
{
    // все мои объявления
}
```

В общем случае, список объявлений может включать в себя пространства имен. Так возникают вложенные пространства имен. Это допускается из практических соображений и согласуется с идеей, что конструкции могут быть вложенными всегда, если только нет серьезных возражений против этого. Вот пример:

```
void h () ;
namespace X
{
    void g () ;
    // ...
    namespace Y
    {
        void f () ;
        void ff () ;
        // ...
    }
}
```

Применяются обычные правила видимости и квалификации:

```
void X: :Y: :ff()
{
    f(); g(); h();
}

void X: :g()
{
    f();           // error: нет f() в X
    Y: :f();       // ok
}

void h()
{
    f();           // error: нет глобальной f()
    Y: :f();       // error: нет глобального Y
    X: :f();       // error: нет f() в X
    X: :Y: :f();   // ok
}
```

С.10.3. Пространства имен и классы

Пространство имен — это именованная область видимости. Класс — это тип с определенной областью видимости, которая описывает, как могут объекты класса создаваться и использоваться. Таким образом, пространство имен — это более простая концепция, нежели класс, и было бы идеально определить класс как пространство имен плюс несколько дополнительных средств. Это почти так. Пространство имен открыто (§8.2.9.3), а класс закрыт. Это различие проистекает из соображения, что класс должен определять раскладку объектов, а это лучше делать в одном месте. Кроме того, *using*-объявления и *using*-директивы применимы к классам весьма ограниченным образом (§15.2.2).

Пространства имен предпочтительнее классов, если все, что требуется — это инкапсуляция имен. В таком случае аппарат классов для проверки типов и создания объектов не нужен; достаточно более простой концепции пространств имен.

С.11. Управление режимами доступа

В настоящем разделе приведено некоторое количество технических примеров, иллюстрирующих управление режимами доступа в дополнение к примерам из §15.3.

С.11.1. Доступ к членам класса

Рассмотрим фрагмент кода:

```
class X
{
    int priv;

protected:
    int prot;
```

```
public:
    int publ;
    void m () ;
};
```

Здесь $X::m()$ имеет неограниченный доступ:

```
void X::m ()
{
    priv = 1;    // ok
    prot = 2;    // ok
    publ = 3;    // ok
}
```

Члены производного класса имеют доступ к открытым и защищенным членам базового класса (§15.3):

```
class Y: public X
{
    void mderived () ;
};

void Y::mderived ()
{
    priv = 1;    // error: priv - private
    prot = 2;    // ok: prot - protected и mderived() есть функция-член производного класса
    publ = 3;    // ok
}
```

Глобальные функции могут обращаться только к открытым членам:

```
void f(Y*p)
{
    p->priv = 1; // error: priv - private
    p->prot = 2; // error: prot - protected, а f() - не друг и не член X или Y
    p->publ = 3; // ok:
}
```

С.11.2. Доступ к базовым классам

Как и члены класса, базовые классы также могут быть объявлены с ключевыми словами *private*, *protected* и *public*:

```
class X
{
public:
    int a;
    // ...
};

class Y1: public X {};
class Y2: protected X {};
class Y3: private X {};
```

Поскольку X является открытым базовым классом для $Y1$, то любая функция может воспользоваться неявным преобразованием от $Y1^*$ к X^* там, где это нужно. Например:

```
void f(Y1* pyl, Y2* py2, Y3* py3)
{
    X* px = pyl; // ok: X - открытый базовый класс для Y1
    pyl->a = 7; // ok

    px = py2; // error: X - защищенный базовый класс для Y2
    py2->a = 7; // error

    px = py3; // error: X - закрытый базовый класс для Y3
    py3->a = 7; // error
}
```

Рассмотрим следующий фрагмент:

```
class Y2: protected X {};
class Z2: public Y2 {void f(Y1*, Y2*, Y3*);};
```

Так как *X* является защищенным базовым классом для *Y2*, только члены и друзья класса *Y2* (а также члены и друзья классов, производных от *Y2*, например *Z2*) могут воспользоваться неявным преобразованием от *Y2** к *X** там, где это нужно, как они могут получить доступ к открытым и защищенным членам класса *X*. Например:

```
void Z2::f(Y1* pyl, Y2* py2, Y3* py3)
{
    X* px = pyl; // ok: X - открытый базовый класс для Y1
    pyl->a = 7; // ok
    px = this; // ok: X - защищенный базовый класс для Y2, а Z2 наследует Y2
    a = 7; // ok
    px = py2; // error: X - защищенный базовый класс для Y2, а Z2 наследует Y2,
              // но мы не знаем, представляет ли py2 объект Z2,
              // или как Y2::X используется в не-Z2 объектах
    py2->a = 7; // error: Z2 не знает, как Y2::a используется в не-Z2 объектах
    px = py3; // error: X - закрытый базовый класс для Y3
    py3->a = 7; // error
}
```

Наконец рассмотрим

```
class Y3: private X {void f(Y1*, Y2*, Y3*);};
```

Поскольку *X* является закрытым базовым классом для *Y3*, только члены и друзья класса *Y3* могут воспользоваться неявным преобразованием от *Y3** к *X** (где это нужно), как они могут обращаться к открытым и защищенным членам класса *X*. Например:

```
void Y3::f(Y1* pyl, Y2* py2, Y3* py3)
{
    X* px = pyl; // ok: X - открытый базовый класс для Y1
    pyl->a = 7; // ok
    px = py2; // error: X - защищенный базовый класс для Y2
    py2->a = 7; // error
    px = py3; // ok: X - закрытый базовый класс для Y3,
              // и Y3::f() - член класса Y3
    py3->a = 7; // ok
}
```

С.11.3. Доступ ко вложенным классам

Члены вложенного класса не имеют никаких специальных прав на доступ к членам объемлющего класса. Аналогично, члены объемлющего класса не имеют никаких особых прав на доступ к членам вложенного класса — всюду работают обычные правила доступа (§10.2.2). Например:

```
class Outer
{
    typedef int T;
    int i;

public:
    int i2;
    static int s;

    class Inner
    {
        int x;
        T y;           // error: Outer::T - закрытый

    public:
        void f(Outer* p, int v) ;
    };

    int g(Inner* p) ;
};

void Outer::Inner::f(Outer* p, int v)
{
    p->i = v;           // error: Outer::i - закрытый
    p->i2 = v,          // ok: Outer::i2 - открытый
}

int Outer::g(Inner* p)
{
    p->f(this, 2) ; // ok: Inner::f() - открытый
    return p->x;    // error: Inner::x - закрытый
}
```

Часто, однако, бывает полезным предоставить членам вложенного класса неограниченный доступ к членам объемлющего класса. Для этого достаточно объявить вложенный класс другом объемлющего класса. Например:

```
class Outer
{
    typedef int T;
    int i;

public:
    class Inner;    // предварительное объявление класса
    friend class Inner;

    class Inner
    {
        int x;
        T y;        // ok: Inner - друг
    };
};
```

```

public:
    void f(Outer* p, int v) ;
};

void Outer::Inner::f(Outer* p, int v)
{
    p->i = v;      // ok: Inner - друг
}

```

С.11.4. Отношение «дружбы»

Отношение дружбы не транзитивно и не наследуемо. Например:

```

class A
{
    friend class B;
    int a;
};

class B
{
    friend class C;
};

class C
{
    void f(A* p)
    {
        p->a++;      // error: C - не друг A, а друг друга A
    }
};

class D: public B
{
    void f(A* p)
    {
        p->a++;      //error: D - не друг A, а наследует от друга A
    }
};

```

С.12. Указатели на члены классов

Естественно, что понятие указателя на члены класса (§15.5) применяется как к членам данных, так и к функциям-членам, имеющим аргументы и возвращаемые значения. Например:

```

struct C
{
    const char* val;
    int i;
    void print(int x) { cout << val << x << '\n' ; }
    int fl(int) ;
}

```

```

void f2 () ;
C (const char* v) { val = v; }
};

typedef void (C::*PMFI) (int) ; // указатель на функцию-член класса C с аргументом
int
typedef const char* C::*PM;    // указатель на поле данных типа const char* класса C
void f(C& z1, C& z2)
{
    C* p = &z2;
    PMFI pf= &C::print;
    PM pm = &C::val;

    z1.print(1) ;
    (z1.*pf) (2) ;
    z1.*pm = "nv1";
    p->*pm = "nv2";
    z2.print(3) ;
    (p->*pf) (4) ;

    pf = &C::f1;           // error: неправильный тип возврата
    pf = &C::f2;           // error: неправильный тип аргумента
    pm = &C::i;            // error: несоответствие типа
    pm = pf;              // error: несоответствие типа
}

```

Тип указателя на функцию проверяется точно так же, как любой другой тип.

С.13. Шаблоны

Шаблон класса описывает, как можно сгенерировать класс по заданному подходящему набору аргументов шаблона. Аналогично, шаблон функции описывает, как можно сгенерировать функцию по заданному подходящему набору аргументов шаблона. Таким образом, шаблоны используются для генерирования типов и исполнимого кода. Вместе с такой выразительной мощностью возникают и некоторые сложности, связанные по большей части со множеством контекстов определения и использования шаблонов.

С.13.1. Статические члены

Шаблон класса может иметь статические члены. Каждый класс, сгенерированный из шаблона, имеет свою собственную копию статических членов. Статические члены должны определяться отдельно и могут иметь специализации. Например:

```

template<class T> class X
{
    // ...
    static T def_val;
    static T* new_X(T a = def_val) ;
}

```

```

template<class T> T X<T>::def_val;    // инициализируется X<T>()
template<class T> T* X<T>::new_X(T a) { /* ... */ }

template<> int X<int>::def_val = 0;
template<> int* X<int>::new_X(int i) { /* ... */ }

```

Если вы хотите сделать объект или функцию общими для всех членов любого сгенерированного из шаблона класса, вы можете разместить их в нешаблонном базовом классе. Например:

```

struct B
{
    static B* nil; // общий нулевой указатель для всех классов, производных от B
};

template<class T> class X: public B
{
    // ...
};

B* B::nil = 0;

```

С.13.2. Друзья

Шаблоны классов могут иметь друзей. Рассмотрим, например, *Matrix* и *Vector* из §11.5. В общем случае они будут представляться шаблонами:

```

template<class T> class Matrix;

template<class T> class Vector
{
    T v[4];

public:
    friend Vector operator* (const Matrix<T>&, const Vector&) ;
    // ...
};

template<class T> class Matrix
{
    Vector<T> v[4];

public:
    friend Vector<T> operator* (const Matrix&, const Vector<T>&) ;
    // ...
};

```

Затем можно определить операцию умножения, имеющую прямой доступ к данным из *Matrix* и *Vector*:

```

template<class T> Vector<T> operator* (const Matrix<T>& m, const Vector<T>& v)
{
    // ... используем m.v[i] и v.v[i] для прямого доступа к элементам
}

```

Друзья не оказывают влияния на область видимости, в которой определен шаблонный класс; также они не влияют на область видимости, в которой этот шаблон

используется. Функции-друзья и операции-друзья находятся путем поиска, базирующегося на типе их аргументов (§11.2.4, §11.5.1). Как и функции-члены, функции-друзья конкретизируются (§С.13.9.1) только, если осуществляется их вызов.

С.13.3. Шаблоны в качестве параметров шаблонов

Бывает полезным передать шаблон (а не класс или объект) в качестве аргумента шаблона. Например:

```
template<class T, template<class> class C> class Xrefd
{
    C<T> mems;
    C<T*> refs;
    // ...
};

Xrefd<Entry, vector> x1; // хранит перекрестные ссылки для Entry в контейнере типа vector
Xrefd<Record, set> x2;  // хранит перекрестные ссылки для Record в контейнере типа set
```

Чтобы объявить шаблон как параметр шаблона, нужно указать требующиеся ему аргументы. Например, мы объявляем параметр *C* шаблона *Xrefd* как шаблонный класс, которому требуется один аргумент-тип. Если мы этого не сделали бы, то не смогли бы использовать специализации *C*. Причина нашего стремления использовать шаблон в качестве параметра шаблона заключается в том, что мы хотели бы конкретизировать его разными типами аргументов (такими как *T* и *T** в предыдущем примере). То есть мы хотели бы выражать объявления членов шаблона в терминах другого шаблона, который был бы параметром и задавался пользователем.

Когда шаблону требуется контейнер для хранения элементов с типом аргумента шаблона, лучше передать ему тип контейнера (§13.6, §17.3.1).

Только классовые шаблоны могут выступать в роли аргументов другого шаблона.

С.13.4. Логический вывод аргументов функциональных шаблонов

Компилятор может осуществить логический вывод типа аргумента шаблона, *T* или *TT*, и нетипового аргумента шаблона — *I*, исходя из аргумента шаблона функции с типом, составленным из следующих конструкций:

<i>T</i>	<i>const T</i>	<i>volatile T</i>
<i>T*</i>	<i>T&</i>	<i>T[constant_expression]</i>
<i>type [I]</i>	<i>class _template_name<T></i>	<i>class _template name<I></i>
<i>TT<T></i>	<i>T<I></i>	<i>T<></i>
<i>T type::*</i>	<i>T T::*</i>	<i>type T::*</i>
<i>T (*) (args)</i>	<i>type (T::*) (args)</i>	<i>T (type::*) (args)</i>
<i>type (type::*) (args_TT)</i>	<i>T (T::*) (args_TT)</i>	<i>type (T::*) (args_TT)</i>
<i>T (type::*) (args_TT)</i>	<i>type (*) (args_TT)</i>	

Здесь *args_TT* — это список параметров, по которому можно определить *T* или *I* согласно рекурсивной процедуре применения этих правил, а *args* — это список параметров, не допускающий логического вывода. Если таким образом можно вывести не все параметры, вызов считается неоднозначным. Например:

```

template<class T, class U> void f(const T*, U(*) (U)) ;
int g (int) ;
void h (const char* p)
{
    f(p, g) ;    // T есть char, U есть int
    f(p, h) ;    // error: невозможно вывести U
}

```

Глядя на аргументы первого вызова *f()*, мы легко выводим все аргументы функционального шаблона. Для второго же вызова *f()* мы видим, что *h()* не соответствует образцу *U(*) (U)*, поскольку тип аргумента функции *h()* отличается от типа возвращаемого ею значения.

Если параметр шаблона выводится из более чем одного аргумента функции, результатом каждого вывода должен быть один и тот же тип. В противном случае вызов считается ошибочным. Например:

```

template<class T> void f(T i, T* p) ;
void g (int i)
{
    f(i, &i)                // ok
    f(i, "Remember!") ;    // error, неоднозначность: T есть int, или T есть const char?
}

```

С.13.5. Шаблоны и ключевое слово `typename`

Чтобы сделать обобщенное программирование более простым и более общим, контейнеры стандартной библиотеки предоставляют набор стандартных функций и типов (§16.3.1). Например:

```

template<class T> class vector
{
public:
    typedef T* iterator;

    iterator begin() ;
    iterator end() ;
    // ...
};

template<class T> class list
{
    class link { /* ... */ ;

public:
    typedef link* iterator;

    iterator begin() ;
    iterator end() ;
    // ...
};

```

Это искушает нас написать

```
template<class C> void f(C& v)
{
    C::iterator i = v.begin(); // синтаксическая ошибка
    // ...
}
```

К сожалению, от компилятора нельзя требовать слишком многого и он не понимает, что **C::iterator** — это имя типа. В некоторых частных случаях «умный» компилятор мог бы распознать, относится имя к типу, или к чему-нибудь еще, что не является типом (к функции или шаблону). Но в общем случае это невозможно. Действительно, рассмотрим пример, лишенный подсказок насчет его смысла:

```
int y;

template<class T> void g(T& v)
{
    T::x(y); // вызов функции или объявление переменной?
}
```

Является ли здесь **T::x** функцией, вызванной с **y** в качестве аргумента? Или мы намеревались объявить локальную переменную **y** с типом **T::x**, поставив здесь по странным причинам необязательные круглые скобки? В принципе можно представить себе реальные программы, в которых **X::x(y)** будет функциональным вызовом, а **Y::x(y)** будет объявлением.

Эта неоднозначность разрешается просто: если не указано обратного, то идентификатор относят к чему-то такому, что не является типом или шаблоном. А если мы хотим трактовать идентификатор как имя типа, мы можем это сделать с помощью ключевого слова **typename**:

```
template<class C> void h(C& v)
{
    typename C::iterator i = v.begin();
    // ...
}
```

Ключевое слово **typename** можно поставить перед квалифицированным именем с целью утверждения, что имя относится к типу. В этом отношении оно напоминает ключевые слова **struct** и **class**.

Ключевое слово **typename** необходимо, если имя типа зависит от параметров шаблона. Например:

```
template<class T>
void k(vector<T>& v)
{
    vector<T>::iterator i = v.begin(); // error: мы хотим typename
    typename vector<T>::iterator i = v.begin(); // ok
    // ...
}
```

В этом случае компилятор может быть и смог бы определить, что **iterator** есть имя типа в каждой конкретизации шаблона **vector**, но стандартом от него этого не требуется, так как это было бы непереносимым расширением языка. Единственный контекст, в котором компилятор может определить, что имена, зависящие от аргу-

мента шаблона, являются именами типов — это несколько частных случаев, в которых лишь имена типов разрешены грамматикой. Например, в случае спецификаторов, перечисленных в §А.8.1.

Ключевое слово ***typename*** можно также использовать как альтернативу ключевому слову ***class*** в объявлениях шаблонов:

```
template<typename T> void f(T) ;
```

Поскольку я не слишком быстро работаю с клавиатурой и мне всегда не хватает места на экране, я предпочитаю более короткую запись

```
template<class T> void f(T) ;
```

С.13.6. Ключевое слово ***template*** в качестве квалификатора

Необходимость в квалификаторе ***typename*** возникла потому, что мы можем обращаться к членам классов, которые являются типами, и к членам, которые типами не являются. Аналогично, имеется необходимость отличать шаблонные члены классов от членов, шаблонами не являющихся. Рассмотрим возможный интерфейс к универсальному менеджеру памяти:

```
class Memory
{
public:
    template<class T> T* get_new () ;
    template<class T> void release (T&) ;
    // ...
};

template<class Allocator> void f(Allocator& m)
{
    int* p1 = m.get_new<int> () ;           // error
    int* p2 = m.template get_new<int> () ; // явная квалификация
    // ...
    m.release (p1) ; // параметр шаблона выводится (нет нужды в явной квалификации)
    m.release (p2) ;
}
```

Явная квалификация при вызове ***get_new()*** необходима, поскольку здесь логический вывод параметра невозможен. В этом случае нужно использовать префикс ***template***, чтобы информировать компилятор о том, что имя ***get_new*** относится к шаблонному члену класса и явная квалификация желаемым типом элементов возможна. Без ключевого слова ***template*** будет получено сообщение об ошибке, поскольку знак ***<*** будет воспринят как знак операции сравнения «меньше чем». Это редкий случай, поскольку в большинстве случаев удастся выполнить логический вывод параметров шаблона.

С.13.7. Конкретизация

Генерация корректного машинного кода из определения шаблона и его использования ложится на плечи реализаций языка C++. Из определения шаблона класса и набора аргументов шаблона компилятор должен сгенерировать определение класса и определения тех его функций-членов, которые находят применение в программе.

Из определения шаблона функции компилятор должен генерировать конкретные функции. Эти процессы называются *конкретизацией шаблона* (*template instantiation*)¹.

Сгенерированные классы и функции называются *специализациями* (*specializations*). Когда нужно различить сгенерированные специализации и специализации, написанные программистом вручную (§13.5), применяют более подробные термины — *сгенерированные специализации* (*generated specializations*) и *явные специализации* (*explicit specializations*), соответственно. Последний термин имеет и другие вариации — *специализации, определяемые пользователем* (*user-defined specializations*) или просто *специализации*.

Чтобы использовать шаблоны в нетривиальных программах, программист должен понимать, как имена, используемые в определении шаблона, связываются с объявлениями, и как можно организовать исходный код (§13.7).

По умолчанию, компилятор генерирует классы и функции из шаблонов в соответствии с правилами связывания имен (§C.13.8). То есть программист не обязан явно указывать, какие версии каких шаблонов нужно сгенерировать. Это очень важно, потому что программисту бывает трудно понять, какие именно версии шаблонов нужны. Часто в реализациях библиотек используются шаблоны, о которых программист ничего не слышал, а бывает и что знакомые шаблоны используют аргументы неизвестного ему типа. В общем случае, набор функций, которые требуется сгенерировать, можно выявить посредством рекурсивной проверки шаблонов, используемых в необходимых программе библиотеках. Для выполнения такой задачи компьютеры приспособлены лучше человека.

Иногда, все же, программисту бывает важно точно определить, в каких именно местах должен генерироваться код из шаблона (§C.13.10). Тем самым программист получает полный контроль над контекстом конкретизации. Для большинства компиляторов это также означает получение полного контроля над моментом конкретизации. В частности, явная конкретизация может применяться для получения ошибок компиляции в предсказуемое время, а не в тот момент, что выгоден реализации. В определенных случаях важна предсказуемость процесса построения программы.

С.13.8. Связывание имен

Важно определять шаблоны функций так, чтобы они как можно меньше зависели от нелокальной информации. Дело в том, что потом шаблоны будут использоваться для генерации функций и классов, отталкиваясь от неизвестных типов и в неизвестных контекстах. Всякая тонкая зависимость от контекста всплывет все равно в процессе отладки, а программисту не всегда хочется в деталях разбираться в устройстве шаблонов. Универсальное правило — по-возможности избегать глобальных имен — приобретает особую остроту в случае шаблонов. Поэтому мы стремимся сделать определения шаблонов как можно более самодостаточными и передавать то, что в противном случае было бы глобальным контекстом, в форме параметров шаблона (например, свойств (*traits*); §13.4, §20.2.1).

¹ Встречающийся в литературе термин *инстанцирование* труден для произношения. — Прим. ред.

Но все же, некоторые нелокальные имена использовать приходится. Например, чаще всего пишут целый набор взаимодействующих друг с другом шаблонов функций, а не единственную самодостаточную функцию. Такие взаимодействующие функции могут (но не обязательно) быть членами классов. Иногда наилучшим выбором становятся нелокальные функции. Типичным примером служат вызовы функций *swap()* и *less()* из функции *sort()* (§13.5.2). Алгоритмы стандартной библиотеки предлагают массу примеров на эту тему (глава 18).

Еще одним источником нелокальных имен в определениях шаблонов служат операции с общепринятыми именами и семантикой, например *+*, ***, *[]* и *sort()*. Рассмотрим:

```
#include <vector>

bool tracing;
// ...
template<class T> T sum (std::vector<T>& v)
{
    T t = 0;
    if(tracing) cerr << "sum (" << &v << ") \n";
    for (int i = 0; i<v.size(); i++) t = t + v[i];
    return t;
}
// ...
#include <quad.h>

void f(std::vector<Quad>& v)
{
    Quad c = sum(v);
}
```

Невинно выглядящая шаблонная функция *sum()* зависит от операции *+*. В данном примере операция *+* определена в *<quad.h>*:

```
Quad operator+ (Quad, Quad);
```

Важно, что когда определяется *sum()*, в области видимости нет никаких чисел *Quad*, и автор функции *sum()* может ничего не знать о классе *Quad*. В частности, операция *+* может определяться в программе позже функции *sum()*, и даже позже по времени.

Процесс поиска объявлений имен, явно или неявно используемых в шаблоне, называется *связыванием имен (name binding)*. Главная проблема со связыванием имен в шаблонах состоит в том, что к конкретизации шаблонов имеют отношение три контекста, четко разделить которые невозможно:

1. Контекст определения шаблона
2. Контекст объявления типов аргументов
3. Контекст использования шаблона

С.13.8.1. Зависимые имена

Определяя шаблон функции, мы хотим быть уверены в том, что имеется достаточный контекст этого определения по отношению к фактическим аргументам без

захватывания «случайного мусора» в точке использования. Помогая нам в этом отношении, язык разделяет все имена, используемые в определении шаблона, на две категории:

1. Имена, зависящие от аргумента шаблона. Такие имена связываются в точке конкретизации (§С.13.8.3). В примере с функцией `sum()` операция `+` определяется контекстом конкретизации, поскольку она принимает операнды типа, совпадающего с типом аргументов шаблона.
2. Имена, не зависящие от аргумента шаблона. Эти имена связываются в точке определения шаблона (§С.13.8.2). В примере с функцией `sum()` шаблон `vector` определяется в заголовочном файле `<vector>`, и когда компилятор обрабатывает определение `sum()`, логическая переменная `tracing` находится в области видимости.

Простейшим определением того, что «*N* зависит от шаблонного параметра *T*» было бы «*N* является членом *T*». К сожалению, этого недостаточно; сложение чисел типа `Quad` (§С.13.8) служит примером обратного. Поэтому говорят, что вызов функции зависит от аргумента шаблона, если и только если выполняется одно из двух следующих условий:

1. Тип фактического аргумента зависит от параметра шаблона *T* согласно правилам логического вывода типа (§13.3.1). Например, `f(T(I))`, `f(t)`, `f(g(t))` и `f(&t)` в предположении, что *t* — это *T*.
2. Вызываемая функция имеет формальный параметр, зависящий от *T* согласно правилам логического вывода типа (§13.3.1). Например, `f(T)`, `f(list<T>&)` и `f(const T*)`.

По сути дела, имя в вызываемой функции зависимо тогда, когда его зависимость очевидна при просмотре аргументов или формальных параметров функции.

Вызов, в котором аргумент случайно соответствует фактическому типу параметра шаблона, не является зависимым. Например:

```
template<class T> T f(T a)
{
    return g(1); //error: нет g() в области видимости и g(1) не зависит от T
}

int g(int);
int z=f(2);
```

Не имеет значения, что для вызова `f(2)` параметр *T* оказался `int` и аргумент `g()` также имеет тип `int`. Если бы вызов `g(1)` был зависимым, то его смысл был бы абсолютно непонятен любому, кто читал бы это определение шаблона. Если программист хочет, чтобы вызов связывался с `g(int)`, объявление последнего должно располагаться перед определением шаблона, так чтобы оно попало в область видимости в момент анализа шаблона. В принципе, это правило аналогично соответствующему правилу для нешаблонных функций.

Кроме имен функций имена переменных, типов, констант и т.д. могут быть зависимыми, если их тип зависит от параметра шаблона. Например:

```

template<class T> void fct(const T& a)
{
    typename T::Memtype p = a.p;    // p и Memtype зависят от T
    cout << a.i << ' ' << p->j;    // i и j зависят от T
}

```

С.13.8.2. Связывание в точке определения

Когда компилятор видит определение шаблона, он определяет, какие имена являются зависимыми (§С.13.8.1). Если имя зависимое, поиск его объявления нужно отложить до момента конкретизации (§С.13.8.3).

Имена, не зависящие от аргумента шаблона, должны находиться в области видимости (§4.9.4) в точке определения. Например:

```

int x;

template<class T> T f(T a)
{
    x++;          // ok
    y++;          // error: нет y в области видимости, и y не зависит от T
    return a;
}

int y;
int z = f(2);

```

Если объявление находится, то именно оно и используется, несмотря на то, что впоследствии могло быть найдено и объявление «получше». Например:

```

void g(double);

template<class T> class X: public T
{
public:
    void f() { g(2); } // вызывается g(double);
    // ...
};

void g(int);
class Z {};

void h(X<Z> x)
{
    x.f();
}

```

Когда генерируется определение для $X<Z>: :f()$, $g(int)$ не рассматривается, поскольку его объявление расположено позже определения X . Не важно, что X используется после объявления $g(int)$. Кроме того, вызов, не являющийся зависимым, не может быть отпассован в базовый класс:

```

class Y {public: void g(int);};

void h(X<Y>x)
{
    x.f();
}

```

И снова $X < Y : : f()$ вызовет $g(\textit{double})$. Если бы программист хотел вызвать $g()$ из базового класса T , определение $f()$ должно было бы иметь вид:

```
template<class T> class XX: public T
{
void f() { T::g(2); } // вызывается T::g()
// ...
};
```

Здесь применено эмпирическое правило, гласящее, что «определение шаблона должно быть настолько самодостаточным, насколько это вообще возможно» (§С.13.8).

С.13.8.3. Связывание в точке конкретизации

Каждое применение шаблона для фиксированного набора его аргументов определяет *точку конкретизации (point of instantiation)*. Эта точка находится в ближайшей охватывающей ее глобальной области видимости или области видимости пространства имен, непосредственно *перед объявлением, использующим шаблон*. Например:

```
struct X { X(int); /* ... */ };

void g(X);

template<class T> void f(T a) { g(a); }

void h()
{
extern void g(int);
f(2); // вызывается f(X(2)); то есть f<X>(X(2))
}
```

Здесь точка конкретизации для f расположена перед $h()$, так что функция $g()$, вызываемая из $f()$, есть глобальная $g(X)$, а не локальная $g(int)$.

Из определения точки конкретизации вытекает, что *параметр шаблона никогда не может быть связан с локальным именем или членом класса*. Например:

```
void f()
{
struct X { /* ... */ }; // локальная структура
vector<X> v; // error: нельзя использовать локальную структуру
// в качестве параметра шаблона
// ...
}
```

Кроме того, невалифицированные имена, применяемые в шаблонах, также не могут быть связаны с локальными именами. И наконец, даже если шаблон впервые используется внутри класса, невалифицированные имена из определения шаблона не могут быть связаны с членами этого класса. Такое игнорирование локальных имен существенно для предотвращения неприятного поведения, характерного для макросов:

```
template<class T> void sort(vector<T>& v)
{
sort(v.begin(), v.end()); // используется sort() стандартной биб-ки
```

```

// (без явного указания std::)
}

class Container
{
    vector<int> v;

public:
    void sort()
    {
        : sort(v);    //sort(vector<int>&) вызывает std::sort(), а не Container::sort()
    }
    // ...
};

```

Если бы `sort(vector<T>&)` вызывал `sort()`, используя `std::sort()`, результат был бы тем же, а код — был бы прозрачнее.

Если точка конкретизации для шаблона, определенного в пространстве имен, находится в другом пространстве имен, то для связывания доступны имена из обоих пространств имен. Как обычно, для выбора между именами из разных пространств имен применяются правила разрешения перегрузки (§8.2.9.2).

Еще раз отметим, что шаблон, используемый несколько раз с одним и тем же набором аргументов, имеет несколько точек конкретизации. Если при этом связывание независимых имен различно, программа ошибочна. Однако такую ошибку реализациям выявить нелегко, особенно если разные точки конкретизации находятся в разных единицах трансляции. Лучше всего стараться избегать сложностей со связыванием имен, минимизируя применение нелокальных имен в шаблонах и применяя заголовочные файлы для согласования контекстов их использования.

С.13.8.4. Шаблоны и пространства имен

Когда вызывается функция, ее объявление может быть найдено даже вне текущей области видимости при условии, что она объявлена в том же пространстве имен, что и один из ее аргументов (§8.2.6). Это очень важно для функций, вызываемых из определений шаблонов, поскольку при помощи этого механизма находятся зависимые функции в момент конкретизации.

Специализации шаблонов могут генерироваться в любой точке конкретизации (§С.13.8.3), в любой точке, следующей за ней в той же единице трансляции, или в единице трансляции, специально созданной для генерации специализаций. Это отражает три очевидные стратегии, которые могут использоваться в реализациях C++ для генерирования специализаций:

1. Генерировать специализацию сразу, как только первый раз встречается соответствующий вызов.
2. В конце единицы трансляции генерировать все специализации, необходимые для этой единицы трансляции.
3. Когда просмотрены все единицы трансляции, генерировать все специализации, необходимые программе.

Перечисленные стратегии имеют свои преимущества и недостатки; допускаются также и комбинации этих стратегий.

В любом случае, связывание независимых имен производится в точке определения шаблона. Связывание зависимых имен выполняется путем просмотра:

1. Имен в области видимости в точке, где определяется шаблон.
2. Имен из пространства имен аргумента зависимого вызова (глобальные функции рассматриваются в пространстве имен встроенных типов).

Например:

```
namespace N
{
    class A { /* ... */ };
    char f(A) ;
}

char f(int) ;

template<class T> char g(T t) {return f(t) ;}
char c = g(N::A()) ;           // приводит к вызову N::f(N::A)
```

Здесь вызов $f(t)$ очевидно зависим, так что мы не можем связать f с $f(N::A)$ или с $f(int)$ в точке определения. Чтобы сгенерировать специализацию для $g<N::A>(N::A)$, реализация ищет функции с именем $f()$ в пространстве имен N и находит $N::f(N::A)$.

Программу следует считать неправильной, если можно получить разные результаты, выбрав разные точки конкретизации или разное содержимое пространство имен в разных контекстах генерации специализации. Например:

```
namespace N
{
    class A { /* ... */ };
    char f(A, int) ;
}

template<class T, class T2> char g(T t, T2 t2) {return f(t, t2) ;}

char c = g(N::A(), 'a') ;    // error: возможны разные варианты разрешения f(t)

namespace N                  // добавка к пространству имен N (§8.2.9.3)
{
    void f(A, char) ;
}
```

Мы могли бы сгенерировать специализацию в точке конкретизации и получить вызов $f(N::A, int)$. Или могли бы подождать и сгенерировать специализацию в конце единицы трансляции и получить вызов $f(N::A, char)$. Следовательно, вызов $g(N::A(), 'a')$ является ошибкой.

Вообще-то, это очень неряшливое программирование — вызывать перегруженную функцию между двумя ее объявлениями. В большой программе программист может и не обнаружить подвоха. В нашем конкретном случае компилятор вылавливает неоднозначность. Но если такие вещи встречаются в разных единицах трансляции, то обнаружить проблему будет намного сложнее — реализации не обязаны вылавливать подобные ошибки.

Большинство проблем с неоднозначным разрешением вызовов функций связано со встроенными типами. Поэтому большинство средств лечения проблемы опирается на более осторожное использование аргументов встроенных типов.

Встроенные типы не имеют ассоциированного с ними пространства имен. Следовательно, разрешение зависимых имен не выполняет разрешения перегрузки объявлений, видимых до и после определения шаблона. Например:

```
int f(int) ;
void ff(int) ;
void ff(char) ;
template<class T> T g(T t) {ff(t) ; return f(t) ; }
char f(char) ;
char c = g('a') ;           //вызывается ff(char);
```

Очевидно, что подобных хитростей лучше избегать.

С.13.9. Когда нужны специализации

Специализацию шаблона класса нужно генерировать лишь тогда, когда требуется определение класса. Например, если используется указатель, то определение класса не нужно. Например:

```
class X;
X* p;           // ok: определения X не требуется
X a;           // error: требуется определение X
```

Шаблонный класс не конкретизируется до тех пор, пока действительно не потребуется его определение. Например:

```
template<class T> class Link
{
    Link* suc;    // ok: нет нужды в определении Link (пока)
    // ...
};
Link<int>* pl;   // не требуется конкретизации Link<int>
Link<int> lnk;   // здесь уже нужно конкретизировать Link<int>
```

Точка конкретизации находится там, где впервые понадобилось определение.

С.13.9.1. Конкретизация шаблона функции

Реализация конкретизирует шаблон функции только при использовании (вызове) функции. В частности, конкретизация шаблона класса не влечет за собой конкретизации всех его членов, и даже членов, полностью определенных в рамках объявления классowego шаблона. Это дает программисту определенную гибкость при определении шаблона класса. Например:

```
template<class T> class List
{
    // ...
    void sort() ;
};
class Glob { /* нет операций сравнения */ ;
```

```
void f(List<Glob>& lb, List<string>& ls)
{
    ls.sort();
    // ... используем операции над lb, но не lb.sort()
}
```

Здесь конкретизируется функция **List<string>::sort()**, но не функция **List<Glob>::sort()**. Это и уменьшает объем работы, и избавляет нас от необходимости перепроектировать программу. Если бы функция **List<Glob>::sort()** была сгенерирована, нам пришлось бы либо добавить в тип **Glob** операции, требуемые функцией **List::sort()**, переопределить **sort()** так, чтобы она не была членом **List**, или применить какой-либо другой контейнер для объектов типа **Glob**.

С.13.10. Явная конкретизация

Запрос на явную конкретизацию состоит в объявлении специализации после ключевого слова **template** (за которым не следует символ <):

```
template class vector<int>; // класс
template int& vector<int>::operator[] (int); // функция-член
template int convert<int, double> (double); // функция
```

Объявление шаблона стартует с **template<**, в то время как просто **template** означает начало явного запроса на конкретизацию. Заметьте, что **template** предваряет полное объявление; констатации одного лишь имени не достаточно:

```
template vector<int>::operator[]; // синтаксическая ошибка
template convert<int, double>; // синтаксическая ошибка
```

Как и в случае вызовов шаблонных функций, аргументы шаблона, которые выводятся из аргументов функций, можно опустить (§13.3.1). Например:

```
template int convert<int, double> (double); // ok (избыточно)
template int convert<int> (double); // ok
```

Когда явно конкретизируется шаблон класса, конкретизируются и все его функции-члены.

Заметьте, что явную конкретизацию можно применить для проверки ограничений (§13.6.2). Например:

```
template<class T> class Calls_foo
{
    void constraints (T t) {foo(t);} // call from every constructor
    // ...
};

template class Calls_foo<int>; // error: foo(int) не определена
template void Calls_foo<Shape*>::constraints (Shape*); // error: foo(Shape*) не определена
```

Запросы на конкретизацию могут сильно влиять на время компоновки и эффективность повторных компиляций. Я знаю примеры, когда помещение большинства конкретизаций шаблонов в одну единицу трансляции сокращало время компиляции с нескольких часов до нескольких минут.

Иметь два определения для одной и той же специализации — ошибка. Не имеет значения, являются ли они специализациями, определяемыми пользователем (§13.5), неявно сгенерированными (§С.13.7) или явно запрошенными. Однако компилятор не обязан диагностировать множественные конкретизации в разных единицах трансляции. Это позволяет «умным реализациям» игнорировать избыточные конкретизации и, тем самым, избегать проблем, связанных с применением библиотек, использующих явную конкретизацию (§С.13.7). Реализации, однако, не обязаны быть умными. Пользователи «менее умных» реализаций должны сами избегать множественных конкретизаций. Самое плохое, что может случиться в противном случае — это отказ запуска программы.

Язык С++ не требует применения явной конкретизации. Явная конкретизация — это дополнительный механизм для «ручной» оптимизации процессов компиляции и компоновки (§С.13.7).

С.14. Советы

1. Фокусируйтесь на разработке программного продукта, а не на мелких технических деталях; §С.1.
2. Следование стандарту само по себе не гарантирует переносимости; §С.2.
3. Избегайте неопределенного поведения (включая нестандартные расширения); §С.2.
4. Локализируйте поведение, зависящее от реализации; §С.2.
5. Используйте ключевые слова и диграфы в системах, в которых отсутствуют {, }, [,], | и !, а где отсутствует \ — применяйте триграфы; §С.3.1.
6. Для расширения круга восприятия текста программы используйте лишь символы ANSI; §С.3.3.
7. Числовому представлению символов предпочитайте escape-последовательности; §С.3.2.
8. Не полагайтесь на знаковый или беззнаковый характер типа *char*; §С.3.4.
9. Если не уверены в типе целого литерала, явно применяйте необходимый суффикс; §С.4.
10. Избегайте неявных преобразований типа, приводящих к потере информации; §С.6.
11. Массивам предпочитайте тип *vector*; §С.7.
12. Избегайте объединений; §С.8.2.
13. Применяйте битовые поля для представления частей заданной извне раскладки памяти; §С.8.1.
14. Выбирая между разными способами использования памяти, сопоставляйте друг с другом их достоинства и недостатки; §С.9.
15. Не засоряйте глобальное пространство имен; §С.10.1.
16. Если требуется лишь ограниченная область видимости (модуль), а не тип, используйте пространства имен, а не классы; §С.10.3.

17. Не забывайте определять статические члены классовых шаблонов; §С.13.1.
18. Явно маркируйте типы в членах параметров шаблона с помощью ключевого слова ***typename***; §С.13.5.
19. Если возникла необходимость в явной квалификации аргументами шаблона, применяйте ключевое слово ***template*** для явного обозначения шаблонных членов класса; §С.13.6.
20. Определяйте шаблоны, минимизируя их зависимость от контекста конкретизации; §С.13.8.
21. Если конкретизация шаблона занимает слишком много времени, попробуйте конкретизировать их явным образом; §С.13.10.
22. Если нужна предсказуемая последовательность компиляции частей программы, попробуйте применить явную конкретизацию; §С.13.10.

Приложение D

Локализация

*Находясь в Риме — поступай как Римлянин.
— Поговорка*

Национальные особенности — класс **locale** — именованные локализации — создание локализаций — копирование и сравнение локализаций — контексты локализации **global()** и **classic()** — класс **facet** — доступ к фасетам локализаций — простой пользовательский фасет — стандартные фасеты — сравнение строк — ввод/вывод чисел — ввод/вывод денежных величин — ввод/вывод дат и времени — низкоуровневые операции со временем — класс **Date** — классификация символов — преобразование кодов символов — сообщения — советы — упражнения.

D.1. Национальные особенности

Объекты типа **locale** (объекты локального контекста) призваны учитывать местные и национальные особенности программ, такие как сравнение и сортировка строк, формат вывода чисел и представление символов на внешних носителях. Для более глубокой локализации программ можно расширять **locale** новыми *фасетами (facet)*¹, отражающими дополнительные национальные особенности, которые исходно не включены в стандартную библиотеку, например, почтовые индексы или телефонные номера. Объекты локализации в стандартной библиотеке в основном применяются для управления отображением выводимых в поток **ostream** данных, а также для контроля формата данных, вводимых в рамках потока **istream**.

В разделе §21.7 рассказано, как изменить локализацию для потока; в данном приложении рассматривается, как объекты локализации конструируются из фасетов, а также объясняются механизмы влияния локализации на поток. Также здесь рассматривается определение фасетов, перечисляются стандартные фасеты, задающие специфические свойства потоков, изложены технологии реализации и приме-

¹ Фасет — грань, аспект чего-либо. — Прим. ред.

нения объектов *locale* и *facet*. Стандартные библиотечные средства для представления даты и времени рассматриваются в контексте ввода/вывода.

Весь этот материал организован следующим образом:

- §D.1. Описывает основные идеи, касающиеся локализации программ под национальные особенности с помощью объектов *locale*.
- §D.2. Представляет класс *locale*.
- §D.3. Представляет класс *facet*.
- §D.4. Содержит обзор стандартных фасетов с описанием каждого из них:
 - §D.4.1. Сравнение строк.
 - §D.4.2. Ввод и вывод числовых значений.
 - §D.4.3. Ввод и вывод финансовой информации.
 - §D.4.4. Ввод и вывод дат и времени.
 - §D.4.5. Классификация символов.
 - §D.4.6. Преобразование кодов символов.
 - §D.4.7. Сообщения.

Общее понятие локализации программ не относится непосредственно к языку C++. Операционные системы и среды разработки также оперируют понятием локализации. В принципе, локализация системы относится ко всем программам независимо от того, на каком языке они написаны. Таким образом, понятие контекста локализации стандартной библиотеки C++ можно рассматривать как стандартный и переносимый способ доступа к информации, имеющей существенно различное представление в разных системах.

D.1.1. Программирование национальных особенностей

Рассмотрим вопрос о написании программ, которые могут использоваться в разных странах. Особенности написания таких программ часто называют *интернационализацией* (*internationalization*) — когда внимание фокусируется на попытках обеспечить применение программы в нескольких странах), или *локализацией* (*localization*) — когда внимание фокусируется на попытке обеспечить наилучшую адаптацию программы к конкретным локальным (национальным) особенностям. Многие объекты, которыми программа манипулирует, в разных странах принято выводить по-разному. Мы можем справиться с этой проблемой, учтя эти особенности в наших процедурах ввода/вывода:

```
void print_date (const Date& d)      // print in the appropriate format
{
    switch (where_am_I)               // пользовательский индикатор стиля
    {
        case DK:                     // например, 7. marts 1999
            cout << d.day () << " . " << dk_month [d.month ()] << " " << d.year () ;
            break;
        case UK:                     // например, 7 / 3 / 1999
            cout << d.day () << " / " << d.month () << " / " << d.year () ;
            break;
        case US:                     // например, 3/7/1999
```

```

    cout << d.month() << "/" << d.day() << "/" << d.year();
    break;
    // ...
}
}

```

Этот подход решает задачу. Однако он достаточно неуклюж, к тому же его нужно последовательно применять ко всему выводу, чтобы полностью выполнить адаптацию под локальные условия. Но хуже всего то, что если нам понадобится новый способ вывода даты, придется модифицировать исходный код. Казалось бы, с проблемой можно справиться, создав иерархию классов (§12.2.4). Однако информация в *Date* не зависит от способа, которым мы хотим ее вывести. Следовательно, нам не подходит иерархия типов *Date*, например, *US_date*, *UK_date* и *JP_date*. Скорее, нам нужны разные способы вывода дат: например, вывод в американском стиле, вывод в британском стиле или вывод в японском стиле; см. §D.4.4.5.

Иные проблемы возникают в рамках подхода «пусть пользователь сам пишет функции ввода/вывода, которые учитывают необходимые национальные особенности»:

1. Без помощи стандартной библиотеки прикладной программист вряд ли легко сможет изменить представление встроенных типов эффективным и переносимым образом.
2. Нахождение каждой операции ввода/вывода (и каждой операции подготовки ввода/вывода с учетом национальных особенностей) в большой программе не всегда достижимо.
3. Иногда у нас нет возможности переписать программу для учета нового соглашения — а если и есть, то вряд ли это для нас будет приятным занятием.
4. Заставлять всех пользователей реализовывать учет разных национальных особенностей весьма накладно.
5. Разные программисты будут по-разному реализовывать низкоуровневые средства учета национальных особенностей, так что программы, работающие с одинаковой информацией будут различаться по случайным причинам. Из-за этого программистам, поддерживающим код, полученный из разных источников, придется знакомиться с самыми разными программными решениями, что не только утомительно, но и чревато ошибками.

Как следствие, стандартная библиотека обеспечивает расширяемый способ учета национальных соглашений. На эту схему работы опирается библиотека ввода/вывода (§21.7) как для встроенных типов, так и для типов, определяемых пользователем. Рассмотрим для примера простой цикл копирования пар (*Date*, *double*), которые могут представлять совокупность измерений или транзакций:

```

void cpy(istream& is, ostream& os) // copy (Date,double) stream
{
    Date d;
    double volume;

    while (is>>d>>volume) os << d << ' ' << volume << '\n';
}

```

Само собой разумеется, что настоящая программа будет что-нибудь делать с записями, а также будет более внимательна к обработке ошибочных ситуаций.

Как заставить эту программу прочитать файл с французскими соглашениями (дробная часть отделяется с помощью запятой; например, 12,5 означает «двенадцать с половиной») и записать его в американском формате? Для этого мы можем предоставить объекты локального контекста и операции ввода/вывода так, что функция `cpu()` будет выполнять преобразование между указанными соглашениями:

```
void f(istream& fin, ostream& fout, istream& fin2, ostream& fout2)
{
    fin.imbue(locale("en_US")); // американский английский
    fout.imbue(locale("fr"));   // французский
    cpu(fin, fout);             // читаем американский английский, пишем французский

    fin2.imbue(locale("fr"));   // французский
    fout2.imbue(locale("en_US")); // американский английский
    cpu(fin2, fout2);           // читаем французский, пишем американский английский
}
```

Тогда для потоков

```
Apr 12, 1999 1000.3
Apr 13, 1999 345.45
Apr 14, 1999 9688.321
...
3 juillet 1950 10,3
3 juillet 1951 134,45
3 juillet 1952 67,9
```

эта программа породит вывод

```
12 avril 1999 1000,3
13 avril 1999 345,45
14 avril 1999 9688,321
...
July 3 1950 10.3
July 3, 1951 134.45
July 3, 1952 67.9
```

Оставшаяся часть настоящего приложения в основном описывает механизмы, делающие подобную схему работы возможной, и объясняет, как их нужно применять. Отметим, что у большинства программистов нет причин разбираться в мельчайших деталях локальных контекстов. Многие программисты никогда не манипулируют объектами *locale* явным образом, а те, что все-таки выполняют такую работу, просто извлекают стандартный *locale* и закрепляют его за потоком (§21.7). В то же время, механизмы, обеспечивающие создание таких объектов и возможность их простого использования представляют из себя небольшой язык программирования.

А если программа или система оказались успешными, они в обязательном порядке будут использоваться людьми, потребности и предпочтения которых проектировщики не предвидели. Большинство успешных программ будут использоваться в странах, естественные языки и наборы символов в которых отличаются от таковых для оригинальных (исходных) проектировщиков и программистов. Широкое

применение программы является признаком успеха программы, так что проектирование и программирование, нацеленные на преодоление языковых и культурных барьеров, являются необходимыми предпосылками для успешной работы.

Концепция локализации (интернационализации) проста. Но многочисленные практические детали изрядно запутывают дизайн и реализацию объектов *locale*:

1. Объекты *locale* должны инкапсулировать национальные стандарты, такие как, например, форма отображения дат. Подобные национальные соглашения варьируются непостижимым и нерегулярным образом. Они не имеют ничего общего с языками программирования, и поэтому язык программирования не может стандартизовать их.
2. Концепция локализации должна быть расширяемой, поскольку невозможно исчерпывающе перечислить все особенности, присущие всем национальным культурам, и представляющие интерес для пользователей C++.
3. Объекты *locale* используются в операциях ввода/вывода, от которых требуется высокая эффективность на этапе выполнения.
4. Локализация должна быть прозрачна для большинства программистов, желающих просто воспользоваться правильно работающим вводом/выводом, и не желающих разбираться в том, как все это в точности устроено.
5. Локализация должна быть доступна разработчикам средств, имеющих дело с зависящей от национальных особенностей информацией в рамках, выходящих за пределы стандартных потоков ввода/вывода.

Проектирование программы, имеющей дело с вводом/выводом, имеет выбор между управлением форматированием при помощи «обычного кода», или при помощи объектов *locale*. Первый (более традиционный) подход приемлем в случаях, когда мы уверены, что каждая операция ввода может быть легко преобразована из одной системы соглашений в другую. Однако если нужно менять формы отображения встроенных типов, или если требуются разные символьные наборы, или набор соглашений может расширяться, то становится привлекательным подход с объектами *locale*.

В общем случае *locale* составляется из фасетов, контролирующих индивидуальные аспекты соглашений, таких как символы-разделители для отображения чисел с плавающей запятой (*decimal_point*() ; §D.4.2) или формат, применяемый для чтения денежных величин (*money_punct*; §D.4.3). Фасет является объектом класса, производного от класса *locale*: *:facet* (§D.3). Мы можем рассматривать *locale* как контейнер фасетов (§D.2, §D.3.1).

D.2. Класс `locale`

Класс *locale* и связанные с ним средства представлены в заголовочном файле `<locale>`:

```
class std : locale
{
public:
    class facet;           // тип для представления фасетов; §D.3
    class id;              // тип для идентификации локализации; §D.3
    typedef int category;  // тип для разбиения фасетов на категории
```

```

static const category // настоящие значения зависят от реализации
    none = 0,
    collate = 1,
    ctype = 1<<1,
    monetary = 1<<2,
    numeric = 1<<3,
    time = 1<<4,
    messages = 1<<5,
    all = collate | ctype | monetary | numeric | time | messages;

locale () throw () ; // копирует глобальную локализацию (§D.2.1)
locale (const locale& x) throw () ; // копирует x
explicit locale (const char* p) ; // копия локализации с именем p (§D.2.1)

~locale () throw () ;

locale (const locale& x, const char* p, category c) ; // копия x плюс фасеты с из p
locale (const locale& x, const locale& y, category c) ; // копия x плюс фасеты с из y

template<class Facet> locale (const locale& x, Facet* f) ; // копия x плюс фасет f
template<class Facet> locale combine (const locale& x) ; // копия *this плюс Facet из x

const locale& operator= (const locale& x) throw () ;

bool operator== (const locale&) const ; // сравнение локализаций
bool operator!= (const locale&) const ;

string name () const ; // имя данной локализации (§D.2.1)

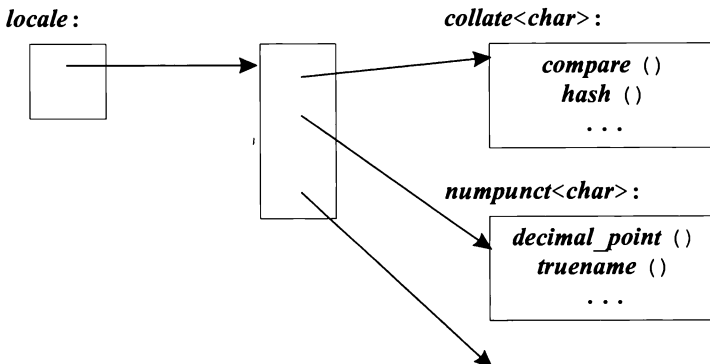
template<class Ch, class Tr, class A> // сравнение строк в данной локализации
bool operator () (const basic_string<Ch, Tr, A>&, const basic_string<Ch, Tr, A>&) const ;

static locale global (const locale&) ; // уст-ка новой глоб-ой локал-ии и возвр. старой
static const locale& classic () ; // "классическая" локализация в C-стиле

private:
    // representation
};

```

Тип `locale` можно рассматривать как интерфейс к `map<id, facet*>`, то есть как нечто, позволяющее нам использовать `locale::id` для нахождения соответствующего объекта класса, производного от `locale::facet`. Истинная реализация класса `locale` является эффективным вариантом этой идеи. Схема выглядит примерно следующим образом:



Здесь *collate<char>* и *num_punct<char>* являются стандартными библиотечными фасетами (§D.4). Как и любые фасеты, они наследуют от *locale: :facet*.

Полагается, что объекты *locale* (объекты локализации, объекты локального контекста) можно легко и дешево копировать. Отсюда следует, что класс *locale* почти наверняка является дескриптором к специализированному *map<id, facet*>*, представляющему собой основную часть реализации. Доступ к фасетам объектов локализации должен осуществляться быстро и эффективно, так что специализированный *map<id, facet*>* должен быть оптимизирован для быстрого доступа наподобие массива. Доступ к фасету объекта *locale* выполняется в нотации *use_facet<Facet>(loc)* (см. §D.3.1).

Стандартная библиотека предоставляет богатый набор фасетов. Чтобы помочь программисту ориентироваться в стандартных фасетах, они группируются в категории, такие как *numeric* или *collate* (§D.4).

Программист может заменять фасеты в существующих категориях (§D.4, §D.4.2.1). Нет, однако, возможности добавлять новые категории (нет возможности определить новую категорию программным путем). Понятие категории фасетов относится только к фасетам стандартной библиотеки и не является расширяемым. Фасет не обязан принадлежать к какой-либо категории, так что многие пользовательские фасеты и не принадлежат им.

По большей части локализация (неявно) используется в потоках ввода/вывода. У каждого *istream* и *ostream* есть свой *locale*. По умолчанию, поток получает глобальный *locale* (§D.2.1) в момент создания потока. Задать локализацию потока можно при помощи функции *imbue()*, а извлекается копия *locale* потока при помощи функции *getloc()* (§21.6.3).

D.2.1. Локальные контексты с заданными именами

Объекты *locale* конструируются из других *locale* и фасетов. Самым простым способом их создания является копирование. Например:

```
locale loc0;                                // копия текущей глобальной локализации (§D.2.3)
locale loc1 = locale();                    // копия текущей глобальной локализации (§D.2.3)
locale loc2 ("");                          // копия "локализации, предпочитаемой пользователем"
locale loc3 ("C");                          // копия "C"-локализации
locale loc4 = locale::classic();           // копия "C"-локализации
locale loc5 ("POSIX");                     // копия "POSIX"-локализации, задаваемой реализацией
```

Смысл *locale("C")* задается стандартом как классический контекст локализации языка C; именно он подразумевался на протяжении всей книги. Остальные имена контекстов локализации определяются реализациями.

Полагается, что *locale("")* задает контекст локализации, предпочтительный для пользователя системы, который формируется не языковыми, а системными средствами.

В большинстве операционных систем имеются свои собственные средства формирования контекста локализации программы. Часто локализация фиксируется в момент начала работы с системой. Например, можно предположить, что пользователь, выбравший аргентинский испанский в качестве рабочего языка по умолчанию, получит для *locale("")* именно *locale("es_AR")*. Беглый взгляд на одну из

моих систем обнаружил 51 мнемоническое имя для контекстов локализации, таких как **POSIX**, **de**, **en_UK**, **en_US**, **es**, **es_AR**, **fr**, **sv**, **da**, **pl** и **iso_8859_1**. Стандарт **POSIX** рекомендует начинать их именем языка (буквами нижнего регистра), за которыми может следовать название страны прописными буквами, после чего может следовать спецификатор кодировки, например, **jp_JP.jit**. Однако этот стандарт не распространяется на все платформы, на некоторых из которых, среди прочего, обнаружил имена **g**, **uk**, **us**, **s**, **fr**, **sw** и **da**. Стандарт языка C++ не определяет смысла объектов локализации для заданных стран или конкретных естественных языков, что, однако, может стандартизоваться в рамках тех или иных платформ. Поэтому при обращении к именованным контекстам локализации программисту нужно ознакомиться с соответствующей документацией по его системе (и, возможно, поэкспериментировать).

Как правило, нежелательно непосредственно в тексте программы располагать строки с именами локализаций. Это ограничивает переносимость программы и заставляет программиста просматривать весь ее текст при необходимости адаптации программы к новой среде. Вместо этого лучше динамически считать системную информацию о локализации с помощью **locale** (""), или предоставить опытному пользователю возможность выбора альтернативной локализации путем ввода соответствующей строки. Например:

```
void user_set_locale (const string& question_string)
{
    cout << question_string;           // "Нужна другая локализация? - введите ее имя"

    string s;
    cin >> s;
    locale : : global (locale (s.c_str()) ); // установить указанную пользователем
                                           // локализацию в качестве глобальной
}
```

Менее подготовленному пользователю желательно предоставить выбор возможных альтернатив из заранее сформированного списка строк с допустимыми именами. Программа должна знать, где система хранит сведения о локализации.

Если строковый аргумент ссылается на недействительную локализацию, конструктор генерирует исключение **runtime_error** (§14.10). Например:

```
void set_loc (locale& loc, const char* name)
try
{
    loc = locale (name);
}
catch (runtime_error)
{
    cerr << "locale \"\" << name << "\" isn't defined\n";
    // ...
}
```

Если контекст локализации именованный, то функция **name()** вернет его имя, а если неименованный — то **string("")**. Строку с именем можно использовать для отладки. Например:

```
void print_locale_names (const locale& my_locale)
{
    cout<< "name of current global locale: " << locale().name() << "\n";
    cout<< "name of classic C locale: " << locale::classic().name() << "\n";
    cout<< "name of ' ' user's preferred locale' ' : " << locale("").name() << "\n";
    cout<< "name of my locale: " << my_locale.name() << "\n";
}
```

Контексты локализации с идентичными именами (отличными от значения по умолчанию *string*("")) считаются совпадающими. В то же время, операции == или != предоставляют более непосредственный способ их сравнения.

Вызов *locale*(*loc*, "Foo", *cat*) создает контекст, подобный *loc*, но фасеты берутся из категории *cat* контекста *locale*("Foo"). Результирующий контекст имеет строку имени лишь в том случае, если она есть у исходного контекста *loc*. Стандарт не определяет, какова эта строка на самом деле, но предполагается, что она будет отличаться от таковой у контекста *loc*. Одно из возможных решений — формирование строки из исходной строки контекста *loc* плюс имя "Foo". Например, если у *loc* строка имени есть "en_UK", то новый контекст получит строку "en_UK: Foo".

Обобщим сведения о строке имени вновь создаваемого контекста следующим образом:

Локализации	Строка имени
<i>locale</i> ("Foo")	"Foo"
<i>locale</i> (<i>loc</i>)	<i>loc.name</i> ()
<i>locale</i> (<i>loc</i> , "Foo", <i>cat</i>)	Новая строка имени, если у <i>loc</i> есть строка имени; иначе — <i>string</i> ("")
<i>locale</i> (<i>loc</i> , <i>loc2</i> , <i>cat</i>)	Новая строка имени, если у <i>loc</i> и <i>loc2</i> есть строка имени; иначе — <i>string</i> ("")
<i>locale</i> (<i>loc</i> , <i>Facet</i>)	<i>string</i> ("")
<i>loc.combine</i> (<i>loc2</i>)	<i>string</i> ("")

Не существует программных средств, позволяющих для вновь созданного контекста задать строку имени локализации в С-стиле. Строки имен либо определяются в рамках системы, либо создаются в виде комбинаций таких имен конструкторами объектов *locale*.

D.2.1.1. Конструирование новых объектов локализации

Новый контекст локализации создается на основе существующего посредством добавления или замены фасетов. В типичном случае это приводит к незначительной модификации исходного контекста. Например:

```
void f(const locale& loc, const My_money_io* mio) // My_money_io определен в §D.4.3.1
{
    locale loc1 (locale ("POSIX"), loc, locale::monetary); // monetary фасеты из loc
    locale loc2 = locale (locale::classic(), mio); // classic плюс mio
    ..
}
```

Здесь **loc1** является модифицированной копией контекста **POSIX**, в которую добавляются «денежные» фасеты из **loc** (§D.4.3). Аналогично, **loc2** является копией контекста в C-стиле, модифицированной для использования **My_money_io** (§D.4.3.1). Если аргумент типа **Facet*** (в нашем случае это **My_money_io**) равен 0, то результирующий контекст будет просто копией аргумента типа **locale**.

В записи

```
locale (const locale& x, Facet* f) ;
```

аргумент **f** должен идентифицировать конкретный тип фасета. Просто **facet*** не проходит. Например:

```
void g (const locale : facet* mio1, const My_money_io* mio2)
{
    locale loc3 = locale (locale : classic (), mio1) ;    // error: не известен тип фасета
    locale loc4 = locale (locale : classic (), mio2) ;    // ok: тип фасета известен
    // ...
}
```

Причина в том, что **locale** использует аргумент **Facet*** для определения типа фасета на этапе компиляции. А именно, реализация **locale** использует тип, идентифицирующий фасет, **facet** : **id** (§D.3), для поиска фасета в контексте локализации.

Обратите внимание, что конструктор

```
template<class Facet> locale (const locale& x, Facet* f) ;
```

является единственным языковым механизмом, позволяющим программисту задать фасет, который будет использоваться в объекте локализации. Именованные контексты локализации предоставляются непосредственно реализациями (§D.2.1). Эти именованные локализации можно извлекать из системной среды. Если программист сможет изучить и понять эти механизмы, он сможет таким способом добавлять новые контексты локализации (§D.6[11,12]).

Набор конструкторов для **locale** спроектирован таким образом, чтобы тип каждого фасета становился известен либо путем выведения типа (из параметра **Facet** шаблона), либо благодаря тому, что он достается от другого объекта локализации (и который знает этот тип). Задание аргумента **category** косвенно определяет тип фасета, потому что в контексте локализации типы фасетов в категориях известны. Это подразумевает, что в классе **locale** типы фасетов известны, что позволяет манипулировать ими с минимальными накладными расходами.

Для идентификации типов фасетов класс **locale** использует поле **locale** : **id** (§D.3).

Бывает полезно создать объект локализации, во всем являющийся копией другого объекта локализации за исключением того, что некоторые его фасеты взяты из третьего контекста локализации. Это реализуется шаблонной функцией-членом **combine** (). Например:

```
void f (const locale& loc, const locale& loc2)
{
    locale loc3 = loc.combine<My_money_io> (loc2) ;
    //
```

Результирующий объект *loc3* ведет себя как *loc*, только он использует копию *My_money_io* (§D.4.3.1) из *loc2* для форматирования ввода/вывода денежных величин. Если в *loc2* нет *My_money_io*, то функция *combine()* сгенерирует исключение *runtime_error* (§14.10). Результат работы функции *combine()* не имеет строки имени.

D.2.2. Копирование и сравнение контекстов локализации

Объекты *locale* копируются при инициализации и в операциях присваивания. Например:

```
void swap(locale& x, locale& y)      // как std::swap()
{
    locale temp = x;
    x = y;
    y = temp;
}
```

Копия объекта *locale* считается равной оригиналу, но при этом сама она является отдельным независимым объектом. Например:

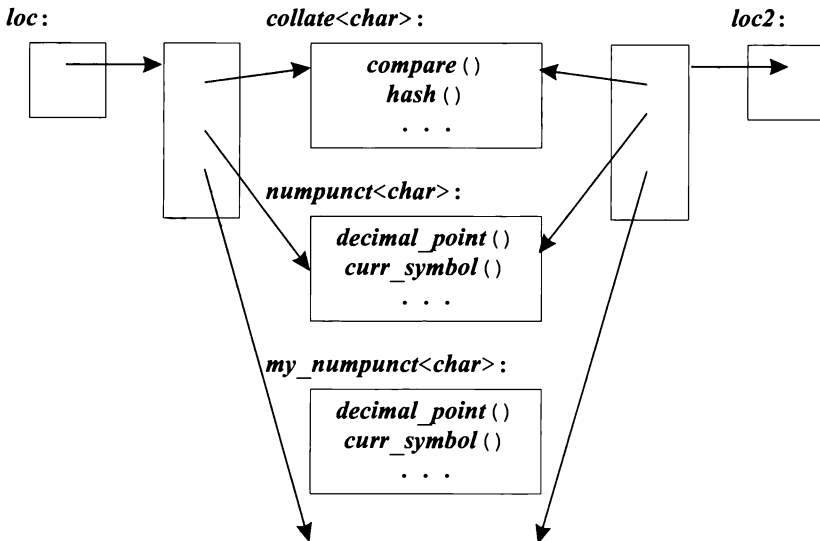
```
void f(locale* my_locale)
{
    locale loc = locale::classic();      // "C" locale

    if(loc != locale::classic())
    {
        cerr << "implementation error: send bug report to vendor\n";
        exit(1);
    }

    if(&loc != &locale::classic()) cout << "no surprise: addresses differ\n";
    locale loc2 = locale(loc, my_locale, locale::numeric);

    if(loc == loc2)
    {
        cout << "my numeric facets are the same as classic() 's numeric facets\n";
        // ...
    }
    // ...
}
```

Если *my_locale* имеет отвечающий за числовую пунктуацию фасет *my_num_punct<char>*, отличный от стандартного *num_punct<char>* из *classic()*, то результирующие контексты локализации можно отобразить следующим образом:



Не существует способа модификации **locale**. Вместо этого операции класса **locale** позволяют создавать новые объекты локализации на базе уже существующих. Тот факт, что объекты локализации неизменяемы после создания, очень важен для обеспечения эффективности на этапе выполнения. Это позволяет пользователю вызывать виртуальные функции фасетов и кэшировать возвращаемые значения. Например, **istream** может знать, какой символ используется в качестве десятичной запятой, или как представляется значение **true**, не вызывая **decimal_point()** каждый раз при чтении числа и **truename()** при чтении в логическую переменную (§D.4.2). Только вызов **imbue()** для потока может заставить изменить значения, возвращаемые этими вызовами.

D.2.3. Контексты **global()** и **classic()**

Понятие текущей локализации программы поддерживается функцией **locale()**, возвращающей копию текущей локализации, а также функцией **locale::global()**, устанавливающей контекст локализации в качестве текущего. Текущую локализацию обычно называют глобальной, имея в виду то, что чаще всего она реализуется с помощью глобального (или статического) объекта.

Глобальная локализация используется неявным образом при инициализации потока, то есть к каждому вновь создаваемому потоку прикрепляется (**imbue()**; §21.1, §21.6.3) копия **locale()**. Изначально, глобальная локализация есть стандартная локализация языка C, **locale::classic()**.

Статическая функция-член **locale::global()** позволяет программисту задать контекст локализации, который будет использоваться как глобальный. Эта функция возвращает предыдущий контекст локализации, использовавшийся в качестве глобального контекста, что позволяет при необходимости восстановить его позже. Например:

```
void f(const locale& my_loc)
{
    ifstream fin1(some_name);
```

```
// к fin1 прикреплена глобальная локализация
```

```

locale old_global = locale : :global (my_loc) ; //устанавливаем новую глоб-ную локал-цию
ifstream fin2 (some_other_name) ; // к fin2 прикреплена локал-ция my_loc
// ...
locale : :global (old_global) ; // восстанавливаем стар. глоб-ную локал-цию
}

```

Если в локализации *x* имеется строка имени, то **locale : :global (x)** также устанавливает глобальную локализацию языка *C*. Это означает, что если программа на *C++* вызывает зависимую от контекста локализации стандартную библиотечную функцию языка *C*, то в такой смешанной среде трактовка локализации будет согласованной.

Однако если локализация *x* строки имени не содержит, то неизвестно, влияет ли вызов **locale : :global ()** на глобальный контекст локализации языка *C*. Это означает, что в таком случае программа на языке *C++* не может надежным и переносимым образом установить для средств языка *C* локализацию, которая не была получена от системной среды. Также не существует стандартного способа, которым программа на языке *C* могла бы установить глобальную локализацию *C++* (кроме как вызвать для этого функцию *C++*). В общем, в смешанной среде программ на языках *C* и *C++* использование для языка *C* глобальной локализации, отличной от **global ()**, чревато ошибками.

Задание глобальной локализации не влияет на уже существующие потоки ввода/вывода; они по-прежнему используют те локализации, которые были закреплены за ними. Например, на поток **fin1** не повлияли последующие манипуляции с глобальной локализацией, в результате которых за потоком **fin2** была закреплена локализация **my_loc**.

Манипуляции с глобальной локализацией страдают общим пороком работы с глобальными объектами — очень трудно (если возможно) проконтролировать, на что может повлиять изменение этих объектов. Поэтому вызовы **global ()** лучше всего минимизировать и четко локализовать их в нескольких разделах кода, подчиняющихся простой стратегии этих изменений. Возможность прикреплять специфические контексты локализации к отдельным потокам (**imbue ()**; §21.6.3) существенно упрощает дело, но все равно, изобилие разбросанных по программе явных ссылок на контексты локализации сильно затрудняет сопровождение программы.

D.2.4. Сравнение строк

Сравнение двух строк в соответствии с конкретной локализацией является, наверное, самой распространенной задачей, явно использующей контекст локализации. Поэтому данная операция непосредственно предоставляется классом **locale**, чтобы избавить пользователей от необходимости создавать собственные функции сравнения из фасета **collate** (§D.4.1). Чтобы функцию сравнения из класса **locale** можно было использовать непосредственно в качестве предиката (§18.4.2), она определена там как **operator () ()**. Например:

```

void f (vector<string>& v, const locale& my_locale)
{
    sort (v.begin (), v.end () ) ; // sort с применением < для сравнения элементов
    // ...
    sort (v.begin () , v.end () , my_locale) ; // sort согласно правилам my_locale
    //

```

По умолчанию, стандартная библиотечная функция **sort**() использует операцию < с числовыми значениями символического набора конкретной реализации (§18.7, §18.6.3.1).

Заметьте, что в рамках контекстов локализации сравниваются объекты типа **basic_string**, а не C-строки.

D.3. Фасеты

Фасеты являются объектами класса, производного от класса **facet** (объявленного в классе **locale**):

```
class std::locale::facet
{
protected:
    explicit facet(size_t r = 0); // "r==0" -локаль-я управляет временем жизни этого фасета
    virtual ~facet();           // обратите внимание: protected деструктор

private:
    facet(const facet&);          // не определен
    void operator=(const facet&); // не определен
    // представление класса
}
```

Операции копирования объявлены закрытыми и оставлены неопределенными для предотвращения копирования (§11.2.2).

Класс **facet** спроектирован в качестве базового, и в нем нет открытых функций. Его конструктор объявлен как **protected**, чтобы нельзя было создать объекты типа «просто **facet**», а деструктор объявлен виртуальным для обеспечения корректного уничтожения объектов производных классов.

Предполагается, что оперировать фасетами будут объекты **locale** при помощи указателей. Нулевой аргумент конструктора класса **facet** означает, что **locale** обязан удалить фасет после исчезновения последней ссылки на него. Напротив, ненулевой аргумент конструктора гарантирует, что **locale** никогда не удалит фасет. Это значение аргумента предназначено для редкого случая, когда сам программист управляет временем жизни фасета (а не локализация). Например, мы можем попытаться создать объекты стандартного класса фасетов **collate_byname<char>** (§D.4.1.1) следующим образом:

```
void f(const string& s1, const string& s2)
{
    // нормальный случай: (умолчательный) аргумент 0 - locale отвечает за удаление:
    collate<char>* p = new collate_byname<char>("pl");
    locale loc(locale(), p);

    // редкий случай: аргумент 1 - пользователь отвечает за удаление:
    collate<char>* q = new collate_byname<char>("ge", 1);

    collate_byname<char> bug1("sw"); // error: невозм-но унич-ить лок-ную перем-ую
    collate_byname<char> bug2("no", 1); // error: невозм-но унич-ить лок-ную перем-ую
    // ...

    // q нельзя удалить: у collate_byname<char> деструктор protected
    // нельзя delete p; locale управляет удалением *p
}
```

Стандартные фасеты полезны, когда управляются контекстами локализации (как базовые классы), и очень редко — в других случаях.

Фасет *_byname()* является фасетом из именованного локального контекста системной среды (§D.2.1).

Чтобы можно было осуществлять поиск фасета в рамках контекста локализации функциями *has_facet()* и *use_facet()*, для каждого типа фасета должен определяться идентификатор *id*:

```
class std::locale::id
{
public:
    id();

private:
    id(const id&);           // не определен
    void operator=(const id&); // не определен
    // представление
};
```

Операции копирования объявлены закрытыми и оставлены неопределенными для предотвращения копирования (§11.2.2).

Класс *id* задуман для того, чтобы пользователь мог определить статический член типа *id* в каждом классе, предоставляющем новый интерфейс фасета (например, см. §D.4.1). Внутренние механизмы класса *locale* используют *id* для идентификации фасетов (§D.2, §D.3.1). В очевидной реализации *id* используется как индекс вектора указателей на фасеты, реализуя тем самым эффективный *map<id, facet*>*.

Поля данных, необходимые для формирования (производного) фасета определяются в производных классах, а не в самом базовом классе *facet*. Это означает, что программист, определяющий фасет, имеет полный контроль над данными и что любое количество данных может быть задействовано для реализации концепции фасета.

Заметьте, что все функции-члены определяемого пользователем фасета должны быть константными. В общем случае, фасеты полагаются неизменяемыми (§D.2.2).

D.3.1. Доступ к фасетам класса *locale*

Доступ к фасетам класса *locale* осуществляется с помощью функционального шаблона *use_facet*. С помощью функционального шаблона *has_facet* мы можем узнать, имеется ли конкретный фасет в заданном контексте локализации:

```
template<class Facet> bool has_facet(const locale&) throw();
template<class Facet> const Facet& use_facet(const locale&); // может генер-ть bad_cast
```

Можно считать, что эти шаблонные функции осуществляют поиск параметра шаблона *Facet* в аргументе *locale*. Можно, однако, взглянуть на функциональный шаблон *use_facet* как на своего рода явное приведение типа от *locale* к конкретному фасету. Эта возможность базируется на том, что у *locale* только один фасет данного типа. Например:

```

void f(const locale& my_locale)
{
    char c = use_facet<num_punct<char>> (my_locale) .decimal_point(); // используем
                                                                    // стандартный фасет
    // ...

    If(has_facet<Encrypt> (my_locale)) // в my_locale есть фасет Encrypt?
    {
        const Encrypt& f = use_facet<Encrypt> (my_locale); // извлекаем фасет Encrypt
        const Crypto c = f.get_crypto(); // используем фасет Encrypt
        // ...
    }
    // ...
}

```

Обратите внимание на то, что **use_facet** возвращает ссылку на константный фасет, так что мы не можем присвоить этот возврат неконстантным переменным. Это разумно, поскольку фасеты полагаются неизменяемыми и содержащими лишь константные методы.

Если мы попытаемся вызвать **use_facet<X> (loc)**, а **loc** не содержит фасет **X**, то **use_facet** сгенерирует исключение **bad_cast** (§14.10). Так как гарантируется, что стандартные фасеты доступны во всех контекстах локализации (§D.4), то в этом случае нет необходимости обращаться к **has_facet**. Для стандартных фасетов **use_facet** никогда не сгенерирует исключение **bad_cast**.

Как могут быть реализованы **use_facet** и **has_facet**? Вспомним, что **locale** можно рассматривать как **map<id, facet*>** (§D.2). Отталкиваясь от полученного в параметре шаблона **Facet** типа фасета, реализации **use_facet** или **has_facet** могут обратиться к **Facet::id** и использовать его для поиска соответствующего фасета. Упрощенные реализации **has_facet** и **use_facet** могут выглядеть следующим образом:

```

// псевдореализация: пусть locale содержит map<id, facet*> названный facet_map
template<class Facet> bool has_facet(const locale& loc) throw()
{
    const locale::facet* f = loc.facet_map[Facet::id];
    return f ? true : false;
}

template<class Facet> const Facet& use_facet(const locale& loc)
{
    const locale::facet* f = loc.facet_map[Facet::id];
    if(f) return static_cast<const Facet&>(*f);
    throw bad_cast();
}

```

Взглянув по-другому на механизм **facet::id**, можно реализовать некую форму полиморфизма на этапе компиляции. Очень близкий к **use_facet** результат можно получить применением операции **dynamic_cast**. Однако специализированный шаблон **use_facet** может быть реализован гораздо эффективнее общего механизма на базе операции **dynamic_cast**.

На самом деле **id** идентифицирует скорее интерфейс и поведение, нежели класс. То есть если интерфейсы двух классов фасетов в точности совпадают при одинаково-

вой семантике (по отношению к контексту локализации), они должны идентифицироваться одним и тем же *id*. Например, *collate<char>* и *collate_byname<char>* для *locale* взаимозаменяемы, и потому оба идентифицируются при помощи одного и того же *collate<char>::id* (§D.4.1).

Если мы определяем фасет с новым интерфейсом — например *Encrypt* в функции *f()* — мы должны определить и соответствующий *id* для его идентификации (см. §D.3.2 и §D.4.1).

D.3.2. Простой пользовательский фасет

Стандартная библиотека предоставляет стандартные фасеты, учитывающие наиболее важные аспекты национальных особенностей, такие как наборы символов и ввод/вывод чисел. Чтобы познакомиться с механизмом фасетов вне их связи с широко применяемыми типами, влекущими за собой всякие сложности, а также вне забот о высокой эффективности, которые их сопровождают, рассмотрим фасет для тривиального пользовательского типа:

```
enum Season {spring, summer, fall, winter};
```

Это простейший пользовательский тип, который я вообще могу себе представить. Показанный ниже стиль ввода/вывода может использоваться (с минимальными изменениями) для большинства простых пользовательских типов:

```
class Season_io : public locale::facet
{
public:
    Season_io(int i = 0) : locale::facet(i) {}
    ~Season_io() {} // обеспечивает возможность уничтожения объектов Season_io (§D.3)
    virtual const string& to_str(Season x) const = 0; // строковое представление для x
    // place Season corresponding to s in x:
    virtual bool from_str(const string& s, Season& x) const = 0;
    static locale::id id; // объект идентификации фасета (§D.2, §D.3, §D.3.1)
};

locale::id Season_io::id; // определяем идентифицирующий объект
```

Для простоты данный фасет ограничивается представлениями, использующими лишь *char*.

Класс *Season_io* обеспечивает общий абстрактный интерфейс ко всем фасетам иерархии наследования. Чтобы определить вывод объектов типа *Season* для конкретного контекста локализации, мы наследуем класс от *Season_io*, определяя при этом надлежащим образом функции *to_str()* и *from_str()*.

Вывести *Season* просто. Если у потока имеется фасет *Season_io*, то мы можем использовать его для преобразования времени года¹ в строку. В противном случае мы можем вывести целое значение:

```
ostream& operator<<(ostream& s, Season x)
{
    const locale& loc = s.getloc(); // извлекаем stream's locale (§21.7.1)
```

¹ Время года по-английски — season. — Прим. ред.

```

if (has_facet<Season_io> (loc) ) return s << use_facet<Season_io> (loc) .to_str (x) ;
return s << int (x) ;
}

```

Обратите внимание, что данная операция << реализована посредством вызова этой же операции, но для других типов. Таким образом, мы извлекаем пользу из простоты вызова операций << по сравнению с прямым доступом к буферам потока **ostream**, из учета этими операциями контекста локализации и из реализованной ими обработки ошибок. Стандартные фасеты напрямую обращаются к буферам потоков (§D.4.2.2, §D.4.2.3) ради максимальной эффективности и гибкости, но для многих простых пользовательских типов нет никакой необходимости опускаться на уровень абстракции **streambuf**.

Как обычно, ввод немного сложнее вывода:

```

istream& operator>> (istream& s, Season& x)
{
    const locale& loc = s.getloc () ;           // извлекаем stream's locale (§21.7.1)
    if (has_facet<Season_io> (loc) )           // читаем алфавитное представление
    {
        const Season_io& f = use_facet<Season_io> (loc) ;
        string buf;
        if ( ! (s>>buf && f.from_str (buf, x) ) ) s.setstate (ios_base::failbit) ;
        return s ;
    }
    ,

    int i;                                     // читаем числовое представление
    s >> i;
    x = Season (i) ;
    return s ;
}

```

Обработка ошибок проста и следует стилю обработки ошибок для встроенных типов. То есть если вводимая строка не соответствует стилю **Season** в рамках текущего контекста локализации, поток устанавливается в состояние **fail**. Если допускаются исключения, то следует обрабатывать исключения **ios_base::failure** (§21.3.6).

Вот тривиальная тестовая программа:

```

int main ()
{
    Season x;

    // Используем локал-цию по умолчанию (нет фасета Season_io) - ввод/вывод целых:
    cin >> x;
    cout << x << endl;

    locale loc (locale () , new US_season_io) ;
    cout.imbue (loc) ;           // используем локализацию с фасетом Season_io
    cin.imbue (loc) ;           // используем локализацию с фасетом Season_io

    cin >> x;
    cout << x << endl;
}

```

Для входа

2

summer

эта программа отреагирует следующим выводом:

2

summer

Для достижения этого результата мы должны определить *US_season_io* с целью задания строкового представления времен года и замещения функций интерфейса *Season_io*, которые преобразуют строковые представления в перечисления и обратно:

```
class US_season_io: public Season_io
{
    static const string seasons[];

public:
    const string& to_str(Season) const;
    bool from_str(const string&, Season&) const;
    // обратите внимание: отсылчаем US_season_io::id
};

const string US_season_io::seasons[] = {"spring", "summer", "fall", "winter"};

const string& US_season_io::to_str(Season x) const
{
    if(x < spring || winter < x)
    {
        static const string ss = "no-such-season";
        return ss;
    }
    return seasons[x];
}

bool US_season_io::from_str(const string& s, Season& x) const
{
    const string* beg = &seasons[spring];
    const string* end = &seasons[winter] + 1;
    const string* p = find(beg, end, s); // §3.8.1, §18.5.2

    if(p == end) return false;
    x = Season(p - beg);
    return true;
}
```

Заметьте, что поскольку *US_season_io* является просто реализацией интерфейса *Season_io*, я не определял *id* для *US_season_io*. Вообще, если мы хотим использовать *US_season_io* в качестве *Season_io*, мы не можем задавать для *US_season_io* его собственный *id*. Операции класса *locale*, например *has_facet* (§D.3.1), полагаются на то, что фасеты, реализующие одну и ту же концепцию, идентифицируются одним и тем же *id* (§D.3).

Единственным вопросом в данной реализации остается вопрос, что делать, когда для вывода предоставляется некорректный *Season*? Случаи появления некор-

ректных значений для простых пользовательских типов не столь уж необычны, так что стоит заранее принять во внимание подобную ситуацию. Я мог бы сгенерировать исключение. Но когда имеешь дело с простым выводом, предназначенным для человеческих глаз, полезно подготовить специальное визуальное представление для некорректных значений. Обратите внимание, что при вводе стратегия обработки ошибок передана операции `<<`, в то время как при выводе эта стратегия оставляется функции фасета `to_str()`. Это было сделано с целью проиллюстрировать возможные варианты проектирования. В настоящих, промышленных проектах функции фасета либо реализуют обработку ошибок как ввода, так и вывода, либо перепоручают все это операциям `>>` и `<<`.

Данный проект для *Season_io* полагается на то, что производные классы предоставляют специфичные для локализации строки. В альтернативном дизайне *Season_io* сам мог бы извлекать строки из соответствующего хранилища (см. §D.4.7). Создание класса *Season_io*, которому строки времен года передаются в качестве аргументов конструктора, оставляется в качестве упражнения (§D.6[2]).

D.3.3. Использование локализаций и фасетов

В рамках стандартной библиотеки локализации в первую очередь принимаются во внимание в операциях ввода/вывода. Тем не менее, механизм локализации является общим и расширяемым средством представления информации, чувствительной к локальным и национальным особенностям. Класс *messages* (§D.4.7) является примером фасета, не имеющего никакого отношения ко вводу/выводу. Расширения библиотеки потокового ввода/вывода и средства ввода/вывода, не опирающиеся на потоки, также могут использовать объекты *locale*. Пользователь может использовать контексты локализации для организации произвольной информации, зависимой от национальных и культурных особенностей.

Благодаря общности механизмов локализаций и фасетов возможности определяемых пользователем фасетов не ограничены. Вероятными кандидатами на представление фасетами являются даты, часовые пояса, телефонные номера, номера социального страхования, товарные коды, температурные величины, произвольные пары измеряемых величин (единица измерения, значение), почтовые коды (zip-коды), размеры одежды и т.д.

Как и другие мощные механизмы, фасеты следует применять с определенной осторожностью. Из одного лишь факта, что нечто может быть представлено в виде фасета, не следует, что такое представление будет наилучшим. Для выбора ключевыми, как и всегда, являются следующие факторы: как различные решения влияют на сложность написания кода, легко или сложно читать этот код, насколько сложно поддерживать код в дальнейшем в процессе его эксплуатации и неизбежных модификаций, какова эффективность операций ввода/вывода в плане быстродействия и используемой памяти.

D.4. Стандартные фасеты

Стандартная библиотека предоставляет в заголовочном файле `<locale>` следующие фасеты для локализации *classic()*:

Стандартные фасеты (в локализации <code>classic()</code>)			
	Категория	Назначение	Фасеты
§D.4.1	<i>collate</i>	сравнение строк	<i>collate</i> < <i>Ch</i> >
§D.4.2	<i>numeric</i>	числовой ввод/вывод	<i>numpunct</i> < <i>Ch</i> >
			<i>num_get</i> < <i>Ch</i> >
			<i>num_put</i> < <i>Ch</i> >
§D.4.3	<i>monetary</i>	ввод/вывод денежных сумм	<i>money_punct</i> < <i>Ch</i> >
			<i>money_punct</i> < <i>Ch</i> , <i>true</i> >
			<i>money_get</i> < <i>Ch</i> >
			<i>money_put</i> < <i>Ch</i> >
§D.4.4	<i>time</i>	ввод/вывод времени	<i>time_get</i> < <i>Ch</i> >
			<i>time_put</i> < <i>Ch</i> >
§D.4.5	<i>ctype</i>	классификация символов	<i>ctype</i> < <i>Ch</i> >
			<i>codecvt</i> < <i>Ch</i> , <i>char</i> , <i>mbstate_t</i> >
§D.4.7	<i>messages</i>	извлечение сообщений	<i>messages</i> < <i>Ch</i> >

В этой таблице ***Ch*** означает ***char*** или ***wchar_t***. Пользователь, которому требуется, чтобы стандартный ввод/вывод работал с другим символьным типом *X*, должен предоставить подходящие варианты фасетов для *X*. Например, может потребоваться ***codecvt***<*X*, *char*, *mbstate_t*> (§D.4.6) для управления преобразованиями между *X* и *char*. Тип *mbstate_t* используется для представления состояния сдвига в многобайтных кодировках символов (§D.4.6); *mbstate_t* определен в ***cwchar*** и ***wchar.h***. Эквивалентом *mbstate_t* для произвольного символьного типа *X* является ***char_traits***<*X*>::*state_type*.

В заголовочном файле ***<locale>*** стандартная библиотека дополнительно предоставляет следующие фасеты:

Стандартные фасеты			
	Категория	Назначение	Фасеты
§D.4.1	<i>collate</i>	сравнение строк	<i>collate_byname</i> < <i>Ch</i> >
§D.4.2	<i>numeric</i>	ввод/вывод чисел	<i>num_punct_byname</i> < <i>Ch</i> >
			<i>num_get</i> < <i>C</i> , <i>In</i> >
			<i>num_put</i> < <i>C</i> , <i>Out</i> >
§D.4.3	<i>monetary</i>	ввод/вывод денег	<i>money_punct_byname</i> < <i>Ch</i> , <i>International</i> >
			<i>money_get</i> < <i>C</i> , <i>In</i> >
			<i>money_put</i> < <i>C</i> , <i>Out</i> >
§D.4.4	<i>time</i>	ввод/вывод времени	<i>time_put_byname</i> < <i>Ch</i> , <i>Out</i> >
§D.4.5	<i>ctype</i>	классификация символов	<i>ctype_byname</i> < <i>Ch</i> >
§D.4.7	<i>messages</i>	извлечение сообщений	<i>messages_byname</i> < <i>Ch</i> >

При конкретизации фасетов из данной таблицы **Ch** может быть **char** или **wchar_t**, **C** может быть любым символьным типом (§20.1). Параметр **International** может иметь значение **true** или **false**; **true** означает, что используется международное четырехсимвольное представление валютного знака (§D.4.3.1). Тип **mbstate_t** определен в **<cwchar>** и **<wchar.h>**.

Параметры **In** и **Out** — это итераторы ввода и вывода, соответственно (§19.1, §19.2.1). Придание фасетам с суффиксами **_put** и **_get** этих параметров позволяет программисту реализовывать фасеты с прямым доступом к нестандартным буферам (§D.4.2.2). Буфера, ассоциированные с **iostream**, являются буферами потоков, так что тип связанных с ними итераторов есть **ostreambuf_iterator** (§19.2.6.1, §D.4.2.2). Как следствие, для обработки ошибок доступна функция **failed()** (§19.2.6.1).

Фасет **F_byname** является производным от фасета **F**. Фасет **F_byname** предоставляет такой же интерфейс, что и **F**, за исключением того, что добавлен конструктор со строковым аргументом для задания имени локализации (см. §D.4.1). Фасет **F_byname(name)** предоставляет соответствующую семантику для **F**, определенную в **locale(name)**. Идея заключается в том, чтобы извлечь версию стандартного фасета из именованной локализации (§D.2.1) системной среды выполнения программы. Например:

```
void f(vector<string>& v, const locale& loc)
{
    locale d1(loc, new collate_byname<char>("da")); // сравнение датских строк
    locale dk(d1, new ctype_byname<char>("da"));    // классиф-я датских символов
    sort(v.begin(), v.end(), dk);
    // ...
}
```

Новый контекст локализации **dk** будет пользоваться датскими текстовыми строками, но прежними умолчательными соглашениями по числам. Отметим, что поскольку умолчательное значение второго аргумента фасета есть **0**, то **locale** будет управлять временем жизни фасета, созданного операцией **new** (§D.3).

Подобно конструкторам класса **locale** со строковыми аргументами, конструкторы фасетов с суффиксом **_byname** осуществляют доступ к системной среде выполнения программы. Это означает, что они существенно медленнее конструкторов, которым не нужен такой доступ. Почти всегда быстрее сначала создать объект локализации, и лишь после этого выполнять доступ к его фасетам, чем пользоваться фасетами с суффиксом **_byname** во многих местах программы. Таким образом, хорошей идеей является однократное чтение фасета из системной среды с последующим многократным использованием его копии во многих местах программы. Например:

```
locale dk("da"); // читаем датскую локаль (включая все фасеты) единожды
                // а затем используем dk и его фасеты по необходимости

void f(vector<string>& v, const locale& loc)
{
    const collate<char>& col = use_facet<collate<char>>(dk);
    const ctype<char>& ctyp = use_facet<ctype<char>>(dk);

    locale d1(loc, col); // сравнение датских строк
    locale d2(d1, ctyp); // сравнение датских строк и классификация датских символов
}
```

```
sort(v.begin(), v.end(), d2);
// ...
}
```

Понятие категории предоставляет более простой способ манипулирования стандартными фасетами в контекстах локализации. Например, располагая локализацией **dk**, мы можем создать контекст локализации, в рамках которого строки читаются и сравниваются в соответствии с правилами датского языка (в котором есть три лишних гласных по сравнению с английским языком), а числа сохраняют синтаксис языка C++:

```
locale dk_us(locale::classic(), dk, collate|ctype); // датские буквы, обычные числа
```

Ниже при описании конкретных стандартных фасетов даются дополнительные примеры их использования. В частности, обсуждение **collate** (§D.4.1) проявляет многие структурные (архитектурные) аспекты фасетов.

Обратите внимание на то, что стандартные фасеты часто зависят друг от друга. Например, **num_put** зависит от **num_punct**. Чтобы успешно комбинировать и сопоставлять фасеты, а также создавать новые стандартные фасеты, вы должны хорошо знать их индивидуальные свойства. Другими словами, за исключением простых операций, рассмотренных в §21.7, механизмы контекстов локализации не предназначены для их непосредственного использования новичками.

Разработка конкретного фасета весьма трудоемка. Отчасти причина заключается в том, что конкретные национальные и культурные соглашения сами по себе весьма запутанные, а кроме того важно, чтобы средства стандартной библиотеки C++ были совместимы со средствами стандартной библиотеки языка C и со многими другими системными соглашениями и стандартами. Например, разработчику библиотеки было бы неразумно игнорировать средства локализации POSIX.

С другой стороны, среда, формируемая с помощью **locale** и фасетов, весьма гибкая и общая. Можно сконструировать фасет для любых данных, а операции фасета позволят выполнять с этими данными любые специфические для них действия. Если фасет не будет слишком стеснен существующими соглашениями, то его дизайн и реализация могут быть простыми и ясными (§D.3.2).

D.4.1. Сравнение строк

Стандартный фасет **collate** обеспечивает сравнение массивов символов типа **Ch**:

```
template<class Ch>
class std::collate: public locale::facet
{
public:
    typedef Ch char_type;
    typedef basic_string<Ch> string_type;

    explicit collate(size_t r = 0);

    int compare(const Ch* b, const Ch* e, const Ch* b2, const Ch* e2) const
    { return do_compare(b, e, b2, e2); }

    long hash(const Ch* b, const Ch* e) const { return do_hash(b, e); }
    string_type transform(const Ch* b, const Ch* e) const { return do_transform(b, e); }
```

```

static locale : : id id;    // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected :
~collate () ;              // внимание: protected деструктор

virtual int do_compare (const Ch* b, const Ch* e, const Ch* b2, const Ch* e2)
const;
virtual string_type do_transform (const Ch* b, const Ch* e) const;
virtual long do_hash (const Ch* b, const Ch* e) const;
};

```

Как и любые фасеты, **collate** наследуется от **facet** и предоставляет конструктор с аргументом, указывающим, отвечает ли класс **locale** за время жизни фасета (§D.3).

Обратите внимание на то, что деструктор объявляется в секции **protected**. Фасет **collate** не предназначен для непосредственного использования, а служит базовым классом для наследования от него конкретных классов сравнения строк под управлением **locale** (§D.3). В рамках интерфейса **collate** прикладные программисты и разработчики библиотек могут писать свои собственные фасеты сравнения строк.

Функция **compare()** выполняет базовую операцию сравнения строк в рамках правил, установленных для конкретного **collate**; она возвращает **1**, если первая строка лексикографически больше второй; возвращает **0**, если строки идентичны, и возвращает **-1** в случае, когда вторая строка больше первой. Например:

```

void f (const string& s1, const string& s2, collate<char>& cmp)
{
    const char* cs1 = s1.data () ;    // поскольку compare() работает с char[]
    const char* cs2 = s2.data () ;

    switch (cmp.compare (cs1, cs1+s1.size () , cs2, cs2+s2.size () ) )
    {
        case 0 :                                // строки идентичны согласно cmp
            // ...
            break;
        case -1 :                               // s1 < s2
            // ...
            break;
        case 1 :                               // s1 > s2
            // ...
            break;
    }
}

```

Обратите внимание, что функции-члены **collate** сравнивают массивы элементов типа **Ch**, а не строки **basic_string** или C-строки с терминальным нулем. В частности, числовое значение **0** для **Ch** воспринимается как обычный символ, а не как терминальный нуль. Кроме того, **compare()** отличается от **strcmp()** тем, что возвращает именно **1**, **0** и **-1**, а не нуль и произвольные положительные и отрицательные значения (§20.4.1).

Стандартный библиотечный тип **string** не чувствителен к локализациям — он выполняет сравнение строк по правилам для набора символов реализации (§C.2). Более того, стандартный тип **string** не позволяет непосредственно задать критерий сравнения (глава 20). Чтобы выполнить сравнение, чувствительное к локализации,

мы можем использовать функцию **compare()** фасета **collate**. С точки зрения простоты записи, лучше использовать **compare()** неявно, через функцию-операцию **operator()** () класса **locale** (§D.2.4). Например:

```
void f(const string& s1, const string& s2, const char* n)
{
    bool b = s1 == s2;

    const char* cs1 = s1.data(); // поскольку compare() работает с char[]
    const char* cs2 = s2.data();

    typedef collate<char> Col;

    const Col& glob = use_facet<Col>(locale()); // из текущей глобальной локализации
    int i0 = glob.compare(cs1, cs1+s1.size(), cs2, cs2+s2.size());

    const Col& my_coll = use_facet<Col>(locale("")); // из предпочтительной локализации
    int i1 = my_coll.compare(cs1, cs1+s1.size(), cs2, cs2+s2.size());

    const Col& coll = use_facet<Col>(locale(n)); // из locale с именем n
    int i2 = coll.compare(cs1, cs1+s1.size(), cs2, cs2+s2.size());

    int i3 = locale()(s1, s2); // сравнение при текущей глобальной локализации
    int i4 = locale("") (s1, s2); // сравнение при предпочтительной локализации
    int i5 = locale(n) (s1, s2); // сравнение при локализации с именем n
    // ...
}
```

Здесь **i0==i3**, **i1==i4** и **i2==i5**. Нетрудно представить случаи, когда **i2**, **i3** и **i4** различаются. Рассмотрим последовательность слов из немецкого словаря:

Dialekt, Diät, dich, dichten, Dichtung

По соглашению, существительные (и только) пишутся с заглавной буквы, но их порядок не зависит от регистра букв.

Зависящая от регистра букв сортировка расположила бы эти слова в таком порядке:

Dialekt, Diät, Dichtung, dich, dichten

Буква **ä** (**a** умяют) интерпретируется как «нечто вроде **a**», и поэтому она стоит раньше буквы **c**. Однако в большинстве символьных кодовых наборов числовое значение для **a** больше числового значения для **c**, то есть **int(ä) > int(c)**, и на выходе простой сортировки на базе числовых значений будет:

Dialekt, Dichtung, Diät, dich, dichten

Написание функции сравнения, которая корректно упорядочивает данную последовательность слов точно по словарю, является интересным упражнением (§D.6[3]).

Функция **hash()** вычисляет *хэш-значение* (§17.6.2.3), которое требуется для построения *хэш-таблиц*.

Функция **transform()** производит из исходной строки некую строку, сравнение с которой третьих строк дает тот же результат, что и их сравнение с исходной строкой. Применение этой функции оптимизирует код, в котором некоторые строки сравниваются с одной и той же строкой. Это характерно для фрагментов, в которых, например, некоторая строка ищется во множестве других строк.

Функции `compare()`, `hash()` и `transform()`, объявленные в секции **public**, в своей реализации вызывают защищенные виртуальные функции `do_compare()`, `do_hash()` и `do_transform()`, соответственно; последние могут замещаться в производных классах. Такая двухслойная стратегия позволяет разработчикам библиотек при реализации неvirtуальных функций обеспечить некоторую общую функциональность для всех вызовов в независимости от того, что будет реализовано в «do-функциях».

Использование виртуальных функций обеспечивает, хотя и несколько накладным образом, полиморфную природу фасетов. Во избежание лишних накладных расходов на эти вызовы *locale* может установить точную природу используемого фасета и кэшировать любые значения, необходимые для повышения производительности (§D.2.2).

Статический член *id* типа `locale::id` используется для идентификации фасета (§D.3). Стандартные функции *has_facet* и *use_facet* полагаются на соответствие этих идентификаторов и фасетов (§D.3.1). Два фасета, предоставляющие локализации один и тот же интерфейс и семантику, должны иметь одно и то же значение *id*. Например, у `collate<char>` и `collate_byname<char>` эти идентификаторы одинаковые. И наоборот, два фасета, выполняющие разные функции по отношению к локализации, должны иметь разные *id*. Например, у `num_punct<char>` и у `num_put<char>` они разные (§D.4.2).

D.4.1.1. Именованные фасеты сравнения

Фасет `collate_byname` является для конкретной локализации вариацией `collate` с именем, заданным строковым аргументом конструктора:

```
template<class Ch>
class std::collate_byname: public collate<Ch>
{
public:
    typedef basic_string<Ch> string_type;

    // конструирование из именованной локализации:
    explicit collate_byname(const char*, size_t r = 0);

    // внимание: ни нового id, ни новых функций

protected:
    ~collate_byname();    // внимание: protected деструктор

    // замещаем виртуальные функции collate<Ch>:
    int do_compare(const Ch* b, const Ch* e, const Ch* b2, const Ch* e2) const;
    string_type do_transform(const Ch* b, const Ch* e) const;
    long do_hash(const Ch* b, const Ch* e) const;
};
```

Таким образом, фасетом `collate_byname` можно пользоваться для извлечения фасета `collate` из локализации, именованной в рамках системной среды выполнения программы (§D.4). Очевидным способом хранения фасетов в среде выполнения является запись необходимых данных в файл на диске. Менее гибкой альтернативой будет представление фасета в виде кода и данных в фасете с суффиксом `_byname`.

Класс *collate_byname*<char> является примером фасета, у которого нет собственного *id* (§D.3). В рамках *locale* фасеты *collate_byname*<Ch> и *collate*<Ch> взаимозаменяемы. Фасеты *collate* и *collate_byname* для одной и той же локализации различаются лишь дополнительным конструктором фасета *collate_byname*, а также его семантикой.

Отметим, что деструктор у *collate_byname* защищенный. Это означает, что нельзя определить локальные (в функции) объекты типа *collate_byname*:

```
void f()
{
    collate_byname<char> my_coll(""); ;    // error: невозможно уничтожить my_coll
    // ...
}
```

Это отражает взгляд на то, что контексты локализации и фасеты лучше определять где-нибудь на более высоком уровне, чтобы они могли оказывать влияние на более обширные фрагменты программ. Например, установка глобального контекста (§D.2.3) или прикрепление контекста локализации к потоку (§21.6.3, §D.1). Но при необходимости, мы можем наследовать от *collate_byname* класс с открытым деструктором и создавать локальные объекты такого класса.

D.4.2. Ввод и вывод чисел

Вывод чисел выполняется фасетом *num_put* посредством записи в буфер потока (§21.6.4). Ввод чисел выполняется фасетом *num_get* путем чтения из буфера потока. Формат, используемый фасетами *num_put* и *num_get*, задается «фасетом пунктуации чисел» — *num_punct*.

D.4.2.1. Пунктуация чисел

Фасет *num_punct* определяет формат ввода/вывода встроенных типов, таких как *bool*, *int* и *double*:

```
template<class Ch>
class std::num_punct: public locale::facet
{
public:
    typedef Ch char_type;
    typedef basic_string<Ch> string_type;
    explicit num_punct(size_t r = 0) ;

    Ch decimal_point() const;    // '.' в classic()
    Ch thousands_sep() const;    // ',' в classic()
    string grouping() const;    // "" в classic() - значит, что нет группировки

    string_type truename() const; // "true" в classic()
    string_type falsename() const; // "false" в classic()

    static locale::id id;    // объект идентификации фасета (§D2, §D.3, §D.3.1)
protected:
    ~num_punct();
    // виртуальные "do_"-функции (см. §D.4.1)
```

Символы строки, возвращаемые функцией **grouping()**, читаются как последовательность небольших целых значений, каждое из которых идентифицирует число цифр в группе. Символ с номером **0** соответствует самой правой группе (то есть наименее значимым цифрам), символ с номером **1** — группу слева от предыдущей и т.д. Таким образом, "**\004\002\003**" описывает формат чисел вида **123-45-6789** (при условии, что в качестве разделителя используется знак '-'). При необходимости последнее числовое значение в образце группировки может быть повторено. В нашем примере "**\003**" эквивалентно, например, "**\003\003\003**". Само название функции **thousands_sep()** (разделение тысяч), возвращающей символ разделения, говорит о том, что основное предназначение группировки заключается в том, чтобы представить большие числа в более читаемой форме. Функции **grouping()** и **thousands_sep()** определяют формат и для ввода, и для вывода целых чисел. Они также определяют формат целой части чисел с плавающей запятой, но не цифр после **decimal_point()**.

Можно определить новый стиль пунктуации чисел, создавая производный от **num_punct** класс. Например, я могу определить фасет **My_punct** для записи целых значений тройками цифр с пробелом между ними, и для записи чисел с плавающей запятой с использованием запятой в Европейском стиле, отделяющей дробную часть от целой части числа:

```
class My_punct: public std::num_punct<char>
{
public:
    typedef char char_type;
    typedef string string_type;

    explicit My_punct(size_t r = 0) : std::num_punct<char>(r) {}

protected:
    char do_decimal_point() const {return ',';} // запятая
    char do_thousands_sep() const {return ' ';} // пробел
    string do_grouping() const {return "\003";} // 3-цифровые группы
};

void f()
{
    cout << setprecision(4) << fixed;
    cout << "style A: " << 12345678 << " *** " << 1234.5678 << '\n';

    locale loc(locale(), new My_punct);
    cout.imbue(loc);
    cout << "style B: " << 12345678 << " *** " << 1234.5678 << '\n';
}
```

На выходе получим:

```
style A: 12345678 *** 1234.5678
style B: 12 345 678 *** 1 234, 5678
```

Заметьте, что **imbue()** хранит копию своего аргумента в своем потоке. Следовательно, поток может использовать прикрепленную к нему локализацию даже после того, как ее оригинал будет уничтожен. Если **iostream** имеет установленный флаг **boolalpha** (§21.2.2, §21.4.1), то для символьного представления **true** и **false** использу-

ются возвраты функций **truename()** и **falsename()**, соответственно; в противном случае используются **1** и **0**.

Имеется и соответствующая **_byname** версия для **num_punct** (§D.4, §D.4.1):

```
template<class Ch> class std::num_punct_byname: public num_punct<Ch> { /* ... */ };
```

D.4.2.2. Вывод чисел

При записи в буфер потока (§21.6.4) **ostream** использует фасет **num_put**:

```
template<class Ch, class Out = ostreambuf_iterator<Ch> >
class std::num_put: public locale::facet
{
public:
    typedef Ch char_type;
    typedef Out iter_type;

    explicit num_put(size_t r = 0);

    // поместить значение "v" в позицию буфера "b" потока "s":
    Out put(Out b, ios_base& s, Ch fill, bool v) const;
    Out put(Out b, ios_base& s, Ch fill, long v) const;
    Out put(Out b, ios_base& s, Ch fill, unsigned long v) const;
    Out put(Out b, ios_base& s, Ch fill, double v) const;
    Out put(Out b, ios_base& s, Ch fill, long double v) const;
    Out put(Out b, ios_base& s, Ch fill, const void* v) const;

    static locale::id id;           // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~num_put();
    // виртуальные "do_"-функции (см. §D.4.1)
};
```

Аргумент **Out** (итератор вывода; §19.1, §19.2.1) идентифицирует место в буфере потока **ostream** (§21.6.4), в которое **put()** помещает символы, представляющие выводимое числовое значение. Возврат функции **put()** — это итератор, указывающий на позицию, следующую за последним записанным символом.

Отметим, что умолчательная специализация **num_put** (та, чей итератор для доступа к символам имеет тип **ostreambuf_iterator<Ch>**) является частью стандартного контекста локализации (§D.4). Если вам нужна иная специализация, придется создать ее самостоятельно. Например:

```
template<class Ch>
class String_numput: public std::num_put<Ch, typename basic_string<Ch>::iterator>
{
public:
    String_numput() : std::num_put<Ch, typename basic_string<Ch>::iterator>(1) {}
};

void f(int i, string& s, int pos)    // форматирование i в s с позиции pos
{
    String_numput<char> f;
    ios_base& xxx = cout;           // используем правила форматирования потока cout
    f.put(s.begin() + pos, xxx, ' ', i); // форматируем i в s
}
```

Аргумент *ios_base* используется для получения информации о состоянии форматирования и локализации. Например, если требуется осуществить заполнение определенного пространства, используется символ *fill*, как того требует аргумент *ios_base*. Как правило, буфер потока, в который пишут через итератор *b*, является буфером, ассоциированным с *ostream*, для которого *s* является базовым классом. Заметим, что сконструировать объект типа *ios_base* не просто. Он, в частности, управляет множеством аспектов форматирования, которые нужно согласовывать с целью достижения приемлемого вывода. Соответственно, в *ios_base* нет открытого конструктора (§21.3.3).

Функция *put()* также пользуется аргументом *ios_base* для доступа к локализации потока. Она используется для определения пунктуации (§D.4.2.1), символьного представления булевых значений и для преобразований к *Ch*. Например, если *s* является аргументом типа *ios_base* у функции *put()*, то в исходном тексте этой функции можно было бы встретить следующие фрагменты кода:

```
const locale& loc = s.getloc();
// ...
wchar_t w = use_facet<ctype<char>>(loc).widen(c);           // из char в Ch
// ...
string pnt = use_facet<numput<char>>(loc).decimal_point(); // по умолчанию: '.'
// ...
string flse = use_facet<numput<char>>(loc).falsename();       // по умолчанию: "false"
```

Стандартные фасеты, такие как *num_put<char>*, обычно используются неявно в функциях стандартного ввода/вывода; поэтому большинству программистов нет необходимости знать о них. Тем не менее, использование таких фасетов стандартными библиотечными функциями представляет интерес, поскольку иллюстрирует работу потоков ввода/вывода и способы обращения с фасетами. Как всегда, стандартная библиотека демонстрирует примеры интересных программных технологий.

Используя *num_put*, разработчик *ostream* мог бы написать:

```
template<class Ch, class Tr>
ostream& std::basic_ostream<Ch, Tr>::operator<<(double d)
{
    sentry guard(*this); // см. §21.3.8
    if(!guard) return *this;

    try
    {
        if(use_facet<num_put<Ch>>(getloc()).put(*this, *this, this->fill(), d).failed())
            setstate(badbit);
    }
    catch(...)
    {
        handle_ioexception(*this);
    }
    return *this;
}
```

В этом примере много интересного. Класс *sentry* (часовой) обеспечивает гарантию выполнения всех префиксных и постфиксных операций (§21.3.8). Мы получаем контекст локализации потока *ostream* вызовом его функции-члена *getloc()*

(§21.7). Извлекаем *num_put* из *locale* при помощи *use_facet* (§D.3.1). После этого мы обращаемся к соответствующей функции *put*() для выполнения реальной работы. Итератор *ostreambuf_iterator* можно сконструировать из *ostream* (§19.2.6), а *ostream* можно неявно преобразовать к его базовому классу *ios_base* (§21.2.1), чтобы обеспечить два первых аргумента для функции *put*() .

Функция *put*() возвращает свой аргумент, являющийся итератором вывода. Этот итератор вывода извлекается из *basic_ostream*, так что он имеет тип *ostreambuf_iterator*. Следовательно, для проверки корректности состояния и для установки такого состояния нам доступна функция *failed*() (§19.2.6.1).

Я не использую *use_facet*, поскольку гарантируется, что стандартные фасеты (§D.4) присутствуют в любом контексте локализации. Если эта гарантия нарушается, генерируется исключение *bad_cast* (§D.3.1).

Функция *put*() вызывает виртуальную функцию *do_put*() , так что может быть задействован пользовательский код и *operator<<*() должен быть готов к обработке исключения, генерируемого замещающим вариантом *do_put*() . Кроме того, *num_put* может отсутствовать для некоторых символьных типов, так что *use_facet*() может сгенерировать исключение *std::bad_cast* (§D.3.1). Поведение операции << для встроенных типов, таких как *double*, определено стандартом языка C++. Стало быть вопрос не в том, что должна делать функция *handle_ioexception*() , а в том, как она должна действовать в соответствии со стандартом. Если в исключении у *ostream* установлен флаг *badbit* (§21.3.6), то исключение просто повторно генерируется. В противном случае обработка исключения сводится к установке состояния потока и продолжению работы. В обоих случаях нужно установить флаг *badbit* (§21.3.3):

```
template<class Ch, class Tr>
void handle_ioexception (std::basic_ostream<Ch, Tr>& s) // вызывается из catch-блока
{
    if (s.exceptions() & ios_base::badbit)
    {
        try
        {
            s.setstate (ios_base::badbit);           // может генер-вать basic_ios::failure
        }
        catch (...) {}
        throw;                                       // повторная генерация исключения
    }
    s.setstate (ios_base::badbit);
}
```

Здесь *try*-блок необходим, поскольку *setstate*() может сгенерировать исключение *basic_ios::failure* (§21.3.3, §21.3.6). Однако если в исключительном состоянии потока установлен флаг *badbit*, тогда *operator<<*() должен повторно сгенерировать исключение, приведшее к вызову *handle_ioexception*() (а не просто сгенерировать *basic_ios::failure*).

Реализация операции << для встроенных типов, таких как *double*, должна писать непосредственно в буфер потока. Реализуя операцию << для пользовательских типов, мы можем избежать ненужной сложности путем сведения вывода пользовательских типов к выводу уже существующих типов (§D.3.2).

D.4.2.3. Ввод чисел

При чтении из буфера потока (§21.6.4) **istream** полагается на фасет **num_get**:

```
template<class Ch, class In = istreambuf_iterator<Ch> >
class std::num_get: public locale::facet
{
public:
    typedef Ch char_type;
    typedef In iter_type;

    explicit num_get(size_t r = 0);

    // читаем [b:e) в v по правилам форматирования из s; об ошибках - через r:
    In get(In b, In e, ios_base& s, ios_base::iostate& r, bool& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, long& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, unsigned short& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, unsigned int& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, unsigned long& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, float& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, double& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, long double& v) const;
    In get(In b, In e, ios_base& s, ios_base::iostate& r, void*& v) const;

    static locale::id id;    // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~num_get();
    // виртуальные "do_"-функции (см. §D.4.1)
};
```

В основном **num_get()** реализован так же, как и **num_put** (§D.4.2.2). Поскольку **get()** читает, а не пишет, то ей нужна пара итераторов ввода, а аргумент, обозначающий целевой объект для читаемых данных, представлен ссылкой.

Для уведомления о состоянии потока устанавливается переменная **r** типа **iostate**. Если значение желаемого типа прочитать не удастся, то флаг **failbit** в переменной **r** устанавливается; если был достигнут конец ввода, то в **r** устанавливается **eofbit**. Операция ввода использует **r** для того, чтобы решить, как установить состояние своего потока. Если ошибок не обнаружено, то считанное значение присваивается **v**; в противном случае переменная **v** остается неизменной.

Класс **sentry** (часовой) обеспечивает гарантию выполнения всех префиксных и постфиксных операций потока (§21.3.8). В частности, применение **sentry** гарантирует, что мы начинаем читать, лишь если поток находится в надлежащем для чтения состоянии.

Разработчик **istream** может написать следующее:

```
template<class Ch, class Tr>
istream& std::basic_istream<Ch, Tr>::operator>>(double& d)
{
    sentry guard(*this);           // см. §21.3.8
    if(!guard) return *this;

    iostate state = 0;              // все хорошо
    istreambuf_iterator<Ch> eos;
    double dd;
```

```

try
{
    use_facet<num_get<Ch>>(getloc()) .get(*this, eos, *this, state, dd);
    if(state==0 || state==eofbit) d = dd;
    setstate(state);
}
catch(...)
{
    handle_ioexception(*this);    // см. §D.4.2.2
}
return *this;
}

```

В случае ошибки функция `setstate()` сгенерирует исключения, определенные для `istream` (§21.3.6).

Определив `num_punct`, такой как, например, `My_punct` из §D.4.2, мы можем читать с использованием нестандартной пунктуации. Например:

```

void f()
{
    cout << "style A: ";
    int i1;
    double d1;
    cin >> i1 >> d1;                // читаем по стандартному формату "12345678"

    locale loc(locale::classic(), new My_punct);
    cin.imbue(loc);
    cout << "style B: ";
    int i2;
    double d2;
    cin >> i1 >> d2;                // читаем по формату "12 345 678"
}

```

Чтобы читать по-настоящему необычные числовые форматы, нам нужно заместить `do_get()`. Например, мы могли бы определить `num_get` для чтения римских цифр вроде *XXI* или *MM* (§D.6[15]).

D.4.3. Ввод и вывод финансовой информации

Форматирование количества денег технически аналогично форматированию «просто чисел» (§D.4.2). Однако представление денежных величин еще более чувствительно к национальным стандартам и особенностям. Например, отрицательные значения (убыток, дебет), таких как **−1.25**, должно в некоторых контекстах представляться положительными числами, заключенными в круглые скобки: **(1.25)**. Аналогично, в других контекстах могут применять цвет для наглядного отображения отрицательных сумм денег.

Нет никакого стандартного «денежного типа». Вместо этого предполагается, что программист явным образом применит «денежные фасеты» к числовым величинам, про которые он знает, что они соответствуют денежным суммам. Например:

```

class Money
{
    long int amount;

public:
    Money(long int i) : amount(i) {}
    operator long int() const {return amount;}
};
//...
void f(long int i)
{
    cout << "value= " << i << " amount= " << Money(i) << endl;
}

```

Денежные фасеты призваны упростить написание операции вывода для *Money*, которая выводила бы денежные суммы в соответствии с национальными особенностями (соглашениями) (§D.4.3.2). Результат вывода будет зависеть от того, какой *locale* закреплен за *cout*. Вот возможные варианты форматов вывода:

```

value= 1234567 amount= $12345.67
value= 1234567 amount= 12345,67 DKK
value= -1234567 amount= $-12345.67
value= -1234567 amount= -$12345.67
value= -1234567 amount= (CHF12345,67)

```

Для денежных сумм считается важным учет, по возможности, самых малых монет. Поэтому я принял соглашение о представлении целым значением количества центов (пенсов, центов, копеек и т.д.), а не долларов (фунтов, евро, рублей и т.д.). Это соглашение поддерживается посредством функции *frac_digits()* фасета *money_punct* (§D.4.3.1). Вид «десятичной точки» определяется при помощи *decimal_point()*.

Фасеты *money_get* и *money_put* предоставляют функции, которые выполняют ввод/вывод на основе формата, определяемого фасетом *money_base*.

Наш простой тип *Money* может использоваться для управления форматами ввода/вывода или хранения денежных величин. В первом случае мы приводим к типу *Money* иные типы, хранящие денежные суммы, непосредственно перед их записью, и читаем в переменные типа *Money* перед их конвертированием в другие типы. Во втором случае мы исходно храним денежные суммы в переменных типа *Money*, что менее чревато ошибками, ибо нет необходимости выполнять преобразования при записи в потоки, и не надо преобразовывать прочитанные данные к форматам, учитывающим национальные особенности. Но, к сожалению, не всегда можно внедрить тип *Money* глубоко в программы, которые не разрабатывались с этой целью. Остается выполнять преобразования к типу *Money* в операциях ввода/вывода.

D.4.3.1. Пунктуация денежных величин

Фасет *money_punct*, управляющий представлением денежных величин, похож, естественно, на фасет *num_punct*, управляющий представлением «обычных чисел» (§D.4.2.1):

```

class std : money_base
{
public:
    enum part { none, space, symbol, sign, value }; // части раскладки значения
    struct pattern { char field[4] }; // спецификация раскладки
};

template<class Ch, bool International = false>
class std : moneypunct : public locale::facet, public money_base
{
public:
    typedef Ch char_type;
    typedef basic_string<Ch> string_type;
    explicit moneypunct (size_t r = 0) ;

    Ch decimal_point () const; // '.' в classic()
    Ch thousands_sep () const; // ',' в classic()
    string grouping () const; // "" в classic() - означает "нет группировки"

    string_type curr_symbol () const; // "$" в classic()
    string_type positive_sign () const; // "" в classic()
    string_type negative_sign () const; // "-" в classic()

    int frac_digits () const; // число цифр после десятичной точки; 2 в classic()

    pattern pos_format () const; // { symbol, sign, none, value } в classic()
    pattern neg_format () const; // { symbol, sign, none, value } в classic()

    static const bool intl = International; // применяем международ. денеж. форматы
    static locale::id id; // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~moneypunct () ;
    // виртуальные "do_"-функции (см. §D.4.1)
};

```

Средства, которые предлагает фасет **moneypunct**, предназначены, в первую очередь, для разработчиков реализаций фасетов **money_put** и **money_get** (§D.4.3.2, §D.4.3.3).

Функции-члены **decimal_point()**, **thousands_sep()** и **grouping()** ведут себя так же, как их эквиваленты из **numpunct**.

Функции-члены **curr_symbol()**, **positive_sign()** и **negative_sign()** возвращают строку, изображающую валюту (например, **\$**, **FRF**, **DKK** и т.д.), плюсовой знак и минусовой знак, соответственно. Если аргумент **International** шаблона равен **true**, то член **intl** также будет равен **true**, что повлечет за собой использование международных обозначений валютных символов. Такие международные обозначения являются четырехсимвольными строками. Например:

"USD"

"DKK"

"EUR"

Последним символом является терминальный ноль. Стандарт ISO-4217 определяет трехбуквенные идентификаторы валют. Когда **International** равен **false**, можно использовать локальные обозначения валют, вроде **\$** и т.д.

Объект типа *pattern*, возвращаемый функциями *pos_format()* или *neg_format()*, состоит из четырех частей, определяющих последовательность, в которой отображаются числовое значение, валютный символ, символ знака (плюс или минус) и заполнители (пробельные символы). Наиболее общие форматы задаются тривиально с применением этого простого образца (*pattern*) формата. Например:

```
+$123.45      // { sign, symbol, space, value } где positive_sign() возвращает "+"
$+123.45      // { symbol, sign, value, none } где positive_sign() возвращает "+ "
$123.45       // { symbol, sign, value, none } где positive_sign() возвращает ""
$123.45-      // { symbol, value, sign, none }
-123.45 DKK   // { sign, value, space, symbol}
($123.45)     // { sign, symbol, value, none } где negative_sign() возвращает "()"
(123.45DKK)   // { sign, value, symbol, none } где negative_sign() возвращает "()"
```

Представление отрицательных значений с использованием скобок достигается, когда возврат функции *negative_sign()* содержит пару символов `()`. Первый символ этой строки помещается туда, где часть *sign* располагается в образце, а остальная часть строки — после всех остальных частей образца. Наиболее распространенным применением этого средства является принятое в финансовых кругах заключение отрицательных денежных сумм в круглые скобки, но возможны и иные варианты применения. Например:

```
-$123.45      // {sign,symbol,value,none} где negative_sign() возвращает "-"
*$123.45 silly // {sign,symbol,value,none} где negative_sign() возвращает "** silly"
```

Каждое из значений *sign*, *value* и *symbol* должно появиться в образце ровно один раз. Оставшееся значение может быть *space* или *none*. В том месте, где в образце расположен *space*, в отображаемом значении может появиться один или более пробельных символов. А где расположен *none* (но не в конце образца) — там может быть ноль или более пробельных символов.

Обратите внимание на то, что эти строгие правила запрещают такие на первый взгляд разумные образцы, как

```
pattern pat = {sign, value, none, none};    // error: нету symbol
```

Функция *frac_digits()* указывает, где помещается *decimal_point()*. Часто денежные суммы представляются в самых мелких валютных единицах (§D.4.3). Такая единица обычно в сто раз меньше основной валютной единицы (доллара, фунта, рубля и т.д.), и поэтому *frac_digits()* чаще всего возвращает 2.

Приведем пример простого формата, оформленного в виде фасета:

```
class My_money_io : public moneypunct<char, true>
{
public:
    explicit My_money_io (size_t r = 0) : moneypunct<char, true> (r) {}

    char_type do_decimal_point() const {return '.';};
    char_type do_thousands_sep() const {return ',';};
    string do_grouping() const {return "\003\003\003";};

    string_type do_curr_symbol() const {return "USD";};
    string_type do_positive_sign() const {return "";};
    string_type do_negative_sign() const {return "()";};
```

```

int do_frac_digits() const {return 2;} // две цифры после десятичной точки
pattern do_pos_format() const
{
    static pattern pat = {sign, symbol, value, none};
    return pat;
}

pattern do_neg_format() const
{
    static pattern pat = {sign, symbol, value, none};
    return pat;
}
};

```

Этот фасет используется в операциях ввода/вывода *Money*, определенных в §D.4.3.2 и §D.4.3.3.

Для `money_punct` имеется и версия с суффиксом `_byname` (§D.4, §D.4.1):

```

template<class Ch, bool Intl = false>
class std::money_punct_byname: public money_punct<Ch, Intl> { /* ... */ };

```

D.4.3.2. Вывод денежных величин

Фасет `money_put` записывает денежные суммы в соответствии с форматом, заданным `money_punct`. Более точно, `money_put` предоставляет функции `put()`, которые помещают надлежащим образом отформатированное представление символов в буфер потока:

```

template<class Ch, class Out = ostreambuf_iterator<Ch> >
class std::money_put: public locale::facet
{
public:
    typedef Ch char_type;
    typedef Out iter_type;
    typedef basic_string<Ch> string_type;

    explicit money_put(size_t r = 0);

    // поместить значение "v" в позицию буфера "b":
    Out put(Out b, bool intl, ios_base& s, Ch fill, long double v) const;
    Out put(Out b, bool intl, ios_base& s, Ch fill, const string_type& v) const;

    static locale::id id; // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~money_put();
    // виртуальные "do_"-функции (см. §D.4.1)
};

```

Аргументы `b`, `s`, `fill` и `v` используются так же, как в функциях `put()` фасета `num_put` (§D.4.2.2). Аргумент `intl` указывает, используется ли стандартный международный четырехсимвольный знак валюты, или же с этой целью используется какой-нибудь «местный» символ (§D.4.3.1).

Исходя из `money_put`, мы можем определить операцию вывода для *Money* (§D.4.3):

```
ostream& operator<< (ostream& s, Money m)
{
    ostream : : sentry guard (s) ;                // см. §21.3.8
    if (!guard) return s ;

    try
    {
        const money_put<char>& f = use_facet<money_put<char>> (s.getloc () ) ;
        if (m==static_cast<long double> (m) )    // m можно представить как long double
        {
            if ( f.put (s, true, s, s.fill () , m) .failed () ) s.setstate (ios_base : : badbit) ;
        }
        else
        {
            ostringstream v ;
            v << m ;                                // преобразуем в строковое представление
            if ( f.put (s, true, s, s.fill () , v.str () ) .failed () ) s.setstate (ios_base : : badbit) ;
        }
    }
    catch ( . . . )
    {
        handle_ioexception (s) ;                    // см. §D.4.2.2
    }
    return s ;
}
```

Если для точного представления денежных сумм не хватает типа **long double**, я преобразовываю значение в его строковое представление и вывожу это функцией **put()**, принимающей строковый аргумент.

D.4.3.3. Ввод денежных величин

Фасет **money_get** читает денежные суммы в соответствии с форматом, заданным с помощью **moneyupunct**. Более точно, **money_get** предоставляет функции **get()**, которые извлекают надлежащим образом отформатированное представление символов из буфера потока:

```
template<class Ch, class In = istreambuf_iterator<Ch> >
class std : : money_get : public locale : : facet
{
public :
    typedef Ch char_type ;
    typedef In iter_type ;
    typedef basic_string<Ch> string_type ;

    explicit money_get (size_t r = 0) ;

    // читаем [b:e) в v по форматным правилам из s; ошибки - устанавливаем r:
    In get (In b, In e, bool intl, ios_base& s, ios_base : : iostate& r, long double& v) const ;
    In get (In b, In e, bool intl, ios_base& s, ios_base : : iostate& r, string_type& v) const ;

    static locale : : id id ;    // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected :
    ~money_get () ;
    // виртуальные "do_"-функции (см. §D.4.1)
    ,
```

Аргументы *b*, *e*, *s*, *fill* и *v* используются так же, как в функциях *get()* фасета *num_get* (§D.4.2.3). Аргумент *intl* указывает, используется ли стандартный международный четырехсимвольный знак валюты, или же с этой целью используется какой-нибудь «местный» символ (§D.4.3.1).

Правильно определенная пара фасетов *money_get* и *money_put* обеспечит вывод в форме, которая может быть считана обратно без ошибок и потери информации. Например:

```
int main ()
{
    Money m;
    while (cin >> m) cout << m << "\n";
}
```

Вывод этой программы может быть принят ею в качестве ввода. Более того, вывод, полученный в результате второго запуска этой программы с вводом, равным выводу первого запуска, должен быть идентичен исходному вводу.

Операция ввода для *Money* может выглядеть следующим образом:

```
istream& operator>> (istream& s, Money& m)
{
    istream::sentry guard(s); // см. §21.3.8
    if (guard)
        try
        {
            ios_base::iostate state = 0; // все хорошо
            istreambuf_iterator<char> eos;
            string str;

            use_facet<money_get<char>>>(s.getloc()).get(s, eos, true, state, str);
            if (state == 0 || state == ios_base::eofbit)
            {
                long int i = strtol(str.c_str(), 0, 0); // npo strtol() см. §20.4.1
                if (errno == ERANGE)
                    state |= ios_base::failbit;
                else
                    m = i;
                s.setstate(state);
            }
        }
    catch (...)
    {
        handle_ioexception(s); // см. §D.4.2.2
    }
    return s;
}
```

Здесь использована функция *get()*, которая читает в строковую переменную, поскольку чтение в переменную типа *double* с последующим преобразованием в *long int* может привести к потере точности.

D.4.4. Ввод и вывод дат и времени

К сожалению, стандартная библиотека C++ не предоставляет удовлетворительного типа для даты. Из стандартной библиотеки языка C она получает по наследству низкоуровневые средства работы с датами и интервалами времени. Эти средства языка C являются фундаментом для средств C++, позволяющим работать со временем удобным образом, не зависящим от системных особенностей.

В следующих разделах показывается, как представление дат и времени дня можно сделать чувствительными к контексту локализации. Приводится пример того, как пользовательский тип *Date* может быть вписан в архитектуру потоков (глава 21) и контекстов локализации (§D.2). Реализация *Date* демонстрирует технологии, полезные при обращении со временем в том случае, если у вас нет типа *Date*.

D.4.4.1. Часы и таймеры

На самом низком уровне большинство систем имеет таймер, отсчитывающий малые интервалы времени. Стандартная библиотека предоставляет функцию *clock()*, которая возвращает значение зависящего от реализации арифметического типа *clock_t*. Возврат функции *clock()* может быть откалиброван макросом *CLOCKS_PER_SEC*. Если у вас нет доступа к надежной утилите измерения времени, можно написать свою собственную измерительную программу:

```
int main (int argc, char* argv[]) // §6.1.7
{
    int n = atoi(argv[1]); // §20.4.1
    clock_t t1 = clock();

    if(t1 == clock_t(-1)) // clock_t(-1) означает, что clock() не работает
    {
        cerr << "sorry, no clock\n";
        exit(1);
    }

    for(int i = 0; i < n; i++) do_something(); // измерительный цикл
    clock_t t2 = clock();
    if(t2 == clock_t(-1))
    {
        cerr << "sorry, clock overflow\n";
        exit(2);
    }

    cout << "do_something() "<<n<<" times took "
         << double(t2-t1) / CLOCKS_PER_SEC << " seconds"
         << " (measurement granularity: " << CLOCKS_PER_SEC << " of a second)\n";
}
```

Явное преобразование *double(t2-t1)* перед операцией деления необходимо, поскольку значение *clock_t* может быть целым. Точное значение, когда начинает работать функция *clock()*, зависит от реализации; предполагается, что *clock()* измеряет интервалы времени в течение одного запуска программы. Для величин *t1* и *t2*, возвращаемых функцией *clock()*, выражение *double(t2-t1) / CLOCKS_PER_SEC* является наилучшей оценкой в секундах для времени между двумя вызовами.

Если для конкретного процессора таймер не реализован, или если интервал времени слишком велик для измерения, то **clock()** возвращает значение **clock_t(-1)**.

Функция **clock()** предназначена для измерения интервалов времени длительно-стью от долей секунды до нескольких секунд. Например, если тип **clock_t** есть 32-битовый знаковый **int** и **CLOCKS_PER_SEC** равно 1000000, то можно использовать **clock()** для измерения в микросекундах интервалов от 0 до 2000 секунд (порядка получаса).

Обратите внимание, что получение достоверных результатов измерения времени работы программы может оказаться нетривиальным делом. Другие исполняемые на машине программы могут влиять на это время; кэш и взаимодействие между процессами являются труднопрогнозируемыми, да и алгоритмы обработки могут зависеть от данных неожиданным образом. Если вы уж пытаетесь замерить что-либо, то хотя бы делайте это несколько раз и отбрасывайте сильно отличающиеся результаты как недостоверные.

Для работы с более продолжительными интервалами и календарным временем, стандартная библиотека предоставляет тип **time_t** для представления момента времени и структуру **tm** для “препарирования” этого момента:

```
typedef implementation_defined time_t; // зависящий от реализации арифметический тип
// (§4.1.1), чаще всего это 32-bit целое
```

```
struct tm
{
    int tm_sec;      // секунда минуты [0,61]; 60 и 61 - корректировочные секунды
    int tm_min;     // минута часа [0,59]
    int tm_hour;    // час дня [0,23]
    int tm_mday;    // день месяца [1,31]
    int tm_mon;     // месяц года [0,11]; 0 значит January
    int tm_year;    // год после 1900; 0 значит 1900, а 102 значит 2002
    int tm_wday;    // день после Sunday [0,6]; 0 значит Sunday
    int tm_yday;    // день после January 1 [0,365]; 0 значит January 1
    int tm_isdst;   // часы летнего времени
};
```

Отметим, что стандарт гарантирует содержание в **tm** указанных здесь целочисленных полей. Стандарт не гарантирует, что они расположены именно в указанном порядке, или же что нет других полей.

Типы **time_t** и **tm** вместе со стандартными средствами их использования представлены в **<ctime>** и **<time.h>**. К примеру:

```
clock_t clock();           // кол-во "тиков" с момента начала работы программы

time_t time(time_t* pt);   // текущее календарное время
double difftime(time_t t2, time_t t1); // t2-t1 в секундах

tm* localtime(const time_t* pt); // местное время для *pt
tm* gmtime(const time_t* pt);    // GMT (Greenwich Mean Time) для *pt, или 0
// (официально называется Coordinated Universal Time, UTC)

time_t mktime(tm* ptm);       // time_t для *ptm, или time_t(-1)
char* asctime(const tm* ptm);  // представление *ptm в стиле C-строки
// например, "Sun Sep 16 01:03:52 1973\n"

char* ctime(const time_t* t) {return asctime(localtime(t));}
```

Будьте осторожны — как *localtime()*, так и *gmtime()* возвращают указатель *tm** на статически размещенный объект, так что последовательные вызовы функции изменяют значение этого объекта. Так что, либо немедленно используйте возвращаемое значение, либо скопируйте *tm* в управляемую вами область памяти. Аналогично, *asctime()* возвращает указатель на статически размещенный массив символов.

Структура *tm* позволяет представить даты в диапазоне по крайней мере десятков тысяч лет (около [-32000,32000] для минимального размера типа *int*). В то же время, *time_t* — это чаще всего 32-битовое *long int*, что позволяет типу *time_t* представлять в секундах диапазон длиной в 68 лет в обе стороны от базового года. Базовым годом обычно является год 1970; точным базовым временем при этом является 0:00, 1 января по Гринвичу. Если же *time_t* — это 32-битное *int*, то уже к 2038 году мы окажемся «вне времени», если, конечно, не перейдем на более длинные *int*, как это уже сделано на некоторых системах.

Тип *time_t* предназначен для работы с относительно близкими датами, так что нельзя гарантировать его работу вне диапазона [1902,2038]. Еще хуже то, что не все реализации функций для работы со временем одинаково работают с отрицательными значениями. Для переносимости требуется, чтобы значение, которое нужно представлять и как *time_t*, и как *tm*, лежало в диапазоне [1902,2038]. Для представления дат вне этого диапазона нужен какой-либо иной механизм.

Одним из следствий вышесказанного является тот факт, что функция *mktime()* может отработать некорректно. Для аргументов, которые не представимы типом *time_t*, она возвращает индикатор ошибки *time_t(-1)*.

Хронометраж в программах с большим временем работы можно выполнить следующим образом:

```
int main (int argc, char* argv[])    // §6.1.7
{
    time_t t1 = time (0) ;
    do_a_lot (argc, argv) ;
    time_t t2 = time (0) ;
    double d = difftime (t2, t1) ;
    cout << "do_a_lot() took" << d << " seconds\n" ;
}
```

Если аргумент-указатель функции *time()* не 0, то результирующее время записывается и по этому указателю. Если календарное время не реализовано (например, на специализированных процессорах), то возвращается значение *time_t(-1)*. Можно попытаться определить текущую дату следующим образом:

```
int main ()
{
    time_t t ;
    if (time (&t) == time_t (-1))    // time_t(-1) означает, что time() не работает
    {
        cerr << "Bad time\n" ;
        exit (1) ;
    }

    tm* gt = gmtime (&t) ;
    cout << gt->tm_mon+1 << '/' << gt->tm_mday << '/' << 1900+gt->tm_year<<endl ;
}
```

D.4.4.2. Класс *Date*

Как упоминалось в §10.3, маловероятно, чтобы единственный тип *Date* мог удовлетворить все потребности. Различные способы использования дат диктуют разные способы их представления, а календарная информация до 19 столетия сильно зависит от капризов истории. Тем не менее, в качестве примера мы можем определить тип *Date* примерно как в §10.3, используя для реализации *time_t*:

```
class Date
{
public:
    enum Month {jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};

    class Bad_date {};

    Date(int dd, Month mm, int yy);
    Date();

    friend ostream& operator<<(ostream& s, const Date& d);
    // ...

private:
    time_t d; // стандартное представление даты и времени
};

Date::Date(int dd, Month mm, int yy)
{
    tm x = {0};
    if(dd<0 || 31<dd) throw Bad_date(); // переупрощено: см. §10.3.1
    x.tm_mday = dd;
    if(mm<jan || dec<mm) throw Bad_date();
    x.tm_mon = mm-1; // tm_mon отсчитывается с нуля
    x.tm_year = yy-1900; // tm_year отсчитывается от 1900
    d = mktime(&x);
}

Date::Date()
{
    d = time(0); // умолчательный Date: сегодня
    if(d == time_t(-1)) throw Bad_date();
}
```

Теперь перед нами встает задача определить операции << и >> для класса *Date*, которые были бы чувствительны к локализации.

D.4.4.3. Вывод дат и времени

Как и *num_put* (§D.4.2), фасет *time_put* предоставляет функции *put()* для записи в буферы через итераторы:

```
template<class Ch, class Out = ostreambuf_iterator<Ch> >
class std::time_put: public locale::facet
{
public:
    typedef Ch char_type;
    typedef Out iter_type;
```

```
explicit time_put (size_t r = 0) ;
```

```
// помещаем t в буфер потока s через b по формату fmt:
```

```
Out put (Out b, ios_base& s, Ch fill, const tm* t,
```

```
const Ch* fmt_b, const Ch* fmt_e) const;
```

```
Out put (Out b, ios_base& s, Ch fill, const tm* t, char fmt, char mod = 0) const  
{ return do_put (b, s, fill, t, fmt, mod) ; }
```

```
static locale : : id id; // объект идентификации facets (§D.2, §D.3, §D.3.1)
```

```
protected:
```

```
~time_put () ;
```

```
virtual Out do_put (Out, ios_base&, Ch, const tm*, char, char) const;
```

```
};
```

Вызов `put (b, s, fill, t, fmt_b, fmt_e)` помещает информацию о дате из `t` в буфер потока `s` через `b`. Символ `fill` используется в качестве заполняющего. Формат вывода специфицируется `printf()`-подобной строкой `[fmt_b, fmt_e]`. Этот `printf()`-подобный формат (§21.8) используется для порождения настоящего вывода и может содержать следующие форматные спецификаторы

- `%a` сокращение для дня недели (например, Sat)
- `%A` полное название дня недели (например, Saturday)
- `%b` сокращенное название месяца (например, Feb)
- `%B` полное название месяца (например, February)
- `%c` дата и время (например, Sat Feb 06 21:46:05 1999)
- `%d` число [01,31] (например, 06)
- `%H` 24-часовой формат времени [00,23] (например, 21)
- `%I` 12-часовой формат времени [01,12] (например, 09)
- `%j` день года [001,366] (например, 037)
- `%m` месяц [01,12] (например, 02)
- `%M` минута часа [00,59] (например, 48)
- `%p` а.м./п.м индикатор для 12-часового формата (например, PM)
- `%S` секунда в минуте [00,61] (например, 40)
- `%U` неделя года [00,53] начиная с воскресенья (например, 05)
- `%w` день недели [0,6], где 0 это воскресенье (например, 6)
- `%W` неделя года [00,53] с понедельника (например, 05)
- `%x` дата (например, 02/06/99)
- `%X` время (например, 21:48:40)
- `%y` год без века [00,99] (например, 99)
- `%Y` год (например, 1999)
- `%Z` индикатор часового пояса (например, EST), если он известен

Этот длинный список форматных спецификаторов может послужить наглядным аргументом в пользу расширяемой системы ввода/вывода. Однако как и всякая специализированная система обозначений она точна и даже, до некоторой степени, удобна.

Кроме перечисленных директив форматирования многие реализации поддерживают «модификаторы», такие как, например, целые модификаторы ширины поля вывода (§21.8), `%10X`. Модификаторы форматов дат и времени не являются частью стандарта C++, в то время как стандарты некоторых платформ, например POSIX, требуют их. Поэтому избежать применения модификаторов трудно, несмотря на то, что их применение снижает переносимость программы.

Функция `strftime()` из `<ctime>` или `<time.h>`, подобная функции `sprintf()` (§21.8), производит вывод в соответствии с директивами форматирования дат и времени:

```
size_t strftime(char* s, size_t max, const char* format, const tm* tmp);
```

Эта функция помещает максимум *max* символов из **tmp* в **s* согласно формату *format*. Например:

```
int main()
{
    const int max = 20;    // надеемся, что strftime() не произведет более 20 символов
    char buf[max];

    time_t t = time(0);
    strftime(buf, max, "%A\n", localtime(&t));
    cout << buf;
}
```

Для дня недели «среда» эта программа выведет **Wednesday** в контексте локализации «по умолчанию» `classic()` (§D.2.3), и **onsdag** для датской локализации.

Символы, не являющиеся частью формата, такие как «перевод строки» в данном примере, просто копируются в **s*.

Когда `put()` обнаруживает символ форматирования *f* (и необязательный модификатор *m*), то для выполнения форматирования она вызывает виртуальную функцию `do_put()` с соответствующими параметрами: `do_put(b, s, fill, t, f, m)`.

Вызов `put(b, s, fill, t, f, m)` является упрощенным вариантом `put()`, в котором символ форматирования *f* и модификатор *m* задаются явным образом. Таким образом, фрагмент

```
const char fmt[] = "%10X";
put(b, s, fill, t, fmt, fmt+sizeof(fmt));
```

можно сократить до

```
put(b, s, fill, t, 'X', 10);
```

Если формат содержит мультибайтные символы, он должен и начинаться, и заканчиваться в состоянии «по умолчанию» (§D.4.6).

Мы можем применить функцию `put()`, чтобы реализовать зависящую от контекста локализации операцию вывода для типа **Date**:

```
ostream& operator<<(ostream& s, const Date& d)
{
    ostream::sentry guard(s);    // см. §21.3.8
    if(!guard) return s;

    tm* tmp = localtime(&d.d);
    try
```

```

{
    if (use_facet<time_put<char>> (s.getloc()) .put(s,s,s.fill(),tmp, 'x') .failed())
        s.setstate (ios_base::failbit);
}
catch (...)
{
    handle_ioexception(s);          // см. §D.4.2.2
}
return s;
}

```

Поскольку никакого стандартного типа **Date** не существует, нет и стандартного формата «по умолчанию» для ввода/вывода дат. Здесь я задал формат `%x`, передавая символ `'x'` в качестве символа форматирования. Поскольку формат `%x` является форматом «по умолчанию» для `get_time()` (§D.4.4.4), то он является по сути дела «наиболее стандартным».

Имеется и версия `time_put` с суффиксом `_byname` (§D.4, §D.4.1):

```

template<class Ch, class Out = ostreambuf_iterator<Ch> >
class std::time_put_byname: public time_put<Ch, Out> { /* ... */ };

```

D.4.4.4. Ввод дат и времени

Как всегда, ввод несколько сложнее вывода. Когда мы пишем код для вывода значения, у нас обычно есть выбор между несколькими разными форматами. А когда мы пишем код для ввода, мы должны в первую очередь следить за ошибками и, лишь иногда, выбирать из нескольких форматных альтернатив.

Фасет `time_get` реализует ввод дат и времени. Идея заключается в том, что фасет `time_get` может читать форматы времени и дат, которые в данной локализации порождает фасет `time_put`. Ввиду отсутствия стандартных классов времени и дат, программист в рамках контекста локализации может осуществлять вывод согласно множеству форматов. Например, все приведенные ниже представления могут быть получены с помощью одной и той же операции вывода, использующей фасет `time_put` (§D.4.4.5) разных локализаций:

```

January 15th 1999
Thursday 15th January 1999
15 Jan 1999AD
Thurs 15/1/99

```

Стандарт C++ поощряет разработчиков `time_get` воспринимать форматы дат и времени в соответствии с POSIX и иными распространенными промышленными стандартами. Главная трудность состоит в том, что почти невозможно стандартизировать намерение читать даты и время во всех мыслимых форматах, принятых в рамках разнообразных местных и национальных (шире — культурных) соглашений. Можно поэкспериментировать, чтобы поточнее познакомиться с возможностями конкретного контекста локализации (§D.6[8]). Если формат не воспринимается, программист может реализовать необходимый ему альтернативный фасет `time_get` сам.

Стандартный фасет для ввода времени, `time_get`, наследуется от класса `time_base`:

```

struct std::time_base
{
    enum dateorder
    {
        no_order,           // не упорядочено
        dmy,                // день-месяц-год
        mdy,                // месяц-день-год
        ymd,                // год-месяц-день
        ydm                 // год-день-месяц
    };
};

```

Разработчик может воспользоваться этим перечислением для упрощения анализа форматов даты.

Подобно *num_get*, фасет *time_get* осуществляет доступ к своему буферу посредством пары итераторов ввода:

```

template<class Ch, class In = istreambuf_iterator<Ch> >
class time_get: public locale::facet, public time_base
{
public:
    typedef Ch char_type;
    typedef In iter_type;

    explicit time_get(size_t r = 0);

    dateorder date_order() const { return do_date_order(); }

    // читаем [b,e) в d по форматным правилам из s; ошибки - установка r:
    In get_time(In b, In e, ios_base& s, ios_base::iostate& r, tm* d) const;
    In get_date(In b, In e, ios_base& s, ios_base::iostate& r, tm* d) const;
    In get_year(In b, In e, ios_base& s, ios_base::iostate& r, tm* d) const;

    In get_weekday(In b, In e, ios_base& s, ios_base::iostate& r, tm* d) const;
    In get_monthname(In b, In e, ios_base& s, ios_base::iostate& r, tm* d) const;

    static locale::id id;    // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~time_get();
    // виртуальные "do_"-функции (см. §D.4.1)
};

```

Функция *get_time()* вызывает *do_get_time()*. По умолчанию *get_time()* читает время так, как оно выводится функцией *time_put::put()* в рамках контекста локализации, используя формат *%x* (§D.4.4).

Таким образом, простейший вариант операции ввода для типа *Date* выглядит примерно так:

```

istream& operator>>(istream& s, Date& d)
{
    istream::sentry guard(s);    // см. §21.3.8
    if(!guard) return s;

    ios_base::iostate res = 0;
    tm x = {0};
    istreambuf_iterator<char, char_traits<char> > end;

```

```

try
{
    use_facet<time_get<char>> > (s.getloc()) .get_date(s, end, s, res, &x);
    if(res==0 || res==ios_base::eofbit)
        d = Date(x.tm_mday, Date::Month(x.tm_mon+1), x.tm_year+1900);
    else
        s.setstate(res);
}
catch(...)
{
    handle_ioexception(s);    // см. §D.4.2.2
}
return s;
}

```

Вызов `get_date(s, end, s, res, &x)` полагается на два неявных приведения типа от типа *istream*: первый аргумент *s* используется для создания *istreambuf_iterator*; третий аргумент *s* конвертируется к типу *ios_base* (базовый класс для *istream*).

Эта операция ввода будет работать корректно для дат из диапазона значений, представимых с помощью *time_t*. Вот несложный тестировочный фрагмент:

```

int main()
try
{
    Date today;
    cout << today << endl;           // пишем в формате %x
    Date d(12, Date::may, 1998);

    cout << d << endl;
    Date dd;
    while(cin >> dd) cout << dd << endl;    // читаем даты в формате %x
}
catch(Date::Bad date)
{
    cout << "exit: bad date caught\n";
}

```

Имеется и версия *time_get* с суффиксом *_byname* (§D.4, §D.4.1):

```

template<class Ch, class In = istreambuf_iterator<Ch>>
class std::time_get_byname: public time_get<Ch, In> { /* ... */ };

```

D.4.4.5. Более гибкий класс Date

Если вы попытаетесь использовать класс *Date* из §D.4.4.2 с вводом/выводом из §D.4.4.3 и §D.4.4.4, то вы обнаружите его ограниченность:

1. Он работает только с датами, представимыми с помощью типа *time_t*, что в типичном случае означает диапазон [1970,2038].
2. Он воспринимает даты только в стандартном формате — каков бы он ни был.
3. Его реакция на ошибки ввода неприемлема.
4. Он поддерживает лишь потоки *char* (а не произвольных символьных типов).

Более интересная и полезная операция ввода должна воспринимать более широкий диапазон дат, распознавать несколько распространенных форматов и надежным образом реагировать на ошибки ввода. Для этого нужно уйти от использования типа `time_t`:

```
class Date
{
public:
    enum Month {jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};

    struct Bad_date
    {
        const char* why;
        Bad_date(const char* p) : why(p) {}
    };

    Date(int dd, Month mm, int yy, int day_of_week = 0);
    Date();

    void make_tm(tm* t) const;           // поместить tm представление Date в *t
    time_t make_time_t() const;         // возврат time_t представления Date

    int year() const {return y;}
    Month month() const {return m;}
    int day() const {return d;}
    // ...

private:
    char d;
    Month m;
    int y;
};
```

Для простоты я вернулся к представлению (d, m, y) (§10.2).

Конструктор можно определить следующим образом:

```
Date::Date(int dd, Month mm, int yy, int day_of_week)
    : d(dd), m(mm), y(yy)
{
    if(d==0 && m==Month(0) && y==0) return;           // Date(0,0,0) - "нулевая дата"
    if(mm<jan || dec<mm) throw Bad_date("bad month");

    if(dd<1 || 31<dd)                                  // переупрощено; см. §10.3.1
        throw Bad_date("bad day of month");

    if(day_of_week && day_in_week(yy, mm, dd) != day_of_week)
        throw Bad_date("bad day of week");
}

Date::Date() : d(0), m(0), y(0) {}                    // "нулевая дата"
```

Алгоритм `day_in_week()` весьма нетривиален и не имеет непосредственного отношения к локализациям, так что я отставил его в сторону. Если он вам потребуется, поищите его в рамках вашей системы.

Операции сравнения всегда полезны для типов вроде `Date`:

```
bool operator==(const Date& x, const Date& y)
{
    return x.year() == y.year() && x.month() == y.month() && x.day() == y.day();
}
```

```
bool operator!=(const Date& x, const Date& y)
{
    return !(x==y);
}
```

Поскольку мы расстались со стандартными типами *tm* и *time_t*, нам нужны будут операции преобразования к этим типам для обеспечения взаимодействия с программами, ожидающими эти типы:

```
void Date::make_tm(tm* p) const
{
    tm x = {0};
    *p = x;
    p->tm_year = y-1900;
    p->tm_mday = d;
    p->tm_mon = m-1;
}

time_t Date::make_time_t() const
{
    if(y<1970 || 2038<y) // переупрощено
        throw Bad_date("date out of range for time_t");

    tm x;
    make_tm(&x);
    return mktime(&x);
}
```

D.4.4.6. Задание формата даты

Язык C++ не определяет стандартного формата вывода дат (%x является всего лишь наиболее вероятным претендентом; §D.4.4.3). Однако даже если бы и существовал стандартный формат, нам все равно потребовались бы альтернативы. Это можно реализовать, предоставив формат по умолчанию и способ его замены. Например:

```
class Date_format
{
    static char fmt[]; // умолчательный формат
    const char* curr; // текущий формат
    const char* curr_end;

public:
    Date_format() : curr(fmt), curr_end(fmt+strlen(fmt)) {}
    const char* begin() const {return curr;}
    const char* end() const {return curr_end;}

    void set(const char* p, const char* q) {curr=p; curr_end=q;}
    void set(const char* p) {curr=p; curr_end=curr+strlen(p);}

    static const char* default_fmt() {return fmt;}
};
```

```
const char Date_format::fmt[] = "%A, %B%d, %Y"; //например, Friday, February 5 1999
Date_format date_fmt;
```

Чтобы иметь возможность использовать формат *strftime()* (§D.4.4.3), я воздержался от параметризации класса *Date_format* по типу используемых символов. Таким образом, данное решение допускает только те форматы дат, которые представимы с помощью *char[]*. Кроме того, я воспользовался глобальным объектом форматирования *date_fmt* для представления формата *Date* по умолчанию. Поскольку значение *date_fmt* можно изменять, то мы получаем грубый способ управления форматом *Date*, аналогичный применению *global()* (§D.2.3).

Более общим решением было бы определение фасетов *Date_in* и *Date_out* для управления чтением и записью в потоки. Этот подход представлен в §D.4.4.7.

Располагая *Date_format*, можно написать операцию *Date::operator<<()* следующим образом:

```
template<class Ch, class Tr>
basic_ostream<Ch, Tr>& operator<< (basic_ostream<Ch, Tr>& s, const Date& d)
// запись согласно пользовательскому формату
{
    typename basic_ostream<Ch, Tr>::sentry guard(s); // см. §21.3.8
    if(!guard) return s;

    tm t;
    d.make_tm(&t);

    try
    {
        const time_put<Ch>& f = use_facet<time_put<Ch>>(s.getloc());
        if(f.put(s, s, s.fill(), &t, date_fmt.begin(), date_fmt.end()).failed())
            s.setstate(ios_base::failbit);
    }
    catch(...)
    {
        handle_ioexception(s); // см. §D.4.2.2
    }
    return s;
}
```

Я мог бы воспользоваться *use_facet* для проверки того, что фасет *time_put<Ch>* имеется в контексте локализации потока *s*. Однако в данном случае проще выглядит вариант с перехватом исключения, генерируемого из *use_facet*.

Приведем простую тестовую программу, управляющую форматом вывода с помощью *date_fmt*:

```
int main()
try
{
    while(cin >> dd && dd != Date()) cout << dd << endl; // использ-ем умолчат. date_fmt
    date_fmt.set("%Y/%m/%d");
    while(cin >> dd && dd != Date()) cout << dd << endl; // используем "%Y/%m/%d"
}
catch(Date::Bad_date e)
```

```
cout << "bad date caught: " << e.why << endl;
}
```

D.4.4.7. Фасет ввода даты

Как всегда, ввод немного сложнее вывода. Но поскольку интерфейс доступа к низкоуровневому вводу ограничен функцией *get_date()* и поскольку операция ввода *operator>>()*, определенная для *Date* в §D.4.4.4, не имеет прямого доступа к представлению *Date*, мы можем воспользоваться этой операцией без изменений. Вот параметризованная версия операции ввода, соответствующая операции вывода *operator<<()* из §D.4.4.6:

```
template<class Ch, class Tr>
istream<Ch, Tr>& operator>> (istream<Ch, Tr>& s, Date& d)
{
    typename istream<Ch, Tr>::sentry guard(s);
    if(guard)
        try
        {
            ios_base::iostate res = 0;
            tm x = {0};
            istreambuf_iterator<Ch, Tr> end;

            use_facet<time_get<Ch>>(s.getloc()).get_date(s, end, s, res, &x);

            if(res==0 || res==ios_base::eofbit)
                d =
                Date(x.tm_mday, Date::Month(x.tm_mon+1), x.tm_year+1900, x.tm_wday);
            else
                s.setstate(res);
        }
        catch(...)
        {
            handle_ioexception(s); // см. §D.4.2.2
        }
    return s;
}
```

Эта операция ввода для типа *Date* вызывает функцию *get_date()* из фасета *time_get* (§D.4.4.4) для потока *istream*. Поэтому можно обеспечить иную и более гибкую форму ввода, определив новый фасет, производный от *time_get*:

```
template<class Ch, class In = istreambuf_iterator<Ch>>
class Date_in: public std::time_get<Ch, In>
{
public:
    Date_in(size_t r = 0): std::time_get<Ch>(r) {}

protected:
    In do_get_date(In b, In e, ios_base& s, ios_base::iostate& r, tm* tmp) const;

private:
    enum Vtype {novalue, unknown, dayofweek, month};
    In getval(In b, In e, ios_base& s, ios_base::iostate& r, int* v, Vtype* res) const;
```

Функция **getval()** должна прочесть год, месяц и день (месяца), а также, возможно, и день недели, после чего составить из этого **tm**.

Названия месяцев и дней недели зависят от конкретного контекста локализации. Следовательно, мы не можем упоминать их непосредственно в нашей функции ввода. Вместо этого мы будем распознавать месяцы и дни посредством вызова функций, которые фасет **time_get** предоставляет для этой цели: **get_monthname()** и **get_weekday()** (§D.4.4.4).

Год, день месяца и, возможно, месяц представляются целыми числами. К сожалению, само число не указывает, относится ли оно к месяцу, дню или к чему-либо еще. Например, 7 может означать июль месяц, или 7-е число, или даже 2007 год. Настоящей целью функции **date_order()** из фасета **time_get** является разрешение таких двусмысленностей.

Стратегия фасета **Date_in** состоит в том, чтобы читать значения, классифицировать их и вызывать **date_order()**, чтобы узнать, имеют ли смысл (и какой) введенные значения. Закрытая функция **getval()** осуществляет непосредственное чтение буфера потока ввода и начальную классификацию:

```
template<class Ch, class In>
In Date_in<Ch, In>::getval(In b, In e, ios_base& s, ios_base::iostate& r,
                           int* v, Vtype* res) const
// Читаем части Date: число, день недели, или месяц.
// Пропускаем пробелы и пунктуацию.
{
    const ctype<Ch>& ct = use_facet<ctype<Ch>>(s.getloc()); // ctype - см. §D.4.5
    Ch c;

    *res = novalue; // значение не найдено

    for(;;) // пропускаем пробелы и пунктуацию
    {
        if(b == e) return e;
        c = *b;
        if(! (ct.is(ctype_base::space, c) || ct.is(ctype_base::punct, c))) break;
        ++b;
    }

    if(ct.is(ctype_base::digit, c)) // читаем целое, не обращая внимание на mpunct
    {
        int i = 0;

        do // переводим цифру из произвольного символьного типа в десятичное значение:
        {
            static char const digits[] = "0123456789";
            i = i*10 + find(digits, digits+10, ct.narrow(c, ' ')) - digits;
            c = ++b;
        }
        while(ct.is(ctype_base::digit, c));

        *v = i;
        *res = unknown; // целое значение, но неизвестно, чему оно соответствует
        return b;
    }
}
```

```

if( ct.is( ctype_base::alpha, c ) ) // поиск названия месяца или дня недели
{
    basic_string<Ch> str;
    while( ct.is( ctype_base::alpha, c ) ) // читаем символы в строку
    {
        str += c;
        if( ++b == e ) break;
        c = *b;
    }

    tm t;
    basic_stringstream<Ch> ss( str );
    typedef istreambuf_iterator<Ch> SI; // итераторный тип для буфера ss
    get_monthname( ss.rdbuf(), SI(), s, r, &t ); // читаем из буфера потока в памяти
    if( ( r & ( ios_base::badbit | ios_base::failbit ) ) == 0 )
    {
        *v = t.tm_mon;
        *res = month;
        return b;
    }

    r = 0; // очистка состояния перед повторным чтением
    get_weekday( ss.rdbuf(), SI(), s, r, &t ); // читаем из буфера потока в памяти
    if( ( r & ( ios_base::badbit | ios_base::failbit ) ) == 0 )
    {
        *v = t.tm_wday;
        *res = dayofweek;
        return b;
    }
}

r |= ios_base::failbit;
return b;
}

```

Главный трюк здесь заключается в том, чтобы отличить месяцы от дней недели. Мы читаем с помощью итераторов ввода, так что мы не можем дважды прочитать интервал `[b, e)`, сначала отыскивая месяц, а затем день недели. С другой стороны, мы не можем принять решение, глядя лишь на один символ, потому что только функции `get_monthname()` и `get_weekday()` знают, какая последовательность символов означает название месяца и дня недели в данной локализации. Я принял решение читать строки алфавитных символов в тип `string`, создать поток `stringstream` из `string`, после чего вычитывать уже `streambuf` этого потока.

Обработка ошибок состоит в непосредственном установлении битов состояния, таких как `ios_base::badbit`. Это необходимо из-за того, что более удобные функции манипулирования состоянием потока, такие как `clear()` и `set_state()`, определены в `basic_ios`, а не в его базовом классе `ios_base` (§21.3.3). При необходимости операция `>>` использует информацию об ошибках, получаемую от `get_date()`, для переустановки состояния потока ввода.

Располагая функцией `getval()`, мы можем сначала прочитать значения, и лишь после этого проверять, имеют ли они смысл. Функция `date_order()` критически важна:

```

template<class Ch, class In>
In Date_in<Ch, In>::do_get_date(In b, In e, ios_base& s, ios_base::iostate& r,
                                tm* tmp) const
// необязательный день недели, за которым идут ymd, dmy, mdy, или ydm
{
    int val[3]; // для day, month и year в некотором порядке
    Vtype res[3] = {novalue}; // для классификации значений

    for (int i=0; b!=e && i<3; ++i) // читаем day, month и year
    {
        b = getval(b, e, s, r, &val[i], &res[i]);
        if(r) return b; // oops: error
        if(res[i]==novalue) // незавершенная дата
        {
            r |= ios_base::badbit;
            return b;
        }
        if(res[i]==dayofweek)
        {
            tmp->tm_wday = val[i];
            --i; // oops: не day, month или year
        }
    }

    time_base::dateorder order = date_order(); // попытаемся придать смысл
                                                // прочитанным значениям

    if(res[0]==month) // mdy или error
    {
        // ...
    }
    else if(res[1]==month) // dmy или ymd или error
    {
        tmp->tm_mon = val[1];
        switch (order)
        {
            case dmy:
                tmp->tm_mday = val[0];
                tmp->tm_year = val[2];
                break;
            case ymd:
                tmp->tm_year = val[0];
                tmp->tm_mday = val[2];
                break;
            default:
                r |= ios_base::badbit;
                return b;
        }
    }
    else if(res[2]==month) // ydm или error

```

```

else                                     // полагаемся на dateorder или error
{
    // ...
}

tmp->tm_year -= 1900;                    // подгоняем базовый год под tm-соглашения
return b;
}

```

Я опустил часть кода, которая не вносит ничего нового в понимание контекстов локализации, дат, или же обработки ввода. Оставляем в качестве упражнения написание более универсальных функций ввода дат (§D.6[9-10]).

Вот простая тестовая программа:

```

int main ()
try
{
    cin.imbue (loc (locale () , new Date_in ) );    // читаем даты, используя Date_in
    while (cin >> dd && dd != Date () ) cout << dd << endl;
}
catch (Date : : Bad_date e)
{
    cout << "bad date caught: " << e.why << endl;
}

```

Заметьте, что **do_get_date()** примет и бессмысленные даты, такие как

Thursday October 7, 1998

и

1999/Feb/31

Проверка согласованности года, месяца, числа и, возможно, дня недели, выполняется в конструкторе **Date**. Именно класс **Date** должен знать, что является корректной датой, а **Date_in()** не обязан разделять это знание.

Можно написать **getval()** или **do_get_date()** таким образом, чтобы они пытались угадывать смысл числовых значений. Например, очевидно, что

12 May 1922

не означает 1922 число 12 года. То есть мы можем догадаться, что число, которое не может быть месяцем, является годом. Подобного рода догадки бывают полезными в конкретных ограниченных смысловых контекстах. Но в более общих случаях польза от таких догадок сомнительна. Например:

12 May 15

может соответствовать любому году из следующих: 12, 15, 1912, 1915, 2012 или 2015. Часто бывает полезным снабжать числа подсказками. Например, *1-ое* и *15-ое* — это дни, а *"751 до н.э."* и *"1453 н.э."* — это годы.

D.4.5. Классификация символов

При чтении символов часто возникает задача их классификации с целью придания смысла прочитанному. Например, чтобы прочесть число, процедура ввода должна знать, какие символы являются цифрами. Это похоже на пример из §6.1.2, который демонстрирует классификацию символов стандартными классифицирующими функциями с целью разбора вводимых выражений.

Естественно, классификация символов зависит от используемого алфавита. Как следствие, фасет **ctype** используется для классификации символов в рамках контекста локализации.

Символы классифицируются в соответствии с перечислением **mask**:

```
class std : ctype_base
{
public:
    enum mask                // реальные значения зависят от реализации
    {
        space = 1,          // пробельные символы (в "C" locale: ' ', '\n', '\t', ...)
        print = 1<<1,       // символы печати
        cntrl = 1<<2,        // управляющие символы
        upper = 1<<3,        // символы в верхнем регистре
        lower = 1<<4,        // символы в нижнем регистре
        alpha = 1<<5,        // алфавитные символы
        digit = 1<<6,        // десятичные цифры
        punct = 1<<7,        // символы пунктуации
        xdigit = 1<<8,       // шестнадцатеричные цифры
        alnum=alpha | digit, // алфавитно-цифровые символы
        graph=alnum | punct
    };
};
```

Тип **mask** не зависит от конкретного типа символов. Как следствие, это перечисление помещается в нешаблонный базовый класс.

Очевидно, что **mask** отражает традиционную для C и C++ классификацию (§20.4.1). Однако для разных наборов символов, разные символы попадут в разные группы. Например, для набора ASCII целое значение 125 соответствует символу '}', означающему знак пунктуации (**punct**). А в датском национальном наборе символов этот числовой код соответствует гласной 'å', которая в датской локализации должна классифицироваться как **alpha**.

Данная классификация получила название **mask** (маска), поскольку традиционной эффективной реализацией для классификации небольших наборов символов является таблица, в которой каждый вход (строка, элемент) хранит битовое представление классификации. Например:

```
table['a'] == lower | alpha | xdigit
table['1'] == digit
table[' '] == space
```

При такой реализации **table[c] & m** отлично от нуля, если символ **c** есть **m**, и 0 в противном случае.

Фасет **ctype** определяется следующим образом:

```

template<class Ch>
class std::ctype: public locale::facet, public ctype_base
{
public:
    typedef Ch char_type;
    explicit ctype (size_t r = 0);

    bool is (mask m, Ch c) const;           // "c" есть "m"?

    // поместить классификацию каждого Ch из [b:e) в v:
    const Ch* is (const Ch* b, const Ch* e, mask* v) const;

    const Ch* scan_is (mask m, const Ch* b, const Ch* e) const; // найти m,
    const Ch* scan_not (mask m, const Ch* b, const Ch* e) const; // найти не-m

    Ch toupper (Ch c) const;
    const Ch* toupper (Ch* b, const Ch* e) const;           // convert [b:e)
    Ch tolower (Ch c) const;
    const Ch* tolower (Ch* b, const Ch* e) const;

    Ch widen (char c) const;
    const char* widen (const char* b, const char* e, Ch* b2) const;
    char narrow (Ch c, char def) const;
    const Ch* narrow (const Ch* b, const Ch* e, char def, char* b2) const;

    static locale::id id; // объект идентификации фасета (§D.2, §D3, §D.3.1)
protected:
    ~ctype ();
    // виртуальные "do_"-функции (см. §D.4.1)
};

```

Вызов `is(m, c)` проверяет, принадлежит ли символ `c` классификации `m`. Например:

```

int count_spaces (const string& s, const locale& loc)
{
    const ctype<char>& ct = use_facet<ctype<char>>>(loc);
    int i = 0;

    for (string::const_iterator p = s.begin(); p != s.end(); ++p)
        if (ct.is (ctype_base::space, *p)) ++i; // символ пробела в соответствии с ct

    return i;
}

```

Обратите внимание, что `is()` можно использовать для проверки принадлежности символа к одной из нескольких классификаций. Например:

```

ct.is (ctype_base::space | ctype_base::punct, c); // c в ct -пробел,сим-л или знак пунктуации?

```

Вызов `is(b, e, v)` выявляет классификацию каждого символа из `[b, e)` и помещает ее в соответствующую позицию массива `v`.

Вызов `scan_is(m, b, e)` возвращает указатель на первый символ из `[b, e)`, который есть `m`. Если нет символов, относящихся к классификации `m`, то возвращается `e`. Как всегда для стандартных фасетов, открытые функции-члены реализованы путем вызова своих виртуальных "do-функций". Простая реализация может выглядеть следующим образом:

```
template<class Ch>
const Ch* std::ctype<Ch>::do_scan_is(mask m, const Ch* b, const Ch* e) const
{
    while (b!=e && !is(m, *b)) ++b;
    return b;
}
```

Вызов **scan_not**(*m*, *b*, *e*) возвращает указатель на первый символ из [*b*, *e*), который не есть *m*. Если все символы относятся к классификации *m*, то возвращается *e*.

Вызов **toupper**(*c*) возвращает *c* в верхнем регистре, если это допустимо в используемом наборе символов; или сам *c* в противном случае.

Вызов **toupper**(*b*, *e*) конвертирует каждый символ из интервала [*b*, *e*) в верхний регистр и возвращает *e*. Простая реализация может выглядеть следующим образом:

```
template<class Ch>
const Ch* std::ctype<Ch>::to_upper(Ch* b, const Ch* e)
{
    for (; b!=e; ++b) *b = toupper(*b);
    return e;
}
```

Функция **tolower**() аналогична функции **toupper**(), но только конвертирует символы в нижний регистр.

Вызов **widen**(*c*) преобразует символ *c* в соответствующее значение *Ch*. Если набор символов *Ch* располагает несколькими символами, соответствующими *c*, то согласно стандарту используется «простейшее разумное преобразование». Например:

```
wcout<<use_facet<ctype<wchar_t>>(wcout.getloc()).widen('e');
```

выведет разумный эквивалент символа *e* в контексте локализации потока *wcout*.

С помощью **widen**() можно осуществлять и преобразования между не связанными друг с другом представлениями символов, например ASCII и EBCDIC. Например, предположим, что существует контекст локализации *ebcdic*:

```
char EBCDIC_e = use_facet<ctype<char>>(ebcdic).widen('e');
```

Вызов **widen**(*b*, *e*, *v*) берет каждый символ из интервала [*b*, *e*) и помещает расширенную версию в соответствующую позицию массива *v*.

Вызов **narrow**(*ch*, *def*) выдает значение *char*, соответствующее символу *ch* типа *Ch*. И снова используется «простейшее разумное преобразование». Если не существует *char*, соответствующего *ch*, то возвращается *def*.

Вызов **narrow**(*b*, *e*, *def*, *v*) берет каждый символ из интервала [*b*, *e*) и помещает суженную версию в соответствующую позицию массива *v*.

Общая идея состоит в том, что **narrow**() преобразует из более широкого набора символов в более узкий, а **widen**() — наоборот. Для символа *c* из более узкого набора мы ожидаем, что

```
c == narrow(widen(c), 0) // не гарантируется
```

Это справедливо при условии, что символ, соответствующий *c*, имеет единственное представление в более узком наборе символов. А это не гарантируется. Если символы, представимые с помощью *char*, не составляют подмножества большего

набора символов типа **Ch**, то следует ожидать проблем в коде, работающем с «символами вообще».

Аналогично, мы могли бы ожидать для символа **ch** из более широкого набора символов, что

```
widen (narrow (ch, def) ) == ch || widen (narrow (ch, def) ) == widen (def) // не гарантируется
```

Тем не менее, хотя чаще всего это и так, невозможно дать такую гарантию для символов, имеющих несколько значений в расширенном наборе и лишь одно — в более узком наборе символов. Например, число 7 часто имеет несколько представлений в расширенных наборах. Причина здесь в том, что, как правило, большие символьные наборы содержат несколько обычных наборов символов в качестве подмножеств, и символы из меньших наборов реплицируются для удобства преобразования.

Для каждого символа из базового наборам символов (§C.3.3) гарантируется, что

```
widen (narrow (ch_lit, 0) ) == ch_lit
```

Например:

```
widen (narrow ('x', 0) ) == 'x'
```

Функции **narrow()** и **widen()** соблюдают классификацию символов насколько возможно. Например, если справедливо **is(alpha, c)**, то также справедливы **is(alpha, narrow(c, 'a'))** и **is(alpha, widen(c))**, пока **alpha** является корректной маской для текущего контекста локализации.

Основной причиной использования фасета **ctype** вообще и функций **narrow()** и **widen()** в частности является написание кода, который осуществляет ввод/вывод и манипулирует строками для любого набора символов. Отсюда следует, что реализации **iostream** сильнейшим образом зависят от этих средств. Опираясь на **<iostream>** и **<string>**, пользователь может избежать непосредственного использования фасета **ctype**.

Для фасета **ctype** имеется и **_byname**-версия (§D.4, §D.4.1):

```
template<class Ch> class std::ctype_byname: public ctype<Ch> { /*...*/};
```

D.4.5.1. Вспомогательные шаблоны функций

Наиболее типичным применением фасета **ctype** является проверка принадлежности символа к некоторой классификации. Для решения этой задачи имеется целый набор шаблонов функций:

```
template<class Ch> bool isspace (Ch c, const locale& loc) ;
template<class Ch> bool isprint (Ch c, const locale& loc) ;
template<class Ch> bool iscntrl (Ch c, const locale& loc) ;
template<class Ch> bool isupper (Ch c, const locale& loc) ;
template<class Ch> bool islower (Ch c, const locale& loc) ;
template<class Ch> bool isalpha (Ch c, const locale& loc) ;
template<class Ch> bool isdigit (Ch c, const locale& loc) ;
template<class Ch> bool ispunct (Ch c, const locale& loc) ;
template<class Ch> bool isxdigit (Ch c, const locale& loc) ;
template<class Ch> bool isalnum (Ch c, const locale& loc) ;
template<class Ch> bool isgraph (Ch c, const locale& loc) ;
```

Эти функции тривиально реализуются при помощи фасета *use_facet*. Например:

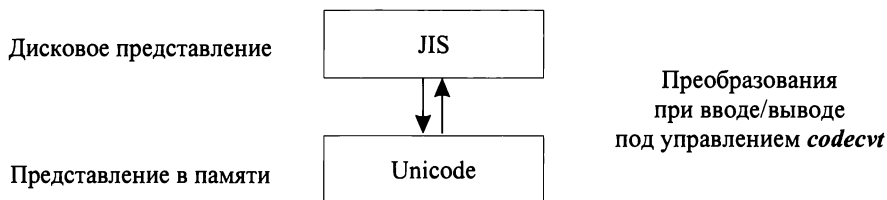
```
template<class Ch>
inline bool isspace (Ch c, const locale& loc)
{
    return use_facet<ctype<Ch> > (loc) .is (space, c) ;
}
```

Версии перечисленных функций с одним аргументом, представленные в §20.4.2, являются теми же функциями для текущего глобального контекста локализации C (но не для *locale*() языка C++). За исключением редких случаев, когда глобальные контексты локализации C и C++ различаются (§D.2.3), мы можем представлять себе одноаргументные версии как двухаргументные, использующие *locale*() . Например:

```
inline int isspace (int i)
{
    return isspace (i, locale ()) ; // почти всегда
}
```

D.4.6. Преобразование кодов символов

Иногда представление хранящихся в файле символов отличается от их желаемого представления в оперативной памяти. Например, часто японские символы хранятся в файлах, в которых используются специальные индикаторы для указания, к которому из четырех распространенных символьных наборов (*kanji*, *katakana*, *hiragana*, *romaji*) они принадлежат. Это немного неудобно, поскольку смысл каждого байта зависит от *состояния смещения (shift state)*. В то же время, это экономит память, поскольку лишь *kanji* требует для представления символов более одного байта. В памяти компьютера символы проще обрабатывать, если они представлены мультибайтными символьными наборами (кодировками), в которых каждому символу соответствует одно и то же количество байт. В таких случаях (например, в случае кодировки Unicode) символы хранятся в памяти в виде расширенных символов (*wchar_t*, §4.3). Вот почему имеется фасет *codecvt* для преобразования символов из одной кодировки в другую при их чтении и записи. Например:



Этот механизм преобразования кодировок является достаточно общим для реализации произвольных преобразований представлений символов. Он позволяет писать программы, использующие удобные внутренние представления символов (в виде *char*, *wchar_t* и т.п.), и воспринимающие различные представления символов потока посредством управления контекстом их локализации. Альтернативой была бы необходимость изменения самой программы или необходимость в промежуточных переконвертациях файлов при их чтении и записи.

Фасет **codecvt** обеспечивает преобразования между различными наборами символов, когда символ перемещается между буфером потока и внешним хранилищем:

```
class std::codecvt_base
{
public:
    enum result {ok, partial, error, noconv}; // индикаторы результата
};

template<class I, class E, class State>
class std::codecvt: public locale::facet, public codecvt_base
{
public:
    typedef I intern_type;
    typedef E extern_type;
    typedef State state_type;

    explicit codecvt(size_t r = 0);

    result in(State&, const E* from, const E* from_end, const E* &from_next, // чтение
              I* to, I* to_end, I* &to_next) const;

    result out(State&, const I* from, const I* from_end, const I* &from_next, // запись
              E* to, E* to_end, E* &to_next) const;

    result unshift(State&, E* to, E* to_end, E* &to_next) const;

    int encoding() const throw(); // свойства перекодирования
    bool always_noconv() const throw(); // можно выполнять ввод/вывод без трансляции?

    int length(const State&, const E* from, const E* from_end, size_t max) const;
    int max_length() const throw();

    static locale::id id; // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~codecvt();
    // виртуальные "do_"-функции (см. §D.4.1)
};
```

Фасет **codecvt** используется **basic_filebuf** (§21.5) для чтения или записи символов. Этот фасет **basic_filebuf** получает из контекста локализации потока (§21.7.1).

Аргумент шаблона **State** является типом, который используется для хранения состояния сдвига подлежащего конвертированию потока. Кроме того, **State** можно использовать для идентификации различных преобразований путем уточняющих специализаций. Последний механизм полезен, поскольку символы разнообразных кодировок могут храниться в объектах одного и того же типа. Например:

```
class JISstate { /* ... */ };
p = new codecvt<wchar_t, char, mbstate_t>; // стандартный char в wchar_t
q = new codecvt<wchar_t, char, JISstate>; // JIS в wchar_t
```

Без аргумента **State** не существовало бы способа сообщить фасету, в какой кодировке воспринимать поток символов **char**. Тип **mbstate_t** из **<wchar.h>** или **<wchar.h>** идентифицирует стандартное системное преобразование между **char** и **wchar_t**.

Можно также создать новый вариант **codecvt** в качестве производного класса. Например:

```
class JIScvt: public codecvt<wchar_t, char, mbstate_t> { /* ... */ };
```

Вызов *in* (*s*, *from*, *from_end*, *from_next*, *to*, *to_end*, *to_next*) читает каждый символ из интервала [*from*, *from_end*) и пытается преобразовать его. Если символ удастся преобразовать, *in* () записывает его преобразованную форму в соответствующую позицию интервала [*to*, *to_end*); если не удастся — *in* () завершает на этом свое выполнение. По завершении, *in* () сохраняет в *from_next* позицию за последним прочитанным символом, а в *to_next* — позицию за последним записанным символом. Значение *result*, возвращаемое функцией *in* (), индицирует выполненный объем работы:

- ok*: преобразованы все символы из интервала [*from*, *from_end*).
- partial*: преобразованы не все символы из интервала [*from*, *from_end*).
- error*: функция *in* () встретила символ, который не смогла преобразовать.
- noconv*: не потребовалось преобразований.

Заметьте, что возврат *partial* не обязательно свидетельствует об ошибке. Возможно, просто нужно прочесть больше символов для завершения многобайтного символа с последующей его записью, или нужно очистить выходной буфер, чтобы в нем оказалось достаточно места для приема символов.

Аргумент *s* типа *State* индицирует состояние входной последовательности символов в момент вызова функции *in* (). Это важно для случаев, когда внешнее представление символов использует состояние сдвига (shift state). Обратите внимание на то, что *s* является неконстантной ссылкой; под конец вызова *s* содержит состояние сдвига входной последовательности. Это позволяет программисту работать с *partial*-результатом преобразований и конвертировать длинные последовательности посредством нескольких вызовов функции *in* ().

Вызов *out* (*s*, *from*, *from_end*, *from_next*, *to*, *to_end*, *to_next*) преобразует [*from*, *from_end*) из внутреннего представления во внешнее так же, как *in* () преобразует из внешнего представления во внутреннее.

Поток символов должен начинаться и заканчиваться в «нейтральном» состоянии (без сдвига — *unshifted state*). Как правило, это состояние *State* (). Вызов *unshift* (*s*, *to*, *to_end*, *to_next*) анализирует *s* и помещает символы в [*to*, *to_end*) таким образом, что последовательность символов возвращается в нейтральное состояние. Возврат *unshift* () и использование аргумента *to_next* вполне аналогичны таковым для функции *out* ().

Вызов *length* (*s*, *from*, *from_end*, *max*) возвращает число символов из [*from*, *from_end*), которое функция *in* () может преобразовать.

Функция *encoding* () возвращает:

- *-1* — если внешнее представление символов использует состояние (есть сдвиг, нет сдвига)
- *0* — если кодировка символов использует переменное число байт (например, отдельный бит может сообщать о том, один или два байта отводятся под символ)
- *n* — если каждый символ во внешнем представлении занимает *n* байт

Вызов *always_noconv* () возвращает *true*, если не требуется преобразований между внутренним и внешним представлениями символов, и возвращает *false* в противном случае. Ясно, что *always_noconv* () == *true* открывает возможность максимально эффективной реализации, которая просто не вызывает функции преобразования.

Вызов `max_length()` возвращает максимальное значение, которое может вернуть `length()` при корректных аргументах.

Простейшее преобразование кодировок, которое я смог придумать, это преобразование входных символов в верхний регистр. Так что ниже представлен возможно простейший вариант `codecvt`, который выполняет все-таки некоторую работу:

```
class Cvt_to_upper: public codecvt<char, char, mbstate_t>
{
    explicit Cvt_to_upper(size_t r = 0) : codecvt(r) {}

protected:
    // читает внешнее представление - пишет внутреннее:
    result do_in(State& s, const char* from, const char* from_end, const char* &
        from_next, char* to, char* to_end, char* & to_next) const;

    // читает внутреннее представление - пишет внешнее:
    result do_out(State& s, const char* from, const char* from_end, const char* &
        from_next, char* to, char* to_end, char* & to_next) const
    {
        return codecvt<char, char, mbstate_t>::do_out
            (s, from, from_end, from_next, to, to_end, to_next);
    }

    result do_unshift(State&, E* to, E* to_end, E* & to_next) const {return ok;}

    int do_encoding() const throw() {return 1;}
    bool do_always_noconv() const throw() {return false;}

    int do_length(const State&, const E* from, const E* from_end, size_t max) const;
    int do_max_length() const throw();
};

codecvt<char, char, mbstate_t>::result
Cvt_to_upper::do_in(State& s, const char* from, const char* from_end,
    const char* & from_next, char* to, char* to_end,
    char* & to_next) const
{
    // ... §D.6[16] ...
}

int main()                // тривиальный тест
{
    locale ulocale(locale(), new Cvt_to_upper);

    cin.imbue(ulocale);

    char ch;
    while(cin >> ch) cout << ch;
}
```

Для `codecvt` имеется и `_byname`-версия (§D.4, §D.4.1):

```
template<class I, class E, class State>
class std::codecvt_byname: public codecvt<I, E, State> { /*...*/ };
```

D.4.7. Сообщения

Неудивительно, что большинство пользователей предпочитает взаимодействовать с программой на своем родном языке. Однако мы не можем обеспечить универсальный стандартный механизм взаимодействия с программой в точном соответствии с контекстом локализации. Вместо этого, стандартная библиотека предоставляет нехитрый механизм хранения зависящих от локализации строк, из которых программист может составлять простые сообщения. По существу, класс *messages* (сообщения) реализует тривиальную базу данных, доступную только для чтения:

```
class std : messages_base
{
public:
    typedef int catalog;      // тип идентификатора каталога
};

template<class Ch>
class std : messages : public locale::facet, public messages_base
{
public:
    typedef Ch char_type;
    typedef basic_string<Ch> string_type;

    explicit messages (size_t r = 0) ;

    catalog open (const basic_string<char>& fn, const locale&) const;
    string_type get (catalog c, int set, int msgid, const string_type& d) const;
    void close (catalog c) const;

    static locale::id id;      // объект идентификации фасета (§D.2, §D.3, §D.3.1)

protected:
    ~messages () ;
    // виртуальные "do_"-функции (см. §D.4.1)
};
```

Вызов *open* (*s*, *loc*) открывает «каталог» сообщений *s* для контекста локализации *loc*. Каталог — это набор строк, организованных зависящим от реализации способом и доступных посредством функции *messages::get* (). Если открыть каталог с именем *s* не удастся, то возвращается отрицательное значение. Каталог должен быть открыт до первого вызова функции *get* ().

Вызов *close* (*cat*) закрывает каталог, идентифицируемый посредством *cat*, и освобождает все связанные с ним ресурсы.

Вызов *get* (*cat*, *set*, *id*, "foo") ищет в каталоге *cat* сообщение, идентифицируемое посредством (*set*, *id*). Если строка находится, то *get* () возвращает эту строку; в противном случае *get* () возвращает строку по умолчанию (здесь это *string* ("foo")).

Приведем пример фасета сообщений для системы, в которой каталог сообщений реализован в виде вектора множеств «сообщений», имеющих тип *string*:

```
struct Set
{
    vector<string> msgs;
```

```

struct Cat
{
    vector<Set> sets;
};

class My_messages : public messages<char>
{
    vector<Cat> & catalogs;

public:
    explicit My_messages (size_t = 0) : catalogs (*new vector<Cat>) {}

    catalog do_open (const string& s, const locale& loc) const; // открыть каталог s
    string do_get (catalog c, int s, int m, const string&) const; // взять сообщ-е (s,m) из c
    void do_close (catalog cat) const
    {
        if (catalogs.size() <= cat) catalogs.erase (catalogs.begin() + cat);
    }

    ~My_messages () { delete &catalogs; }
};

```

Все функции-члены объявлены константными, потому что структура данных каталога (**vector<Set>**) хранится вне фасета.

Сообщения выбираются указанием каталога, множества внутри каталога и строки внутри множества. Строка-аргумент используется в качестве возврата по умолчанию в случае, если сообщение в каталоге не обнаруживается.

```

string My_messages::do_get (catalog cat, int set, int msg, const string& def) const
{
    if (catalogs.size() <= cat) return def;
    Cat& c = catalogs[cat];
    if (c.sets.size() <= set) return def;
    Set& s = c.sets[set];
    if (s.msgs.size() <= msg) return def;
    return s.msgs[msg];
}

```

Открытие каталога включает в себя считывание текстового содержимого с диска в структуру **Cat**. В примере я выбрал представление, читать которое не составляет никакого труда. Множество разграничивается при помощи <<< и >>>, и каждое сообщение есть одна строка текста:

```

messages<char>::catalog My_messages::do_open (const string& n, const locale& loc)
const
{
    string nn = n + locale().name();
    ifstream f (nn.c_str());
    if (!f) return -1;

    catalogs.push_back (Cat());
    Cat& c = catalogs.back();
    string s;
    while (f >> s && s == "<<<")
        // читаем Set

```

```

    c.sets.push_back (Set ( ) ) ;
    Set& ss = c.sets.back ( ) ;
    while (getline (f,s) && s != ">>>") ss.msgs.push_back (s) ;    // читаем сообщение
}
return catalogs.size ( ) -1;
}

```

Вот пример простейшего использования:

```

int main ( )
{
    if (!has_facet<My_messages> (locale ( ) ) )
    {
        cerr << "no messages facet found in " << locale ( ) .name ( ) << '\n' ;
        exit (1) ;
    }

    const messages<char>& m = use_facet<My_messages> (locale ( ) ) ;
    extern string message_directory;    // здесь я храню сообщения
    int cat = m.open (message_directory, locale ( ) ) ;

    if (cat<0)
    {
        cerr << "no catalog found\n" ;
        exit (1) ;
    }

    cout << m.get (cat, 0, 0, "Missed again!") << endl;
    cout << m.get (cat, 1, 2, "Missed again!") << endl;
    cout << m.get (cat, 1, 3, "Missed again!") << endl;
    cout << m.get (cat, 3, 0, "Missed again!") << endl;
}

```

Если каталог содержит

```

<<<
hello
goodbye
>>>
yes
no
maybe
>>>

```

то программа выведет

```

hello
maybe
Missed again!
Missed again!

```

D.4.7.1. Использование сообщений из других фасетов

Помимо того, что сообщения используются в качестве хранилища зависящих от локализации строк, которые используются для общения с пользователем, они могут

содержать строки для других фасетов. Например, фасет **Season_io** (§D.3.2) можно написать следующим образом:

```
class Season_io : public locale::facet
{
    const messages<char>& m;           // директория сообщений
    int cat;                           // каталог сообщений

public:
    class Missing_messages { };

    Season_io (int i = 0) : locale::facet(i),
m (use_facet<Season_messages>(locale())) ,
        cat (m.open{message_directory, locale}())
    {
        if (cat < 0) throw Missing_messages();
    }

    ~Season_io() {} // чтобы можно было уничтожать объекты Season_io (§D.3)

    const string& to_str (Season x) const;           // строковое представление x
    bool from_str (const string& s, Season& x) const; // поместить Season соответ-ий s в x

    static locale::id id; // объект идентификации фасета (§D.2. §D.3, §D.3.1)
};

locale::id Season_io::id; // определяем идентифицирующий объект

const string& Season_io::to_str (Season x) const
{
    return m->get (cat, x, "no-such-season");
}

bool Season_io::from_str (const string& s, Season& x) const
{
    for (int i = Season::spring; i <= Season::winter; i++)
        if (m->get (cat, i, "no-such-season") == s)
        {
            x = Season(i);
            return true;
        }

    return false;
}
```

Это решение, использующее механизм **messages**, отличается от исходного (§D.3.2) тем, что разработчик строк **Season** для нового контекста локализации должен помещать их в каталог **messages**. Это не может представлять сложности для тех, кто в состоянии добавлять в систему новые контексты локализации. Однако из-за того, что **messages** предоставляют механизм доступа «только для чтения», добавление нового набора названий времен года может оказаться не по плечу прикладному программисту.

Имеется и версия **messages** с суффиксом **_byname** (§D.4, §D.4.1):

```
template<class Ch>
class std::messages_byname : public messages<Ch> { /*...*/ };
```

D.5. Советы

1. Будьте готовы к тому, что любая нетривиальная программа или система, непосредственно взаимодействующие с людьми, будут эксплуатироваться в разных странах; §D.1.
2. Не рассчитывайте на то, что все используют тот же набор символов, что и вы; §D.4.1.
3. Используйте *locale*, а не частные проприетарные решения для ввода/вывода, зависящего от национальных и культурных особенностей; §D.1.
4. Не помещайте строковые имена локализаций непосредственно в текст программы; §D.2.1.
5. Минимизируйте применение глобальной форматной информации; §D.2.3, §D.4.4.7.
6. Предпочитайте зависящие от локализации сравнения строк и сортировки; §D.2.4, §D.4.1.
7. Создавайте фасеты неизменяемыми (immutable); §D.2.2, §D.3.
8. Сконцентрируйте модификацию контекстов локализации в четко ограниченных фрагментах программы; §D.2.3.
9. Пусть время жизни фасетов регулируется из контекстов локализации; §D.3.
10. При написании функций ввода/вывода, зависящих от локализации, не забывайте обрабатывать исключения, генерируемые пользовательскими (замещенными) функциями; §D.4.2.2.
11. Используйте простой тип *Money* для хранения денежных сумм; §D.4.3.
12. Используйте простые пользовательские типы для хранения значений, требующих ввода/вывода, чувствительного к местным и национальным особенностям (а не прибегайте к приведению типов ко встроенным типам до и после операций чтения/записи); §D.4.3.
13. Не доверяйте измерениям времени выполнения до тех пор, пока вы не учтете все влияющие на них факторы; §D.4.4.1.
14. Не забывайте об ограничениях типа *time_t*; §D.4.4.1, §D.4.4.5.
15. Используйте процедуры ввода даты, допускающие различные форматы дат; §D.4.4.5.
16. Отдавайте предпочтение тем функциям классификации символов, в которых локализации учитываются (используются) явным образом; §D.4.5, §D.4.5.1.

D.6. Упражнения

1. (*2.5) Определите *Season_io* (§D.3.2) для языка, отличного от американского английского.
2. (*2) Определите класс *Season_io* (§D.3.2), который принимает строки названий времен года в качестве аргументов конструктора, с тем, чтобы названия времен года в разных локализациях могли представляться объектами этого класса.

3. (*3) Напишите функцию *collate<char>::compare()*, которая работает согласно словарному упорядочению. Сделайте это для языков, вроде французского или немецкого, в которых букв больше, чем в английском.
4. (*2) Напишите программу, которая читает и пишет логические значения (типа *bool*) в виде чисел, английских слов или слов любого другого языка по вашему выбору.
5. (*2.5) Определите тип *Time* для представления времени суток. Определите тип *Date_and_time* с помощью типов *Time* и *Date*. Обсудите плюсы и минусы такого подхода в сравнении с *Date* из §D.4.4. Реализуйте чувствительный к локализации ввод/вывод для *Time* и *Date_and_time*.
6. (*2.5) Разработайте и реализуйте фасет почтового индекса. Сделайте это как минимум для двух стран с непохожими соглашениями относительно написания адресов. Например, *NJ 07932* и *CB21QA*.
7. (*2.5) Разработайте и реализуйте фасет телефонных номеров. Сделайте это как минимум для двух стран с непохожими соглашениями относительно написания телефонных номеров. Например, *(973) 360-8000* и *1223 343000*.
8. (*2.5) Поэкспериментируйте и определите, какие форматы ввода/вывода использует ваша система для дат.
9. (*2.5) Определите функцию *get_time()*, которая «угадывает» смысл неоднозначных представлений для дат, таких как 12/5/1995, но по-прежнему отвергает все или почти все ошибочные записи. Решите, какие именно догадки приемлемы и какова вероятность ложных срабатываний.
10. (*2) Определите *get_time()*, которая воспринимает гораздо больше входных форматов, чем вариант из §D.4.4.5.
11. (*2) Составьте список контекстов локализации, поддерживаемых вашей системой.
12. (*2.5) Попробуйте выяснить, где в вашей системе хранятся именованные локализации. Если вы имеете доступ к той части системы, где хранятся контексты локализации, создайте и сохраните новую именованную локализацию. Постарайтесь не испортить существующие локализации.
13. (*2) Сравните две реализации *Season_io* (§D.3.2 и §D.4.7.1).
14. (*2) Напишите и протестируйте фасет *Date_out*, который записывает даты типа *Date* с помощью формата, передаваемого в качестве аргумента конструктору. Обсудите плюсы и минусы такого подхода в сравнении с глобальным форматом дат, предоставляемым *date_fmt* (§D.4.4.6).
15. (*2.5) Реализуйте ввод/вывод римских чисел (вида *XI* или *MDCLII*).
16. (*2.5) Реализуйте и протестируйте *Cvt_to_upper* (§D.4.6).
17. (*2.5) Используйте *clock()* для определения стоимости (1) функционального вызова, (2) вызова виртуальной функции, (3) чтения *char*, (4) чтения *int* из одной цифры, (5) чтения *int* из 5 цифр, (6) чтения *double* из 5 цифр, (7) односимвольного *string*, (8) пятисимвольного *string* и (9) сорокасимвольного *string*.
18. (*6.5) Выучите новый иностранный язык.

Приложение Е

Исключения и безопасность стандартной библиотеки

*Все будет работать так, как вы ожидаете, лишь в случае,
когда ваши ожидания корректны.
— Хайман Роузен*

Исключения и безопасность — технологии реализации безопасности при исключениях — явное управление памятью — присваивание — метод ***push_back()*** — конструкторы и инварианты — гарантии стандартных контейнеров — вставка и удаление элементов — гарантии и компромиссы — функция ***swap()*** — инициализация и итераторы — ссылки на элементы — предикаты — строки, потоки, алгоритмы — типы ***valarray*** и ***complex*** — стандартная библиотека С — рекомендации пользователям стандартной библиотеки — советы — упражнения.

Е.1. Введение

Функции стандартной библиотеки часто используют операции, предоставляемые пользователем в виде аргументов функции или шаблона. Очевидно, что некоторые из этих операций могут иногда генерировать исключения. Также генерировать исключения могут и функции выделения динамической памяти. Рассмотрим пример:

```
void f(vector<X>& v, const X& g)
{
    v[2] = g;           // X-ое присваивание может генерировать исключение
    v.push_back(g);     // аллокатор у vector<X> может генерировать исключение
    sort(v.begin(), v.end()); // X-ая операция < может генерировать исключение
    vector<X> u = v;     // X-ый копирующий конструктор может генерировать исключение
    // ...
    // и здесь уничтожается - нужно гарантировать, что X-ый деструктор сработает
    // корректно
}
```

Что произойдет, если при выполнении присваивания значения *g* будет сгенерировано исключение? Будет ли при этом вектор *v* оставлен в действительном состоянии? Что случится, если конструктор, вызываемый в процессе копирования *g* операцией *v.push_back()*, сгенерирует исключение *std::bad_alloc*? Изменится ли при этом число элементов? Или в контейнер (вектор) добавится недействительный элемент? Что произойдет, если операция «меньше чем» из типа *X* сгенерирует исключение во время сортировки? Элементы будут отсортированы частично? Или операция сортировки удалит элемент из контейнера и не вернет его назад?

Формирование окончательного списка возможных исключений в рассмотренном примере оставлено в качестве самостоятельного упражнения (§E.8[1]). Объяснение того, как может данный пример корректно работать при любом хорошо определенном типе *X* — даже если этот тип генерирует исключения — является одной из задач данного приложения. Львиная доля приложения сводится при этом к объяснению, что такое «корректно работать» и что такое «хорошо определенный тип» в контексте исключений, а также к выработке соответствующей терминологии.

Перечислим цели данного приложения:

1. Рассказать, как пользователь должен проектировать типы, удовлетворяющие требованиям стандартной библиотеки.
2. Сформулировать гарантии, предоставляемые стандартной библиотекой.
3. Сформулировать требования стандартной библиотеки к коду, предоставляемому пользователем.
4. Продемонстрировать эффективные технологии построения эффективных контейнеров, безопасных в контексте исключений.
5. Представить несколько правил программирования безопасного в контексте исключений кода.

Обсуждение проблемы *безопасности в контексте исключений* (*exception safety*) неизбежно сосредотачивается на самых худших случаях. В каких случаях исключения доставляют наибольшие проблемы? Как стандартная библиотека защищает себя и пользователей библиотеки от возникающих при этом неприятностей? Как сами пользователи могут помочь в преодолении проблем? Несмотря на все эти обсуждения, не дайте отвлечь себя от центральной идеи о том, что исключения, тем не менее, являются наилучшим способом сообщений о возникновении ошибочных ситуаций в работающих программах (§14.1, §14.9).

Обсуждение концепций, технологий и гарантий стандартной библиотеки организовано следующим образом:

- §E.2 Содержит обсуждение проблемы безопасности в контексте исключений.
- §E.3 Рассматривает методы реализации контейнеров и операций, безопасных в контексте исключений.
- §E.4 Уточняет гарантии, которые предоставляет стандартная библиотека в отношении контейнеров и операций.
- §E.5 Суммирует вопросы, связанные с безопасностью в контексте исключений, для остальных частей стандартной библиотеки.
- §E.6 Представляет обзор вопросов безопасности в контексте исключений с точки зрения пользователя стандартной библиотеки.

Стандартная библиотека предоставляет наглядные примеры вопросов безопасности, которые нужно принимать к сведению при разработке ответственных приложений. Методы, которые используются в стандартной библиотеке для обеспечения безопасности в контексте исключений, применимы к широкому диапазону проблем.

Е.2. Исключения и безопасность

Говорят, что операция, выполняемая на объекте, *безопасна в контексте исключений* (*exception safe*), если операция оставляет объект в *действительном состоянии* (*valid state*) при генерации исключения. Действительное состояние может быть и ошибочным состоянием, которое для восстановления требует коррекции (очистки — *cleanup*), но оно обязано быть полностью известным (хорошо определенным — *well defined*). Именно последнее обстоятельство позволяет написать код для обработки ошибочного состояния объекта. Этот код обработки исключения может, например, уничтожить объект, восстановить его исходное состояние, повторить операцию, просто продолжить работу программы и т.д.

Иными словами, у объекта есть инвариант (§2.3.7.1), конструкторы устанавливают этот инвариант, дальнейшие операции соблюдают этот инвариант (даже при генерации исключений), а деструктор выполняет заключительную «очистку». Операции должны заботиться о том, чтобы инвариант соблюдался перед генерацией исключений с тем, чтобы объект находился в действительном состоянии. Вполне вероятно, что это действительное состояние может не устраивать приложение. Например, строка может быть пустой, или контейнер может оставаться в неотсортированном состоянии. Таким образом, восстановить объект — значит придать ему значение, устраивающее приложение больше, чем значение, которое объект имел перед началом выполнения неудавшейся операции. В этом отношении для нас самыми интересными объектами стандартной библиотеки являются контейнеры.

Далее мы рассмотрим, при каких условиях операции на контейнерах стандартной библиотеки могут считаться безопасными в контексте возможных исключений. Всего возможны две концептуально чистые стратегии:

1. «*Без гарантий*» — если сгенерировалось исключение, обрабатываемый контейнер возможно остался в испорченном состоянии.
2. «*Сильная гарантия*» — если сгенерировалось исключение, то обрабатываемый контейнер остался в состоянии, которое он имел перед началом выполнения операции стандартной библиотеки.

К сожалению, оба случая неприемлемы для реального использования. Случай [1] неприемлем, потому что в этом случае после возникновения исключения в контейнерной операции к контейнеру уже нельзя обратиться — его даже уничтожить нельзя без опасений спровоцировать ошибку времени выполнения. Случай [2] неприемлем, поскольку он навязывает каждой операции стандартной библиотеки слишком дорогую *семантику отката* (*roll-back semantics*).

Чтобы разрешить эту дилемму, стандартная библиотека формулирует такой набор гарантий безопасности исключений, который позволяет разделить ответственность за создание надежно работающих программ между разработчиками стандартной библиотеки и ее пользователями:

- 3а.** «Базовая гарантия (*basic guarantee*) для всех операций» — поддерживаются базовые инварианты стандартной библиотеки и нет утечек ресурсов, например, памяти.
- 3б.** «Сильная гарантия (*strong guarantee*) для ключевых операций» — в дополнение к базовой гарантии операции либо выполняются успешно, либо не приводят ни к какому эффекту. Эта гарантия обеспечивается для таких ключевых операций, как *push_back()*, одноэлементных *insert()* для списка *list* и *uninitialized_copy()* (§E.3.1, §E.4.1).
- 3с.** «Гарантия отсутствия исключений (*nothrow guarantee*) для некоторых операций» — дополнительно к базовой гарантии устанавливается, что некоторые операции не генерируют исключений. Эта гарантия дается для небольшого числа простых операций типа *swap()* или *pop_back()* (§E.4.1).

Как базовая, так и сильная гарантии обеспечиваются при условии, что предоставляемые пользователем операции (такие как операции присваивания и функции *swap()*) не оставляют элементы контейнера в недействительных состояниях, не допускают утечек ресурсов и что деструкторы не генерируют исключений. Например, рассмотрим два класса, имеющих характер дескрипторных классов (§25.7):

```
template<class T> class Safe
```

```
{
    T* p;

public:
    Safe() : p(new T) {}
    ~Safe() { delete p; }
    Safe& operator=(const Safe& a) { *p = *a.p; return *this; }
    // ...
};
```

```
template<class T> class Unsafe
```

```
{
    T* p;

public:
    Unsafe(T* pp) : p(pp) {}
    ~Unsafe() { if(!p->destructible()) throw E(); delete p; }

    Unsafe& operator=(const Unsafe& a)
    {
        p->~T(); // уничтожаем старое значение (§10.4.11)
        new (p) T(*a.p); // конструируем копию *a.p в *p (§10.4.11)
        return *this;
    }
    // ...
};
```

```
void f(vector<Safe<Some_type>> &vg, vector<Unsafe<Some_type>> &vb)
{
    vg.at(1) = Safe<Some_type>();
    vb.at(1) = Unsafe<Some_type>(new Some_type);
    //
```

В этом примере создание *Safe* выполняется успешно, только если успешно создается *T*. Создание *T* может не состояться из-за неудачи с выделением памяти (с генерацией *std::bad_alloc*) или в случае генерации исключения конструктором *T*. Но в каждом удачно созданном *Safe* поле *p* указывает на успешно созданный *T*; если же конструктор потерпит неудачу, то *T* (а равно и *Safe*) не создается. Точно так же операция присваивания *T* может сгенерировать исключение, заставляя операцию присваивания *Safe* повторно сгенерировать это исключение. Проблем не возникает, если операция присваивания у типа *T* всегда оставляет свои операнды в действительном состоянии. Поэтому *Safe* — корректно ведущий себя класс и, следовательно, каждая операция стандартной библиотеки над *Safe* будет иметь разумный и хорошо определенный результат.

С другой стороны, *Unsafe* написан небрежно (точнее, написан тщательно с целью продемонстрировать, как поступать не нужно). Конструирование *Unsafe* никогда не потерпит неудачи, а операции с *Unsafe*, вроде операций присваивания и уничтожения, оставлены наедине со множеством потенциальных проблем. Операция присваивания может потерпеть неудачу при генерации исключения в копирующем конструкторе *T*. Это оставит *T* в неопределенном состоянии, поскольку старое значение **p* было уничтожено, а новое значение его не заменило. В общем случае результат непредсказуем. Деструктор *Unsafe* демонстрирует непродуманную попытку защититься от нежелательного уничтожения объекта: генерация исключения в процессе обработки исключения приведет к вызову *terminate()* (§14.7), в то время как стандартная библиотека требует нормального возврата из деструктора после уничтожения объекта. Стандартная библиотека не дает, — и не может дать — никаких гарантий, если пользователь предоставляет объекты, которые ведут себя столь плохо.

С точки зрения обработки исключений, типы *Safe* и *Unsafe* различаются тем, что *Safe* использует свой конструктор для установки инварианта (§24.3.7.1), позволяющего реализовать операции просто и эффективно. Если установить инвариант не удастся, то генерируется исключение до того, как будет сконструирован недействительный объект. А тип *Unsafe* действует наобум без какого-либо значимого инварианта, его операции генерируют исключения без привязки к какой-либо единой стратегии обработки ошибок. Естественно, это приводит к нарушению предположений (разумных) стандартной библиотеки относительно поведения типов. Например, *Unsafe* может оставить в контейнере недействительные элементы после генерации исключения из *T: operator=()*, а также может сгенерировать исключение в своем деструкторе.

Отметим, что гарантии стандартной библиотеки касательно предоставляемых пользователем неблагоприятных операций аналогичны гарантиям языка касательно нарушений базовой системы типов. Если базовая операция применяется не в соответствии с ее спецификациями, то результирующее поведение не определено. Например, если в деструкторе элемента контейнера *vector* генерируется исключение, то имеется не больше оснований надеяться на разумный результат, чем в случае разыменования указателя, инициализированного случайным числом:

```
class Bomb
{
public:
    // ...
    ~Bomb() {throw Trouble();}
```

```
vector<Bomb> b (10) ; // приводит к неопределенному поведению
void f()
{
    int* p = reinterpret_cast<int*>(rand()) ; // приводит к неопределенному поведению
    *p = 7;
}
```

Можно твердо заявить, что если вы соблюдаете основные правила языка и стандартной библиотеки, то библиотечные средства будут вести себя корректно, даже когда вы генерируете исключения.

Кроме достижения безопасности исключений как таковой еще стремятся избежать и утечек ресурсов, то есть операция, генерирующая исключение, не только должна оставить свои операнды в действительном состоянии, но и должна обеспечить (в конечном счете) освобождение захваченных ею ресурсов. Например, в точке генерации исключения выделенная ранее память должна либо освободиться, либо находиться под управлением объекта, который должен нести ответственность за ее дальнейшее освобождение.

Стандартная библиотека гарантирует отсутствие утечек ресурсов при условии, что предоставляемые пользователем операции, вызываемые библиотекой, сами не порождают никаких утечек. Рассмотрим пример:

```
void leak (bool abort)
{
    vector<int> v (10) ; // нет утечек
    vector<int>* p = new vector<int> (10) ; // возможны утечки памяти
    auto_ptr<vector<int>> > q (new vector<int> (10) ) ; // нет утечек (§14.4.2)

    if(abort) throw Up() ;
    // ...
    delete p;
}
```

Здесь при генерации исключения вектор *v* и вектор, находящийся под управлением *q*, будут корректно уничтожены так, что их ресурсы освободятся. В то же время, вектор, на который указывает *p*, не защищен от исключений и не будет уничтожен. Чтобы сделать эту часть кода безопасной, мы должны либо явным образом вызвать операцию **delete** для указателя *p* перед генерацией исключения, либо гарантировать, что вектор находится под управлением объекта — такого как **auto_ptr** (§14.4.2) — который сам позаботится о надлежащем уничтожении вектора при возникновении исключения.

Отметим, что языковые правила создания и освобождения объектов по частям гарантируют, что исключения, генерируемые в момент создания подобъектов или членов, будут корректно обработаны без специальных мер со стороны кода стандартной библиотеки (§14.4.1). Это правило лежит в основе всех технологий обработки исключений.

Помните также, что память не является единственным ресурсом, подверженным утечке. Открытые файлы, блокировки, сетевые соединения и *потоки управления (threads)* — вот примеры системных ресурсов, которые функция перед генерацией исключения должна либо освободить, либо передать некоторому объекту для последующего их освобождения.

Е.3. Технологии реализации безопасности при исключениях

Как всегда, стандартная библиотека демонстрирует примеры проблем, которые могут встречаться во многих других контекстах, и широко применимые методы их преодоления. Для написания безопасного кода имеются следующие базисные приемы и методы:

1. *try*-блок (§8.3.1).
2. Технология «выделение ресурса есть инициализация» (§14.4)
3. Не терять информацию, пока ей не найдена замена.
4. Оставлять объекты в действительном состоянии при генерации (или повторной генерации) исключений.

Действуя указанным образом, мы всегда сможем восстановиться после возникновения ошибочных ситуаций. Практическая трудность в следовании данным рекомендациям заключается в том, что невинно выглядящие операции (вроде `<`, `=` и `sort()`) могут генерировать исключения. Требуется опыт, чтобы точно знать, что можно ожидать от приложения.

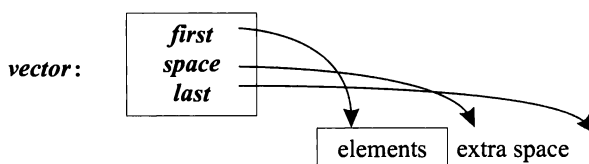
Если вы пишете библиотеку, то в идеале нужно стремиться к сильной гарантии безопасности исключений (§Е.2), безусловно реализуя базовую гарантию. При написании обычных программ о безопасности исключений обычно заботятся в меньшей степени. Например, если я пишу простую программу анализа данных для личного пользования, меня вполне устроит ее аварийное завершение в маловероятном случае исчерпания виртуальной памяти. Однако отметим, что безопасность программы при возникновении исключений и общая ее корректность тесно связаны между собой.

Технологии и методы обеспечения безопасности при исключениях, такие как определение и проверка инвариантов (§24.3.7.1), аналогичны методам, которые делают программу компактной и корректной. Отсюда вытекает, что накладные расходы на обеспечение безопасности исключений (базовая гарантия; §Е.2), даже в случае сильной гарантии, могут оказаться минимальными или даже совсем пренебрежимыми; см. §Е.8[17].

Ниже мы рассмотрим реализацию стандартного контейнера `vector` (§16.3) и выясним, что требуется для достижения этого идеала, и где можно обойтись менее жесткими требованиями к безопасности.

Е.3.1. Простой вектор

Типичная реализация типа `vector` использует представление, содержащее указатели на первый элемент, на элемент, следующий за последним элементом, и на область памяти за выделенным под элементы блоком памяти (§17.1.3) (возможно эквивалентное представление с указателем и смещениями):



Вот упрощенное объявление типа **vector**, призванное показать лишь то, что имеет отношение к обсуждению безопасности при исключениях и к предотвращению утечек ресурсов:

```
template<class T, class A = allocator<T> >
class vector
{
private:
    T* v;           // начало выделенной памяти
    T* space;       // конец последовательности эл-ов (начало свобод. прост-ва)
    T* last;        // конец выделенной памяти
    A alloc;        // аллокатор

public:
    explicit vector (size_type n, const T& val = T(), const A& a = A());
    vector (const vector& a);           // копирующий конструктор
    vector& operator= (const vector& a); // присваивание
    ~vector ();

    size_type size () const {return space-v;}
    size_type capacity () const {return last-v;}

    void push_back (const T&);
    // ...
};
```

Сначала рассмотрим бесхитростную реализацию конструктора:

```
template<class T, class A>
vector<T,A>::vector (size_type n, const T& val, const A& a)
    : alloc (a)           // копируем аллокатор
{
    v = alloc.allocate (n);           // выделяем память под эл-ты (§19.4.1)
    space = last = v+n;
    for (T* p=v, p!=last; ++p) a.construct (p, val); // создаем копию val в *p (§19.4.1)
}
```

Здесь имеются три источника исключений:

1. Функция **allocate**() генерирует исключение, индицирующее отсутствие доступной памяти.
2. Копирующий конструктор аллокатора генерирует исключение.
3. Копирующий конструктор элемента типа **T** генерирует исключение, если не может скопировать **val**.

Во всех случаях никакого объекта не создается, так что деструктор класса **vector** не вызывается (§14.4.1).

Когда функция **allocate**() терпит неудачу, она завершает работу по оператору **throw** прежде, чем будут выделены какие-либо ресурсы, так что все в порядке.

Когда копирующий конструктор типа **T** терпит неудачу, мы уже располагаем выделенной памятью, которую нужно во избежание утечек освободить. Более серьезная проблема состоит в том, что этот копирующий конструктор может сгенерировать исключение после корректного создания некоторого количества элементов, но не всех элементов.

Чтобы справиться с проблемой в этом случае, мы могли бы отслеживать, какие элементы были созданы, и уничтожить их (и только их) в случае ошибки:

```
template<class T, class A>
vector<T,A>::vector (size_type n, const T& val, const A& a)
    : alloc (a) // копируем аллокатор
{
    v = alloc.allocate (n); // выделяем память под эл-ты
    iterator p;

    try
    {
        iterator end = v+n;
        for (p=v; p!=end; ++p) alloc.construct (p, val); // конструируем элемент (§19.4.1)
        last = space = p;
    }
    catch (...)
    {
        for (iterator q = v; q!=p; ++q) alloc.destroy (q); // уничтожаем созданные элементы
        alloc.deallocate (v, n); // освобождаем память
        throw; // повторно генерируем исключение
    }
}
```

Здесь накладные расходы связаны лишь с **try**-блоком. В хорошей реализации C++ эти накладные расходы пренебрежимо малы по сравнению со стоимостью выделения памяти и инициализации элементов. Для реализаций, где организация **try**-блока накладна, заслуживает внимание проверка **if(n)** перед входом в **try**-блок и отдельная обработка случая пустого вектора.

Основную работу для этого конструктора выполняет реализация **uninitialized_fill()**, безопасная в контексте исключений:

```
template<class For, class T>
void uninitialized_fill (For beg, For end, const T& x)
{
    For p;
    try
    {
        for (p=beg; p!=end; ++p)
            new (static_cast<void*> (&*p)) T(x); // создаем копию x в *p (§10.4.11)
    }
    catch (...)
    {
        for (For q = beg; q!=p; ++q) (&*q)->~T(); // §10.4.11
        throw;
    }
}
```

Любопытная конструкция **&*p** «заботится» об итераторах, которые не являются указателями. В этом случае мы берем адрес разыменованного элемента с целью получения указателя. Явное приведение к типу **void*** гарантирует, что используется стандартная библиотечная размещающая функция (§19.4.5), а не какая-либо поль-

зовательская функция `operator new()` для T^* . Этот код оперирует на довольно низком уровне, где написание по настоящему общего кода затруднено.

По счастью, нам нет нужды повторно реализовывать `uninitialized_fill()`, потому что стандартная библиотека обеспечивает для нее желаемую сильную гарантию (§E.2). Часто важно, чтобы инициализирующая операция либо завершалась успешно, проинициализировав все элементы, либо она завершалась неудачей, не оставляя за собой никаких созданных элементов. Поэтому алгоритмы `uninitialized_fill()`, `uninitialized_fill_n()` и `uninitialized_copy()` (§19.4.4) стандартной библиотеки обеспечивают такую сильную гарантию безопасности исключений (§E.4.4).

Обратите внимание на то, что алгоритм `uninitialized_fill()` не защищает от исключений, которые генерируются деструкторами элементов или итераторными операциями (§E.4.4). Такая защита была бы слишком дорогой (см. §E.8[16-17]).

Алгоритм `uninitialized_fill()` может применяться ко множеству видов последовательностей. Соответственно, он использует прямой (однонаправленный) итератор (§19.2.1) и не может гарантировать уничтожения элементов в порядке, обратном их созданию.

Используя `uninitialized_fill()`, мы можем написать:

```
template<class T, class A>
vector<T, A>::vector(size_type n, const T& val, const A& a)
    : alloc(a)
{
    v = alloc.allocate(n);

    try
    {
        uninitialized_fill(v, v+n, val); // копируем элементы
        space = last = v+n;
    }
    catch(...)
    {
        alloc.deallocate(v, n);           // освобождаем память
        throw;                             // повторно генерируем исключение
    }
}
```

Лично я не стал бы вызывать этот код. В следующем разделе будет показано, как его можно существенно упростить.

Обратите внимание на то, что конструктор повторно генерирует перехваченное исключение. Это делается для того, чтобы тип `vector` был прозрачен для исключений и чтобы пользователь мог определить истинную причину исключения. Все контейнеры стандартной библиотеки обладают этим свойством. Прозрачность по отношению к исключениям — часто наилучшая политика для шаблонов и других «тонких» слоев программного обеспечения. Напротив, главные части системы (модули) должны в общем случае брать на себя ответственность за обработку исключений. Разработчик модуля должен перечислить все исключения, которые модуль может сгенерировать. Для этого могут потребоваться группировка исключений (§14.2), отображение исключений низкоуровневых процедур в исключения уровня модуля (§14.6.3) и спецификация исключений (§14.6).

Е.3.2. Явное управление памятью

Опыт показывает, что написание корректного кода, безопасного в контексте исключений, при помощи явного применения *try*-блоков является более трудной задачей, чем это кажется на первый взгляд. Эти трудности даже не всегда оправданы, ибо имеется альтернатива: методология «выделение ресурса есть инициализация» (§14.4) позволяет уменьшить объем кода и сделать его более элегантным. Ключевой ресурс, который требует **vector** — это память, необходимая для хранения его элементов. Если выделить отдельный класс для представления понятия памяти, используемой типом **vector**, можно значительно упростить результирующий код и уменьшить риск допущения утечек памяти:

```
template<class T, class A = allocator<T> >
struct vector_base
{
    A alloc;      // аллокатор
    T* v;         // начало выделенной памяти
    T* space;     // конец последовательности эл-ов (начало свобод. прост-ва)
    T* last;      // конец выделенной памяти

    vector_base(const A& a, typename A::size_type n)
        : alloc(a), v(alloc.allocate(n)), space(v+n), last(v+n) {}

    ~vector_base() { alloc.deallocate(v, last-v); }
};
```

Пока *v* и *last* корректны, **vector_base** может быть уничтожен. Класс **vector_base** имеет дело с памятью для типа *T*, а не с объектами типа *T*. Следовательно, пользователь **vector_base** должен уничтожить все объекты, которые созданы в выделенной **vector_base** памяти, прежде чем уничтожить сам **vector_base**.

Естественно, **vector_base** написан так, что при генерации исключения (копирующим конструктором аллокатора или функцией **allocate()**) никакого объекта **vector_base** не создается и память не теряется.

Нам нужно также уметь обменивать (to swap) друг с другом объекты типа **vector_base**. Умолчательный **swap()**, однако, нашим нуждам не удовлетворяет, поскольку он копирует и уничтожает временные объекты, а так как **vector_base** — это вспомогательный класс специального назначения, которому не придана семантика защищенного копирования, то эти уничтожения приведут к нежелательным побочным эффектам. Как следствие, мы предоставляем специализацию:

```
template<class T> void swap(vector_base<T>& a, vector_base<T>& b)
{
    swap(a.alloc, b.alloc); swap(a.v, b.v);
    swap(a.space, b.space); swap(a.last, b.last);
}
```

Отталкиваясь от **vector_base**, можно определить **vector** следующим образом:

```
template<class T, class A = allocator<T> >
class vector: private vector_base<T, A>
{
    void destroy_elements() {for (T* p = v; p!=space; ++p) p->~T(); space=v; }
```

public:

```
explicit vector (size_type n, const T& val = T() , const A& a = A() ) ;
```

```
vector (const vector& a) ; // копирующий конструктор
```

```
vector& operator= (const vector& a) ; // операция присваивания
```

```
~vector () { destroy_elements () ; }
```

```
size_type size () const { return space-v; }
```

```
size_type capacity () const { return last-v; }
```

```
void push_back (const T&) ;
```

```
// ...
```

```
};
```

Деструктор типа **vector** явно вызывает деструктор типа *T* для каждого элемента. Это означает, что если деструктор элемента генерирует исключение, то уничтожение объекта **vector** терпит неудачу. Это просто катастрофа, если все это происходит при раскрутке стека, вызванной исключением, и вызывается функция **terminate**() (§14.7). В случае обычного уничтожения объекта, генерация исключения в деструкторе приводит к утечке ресурсов и непредсказуемому поведению кода, полагающегося на разумное поведение объектов. Не существует действенной защиты от генерации исключений в деструкторах, так что библиотека не дает никаких гарантий при генерации исключений в деструкторах элементов (§E.4).

Теперь конструктор можно определить весьма просто:

```
template<class T, class A>
```

```
vector<T,A> : : vector (size_type n, const T& val, const A& a)
```

```
    : vector_base<T,A> (a, n) // выделяем память для n элементов
```

```
{
```

```
    uninitialized_fill (v, v+n, val) ; // копируем элементы
```

```
}
```

Копирующий конструктор отличается лишь использованием **uninitialized_copy**() вместо **uninitialized_fill**() :

```
template<class T, class A>
```

```
vector<T,A> : : vector (const vector<T,A>& a)
```

```
    : vector_base<T,A> (a.alloc, a.size () )
```

```
{
```

```
    uninitialized_copy (a.begin () , a.end () , v) ;
```

```
}
```

Заметим, что этот стиль конструктора полагается на фундаментальное правило языка: если исключение генерируется в конструкторе, то уже сконструированные подобъекты (такие как поля данных базового класса) будут должным образом уничтожены (§14.4.1). Алгоритм **uninitialized_fill**() и его собраты (§E.4.4) предоставляют аналогичную гарантию для частично созданных последовательностей.

Е.3.3. Присваивание

Как обычно, присваивание отличается от конструирования (создания) тем, что нужно позаботиться о старом значении. Рассмотрим прямолинейную реализацию:

```

template<class T, class A>
vector<T,A>& vector<T,A>::operator=(const vector& a) // дает сильную гарантию (§E.2)
{
    vector_base<T,A> b(a.alloc, a.size()); // получаем память
    uninitialized_copy(a.begin(), a.end(), b.v); // копируем элементы
    destroy_elements();
    alloc.deallocate(v, last-v); // освобождаем старую память
    vector_base::operator=(b); // размещаем новое представление
    b.v = 0; // предотвращаем освобождение
    return *this;
}

```

Это присваивание безопасно, но оно повторяет много кода из конструкторов и деструктора. Избежать этого можно следующим образом:

```

template<class T, class A>
vector<T,A>& vector<T,A>::operator=(const vector& a) // дает сильную гарантию (§E.2)
{
    vector temp(a); // копируем a
    swap<vector_base<T,A>>(*this, temp); // обмен представлениями
    return *this;
}

```

Старые элементы уничтожаются деструктором переменной **temp**, а память, использованная для их хранения, освобождается деструктором класса **vector_base** при вызове деструктора для **temp**.

Производительность этих двух версий должна быть примерно одинаковой. По сути дела, это лишь два разных способа определения одних и тех же операций, но вторая версия короче и не реплицирует код из родственных функций типа **vector**, так что ее применение менее подвержено ошибкам и сопровождать ее легче.

Обратите внимание на отсутствие традиционной проверки на самоприсваивание (§10.4.4). Показанные реализации присваивания сначала создают копию, а затем меняют местами представления. Очевидно, что при этом корректно обрабатывается и случай самоприсваивания. Я решил, что эффективность, достигаемая наличием проверки редкого случая самоприсваивания, не стоит потерь в общем случае, когда присваивается отличающийся **vector**.

В обоих вариантах упущены две возможности значительной оптимизации кода:

1. Если емкость вектора, которому присваивается значение, достаточно велика для того, чтобы вместить все элементы присваиваемого вектора, то нет необходимости в выделении новой памяти.
2. Присваивание элементов может оказаться эффективнее их уничтожения с последующим конструированием.

Реализуя указанные оптимизации, получаем:

```

template<class T, class A>
vector<T,A>& vector<T,A>::operator=(const vector& a) // дает базовую гарантию (§E.2)
{
    if(capacity() < a.size()) // выделяем память под новое представление:
    {
        vector temp(a); // копируем a
        swap<vector_base<T,A>>(*this, temp); // обмениваем представления
    }
}

```

```

    return *this;
}

if (this == &a) return *this;           // защита от самоприсваивания (§10.4.4)

size_type sz = size();
size_type asz = a.size();

if (asz <= sz)
{
    copy(a.begin(), a.begin() + asz, v);           // присваиваем старые элементы
    for (T* p = v + asz; p != space; ++p) p->~T(); // уничтожаем лишние эл-ты (§10.4.11)
}
else
{
    copy(a.begin(), a.begin() + sz, v);           // присваиваем старые элементы
    uninitialized_copy(a.begin() + sz, a.end(), space); // конструируем дополнит. эл-ты
}

space = v + asz;
return *this;
}

```

Выполненная оптимизация не бесплатна. Алгоритм `copy()` (§18.6.1) не предоставляет сильной гарантии безопасности при исключениях. Он не гарантирует, что оставит свой целевой объект нетронутым, если в процессе копирования будет сгенерировано исключение. Таким образом, если `T::operator=()` сгенерирует исключение в процессе работы `copy()`, то вектор, которому присваивается значение другого вектора, не обязательно будет равен последнему, или не обязательно останется в исходном состоянии. Например, первые пять элементов окажутся копиями элементов присваиваемого вектора, а остальные останутся неизменными. Также вполне вероятно, что тот элемент, который копировался тогда, когда `T::operator=()` сгенерировал исключение, останется со значением, которое и не равно старому значению, и не равно значению копировавшегося элемента. И все же, если `T::operator=()` оставляет свои операнды при генерации исключения в действительном состоянии, то `vector` также будет находиться в действительном состоянии, пусть и не в том, которое мы ожидали.

Стандарт не требует, чтобы каждый аллокатор поддерживал присваивание (§19.4.3); он также не указывает в точности, когда аллокаторы копируются. Здесь я копирую аллокатор всегда, когда копируются элементы, память под которые была выделена этим аллокатором.

Присваивание для `vector` из стандартной библиотеки обеспечивает такую же слабую гарантию безопасности исключений вместе с выигрышем в производительности. Присваивание для стандартного типа `vector` обеспечивает лишь базовую гарантию безопасности исключений, что отвечает требованиям со стороны большинства пользователей. Еще раз подчеркнем, что `vector` не предоставляет при этом сильной гарантии (§E.2). Если же вам нужно, чтобы в случае присваивания для типа `vector` левый операнд оставался в исходном (то есть неизменном) состоянии при возникновении исключения, то вам придется либо воспользоваться такой реализацией стандартной библиотеки, которая предоставляет в этом случае сильную гарантию, либо написать свою собственную версию операции присваивания. Например:

```

template<class T, class A>
void safe_assign (vector<T,A>& a, const vector<T,A>& b)
{
    vector<T,A> temp (b.get_allocator() );
    temp.reserve (b.size() );

    for (typename vector<T,A>::iterator p = b.begin() ; p!=b.end() ; ++p)
        temp.push_back (*p) ;
    swap (a, temp) ;
}

```

При нехватке памяти для создания *temp* размером в *b.size()* элементов сгенерируется исключение *std::bad_alloc* до того, как будут внесены изменения в *a*. Точно так же, если *push_back()* терпит неудачу по любой причине, вектор *a* останется нетронутым, потому что мы применяем *push_back()* к *temp*, а не к *a*. В этом случае элементы *temp*, созданные функцией *push_back()*, будут уничтожены до того как исключение, вызвавшее сбой в работе, будет сгенерировано повторно.

Функция *swap()* не копирует элементы векторов. Она просто меняет местами их поля данных; то есть она обменивает их подобъекты *vector_base* (§Е.3.2). Следовательно, она не генерирует исключений, даже если операции на элементах могли бы это делать (§Е.4.3). В итоге, *safe_assign()* не делает паразитных копий элементов и демонстрирует приличную эффективность.

Как это нередко бывает, имеются альтернативы для очевидных реализаций. Мы можем позволить библиотеке выполнять для нас копирование во временную переменную:

```

template<class T, class A>
void safe_assign (vector<T,A>& a, const vector<T,A>& b)
{
    vector<T,A> temp (b) ;    // копируем элементы b в temp
    swap (a, temp) ;
}

```

На самом деле, можно воспользоваться и передачей параметров по значению (§7.2.):

```

template<class T, class A>
void safe_assign (vector<T,A>& a, vector<T,A> b)    // b передается по значению
{
    swap (a, b) ;
}

```

Е.3.4. Метод *push_back()*

В контексте безопасности исключений, *push_back()* аналогична присваиванию в том смысле, что мы должны позаботиться о неизменности *vector* в случае неудачи с добавлением нового элемента:

```

template<class T, class A>
void vector<T,A>::push_back (const T& x)
{
    if (space == last)
        // закончилось свободное пространство:

```

```

vector_base b (alloc, size () ? 2*size () : 2); // удваиваем размер
uninitialized_copy (v, space, b.v);
new (b.space) T (x); // помещаем копию x в *b.space (§10.4.11)
++b.space;
destroy_elements ();
swap<vector_base<T, A> > (b, *this); // обмениваем представления
return;
}

new (space) T (x); // помещаем копию x в *space (§10.4.11)
++space;
}

```

Разумеется, копирующий конструктор, использованный для инициализации **space*, может сгенерировать исключение. Если это произойдет, значение *vector* останется неизменным, а *space* не увеличится. В этом случае элементы *vector* остались на своих местах в памяти, так что ссылающиеся на них итераторы действительны. Таким образом, эта реализация предоставляет сильную гарантию того, что исключение, сгенерированное аллокатором или даже копирующим конструктором, предоставленным пользователем, оставит *vector* неизменным. Стандартная библиотека предоставляет именно такую гарантию для *push_back()* (§E.4.1).

Обратите внимание на отсутствие *try*-блока (кроме скрытого в *uninitialized_copy()*). За счет тщательного упорядочения операций удалось гарантировать неизменность *vector* при возникновении исключений.

Подход, при котором безопасность исключений достигается путем упорядочения кода и следования методике «выделение ресурса есть инициализация» (§14.4), часто оказывается более элегантным и эффективным, чем явная обработка ошибок с использованием *try*-блоков. Однако при неправильном упорядочении кода возникает больше проблем с достижением безопасности исключений, чем из-за отсутствия особого кода обработки ошибок. Основное правило упорядочения кода — не уничтожать старую информацию до того, как будут полностью созданы новые данные, присваивание которых не подвержено риску возникновения исключений.

Исключения создают почву для неожиданностей по части изменений потока управления (то есть последовательности выполнения операторов программы). В простых локальных фрагментах кода, в которых доминирует линейная последовательность вызовов *operator=()*, *safe_assign()* и *push_back()*, возможностей для подобного рода сюрпризов мало. Однако в больших функциях, содержащих множество циклов и условных операторов, прогноз сделать труднее. Добавление *try*-блоков лишь увеличивает сложность управления, и может служить источником недоразумений и ошибок (§14.4). Поэтому подход с упорядочением кода и следованием методике «выделение ресурса есть инициализация» имеет и здесь преимущество: упрощается контроль за потоком исполнения кода программы. Простой, элегантный код всегда легче понять и отладить.

Отметим, что рассмотренная нами реализация *vector* приведена лишь с целью иллюстрации трудностей, которые могут породить исключения, а также методов их преодоления. Стандарт не требует от реализаций библиотеки следовать этим рекомендациям. Настоящие гарантии стандарта рассмотрены ниже в §E.4.

Е.3.5. Конструкторы и инварианты

С точки зрения безопасности исключений другие операции для типа **vector** или эквивалентны уже рассмотренным (поскольку приобретают и освобождают ресурсы схожим образом), или тривиальны (не требуют соблюдения действительных состояний участвующих в них объектов). Эти операции для большинства классов составляют большую часть кода, а легкость их написания критически зависит от среды, устанавливаемой конструктором для их исполнения. Говоря иначе, все зависит от качества выбора инварианта класса (§24.3.7.1). Изучение тривиальных функций класса поможет нам понять, что делает инвариант класса хорошим, и как нужно писать конструкторы, чтобы они устанавливали хорошие инварианты.

Такие операции, как индексирование векторов (§16.3.3), писать просто, поскольку они могут положиться на инвариант, устанавливаемый конструкторами и поддерживаемый всеми функциями, которые захватывают или освобождают ресурсы. В частности, операция индексирования может положиться на то, что поле **v** ссылается на массив элементов:

```
template<class T, class A>
T& vector<T, A>::operator[] (size_type i)
{
    return v[i];
}
```

Крайне важно, чтобы конструкторы, захватывая ресурсы, устанавливали простой инвариант. Чтобы понять, почему это так важно, рассмотрим альтернативное определение **vector_base**:

```
template<class T, class A = allocator<T> >
class vector_base
{
public:
    A alloc;
    T* v;
    T* space;
    T* last;

    vector_base(const A& a, typename A::size_type n)
        : alloc(a), v(0), space(0), last(0)
    {
        v = alloc.allocate(n);
        space = last = v+n;
    }

    ~vector_base() {if(v) alloc.deallocate(v, last-v); }
};
```

Здесь я создаю **vector_base** в две стадии: сначала я создаю безопасное (надежное) состояние с **v**, **space** и **last** равными нулю, и только после этого пробую выделять память. Все это сделано из необоснованного опасения, что во время выделения памяти под элементы произойдет исключение, и конструктор оставит после себя частично созданный объект. Однако это опасение необосновано, так как невозможно оставить после себя частично созданный объект и позднее обратиться к нему — правила для статических и автоматических объектов, объектов-членов класса и элементов контейнеров

стандартной библиотеки предотвращают такую возможность. Это было возможно для «достандартных» вариантов библиотеки, которые использовали операцию «размещающее new» (§10.4.11) для создания объектов в контейнерах, разработанных без учета безопасности исключений. Привычки, однако, изживаются с трудом.

Отметим, что предпринятая нами попытка написать код «побезопаснее», лишь усложняет инвариант для класса: уже не гарантируется, что `v` указывает на выделенную память. Теперь поле `v` может быть равно `0`, а это немедленно вызывает проблемы, так как в стандартной библиотеке для аллокаторов не гарантируется безопасное освобождение памяти по нулевому указателю (§19.4.1). В этом отношении аллокаторы отличаются от операции ***delete*** (§6.2.6). В итоге, пришлось предусмотреть в деструкторе соответствующую проверку. Кроме того, каждый элемент сначала инициализируется нулем, после чего ему присваивается значение. Стоимость этой лишней работы может оказаться существенной для типов элементов, у которых операция присваивания нетривиальна (вроде ***string*** или ***list***).

Рассмотренный двухступенчатый конструктор не так уж необычен. Иногда этот стиль делают явным, оставляя конструктору лишь простую и безопасную инициализацию, обеспечивающую корректное состояние объекта для его потенциального уничтожения. Реальное же создание объекта оставляется функции ***init***(), которую пользователь должен вызвать явно. Например:

```
template<class T>      // архаичный стиль
class vector_base
{
public:
    T* v;
    T* space;
    T* last;

    vector_base() : v(0), space(0), last(0) {}
    ~vector_base() {free(v); }

    bool init(size_t n)    // возвращаем true в случае успешной инициализации
    {
        if(v = (T*) malloc(sizeof(T) * n))
        {
            uninitialized_fill(v, v+n, T());
            space = last = v+n;
            return true;
        }
        return false;
    }
};
```

Предполагается, что ценность данного стиля состоит в том, что:

1. Конструктор не генерирует исключений, а успешность отработки функции ***init***() можно проконтролировать обычными (то есть без исключений) средствами.
2. Существует тривиальное действительное состояние. При возникновении проблем операция может придать это состояние объекту.
3. Выделение ресурсов откладывается до момента, когда на самом деле возникает нужда в полностью инициализированном объекте.

Все эти пункты изучаются в последующих разделах, где показывается, почему эта двухступенчатая методика создания объектов не приносит ожидаемых выгод. Более того, она может стать источником проблем.

Е.3.5.1. Применение функции *init()*

Первый пункт (использование функции *init()* вместо полноценного конструктора) — это ложная идея. Полноценные конструкторы и обработка исключений — намного более общий и систематический способ работы с ресурсами и ошибками инициализации (§14.1, §14.4). Применение функции *init()* — это пережиток эпохи «достандартного» C++ в отсутствие механизма исключений.

Тщательно составленный код обоих стилей более-менее эквивалентен друг другу. Рассмотрим следующие примеры:

```
int f1 (int n)
{
    vector<X> v;
    // ...
    if (v.init (n) )
    {
        // используем v вектор из n элементов
    }
    else
    {
        // улаживаем проблему
    }
}
```

и

```
int f2 (int n)
try
{
    vector<X> v (n) ;
    // ...
    // используем v вектор из n элементов
}
catch ( . . . )
{
    // улаживаем проблему
}
```

При всем при этом, использование отдельной функции *init()* — это возможность:

1. Забыть вызвать *init()* (§10.2.3).
2. Забыть проверить успешность отработки *init()*.
3. Вызвать *init()* более одного раза.
4. Забыть, что *init()* может сгенерировать исключение.
5. Использовать объект до вызова *init()*.

В хорошей реализации C++ функция *f2()* чуть быстрее функции *f1()*, поскольку она избегает в общем случае проверки.

Е.3.5.2. Полагаемся на действительное состояние по умолчанию

Второй пункт (простое действительное состояние по умолчанию) в общем случае правилен, но для `vector` он ведет к ненужным расходам: теперь **`vector_base`** может иметь `v==0`, так что в реализации типа `vector` нужно всюду защищаться от этого. Например:

```
template<class T>
T& vector<T>::operator[] (size_t i)
{
    if(v) return v[i];
    // улаживаем проблему
}
```

Наличие возможности `v==0` делает стоимость операции индексирования без проверки диапазона равной стоимости операции индексирования с проверкой диапазона индексов:

```
template<class T>
T& vector<T>::at (size_t i)
{
    if(i<v.size()) return v[i];
    throw out_of_range ("vector index");
}
```

Главное здесь то, что допуская возможность `v==0`, я усложнил базовый инвариант для **`vector_base`**, а как следствие усложнился и таковой для **`vector`**. В конечном итоге, весь код **`vector_base`** и **`vector`** усложнился в связи с этим, что является источником возможных ошибок, усложняет поддержку кода и увеличивает накладные расходы на этапе выполнения. Отметим, что условные операторы могут оказаться неожиданно дорогими на современных компьютерных архитектурах. Если производительность критически важна, то ключевые операции (вроде индексирования вектора) следует реализовывать без условных операторов.

Интересно, что первоначальное определение **`vector_base`** уже содержало легко конструируемое действительное состояние: объект **`vector_base`** не мог существовать без успешного выделения памяти. Из-за этого реализация **`vector`** могла содержать следующую «функцию аварийного выхода»:

```
template<class T, class A>
void vector<T,A>::emergency_exit()
{
    space = v;    // устанавливаем размер *this в 0
    throw Total_failure();
}
```

Это немного грубовато, так как при этом не удастся вызвать деструкторы элементов и освободить в них память, отведенную под поля данных базового типа **`vector_base`**. То есть не обеспечивается базовая гарантия (§Е.2). Если же мы намерены доверять значениям `v` и `space` и деструкторам элементов, то мы можем избежать потенциальных утечек ресурсов:

```
template<class T, class A>
void vector<T,A>::emergency_exit()
```

```
{
    destroy_elements ();    // очистка
    throw Total_failure ();
}
```

Отметим, что стандартный **vector** спроектирован настолько аккуратно, что он минимизирует проблемы, вызываемые двухфазным конструированием. Функция **init()** приблизительно эквивалентна функции **resize()**, и почти везде возможность **v==0** закрыта проверками **size()==0**. Отрицательные эффекты двухфазного конструирования становятся особо заметными в случае классов, захватывающих важные ресурсы, такие как сетевые соединения и файлы. Эти классы редко являются частью более общей среды, в рамках которой формулировались бы способы их реализации и использования, как это имеет место для контейнера **vector** в стандартной библиотеке языка C++. Обычно проблемы усиливаются еще и из-за того, что понятия прикладных областей плохо соответствуют описанию реально захватываемых компьютерных ресурсов, необходимых для их реализации. Немного существует классов, концепции которых отображаются на системные ресурсы столь непосредственно, как это имеет место для контейнера **vector**.

Идея «безопасного состояния» в принципе хорошая. Если мы не можем поместить объект в безопасное состояние без риска генерации исключения в процессе выполнения операции, у нас и в самом деле проблемы. Однако «безопасное состояние» должно вытекать из семантики класса, а не быть артефактом реализации, усложняющим инвариант этого класса.

Е.3.5.3. Отложенное выделение ресурсов

Подобно второму пункту (§Е.2), третий пункт (отложить выделение ресурса до момента, когда он на самом деле потребуется) неправильно применяет хорошую идею, так что издержки есть, а выгод нет. Во многих случаях, особенно для контейнеров вроде **vector**, лучший способ отложить выделение ресурсов — это отложить создание объекта. Рассмотрим неоптимальное использование контейнера **vector**:

```
void f{int n}
{
    vector<X> v (n);    // создаем n умолчательных значений типа X
    // ...
    v[3] = X(99);        // реальная "инициализация" v[3]
    // ...
}
```

Создание **X** с единственной целью присвоить ему позже новое значение, является расточительной идеей, особенно если присваивание для типа **X** не дешево. Поэтому двухфазное конструирование **X** может показаться привлекательным. Например, если тип **X** сам является вектором, то можно было бы рассмотреть двухфазное создание вестог с целью эффективного создания пустых векторов. Однако создание умолчательных (пустых) векторов и так эффективно, поэтому усложнение реализации с учетом частного случая пустого вектора представляется бесполезным. Вообще, освобождение кода конструктора элементов от сложной инициализации редко когда является наилучшим решением для реализации фиктивной инициализации. Вместо этих ухищрений пользователь может создавать элементы строго по необходимости. Например:

```
void f2 (int n)
{
    vector<X> v;           // создаем пустой vector
    // ...
    v.push_back (X(99)) ; // конструируем элементы по необходимости
    // ...
}
```

Подводим итог: двухфазное конструирование объектов приводит к более сложным инвариантам, и, как правило, к менее элегантному, чреватому ошибками коду, более сложному в сопровождении. Где это возможно, следует отдавать предпочтение коду, базирующемуся на конструкторах, а не на *init()*-функциях, поскольку первый подход поддерживается языковыми средствами. То есть ресурсы должны выделяться в конструкторах, если только отложенное выделение ресурсов не диктуется унаследованной семантикой класса.

Е.4. Гарантии стандартных контейнеров

Если операция стандартной библиотеки генерирует исключение, она может быть уверена в том, что ее объекты-операнды остаются в действительном состоянии. Например, *at()*, генерирующая исключение *out_of_range* для *vector* (§16.3.3), не создает проблем с обеспечением безопасности исключений для контейнера *vector*. Разработчик операции *at()* не сомневается, что в момент генерации исключения *vector* находится в хорошо определенном состоянии. Трудности для разработчиков библиотеки, ее пользователей, а также людей, пытающихся разобраться в коде, начинаются тогда, когда исключение генерирует функция, предоставленная пользователем.

Контейнеры стандартной библиотеки предоставляют базовую гарантию (§Е.2): соблюдаются основные инварианты библиотеки и нет утечек ресурсов до тех пор, пока пользовательский код ведет себя как надо, то есть пользовательские «операции» не оставляют элементы контейнеров в недействительных состояниях и не генерируют исключений из деструкторов. Под «операциями» я здесь понимаю операции, используемые реализацией стандартной библиотеки, такие как конструкторы, операции присваивания, деструкторы и операции над итераторами (§Е.4.4).

Программисту относительно легко добиться того, чтобы эти операции соответствовали ожиданиям библиотеки. Даже бесхитростно написанный код часто фактически соответствует требованиям библиотеки. Следующие типы определенно удовлетворяют требованиям стандартной библиотеки к типам контейнерных элементов:

1. Встроенные типы — включая указатели.
2. Типы без пользовательских операций.
3. Классы с операциями, не генерирующими исключений и не оставляющими операнды в недействительных состояниях.
4. Классы с деструкторами, не генерирующими исключений, и с операциями (которые использует стандартная библиотека — конструкторы, присваивания, *<*, *==*, *swap()* и т.д.), для которых можно легко убедиться, что они не оставляют операнды в недействительных состояниях.

В любом случае нужно также убедиться, что нет утечек ресурсов. Например:

```
void f(Circle* pc, Triangle* pt, vector<Shape*>& v2)
{
    vector<Shape*> v(10);           // создает vector или генерирует bad_alloc
    v[3] = pc;                     // нет генерации исключений
    v.insert(v.begin()+4, pt);     // или внедряет pt, или не изменяет v
    v2.erase(v2.begin()+3);        // или удаляет v2[3], или не изменяет v2
    v2 = v;                         // копирует v или не изменяет v2
    // ...
}
```

Когда функция $f()$ закончит работу, v будет надлежащим образом уничтожена, а $v2$ останется в действительном состоянии. Из данного фрагмента не ясно, кто отвечает за удаление pc и pt . Если это ответственность функции $f()$, то она может либо перехватывать исключения и выполнять необходимые удаления, либо присваивать указатели локальным переменным типа `auto_ptr`.

Более интересный вопрос заключается в следующем: когда библиотечные операции предлагают сильную гарантию (то есть либо успешно завершаются, либо оставляют операнды нетронутыми)? Например:

```
void f(vector<X>& vx)
{
    vx.insert(vx.begin()+4, X(7)); // добавляем элемент
}
```

Вообще говоря, операции X и аллокатор у $vector<X>$ могут генерировать исключения. Что можно сказать об элементах vx , когда $f()$ завершается по исключению? Базовая гарантия утверждает, что нет утечек памяти, и что элементы vx находятся в действительных состояниях. Но остается вопрос — изменился ли при этом vx ? Не добавилось ли умолчательных значений X ? Мог ли элемент быть удален из-за того, что для `insert()` это единственный способ восстановиться с соблюдением базовой гарантии? Иногда не достаточно знать, что контейнер находится в хорошем состоянии; часто нужно знать, что это за состояние. После перехвата исключения нам обычно интересно знать, являются ли значения элементов теми, что мы ожидаем, или же нужно приступить к восстановлению после ошибок.

Е.4.1. Вставка и удаление элементов

Вставка элемента в контейнер и удаление элемента из контейнера — это очевидные примеры операций, которые в случае возникновения исключений могут оставить контейнер в непредсказуемом состоянии. Причина заключается в том, что вставка или удаление активизируют множество операций, которые могут генерировать исключения:

1. Новое значение копируется в контейнер.
2. Нужно уничтожить элемент, удаленный из контейнера.
3. Под новый элемент требуется выделить память.
4. Элементы `vector` или `deque` нужно переместить в новое место в памяти.
5. Ассоциативные контейнеры вызывают операции сравнения для элементов.
6. Вставки и удаления включают операции с итераторами.

В каждом из перечисленных случаев могут генерироваться исключения.

Если деструктор генерирует исключение, не дается вообще никаких гарантий (§Е.2), так как в этом случае они обошлись бы непозволительно дорого. Но от генерации исключений другими операциями, предоставляемыми пользователем, библиотека защищает и себя, и своих пользователей.

При манипуляциях со связными структурами данных, вроде *list* или *map*, элементы могут добавляться или удаляться, не оказывая влияния на другие элементы этих контейнеров. Иначе обстоит дело для контейнеров вроде *vector* или *deque*, элементы которых распределены в памяти непрерывно; здесь иногда приходится перемещать элементы на новые места.

Дополнительно к базовой гарантии стандартная библиотека предоставляет сильную гарантию для нескольких операций, вставляющих и удаляющих элементы. Поскольку контейнеры, реализованные в виде связных структур данных, ведут себя иначе, чем контейнеры с непрерывным размещением элементов в памяти, стандарт обеспечивает слегка различающиеся гарантии для разных видов контейнеров:

1. Гарантии для контейнеров *vector* (§16.3) и *deque* (§17.2.3):

- Если исключение генерируется в функциях *push_back()* или *push_front()*, то они оставляют контейнер нетронутым.
- Если исключение генерируется в функции *insert()*, но не копирующим конструктором или операцией присваивания типа элемента, то контейнер остается нетронутым.
- Функция *erase()* не генерирует исключений, кроме случаев возникновения исключений в копирующем конструкторе или в операции присваивания типа элемента.
- Функции *pop_back()* и *pop_front()* не генерируют исключений.

2. Гарантии для контейнера *list* (§17.2.2):

- Если исключение генерируется в функциях *push_back()* или *push_front()*, то они оставляют контейнер нетронутым.
- Если исключение генерируется в функции *insert()*, то контейнер остается нетронутым.
- Функции *erase()*, *pop_back()*, *pop_front()*, *splice()* и *reverse()* не генерируют исключений.
- Функции-члены контейнера *list*, такие как *remove()*, *remove_if()*, *unique()*, *sort()* и *merge()*, не генерируют исключений, кроме случаев возникновения исключений в предикате или функции сравнения.

3. Гарантии для ассоциативных контейнеров (§17.4):

- Если исключение генерируется операцией *insert()* при вставке единственного элемента, то контейнер остается нетронутым.
- Функция *erase()* не генерирует исключений

Обратите внимание, что если для операций на контейнере дается строгая гарантия, то при генерации исключения все итераторы, указатели и ссылки на элементы остаются действительными.

Эти правила могут быть объединены в таблицу:

Гарантии операций контейнеров				
	<i>vector</i>	<i>deque</i>	<i>list</i>	<i>map</i>
<i>clear</i> ()	не генерирует (копирование)	не генерирует (копирование)	не генерирует	не генерирует
<i>erase</i> ()	не генерирует (копирование)	не генерирует (копирование)	не генерирует	не генерирует
1-элементный <i>insert</i> ()	сильная (копирование)	сильная (копирование)	сильная	сильная
N-элементный <i>insert</i> ()	сильная (копирование)	сильная (копирование)	сильная	базовая
<i>merge</i> ()	—	—	не генерирует (сравнение)	—
<i>push_back</i> ()	сильная	сильная	сильная	—
<i>push_front</i> ()	—	сильная	сильная	—
<i>pop_back</i> ()	не генерирует	не генерирует	не генерирует	—
<i>pop_front</i> ()	—	не генерирует	не генерирует	—
<i>remove</i> ()	—	—	не генерирует (сравнение)	—
<i>remove_if</i> ()	—	—	не генерирует (предикат)	—
<i>reverse</i> ()	—	—	не генерирует	—
<i>splice</i> ()	—	—	не генерирует	—
<i>swap</i> ()	не генерирует	не генерирует	не генерирует	не генерирует (копир-е сравнения)
<i>unique</i> ()	—	—	не генерирует (сравнение)	—

Где гарантия требует отсутствия исключений в некоторой пользовательской операции, там эта операция указана в круглых скобках под строкой гарантии. Эти требования точно сформулированы в тексте, предшествующем таблице.

Функции *swap* () отличаются от других упомянутых функций тем, что не являются функциями-членами. Гарантия для *clear* () вытекает из таковой для *erase* () (§16.3.6). В таблице перечислены гарантии, предоставляемые в дополнение к базовой гарантии. Поэтому в таблицу не вошли такие операции, как *reverse* () и *unique* () для *vector*, поскольку они предоставляются как алгоритмы для любых последовательностей без дополнительных гарантий.

«Почти контейнер» *basic_string* (§17.5, §20.3) предоставляет базовую гарантию для всех операций. Стандарт также гарантирует, что *erase* () и *swap* () для этого типа не генерируют исключений, и что на функции *insert* () и *push_back* () распространяется сильная гарантия.

Помимо неизменности контейнера, операция, предоставляющая сильную гарантию, также оставляет действительными все итераторы, указатели и ссылки. Например:

```
void update (map<string, X>& m, map<string, X>::iterator current)
{
    X x;
    string S;
    while (cin >> s >> x)
        try
        {
            current = m.insert (current, make_pair (s, x)) ;
        }
        catch (...)
        {
            // здесь current по-прежнему соответствует текущему элементу
        }
}
```

Е.4.2. Гарантии и компромиссы

Затейливая смесь дополнительных гарантий отражает сущность реализации. Программисты предпочитают сильную гарантию с возможно меньшим числом оговорок, но они же настаивают, чтобы каждая операция стандартной библиотеки была как можно более эффективна. Оба пожелания разумны, но для многих операций невозможно удовлетворить их одновременно. Чтобы дать лучшее представление о компромиссах, я исследую способы добавления единственного или нескольких элементов к контейнерам *list*, *vector* и *map*.

Рассмотрим сначала добавление единственного элемента к *list* или *vector*. Как всегда, *push_back()* обеспечивает простейший способ для этого:

```
void f (list<X>& lst, vector<X>& vec, const X& x)
{
    try
    {
        lst.push_back (x) ;           // добавляем элемент к list
    }
    catch (...)
    {
        // lst не изменился
        return ;
    }

    try
    {
        vec.push_back (x) ;           // добавляем элемент к vector
    }
    catch (...)
    {
        // vec не изменился
    }
}
```

```

    return;
}
// lst и vec получили по новому элементу со значением x
}

```

Обеспечение сильной гарантии в таких случаях дается легко и дешево. Она весьма полезна, поскольку представляет собой полностью безопасный в контексте исключений способ добавления элементов. Однако **push_back()** не определяется для ассоциативных контейнеров — у контейнера **map** отсутствует **back()**. В конце концов, последний элемент ассоциативного контейнера определяется отношением порядка, а не позицией.

Гарантии для **insert()** несколько более сложные. Причина состоит в том, что иногда функции **insert()** нужно помещать элемент куда-то в «середину контейнера». Это не проблема для связанных структур данных, таких как **list** или **map**. В то же время, при наличии свободного зарезервированного пространства у контейнера типа **vector**, очевидная реализация **vector<X>::insert()** копирует элементы, расположенные после точки вставки, чтобы освободить место. Это эффективно, но нет простого способа восстановления **vector**, если копирующий конструктор типа **X** или его операция присваивания генерируют исключение (см. §Е.8[10-11]). Следовательно, **vector** обеспечивает гарантию лишь при условии, что копирующие операции элементов не генерируют исключений. А контейнеры **list** и **map** не нуждаются в этом условии; они могут запросто привязать новые элементы уже после выполнения любого копирования.

Для примера предположим, что копирующие операции типа **X** генерируют исключение **X::cannot_copy**, если не получается успешно создать копию:

```

void f(list<X>& lst, vector<X>& vec, map<string,X>& m, const X& x, const string& s)
{
    try
    {
        lst.insert(lst.begin(), x);    // добавляем элемент к list
    }
    catch (...)
    {
        // lst не изменился
        return;
    }

    try
    {
        vec.insert(vec.begin(), x);    // добавляем элемент к vector
    }
    catch (X::cannot_copy)
    {
        // оops: vec может содержать, а может и не содержать новый элемент
        return;
    }
    catch (...)
    {
        // vec не изменился

```

```

    return ;
}

try
{
    m.insert(make_pair(s,x) );    // добавляем элемент к map
}
catch ( . . . )
{
    // m не изменился
    return ;
}
// lst и vec получили по новому элементу со значением x
// m получил элемент со значением (s,x)
} .

```

Если исключение *X*: **cannot_copy** перехвачено, то в этот момент новый элемент либо уже вставлен в **vec**, либо не вставлен. В первом случае объект находится в действительном состоянии, но каково его значение — неизвестно. Возможно, что после генерации исключения *X*: **cannot_copy** некоторый элемент окажется мистическим образом продублированным (см. §E.8[11]). Или же **insert()** реализована таким образом, чтобы удалять некоторые хвостовые элементы с целью быть уверенным в том, что в контейнере не оставлены недействительные элементы.

К сожалению, обеспечение сильной гарантии для **insert()** в контейнере **vector** в отсутствие оговорок про копирующие операции типа элементов непозволительно. Цена полной защиты от исключений при перемещении элементов вектора будет весьма значительной в сравнении со случаем предоставления лишь базовой гарантии.

Типы элементов, генерирующие исключения в процессе выполнения копирующих операций, не столь уж и редки. Вот примеры из самой стандартной библиотеки — **vector<string>**, **vector<vector<double> >** и **map<string,int>**.

Контейнеры **list** и **vector** предоставляют одинаковые гарантии при вставке одного или нескольких элементов. Причина состоит в том, что реализация **insert()** для **list** и **vector** применяет одну и ту же стратегию вставки как одного, так и нескольких элементов. В то же время **map** предоставляет сильную гарантию для одноэлементного **insert()**, и лишь базовую гарантию для многоэлементных **insert()**. Легко реализовать вставку одного элемента в **map** с обеспечением сильной гарантии. Однако типичная реализация многоэлементной вставки в **map** состоит в том, чтобы вставлять элементы один за другим, а в этом случае трудно обеспечить сильную гарантию. Проблема заключается в том, что нет простого способа отката предыдущих успешных вставок в момент, когда терпит неудачу очередная вставка.

Если мы потребуем сильной гарантии, что либо все элементы успешно вставлены, либо операция ни на что не повлияла, мы можем создать новый контейнер с последующим выполнением **swap()**:

```

template<class C, class Iter>
void safe_insert(C& c, typename C::const_iterator i, Iter begin, Iter end)
{
    C tmp(c.begin(), i);           // копируем начальные элементы

```

```

copy (begin, end, inserter (tmp, tmp.end() ) ) ; // копируем новый элемент
copy (i, c.end() , inserter (tmp, tmp.end() ) ) ; // копируем "хвостовые" элементы

swap (c, tmp) ;
}

```

Как всегда, этот код может повести себя некорректно, если деструктор элемента сгенерирует исключение. В то же время, если исключение генерируется копирующими операциями элемента, то аргумент-контейнер остается неизменным.

Е.4.3. Функция `swap()`

Подобно копирующим конструкторам и присваиваниям операции `swap()` необходимы для многих стандартных алгоритмов и часто предоставляются пользователем. Например, `sort()` и `stable_sort()` обычно переупорядочивают элементы с помощью `swap()`. Таким образом, если функция `swap()` генерирует исключение в момент перестановки элементов контейнера, контейнер может так и остаться с непереставленными элементами или с продублированными элементами, а не с парой элементов, поменявшихся местами.

Рассмотрим очевидное определение функции `swap()` из стандартной библиотеки (§18.6.8):

```

template<class T> void swap (T& a, T& b)
{
    T tmp = a;
    a = b;
    b = tmp;
}

```

Ясно, что `swap()` не генерирует исключений, если только этого не делают копирующие операции типа элемента.

За одним незначительным исключением для ассоциативных контейнеров, гарантируется, что функции `swap()` стандартных контейнеров не генерируют исключений. Как правило, обмен контейнерами выполняется путем обмена структурами данных, играющими роль дескрипторов доступа к элементам (§13.5, §17.1.3). Поскольку сами элементы не перемещаются в памяти, то конструкторы и операции присваивания для элементов не вызываются и, соответственно, исключений не генерируют. Кроме того, стандарт гарантирует, что никакая функция `swap()` стандартной библиотеки не портит значений ссылок, указателей и итераторов, ссылающихся на элементы обмениваемых контейнеров. В итоге, остается единственный потенциальный источник исключений: в ассоциативных контейнерах объект сравнения (§17.1.4.1) копируется как часть дескриптора. Его копирующий конструктор и операция присваивания могут генерировать исключение при вызове функций `swap()` для стандартных контейнеров. По счастью, операции копирования объектов сравнения обычно тривиальны и у них нет поводов генерировать исключения.

Пользовательские функции `swap()` должны предоставлять те же самые гарантии. Этого несложно добиться, если программист обменивает дескрипторы, а не тупо копирует всю информацию, на которую дескрипторы ссылаются (§13.5, §16.3.9, §17.1.3).

Е.4.4. Инициализация и итераторы

Выделение памяти под элементы и инициализация этой памяти — основные части любой реализации контейнеров (§Е.3). Поэтому гарантируется, что стандартные алгоритмы для конструирования объектов в неинициализированной памяти — *uninitialized_fill()*, *uninitialized_fill_n()* и *uninitialized_copy()* (§19.4.4) — не оставляют после себя никаких сконструированных объектов в случае генерации ими исключений. Таким образом, эти алгоритмы предоставляют сильную гарантию (§Е.2). Так как иногда при этом приходится уничтожать элементы, то требование к деструкторам элементов не генерировать исключений является для этих алгоритмов существенным (см. §Е.8[14]). Кроме того, требуется, чтобы итераторы, передаваемые этим алгоритмам в качестве аргументов, вели себя корректно: они сами должны быть действительными, должны ссылаться на действительные последовательности, а их операции (такие как ++, != и *) не должны генерировать исключений при работе с действительными итераторами.

Итераторы — это типичные примеры объектов, которые свободно копируются стандартными алгоритмами и операциями над стандартными контейнерами. Соответственно, копирующие операции итераторов не должны генерировать исключений, и стандарт гарантирует, что копирующие операции итераторов, возвращаемых стандартными контейнерами, исключений не генерируют. Например, итератор, возвращаемый посредством *vector<T>::begin()*, можно копировать, не опасаясь исключений.

В то же время, операции ++ и – для итераторов могут генерировать исключения. К примеру, *istreambuf_iterator* (§19.2.6) генерирует исключение для указания на ошибку ввода, а итератор с проверкой диапазона может сгенерировать исключение для указания на попытку выхода за границы действительного диапазона (§19.3). Но исключения не могут генерироваться при перемещении итератора от одного элемента последовательности к другому без нарушения определений операций ++ и – для итераторов. Поэтому *uninitialized_fill()*, *uninitialized_fill_n()* и *uninitialized_copy()* предполагают, что операции ++ и – на их параметрах-итераторах исключений не порождают; если все же исключение генерируется, то это может быть следствием передачи под видом итераторов вовсе и не итераторов (в стандартном понимании), или что итерируемая последовательность не является действительной последовательностью. И вновь подчеркиваем, что стандартные контейнеры не защищают пользователя от порождаемого пользователем неопределенного поведения кода (§Е.2).

Е.4.5. Ссылки на элементы

Когда ссылка, указатель или итератор на элемент контейнера передается некоторому коду, этот код может испортить контейнер, повредив его элемент. Например:

```
void f(const X& x)
{
    list<X> lst;
    lst.push_back(x);
    list<X>::iterator i = lst.begin();
    *i = x;    // копируем x в list
    "
```

Если x испорчен, деструктор списка возможно будет не способен должным образом уничтожить *lst*. Например:

```
struct X
{
    int* p;
    X() {p = new int; }
    ~X() {delete p; }
    // ...
};

void malicious ()
{
    X x;
    x.p = reinterpret_cast<int*>(7);    // порчим x
    f(x);                               // бомба с часовым механизмом
}
```

Когда исполнение функции $f()$ завершается, вызывается деструктор типа *list<X>*, что, в свою очередь, порождает вызов деструктора типа *X* для испорченного значения. Но результат для *delete p*, когда p отличен от нуля и не указывает на действительное значение типа *X*, не определен и может спровоцировать немедленный крах, или будет испорчена свободная память, так что проблемы проявятся неожиданным образом позже и в иных частях программы.

Возможность возникновения таких проблем не должна останавливать нас от манипулирования элементами контейнеров посредством ссылок или итераторов; это зачастую самый простой и наиболее эффективный способ. Однако будет разумным проявить дополнительную предосторожность в работе со ссылками на элементы контейнеров. Когда целостность контейнера имеет особое значение, будет неплохо предложить менее опытным программистам более безопасную альтернативу. Например, можно предоставить операцию, которая проверяет действительность нового элемента перед его копированием в особо важный контейнер. Естественно, осуществить такую проверку без детальных знаний прикладного типа невозможно.

В самом общем случае, порча элемента контейнера может способствовать непредсказуемому поведению последующих операций с контейнером. И это никакая не уникальная особенность контейнеров. Любой объект, оставленный в плохом состоянии, может впоследствии вызвать сбой программы.

Е.4.6. Предикаты

Многие стандартные алгоритмы и многие операции над стандартными контейнерами полагаются на пользовательские предикаты. В частности, все ассоциативные контейнеры нуждаются в предикатах как для поиска, так и для вставки элементов.

Предикат, используемый операцией стандартного контейнера, может генерировать исключения. На этот случай каждая операция стандартного контейнера предоставляет базовую гарантию, а некоторые операции, такие как *insert()* для единственного элемента, обеспечивают сильную гарантию (§Е.4.1). Если предикат генерирует исключение во время операции над контейнером, окончательный набор элементов контейнера может оказаться не совсем таким, какой хотел пользователь,

но это будет набор действительных элементов. Например, если операция сравнения `==` сгенерирует исключение, будучи вызванной из `list::unique()` (§17.2.2.3), пользователь не сможет твердо рассчитывать на отсутствие дубликатов в списке. Он может рассчитывать лишь на то, что каждый элемент в списке является действительным (см. §E.5.3).

По счастью, предикаты редко делают что-нибудь, что может генерировать исключение. Однако пользовательские предикаты `<`, `==` и `!=` все же нужно принимать во внимание при рассмотрении вопросов, связанных с безопасностью кода при исключениях.

Объект сравнения ассоциативного контейнера копируется в процессе выполнения операции `swap()` (§E.4.3). Поэтому имеет смысл позаботиться о том, чтобы копирующие операции предикатов, которые могут использоваться в качестве объектов сравнения, не генерировали бы исключений.

Е.5. Другие части стандартной библиотеки

Главная задача в контексте безопасности исключений — это поддержание согласованности объектов; то есть нужно не только поддерживать инварианты индивидуальных объектов, но и согласовывать их между собой. С точки зрения стандартной библиотеки наибольшую трудность в деле обеспечения безопасности исключений представляют собой объекты-контейнеры, а остальная часть библиотеки менее интересна в этом отношении. В то же время, и встроенные массивы являются контейнерами, которые могут быть испорчены какими-либо опасными операциями.

В общем случае, функции стандартной библиотеки генерируют только те исключения, которые включены в их спецификацию, плюс исключения, которые генерируются при этом вызываемыми пользовательскими операциями. Кроме того, все функции, которые прямо или косвенно выделяют память, могут генерировать исключение, свидетельствующее об исчерпании памяти (как правило, `std::bad_alloc`).

Е.5.1. Строки

Операции над строками могут генерировать множество разных исключений. Однако `basic_string` манипулирует символами посредством функций, предоставляемых `char_traits` (§20.2), а этим функциям запрещено генерировать исключения. Подчеркнем, что именно функции `char_traits` из стандартной библиотеки не генерируют исключений, в то время как пользовательские `char_traits` могут и не соблюдать этого соглашения. Обратите также внимание на то, что типу, который используется в качестве символьного типа (типа элементов) для `basic_string`, запрещено иметь определяемые пользователем операции копирования. Тем самым устраняется важный потенциальный источник исключений.

Тип `basic_string` во многих отношениях похож на стандартный контейнер (§17.5, §20.3). Его элементы фактически составляют последовательность, к которой можно обращаться с помощью `basic_string<Ch, Tr, A>::iterator` и `basic_string<Ch, Tr, A>::const_iterator`. Соответственно, реализация строк стандартной библиотеки предоставляет базовую гарантию (§E.2), а гарантии для `erase()`, `insert()`, `push_back()` и `swap()` (E.4.1) распространяются и на `basic_string`. Например, `basic_string<Ch, Tr, A>::push_back()` обеспечивает сильную гарантию.

Е.5.2. Потоки

Когда нужно, функции типа *iostream* генерируют исключения, чтобы сообщить об изменениях состояния (§21.3.6). Соответствующая семантика хорошо определена и не создает проблем по части безопасности исключений. Если же пользовательские *operator<<<()* или *operator>>>()* генерируют исключения, то это может выглядеть так, что исключения сгенерировала библиотека. Такие исключения, однако, не влияют на состояние потока (§21.3.3). Дальнейшие операции потока может быть и не найдут ожидаемых данных — из-за того, что предшествовавшая операция вместо нормального завершения сгенерировала исключение — но сам по себе поток не поврежден. Как всегда при возникновении проблем ввода/вывода, перед возобновлением чтения или записи может потребоваться вызов функции *clear()* (§21.3.3, §21.3.5).

Подобно *basic_string*, потоки *iostream* в своих манипуляциях с символами полагаются на *char_traits* (§20.2.1, §Е.5.1). Таким образом, реализация может полагаться на то, что операции с символами не генерируют исключений, но не дается никаких гарантий в отношении нарушения этих соглашений пользователем.

Ради возможности серьезной оптимизации делается предположение, что классы *locale* (§D.2) и *facet* (§D.3) не генерируют исключений. Если бы они генерировали исключения, то использующий их поток мог бы быть испорчен. Наиболее вероятное исключение, *std::bad_cast*, может генерироваться в рамках пользовательского кода вне реализации стандартных потоков. В самом худшем случае это может привести к неполному выводу или к неудачному чтению, но не к повреждению потока *ostream* (или *istream*).

Е.5.3. Алгоритмы

Кроме *uninitialized_fill()*, *uninitialized_fill_n()* и *uninitialized_copy()* (§Е.4.4), по стандарту для алгоритмов предлагается лишь базовая гарантия (§Е.2). То есть в случае корректного поведения пользовательских объектов стандартные алгоритмы соблюдают все инварианты стандартной библиотеки и не порождают утечку ресурсов. Во избежание неопределенного поведения пользовательские операции всегда должны оставлять свои операнды в действительных состояниях, а деструкторы не должны генерировать исключений.

Сами алгоритмы не генерируют исключений. Вместо этого они сообщают об ошибках через свои возвращаемые значения. Например, поисковые алгоритмы в качестве сообщения «не найдено» обычно возвращают ссылку на конец последовательности (§18.2). Таким образом, исключения, сгенерированные в процессе работы стандартных алгоритмов, на самом деле генерируются пользовательскими операциями. Исключения эти возникают в таких операциях на элементах, как предикаты (§18.4), присваивания или *swap()*, а также в аллокаторах (§19.4).

Если перечисленные операции генерируют исключение, то алгоритм немедленно завершает работу, а дальнейшие решения по обработке исключения остаются за функциями, вызвавшими алгоритм. Некоторые алгоритмы допускают возникновение исключений в ситуациях, когда контейнер находится в недопустимом с точки зрения пользователя состоянии. Например, некоторые алгоритмы сортировки копируют элементы во временный буфер, а затем обратно помещают их в контейнер. Такой алгоритм *sort()* мог бы скопировать элементы из контейнера (планируя за-

писать их потом в контейнер в отсортированном порядке) и начать писать что-либо поверх них, а тут вот генерируется исключение. С точки зрения пользователя контейнер испорчен, но в то же время все элементы находятся в действительном состоянии, так что здесь возможно достаточно прямолинейное восстановление.

Обратим внимание на то, что стандартные алгоритмы обращаются к последовательностям через итераторы. Таким образом, стандартные алгоритмы не работают с контейнерами напрямую, а работают лишь с их элементами. Тот факт, что стандартные алгоритмы никогда непосредственно не добавляют и не удаляют элементы контейнеров, упрощает анализ последствий исключения. Аналогичным образом, если к некоторой структуре данных обращаются лишь через итераторы, указатели или ссылки на константы (например, с помощью *const Rec**), то легко доказать, что исключения в этом случае не приводят к нежелательным последствиям.

Е.5.4. Типы *valarray* и *complex*

Числовые функции не генерируют исключений явным образом (глава 22). В то же время, *valarray* должен распределять память, что может привести к исключению *std::bad_alloc*. Кроме того, шаблонам *valarray* и *complex* можно передать тип элементов (скалярный тип), который в общем случае генерирует исключения. Как всегда, стандартная библиотека предоставляет базовую гарантию (§Е.2), но никаких специальных гарантий относительно результатов вычислений, прерванных исключением, не дается.

Подобно *basic_string* (§Е.5.1) типы *valarray* и *complex* полагаются на то, что пользовательским типам их аргументов запрещено явным образом определять копирующие операции, так что их копирование побитовое. Как правило, эти числовые типы стандартной библиотеки оптимизированы по быстродействию в предположении, что типы их элементов (скалярные типы) не генерируют исключений.

Е.5.5. Стандартная библиотека языка C

Генерирует ли исключения операция стандартной библиотеки, не имеющая спецификации исключений, или не генерирует, зависит от конкретной реализации. Функции стандартной библиотеки языка C не уполномочены генерировать исключения по определению (в языке C нет исключений), но это могут делать их аргументы-функции. Реализации часто объявляют стандартные библиотечные функции языка C с пустой спецификацией исключений *throw()* с целью помочь компилятору в их оптимизации.

Функции типа *qsort()* и *bsearch()* (§18.11) принимают в качестве аргумента указатель на функцию. Поэтому они могут сгенерировать исключение, если это делают их аргументы. Базовая гарантия (§Е.2) распространяется и на эти функции.

Е.6. Рекомендации пользователям стандартной библиотеки

Можно глядеть на безопасность исключений в контексте стандартной библиотеки таким образом, что все проблемы мы создаем себе сами. Библиотека будет функционировать корректно до тех пор, пока предоставляемые пользователем операции

соответствуют базовым гарантиям библиотеки (§E.2). В частности, исключения, сгенерированные из стандартных контейнерных операций не вызовут утечек памяти и не оставят контейнеры в недействительных состояниях. В итоге, перед пользователем библиотеки встает вопрос: каким образом определять свои типы, чтобы они не вызывали утечек ресурсов и не допускали неопределенного поведения?

Вот основные правила:

1. При модификации объекта не уничтожайте его старое содержимое до того, как новое содержимое полностью сконструировано и может заменить старое содержимое без риска возникновения исключений. Например, см. реализацию `vector::operator=()`, `safe_assign()` и `vector::push_back()` в §E.3.
2. Перед генерацией исключения освободите захваченные ресурсы, которые не принадлежат какому-либо другому объекту.
 - 2a. Методика «выделение ресурса есть инициализация» (§14.4) и языковые правила, гласящие, что частично созданные объекты уничтожаются в той степени, в которой они были созданы (§14.4.1), оказывают здесь большую помощь. К примеру, см. `leak()` в §E.2.
 - 2b. Алгоритм `uninitialized_copy()` и его собратья обеспечивают автоматическое освобождение ресурсов в случае невозможности завершить конструирование набора объектов (§E.4.4).
3. Перед генерацией исключения удостоверьтесь, что все операнды находятся в действительных состояниях. Всегда оставляйте объекты в состоянии, которое позволяет обратиться к ним и уничтожить их, не вызвав неопределенного поведения или генерации исключения из деструктора. К примеру, см. присваивание для `vector` из §E.3.2.
 - 3a. Конструкторы имеют такую особенность, что при генерации в них исключений не остается объектов для последующего уничтожения. Соответственно, перед генерацией исключения нам нет необходимости устанавливать инвариант, но есть необходимость в освобождении ресурсов, захваченных в ходе неудавшегося конструирования.
 - 3b. Деструкторы особенны тем, что сгенерированное в них исключение почти наверняка ведет к нарушению инвариантов и/или вызову `terminate()`.

Следовать этим правилам на практике не так-то легко. Главная причина состоит в том, что исключения иногда появляются оттуда, откуда их совсем не ждут. Хороший пример — `std::bad_alloc`. Любая функция, которая прямо или косвенно использует операцию `new` или обращается к аллокатору для выделения памяти, может сгенерировать `bad_alloc`. В некоторых программах мы можем решить эту частную проблему, стараясь не исчерпать память компьютера. Однако для программ, которые должны работать в течение длительных промежутков времени или должны осуществлять ввод данных произвольного объема, нужно быть готовым к обработке самых разнообразных отказов при выделении принципиально ограниченных компьютерных ресурсов. Каждую функцию нужно подозревать на предмет возможности генерации исключений до тех пор, пока твердо не доказано обратное.

Простой способ попробовать избежать неожиданностей состоит в применении контейнеров элементов, которые не генерируют исключений (вроде контейнеров

указателей или контейнеров элементов простых конкретных типов), или связанных контейнеров типа *list*, предоставляющих сильную гарантию (§E.4). Другой подход — положиться в первую очередь на операции вроде *push_back*(), которые предоставляют сильную гарантию, что операция либо успешно заканчивается, либо ни на что не влияет (§E.2). Однако сами по себе эти подходы недостаточны для предотвращения утечек ресурсов и могут приводить к слишком ограничительным и пессимистическим методам обработки ошибок и восстановления. Например, *vector<T*>* — тривиально безопасный по отношению к исключениям тип в том случае, если операции над типом *T* не генерируют исключений. Однако если используемые объекты нигде не удаляются, исключение из *vector* приведет к утечке ресурсов. Введение вспомогательного класса *Handle* (§25.7), предназначенного для освобождения ресурсов, и применение *vector<Handle<T>* вместо *vector<T*>*, с большой вероятностью улучшит жизнеспособность кода.

Если вы пишете новый код, то можно с самого начала принять систематический план, по которому каждый ресурс представляется классом с инвариантом, предоставляющим базовую гарантию (§E.2). Таким образом идентифицируются наиболее важные объекты системы, для которых можно обеспечить *семантику отката* (*roll-back semantics*), то есть сильную гарантию для операций с ними (возможно, при каких-то специфических условиях).

Большинство приложений содержат структуры данных и код, которые написаны без учета безопасности исключений. При необходимости такой код можно встраивать в более надежный (с точки зрения безопасности исключений) каркас либо после проверки, что он не генерирует исключений (как это было для функций стандартной библиотеки языка C; §E.5.5), либо посредством интерфейсных классов, для которых поведение при исключениях и управление ресурсами может быть точно определено.

Проектируя типы, предназначенные для использования в безопасной по отношению к исключениям среде, нужно обращать особое внимание на операции, используемые стандартной библиотекой: конструкторы, деструкторы, присваивания, сравнения, функции *swap*(), предикаты и операции на итераторах. Наилучшее решение состоит в определении инварианта класса, который устанавливается конструкторами. Иногда нужно проектировать инвариант класса так, чтобы объект можно было поместить в такое состояние, при котором его можно уничтожить, даже если операция потерпела неудачу в самой «неудобной» точке. В идеале, такое состояние не должно быть артефактом реализации, искусственно введенным ради удобства обработки исключений, а должно естественным образом вытекать из семантики типа (§E.3.5).

Рассматривая безопасность исключений, делайте акцент на определении действительных состояний объектов (инвариантов) и на надлежащем освобождении ресурсов. Из-за этого тем более важно представлять ресурсы классами. Простым примером служит *vector_base* (§E.3.2). Конструкторы таких «ресурсных» классов захватывают низкоуровневые ресурсы (типа «сырой» памяти для *vector_base*) и устанавливают инварианты (вроде надлежащей инициализации указателей в *vector_base*). Деструкторы этих классов неявно освобождают низкоуровневые ресурсы. Правила частичного конструирования (§14.4.1) и методика «выделение ресурса есть инициализация» (§14.4) поддерживают такой способ управления ресурсами.

Хорошо написанный конструктор устанавливает инвариант класса для объекта (§24.3.7.1), то есть он присваивает объекту такое значение, которое способствует упрощению написания кода для операций класса и способствует их успешному выполнению. Это подразумевает, что конструкторам часто приходится захватывать ресурсы. Если же это не удастся сделать, то конструкторы могут сгенерировать исключение, чтобы мы попытались справиться с проблемой до того, как объект создан. Такой подход непосредственно поддерживается и языком, и стандартной библиотекой (§Е.3.5).

Требование освободить ресурсы и перевести объект в действительное состояние до того момента, когда будет генерироваться исключение, означает, что время обработки исключения распределяется между генерирующей исключение функцией, функциями, расположенными по цепочке вызовов между последней и обработчиком, и самим обработчиком. Таким образом, если функция генерирует исключение, это не означает, что она никак не связана с его обработкой: все функции, генерирующие и передающие исключение должны освобождать захваченные ими ресурсы и помещать операнды в согласованные состояния. Если они этого не делают, то обработчику исключений скорее всего остается лишь попытаться более-менее изящно завершить работу приложения.

Е.7. Советы

1. Определитесь с тем, какая степень безопасности исключений нужна вам; §Е.2.
2. Безопасность исключений должна быть составной частью общей стратегии отказоустойчивости; §Е.2.
3. Предоставляйте базовую гарантию для всех классов (то есть поддерживайте инвариант и не допускайте утечек ресурсов); §Е.2, §Е.3.2, §Е.4.
4. Где это возможно и позволительно, предоставляйте сильную гарантию (то есть чтобы операция либо успешно выполнялась, либо оставляла операнды нетронутыми); §Е.2, §Е.3.
5. Не генерируйте исключений в деструкторах; §Е.2, §Е.3.2, §Е.4.
6. Не генерируйте исключений в итераторах, которые осуществляют проход по действительной последовательности; §Е.4.1, §Е.4.4.
7. Безопасность исключений требует тщательного изучения отдельных операций; §Е.3.
8. Проектируйте шаблоны так, чтобы они были прозрачными по отношению к исключениям; §Е.3.1.
9. В вопросе приобретения ресурсов отдавайте предпочтение конструкторам, а не функциям *init()*; §Е.3.5.
10. Определяйте инвариант класса так, чтобы было ясно, какое состояние объектов класса является действительным; §Е.2, §Е.6.
11. Убедитесь, что объект всегда можно поместить в действительное состояние без риска генерации исключения; §Е.3.2, §Е.6.

12. Придерживайтесь простых инвариантов; §E.3.5.
13. Перед генерацией исключения оставляйте операнды в действительных состояниях; §E.2, §E.6.
14. Избегайте утечек ресурсов; §E.2, §E.3.1, §E.6.
15. Представляйте ресурсы непосредственно классами; §E.3.2, §E.6.
16. Помните, что *swap*() может быть альтернативой копированию элементов; §E.3.3.
17. Там, где возможно, предпочитайте упорядочение кода явному применению *try*-блоков; §E.3.4.
18. Не уничтожайте «старую» информацию до того, как заменяющая ее информация будет надежно подготовлена; §E.3.3, §E.6.
19. Полагайтесь на методику «выделение ресурса есть инициализация»; §E.3, §E.3.2, §E.6.
20. Удостоверьтесь, что объекты сравнения ассоциативных контейнеров можно копировать; §E.3.3.
21. Выделяйте наиболее важные структуры данных и предоставляйте сильную гарантию для их операций; §E.6.

E.8. Упражнения

1. (*1) Перечислите все исключения, которые могут быть сгенерированы в функции *f*() из §E.1.
2. (*1) Ответьте на вопросы к примеру из §E.1.
3. (*1) Определите класс *Tester*, который иногда генерирует исключения в таких базовых операциях, как копирующий конструктор. С помощью *Tester* проверьте контейнеры вашей реализации стандартной библиотеки.
4. (*1) Найдите ошибку в «неряшливой» версии конструктора *vector* (§E.3.1) и напишите программу, приводящую этот конструктор к краху. Подсказка: сначала реализуйте деструктор для типа *vector*.
5. (*2) Реализуйте простой список, предоставляющий базовую гарантию. Уточните, что список требует от пользователей, чтобы обеспечить указанную гарантию.
6. (*3) Реализуйте простой список, предоставляющий сильную гарантию. Тщательно протестируйте этот список. Как люди могут убедиться в безопасности этого списка?
7. (*2.5) Реализуйте *String* из §11.12 так, чтобы он стал столь же безопасным, как стандартный контейнер.
8. (*2) Сравните производительность различных версий присваивания для *vector* и *safe_assign*() (§E.3.3).
9. (*1.5) Скопируйте аллокатор без использования операции присваивания (как это требуется для улучшения *operator=*() в §E.3.3).

10. (*2) Добавьте к типу **vector** одноэлементные и многоэлементные операции **erase()** и **insert()**, предоставляющие базовую гарантию (§E.3.2).
11. (*2) Добавьте к типу **vector** одноэлементные и многоэлементные операции **erase()** и **insert()**, предоставляющие сильную гарантию (§E.3.2). Сравните стоимость и сложность данного решения с решением из упражнения 10.
12. (*2) Напишите **safe_insert()** (§E.4.2), которая вставляет элементы в существующий **vector** (а не копирует во временную переменную). Какие ограничения придется наложить на операции?
13. (*2) Напишите **safe_insert()** (§E.4.2), которая вставляет элементы в существующий **map** (а не копирует во временную переменную). Какие ограничения придется наложить на операции?
14. (*2.5) Сравните размер, сложность и производительность **safe_insert()** из упражнений 12 и 13 с версией **safe_insert()** из §E.4.2.
15. (*2.5) Напишите усовершенствованный вариант **safe_insert()** исключительно для ассоциативных контейнеров. Используйте **traits** для написания **safe_insert()**, который автоматически выбирает оптимальный вариант для контейнера. Подсказка: §19.2.3.
16. (*2.5) Попробуйте переписать **uninitialized_fill()** (§19.4.4, §E.3.1) так, чтобы он справлялся с деструкторами, генерирующими исключения. Возможно ли это? Если да, то какова стоимость такого решения? Если нет, то почему?
17. (*2.5) Попробуйте переписать **uninitialized_fill()** (§19.4.4, §E.3.1) так, чтобы он справлялся с итераторами, которые генерируют исключения в операциях **-** и **++**. Возможно ли это? Если да, то какова стоимость такого решения? Если нет, то почему?
18. (*3) Выберите контейнер из библиотеки, отличной от стандартной. Ознакомьтесь с документацией и выясните, какие гарантии безопасности исключений он предоставляет. Испытайте этот контейнер, чтобы выяснить, насколько он устойчив к исключениям, генерируемым при выделении памяти, а также кодом, предоставляемым пользователем. Сравните с соответствующим контейнером стандартной библиотеки.
19. (*3) Попробуйте оптимизировать **vector** из §E.3, пренебрегая возможностью исключений. Например, уберите все **try**-блоки. Сравните производительность с версией из §E.3 и с реализацией **vector** из стандартной библиотеки. Сравните также размер и сложность кода этих вариантов типа **vector**.
20. (*1) Определите инварианты для **vector** (§E.3) с возможностью **v==0** и без таковой (§E.3.5).
21. (*2.5) Прочитайте исходный код реализации **vector**. Какие гарантии обеспечены для присваивания, многоэлементных **insert()** и **resize()**?
22. (*3) Напишите версию **hash_map** (§17.6), которая столь же безопасна, что и стандартный контейнер.

Предметный указатель

!

[], 65, 557
>>, 158, 174
#define, 274
#endif, 274
#ifndef, 274
%d, 207
&, 158, 174
, 65
*/, 78
*this, 292
, (запятая), 173
/*, 78
^, 158, 174
__ioinit, 754
|, 158, 174
~, 174
~container(), 558
~ios_base(), 766
>>, 158, 174

A

abort(), 276
abstract node class, 905
– types, 74
access functions, 531
accumulate(), 801
acos(), 779
adapters, 565, 620
adjacent_difference(), 801
– _find(), 609
alert, 964
algorithm, 521, 800
aliases, 234
allocate(n), 675
allocator_type, 556
allocators, 674
and, 926
– _eq, 926
ANSI, 46
appending, 703
application frameworks, 918, 919
argv, 167

argv, 167
array, 559
ASCII, 963
– символы, 690
asin(), 779
asm, 926
assertion, 879
assign(first,last), 558
– (n,x), 558
assignment, 309
– operator, 290
associative array, 97, 576
– containers, 555
at(), 557
atof(), 712
atoi(), 712
atol(), 712
auto, 926
automatic objects, 129

B

back(), 557, 573, 695
– _inserter(), 99, 662
backslash, 117, 964
backspace, 964
backward_copy(), 633
bad(), 729
– _alloc, 467
– _cast, 467
– _exception, 467
– _typeid, 467
base class, 373
basefield, 728
Basic Linear Algebra Subprograms, 786
– _ios, 735, 759
– _ostream, 755
– _streambuf, 764
– _string, 691, 693
– _stringCh, 694
– _stringchar, 693
begin(), 557
BEL, 137

binary_function, 620
 —_negate, 621
 —_search(), 611
 bind1st, 620
 —(), 622
 —(x), 620
 bind2nd, 620
 —(), 622
 —(y), 620
 binder1st, 622
 bitand, 926
 bitor, 926
 bitset, 520, 555, 559
 bitwise logical operators, 174
 bool, 62, 114, 171, 926
 break, 926
 built-in types, 114

C

caching, 293
 callback function, 388
 capacity(), 558, 565
 carriage return, 964
 case, 926
 —ветви, 64
 —метки, 158
 cassert, 521
 casting, 181
 catch, 927
 —(...), 442
 —блок, 69, 437
 cctype, 521-522
 cerr, 165
 cerrno, 521, 779
 cfloat, 522
 char, 62, 64, 114, 171, 927
 —*, 65
 —_BIT, 778
 —_traits, 720, 734
 —_traitschar, 691
 —_traitswchar_t, 692
 character constant, 88
 characters, 690
 cin, 63, 90, 168, 728
 class, 151, 286, 927
 — declaration, 285
 — definition, 285
 — hierarchy, 378, 473
 classes inheritance, 373
 cleanup, 73
 clear(), 557, 707
 climits, 522
 clocale, 522
 cmath, 523, 779
 codecvt<Ch, char, mbstate_t>, 1027
 collate_byname<Ch>, 1027
 collate<Ch>, 1027
 comma operator, 173
 compare(), 692
 compile-time polymorphism, 424
 compl, 927
 complex, 523
 concatenation, 88, 704
 concrete node class, 905
 — types, 74, 303
 conditional-expressions, 186
 const, 50, 256, 927
 —_cast, 927
 —_iterator, 556
 —_mem_fun_ref_t, 621
 —_mem_fun_t, 620
 —_mem_fun1_ref_t, 621
 —_mem_fun1_t, 620
 —_reference, 556
 —_reverse_iterator, 556
 constant expressions, 967
 constructors, 287
 container, 890
 — adapter, 571
 — classes, 82, 379
 —(), 558
 —(first,last), 558
 —(n), 558
 —(n,x), 558
 —(x), 558
 continue, 926
 conversion operators, 343
 copy constructor, 290, 309
 —(), 82, 106, 610, 633
 —_backward(), 610
 copyfmt(), 766
 copy-on-write, 365
 cos(), 108
 count (), 106
 count(), 609
 count_if(), 106, 609
 cout, 63
 crosscast, 495
 csetjmp, 522
 cshift(), 784
 csignal, 522
 cstdarg, 522
 cstdarg, 522
 cstdio, 259, 522, 733
 cstdlib, 277, 521-523, 779

cstring, 521
 ctime, 520, 522
 Ctrl+Z, 166
 ctype_byname<Ch>, 1027
 ctype<Ch>, 1027
 curr_symbol(), 1041
 cwchar, 522, 710
 cwcharp, 521
 cwctype, 521, 713
 C-строки, 90, 711

D

deallocate(p,n), 675
 decimal, 118
 —_point(), 1041
 declarations, 124, 930
 declarator, 125
 default, 926
 —ветвь, 64, 91, 158
 definitions, 124
 delegation, 360, 875
 delete, 180, 926
 deque, 520, 556, 559, 565
 dereference operator, 83
 dereferencing, 134
 design reuse, 833
 destructive copy semantics, 448
 destructor, 73, 305
 diamond-shaped inheritance, 484
 dictionary, 97, 576
 difference_type, 556
 digraphs, 690
 divides, 619
 do, 926
 —_get_time(), 1053
 dot operator, 149
 — product, 803
 double, 62, 114, 119, 926
 — quote, 964
 —ended queue, 570
 —linked list, 565
 downcast, 494
 dynamic memory, 979
 — store, 74
 —_cast, 926

E

ellipsis, 202, 207
 else, 926
 empty string, 137
 —(), 558
 end(), 557

enum, 50, 114, 151, 926
 enumeration, 122
 EOF, 712, 734, 769
 eofbit, 175
 eq(), 692
 equal(), 609
 —_range(), 106, 587, 611
 —_range(k), 559
 —_to, 618
 erase(), 96, 548, 599, 706-707
 —(first,last), 557
 —(p), 557
 errno, 779
 escape-символы, 964
 exception, 242, 467, 521-522
 — handler, 243
 — safety, 1078
 — specification, 457
 —(), 735
 exit(), 276
 explicit, 926
 export, 927
 expression, 930
 —-oriented coding, 176
 extern, 927
 external linkage, 255
 extractor, 720

F

facets, 1007
 factory, 396
 fail(), 729
 false, 64, 115, 927
 fat interface, 529, 891
 fill(), 610
 fill_n(), 610
 find(), 106, 609, 627-628
 find(k), 559
 find_end(), 609
 —_first_of, 609
 —_if(), 106, 609, 628
 float, 113, 119, 927
 floating-point literal, 119
 flush(), 746
 for, 927
 —(), 158
 —_each (), 106, 609
 form feed, 964
 format string, 207
 friend, 927
 front(), 557, 573, 695
 front_inserter(), 662

fstream, 522, 760
 full expression, 318
 function declaration, 195
 – definition, 196
 – objects, 356, 607
 functional, 520
 function-like objects, 616
 –-style cast, 183
 functors, 356, 616

G–H

garbage collection, 308
 generalized slice, 796
 generate(), 610
 generate_n(), 610
 generic programming, 401
 get(), 728, 732
 get_allocator(), 558
 –_temporary_buffer(), 683
 getchar(), 733
 getline(), 709, 732
 gmtime(), 1048
 good(), 729
 goto, 926
 gptr(), 761
 greater, 618
 –_equal, 618
 grouping(), 1041
 gslice_array, 780, 786
 handle, 352, 363, 914
 handles, 895, 914–915, 917
 has-a relationship, 869
 header file, 67
 heap, 74, 648, 979
 – objects, 129
 helper functions, 302, 341, 533, 550
 hex number, 964
 hexadecimal, 118
 horizontal tab, 964

I–K

if, 926
 imbue(), 766
 –(loc), 763
 implementation inheritance, 871
 implicit conversion, 121
 includes(), 611
 increment operator, 83
 incremental changes, 803
 indirect_array, 780, 786
 indirection, 134, 360

inheritance, 80
 initializer, 125
 inline, 50, 256, 926
 – functions, 196
 inner_product(), 801
 inplace_merge(), 611
 input iterators, 102
 insert(), 548
 insert(p,first,last), 557
 –(p,n,x), 557
 –(p,x), 557
 inserter, 720
 –(), 662
 int, 62, 113–114, 171, 926
 –_MAX, 778
 integral promotion, 968
 interface, 75
 – inheritance, 871
 internal linkage, 256
 internationalization, 1008
 invalid_argument, 467
 invariant, 877
 iomanip, 522, 724, 747–748
 ios, 467, 521, 747
 –_base, 759
 –_base::badbit, 759
 –_base::failbit, 759
 –_base::failure, 467
 iosfwd, 521
 iostream, 521, 709, 726
 isalnum(), 164
 isalpha(), 164
 isdigit(), 164
 ISO C++ (ISO/IEC 14882), 47
 –/IEC 14882, 947
 istream, 522, 729, 747, 760
 –_iterator, 102, 666
 istream
 istreamstringstream, 167, 756
 iter_swap(), 610
 iterator, 82, 520, 556
 key, 576
 –_comp(), 559, 582
 –_compare, 556
 –_type, 556

L

labels, 189
 lazy evaluation, 295
 leaf class, 905
 left value, 130

left-associative, 171
 less, 618
 — equal, 618
 lexicographical_compare(), 612, 701
 limits, 522
 linkage, 254-255, 257, 259, 261, 263
 — block, 263
 — convention, 262
 linker, 254
 linking, 930
 Liskov substitution principle, 871
 list, 85, 87, 95, 97, 520, 556, 559, 565
 loader, 254
 locale, 522
 localization, 1008
 localtime(), 1048
 log(), 108
 logical constness, 292
 — and, 618
 — not, 618
 — or, 618
 long, 926
 — double, 119
 lower_bound(k), 559
 — bound(), 611
 — bound(), 587
 lt(), 692
 lvalue, 130

M

make_heap(), 612
 malloc(), 50
 manipulators, 741
 map, 85, 97, 520, 556, 559, 576
 mapped value, 576
 — type, 556
 mask_array, 780, 786
 max(), 612
 — element(), 612
 — size(), 558
 mem_fun(), 620
 — fun_ref(), 621
 — fun_ref_t, 621
 — fun_t, 620
 — fun1_ref_t, 621
 — fun1_t, 620
 member functions, 285
 — initializer list, 310
 memory, 520
 — leak, 310
 — on the stack, 979
 merge(), 106, 566-567, 611

messages_byname<Ch>, 1027
 messages<Ch>, 1027
 min(), 612
 — element(), 612
 minus, 619
 mismatch(), 609
 module, 65
 modulus, 619
 money_get<C,In>, 1027
 — get<Ch>, 1027
 — put<C,Out>, 1027
 — put<Ch>, 1027
 moneypunct<Ch>, 1027
 —<Ch,International>, 1027
 —<Ch,true>, 1027
 multimap, 559
 multiple inheritance, 392, 474
 multiplies, 619
 multiset, 559
 mutable, 926

N

namespace, 50, 66, 926
 narrow(ch,def), 1065
 negate, 619
 negative_sign(), 1041
 nested class, 347, 362
 new, 180, 467, 522, 926
 — [], 180
 — (buf) X, 320
 — _handler(), 675
 newline, 964
 next_permutation(), 612, 650
 node class, 903
 not, 926
 — eq, 927
 — equal_to, 618
 not1(), 621
 not2(), 621
 nth_element(), 611
 null element, 656
 — pointer, 134
 num_get<C,In>, 1027
 — get<Ch>, 1027
 — put<C,Out>, 1027
 — put<Ch>, 1027
 numeric, 523
 — limits, 777
 numpunct_byname<Ch>, 1027
 numpunct<Ch>, 1027

O

object hierarchies, 868
 octal, 118
 – number, 964
 operator, 927
 –[] (k), 559
 – function, 329
 – new [] (), 180
 – (), 180
 –=(x), 558
 or, 927
 –_eq, 927
 ort_heap(), 648
 ostream, 522, 720, 747, 729, 757, 760
 –_iterator, 102, 665
 ostreamstream, 755
 out_of_range, 467
 overflow_error, 467
 overloading, 201
 ownership semantics, 448

P–Q

pair<const Key,T>, 581
 parametric polymorphism, 424
 partial specialization, 418
 –_sort(), 611
 –_sort_copy(), 611
 –_sum(), 801
 partition(), 611
 patterns, 833
 permutations, 650
 placement new, 320
 plus, 619
 point of instantiation, 409
 pointer to member, 507
 –_to_binary_function, 621
 –_to_unary_function, 621
 polymorphic type, 75, 383
 polymorphism, 383
 pop(), 65–66, 572
 –_back(), 542, 548, 557
 –_front(), 557, 573
 –_heap(), 612, 648
 positive_sign(), 1041
 postfix, 125
 pow(), 108
 precedence, 171
 precompiling, 258
 prefix, 125
 prev_permutation(), 612, 650
 priority_queue, 559, 565, 574

private, 72, 927
 promotions, 968
 protected, 927
 ptr_fun(), 621
 pubimbue(), 759
 –(loc), 763
 public, 926
 push(), 65–66, 573
 –_back(), 542, 548, 557, 572–573
 –_front(), 557
 –_heap(), 612, 648
 question mark, 964
 queue, 520, 556, 559, 565, 572

R

random_shuffle(), 610
 –-access iterators, 612
 range, 613
 raw memory, 181, 501, 676
 rbegin(), 557
 read(), 732
 realloc(), 50
 recursive descent, 156
 reference, 144, 556
 – count, 362
 register, 926
 reinterpret_cast, 601, 926
 remove(), 610
 remove_copy(), 610
 –_copy_if(), 610
 –_if(), 610
 rend(), 557
 replace(), 106, 610, 707
 –_copy(), 610
 –_copy_if(), 610
 –_if(), 610
 representation, 914
 – class, 363
 – type, 352
 reptace_if(), 106
 –(), 558, 565
 –(res_arg), 709
 resize(), 558, 599
 resolution rules, 202
 return, 926
 reverse(), 610
 –_copy(), 610
 –_iterator, 556, 663
 right-associative, 171
 rotate(), 610
 –_copy(), 610

run-time errors, 244
– polymorphism, 424
– Type Information, 493, 495, 497, 499, 501, 503

S

scope, 127
– resolution operator, 128
search(), 82, 609
–_n(), 609
sentinel element, 83
sentries, 738
sentry, 738
separate compilation, 67
sequences, 82, 555
– of elements, 98
sequencing operator, 135
set, 520, 559, 588
–_difference(), 611
–_intersection(), 611
–_load(), 599
–_symmetric_difference(), 611
–_terminate(), 462
–_union(), 611
setstate(failbit), 731
shift state, 1067
–(), 784
short, 926
–circuit evaluation, 173
signed, 926
single inheritance, 474
– quote, 964
size(), 558
–_type, 556
sizeof, 926
slice, 786
–_array, 780, 786
smart pointers, 358
sort(), 82, 106, 566, 611
sort_heap(), 612
source file, 254, 930
specialization pattern, 418
specializations, 406
specifiers, 125
splice(), 566-567
sstream, 522, 755
stable_partition(), 611
stable_sort(), 611
stack, 520, 556, 559, 565
stack unwinding, 436, 446
Standard Template Library, 109
state, 729

static, 926
– member, 289
– objects, 129
–_cast, 926
std::set_terminate(), 462
–_unexpected(), 462
stdexcept, 467, 521
stdio.h, 259
STL, 109
streambuf, 521, 751, 756-757, 759, 761
streamoff, 722
strict weak ordering, 562
stride, 787
string, 50, 85, 87, 521, 555, 559
– literal, 86, 136
stringstream, 755, 760
strtol(p,end,b), 712
struct, 286, 927
structure pointer dereference operator, 149
subclass, 373
subtyping, 871
superclass, 373
swap(), 558, 610
–_ranges(), 610
switch, 927
sync_with_stdio(false), 767

T

template, 927
– instantiation, 406
template<class T>, 81
temporary objects, 318
the one-definition rule, 260
– Standard Template Library, 56
this, 927
thousands_sep(), 1041
throw, 927
time(), 1048
–_get<Ch>, 1027
–_put_byname<Ch,Out>, 1027
–_put<Ch>, 1027
to_char_type(), 691
–_int_type(), 691
top(), 572, 574
traits_type::eof(), 733
–_type::to_int_type(), 761
transform(), 610
translation unit, 254
–unit, 930
true, 115, 927
try, 926
–блок, 69, 95, 244-245, 454

type field, 78
 typedef, 256, 926
 typeid, 926
 typeid, 467, 522
 typename, 926

U

uflow(), 761
 unary_function, 620
 — negate, 621
 underflow(), 761
 Unicode, 56, 692
 union, 151, 926, 977
 unique(), 610
 — copy(), 106, 610
 UNIX, 56
 unnamed namespace, 232
 unsetf(ios_base::skipws), 746
 unsigned, 118, 926
 — char, 712
 — int, 118
 upcast, 495
 upper_bound(), 611
 upper_bound(), 587
 — bound(k), 559
 user specializations, 417
 —defined specializations, 417
 —defined types, 71
 using, 926
 —declaration, 224
 —directive, 226
 —объявление, 477
 utility, 520

V–W–X

valarray, 107, 136, 523, 555, 559
 valid iterator, 656
 value_comp(), 559, 582
 — type, 556
 — types, 303
 ve.begin(), 98
 ve.end(), 98
 vector, 50, 85, 97, 136, 520, 556, 559, 565
 vertical tab, 964
 virtual, 75, 125, 926
 — base class, 481
 void, 114, 926
 —*, 50
 volatile, 927
 vtbl, 77
 wchar.h, 710
 wchar_t, 927

wctype.h, 713
 while, 927
 widen(), 1065
 xor, 927
 —eq, 927

A

абстрактные классы, 80, 384–385, 390
 — типы, 74, 895, 900–901
 — — данных, 71
 абстрактный узловой класс, 905
 абстракция данных, 59, 69, 71, 73, 75, 77
 автоматическая сборка мусора, 308
 автоматический объект, 129
 агрегат, 149
 агрегация, 867
 — и наследование, 869
 адаптер queue, 572
 — stack, 571
 адаптеры, 565, 620
 — последовательных контейнеров, 570–571, 573, 575
 алгоритм accumulate(), 801
 — adjacent_difference(), 803
 — binary_search(), 645
 — count(), 100, 629, 658
 — count_if(), 105, 629
 — equal(), 630
 — fill(), 640
 — find(), 100, 104
 — find_end(), 632
 — find_if(), 105
 — for_each, 103
 — for_each(), 626, 635
 — generate(), 640
 — includes(), 647
 — inner_product(), 802
 — iter_swap(), 642
 — lexicographical_compare(), 649
 — merge(), 645
 — mismatch(), 617, 630
 — partial_sum(), 803
 — partition(), 646
 — remove(), 640
 — remove_copy_if(), 640
 — reverse(), 641
 — rotate(), 641
 — rotate_copy(), 641
 — search(), 631
 — search_n(), 632
 — set_difference(), 647
 — set_symmetric_difference(), 647

алгоритм `stable_partition()`, 646
– `swap()`, 642
– `swap_ranges()`, 642
– `transform()`, 634
– `unique()`, 636
– `unique_copy()`, 636
алгоритмы, 59, 98–99, 101, 103, 105, 1109
– `partial_sort()`, 643
– замены элементов, 638
– поиска, 627
– семейства `partition`, 646
– стандартной библиотеки, 608–609, 611
– стандартной библиотеки, 106
– удаления элементов, 640
– функциональный объектов, 607–652
аллокаторы, 655–686
альтернатива «агрегация/наследование», 873
– «включение/наследование», 871
анонимные пространства имен, 257
аргументы командной строки, 167
– по умолчанию, 206
– функциональных шаблонов, 410
арифметика указателей, 50
арифметическая операция `-`, 62
– `%`, 62
– `*`, 62
– `/`, 62
– `+`, 62
арифметические операции, 62–63
– типы, 114
ассоциативные контейнеры, 555, 576–587
– массивы, 45
ассоциативный контейнер `multimap`, 587
– `multiset`, 588
– `set`, 588
– массив, 97, 355, 576
– `map`, 576

Б

базовый алгоритм `sort()`, 643
– класс, 373
– `basic_ios`, 721, 726
базы данных определений типов, 42
беззнаковые целые типы, 118
беззнаковый интегральный тип `size_t`, 172
безопасности в контексте исключений, 1078
бесконечный цикл, 158, 188
библиотека BLAS, 786
библиотечная функция `exit()`, 165
– `printf()`, 90
– `realloc()`, 99

– `strlen()`, 139
библиотечные предикаты, 618
библиотечный класс `map`, 169
– `ostream`, 169
– `string`, 169
– тип `ptrdiff_t`, 658
бинарная операция, 171, 330
бинарные предикаты, 617
бинарный поиск, 644
– предикат, 612
битовые поля, 976
– `bitset`, 589
блок `catch`, 437
– спецификации компоновки, 263
буфера потоков, 666

В

ввод дат и времени, 1052
– и вывод дат и времени, 1046
– `финансовой информации`, 1039
вектор, 45, 97
векторы, 971
виртуальные базовые классы, 480
– конструкторы, 396, 511
виртуальные функции, 59, 77, 381
виртуальный базовый класс, 481
вложенные пространства имен, 984
вложенный класс, 347, 362
внешняя компоновка, 255
внутренняя компоновка, 256
возвращаемое значение, 200
восстановление типа исключения, 461
временные объекты, 318
вспомогательные функции, 533, 550
встроенные типы, 62, 114
встроенный тип, 42, 87
входные диапазоны, 614
– итераторы, 102
вывод денежных величин, 1043
выделение памяти под массивы, 511
вызов `widen(c)`, 1065
выражение, 62
выражения, 169, 930
– и операторы, 155–194
вычитание указателей, 140

Г–Д

гарантии операций контейнеров, 1101
генерация исключений, 95
гибридное проектирование, 843
глобальная переменная `errno`, 434
графический сервер X-System, 48

двойная наклонная черта, 62
 двойные кавычки, 136
 двунаправленный итератор, 565, 612
 двусвязный список, 97, 565
 двусторонняя очередь, 97, 570
 действительный итератор, 656
 декларатор, 125, 128
 — `*const`, 143
 деклараторы, 938
 декремент, 175, 360-361
 делегирования, 875
 дескриптор, 352, 363, 914
 дескрипторные классы, 895, 914-915, 917
 деструктор, 73-74, 305, 982
 диапазоны, 613
 диграфы, 690
 динамическая память, 74, 684, 979
 директива `#endif`, 215
 — `#ifdef`, 216
 — `#include cmath`, 259
 — `extern "C"`, 262-263
 — компилятора `#ifdef identifier`, 215
 директивы `#include`, 254, 257
 — — препроцессора, 166
 — `using`, 204, 226
 — препроцессора, 944-945
 — условной компиляции, 274
 доступ к базовым классам, 491, 986
 — к членам класса, 985
 — к элементам массивов, 139
 — ко вложенным классам, 988
 дружественный класс, 348
 друзья, 991
 — класса, 346-347, 349

Е-Ж-З

единица трансляции, 254, 930
 жирный интерфейс, 529, 890-891
 зависимости внутри иерархии классов, 865
 — типа `"is-a"`, 870
 заголовочные файлы, 166, 954
 — — стандартной библиотеки, 259
 заголовочный файл, 67
 — — `algorithm`, 609
 — — `assert.h`, 879
 — — `cassert`, 879
 — — `cctype`, 712, 734
 — — `climits`, 778
 — — `locale`, 766
 — — `cmath`, 778, 779
 — — `cstddef`, 172, 658
 — — `cstdlib`, 650, 711

— — `ctype.h`, 712
 — — `fstream`, 752
 — — `functional`, 609, 616, 618
 — — `iostream`, 87, 734
 — — `istream`, 726
 — — `limits.h`, 778
 — — `limits`, 776
 — — `list`, 98
 — — `locale`, 765, 1011, 1026
 — — `locale.h`, 766
 — — `map`, 98
 — — `math.h`, 778-779
 — — `memory`, 674
 — — `new`, 180, 320, 684
 — — `numeric`, 612, 800
 — — `queue`, 572
 — — `sstream`, 168
 — — `stdlib.h`, 650, 711
 — — `streambuf`, 756
 — — `string`, 692, 709
 — — `utility`, 578
 — — `valarray`, 780
 — — `vector`, 98
 загрузчик, 254
 закрытие потоков, 753
 замаскированные указатели, 980
 замещение функций виртуальных
 базовых классов, 486
 защита от повторных включений, 274
 защищенные члены классов, 489
 знак `*`, 134
 — `=`, 63
 — `==`, 63
 — `>>`, 90
 знаковые и беззнаковые символы, 966
 знаковый интегральный тип `ptrdiff_t`, 172
 значение `EDOM`, 779
 — `ERANGE`, 779
 — `true`, 105

И

идентификатор, 113, 126
 иерархии классов, 80, 378, 473-514, 862
 иерархия объектов, 868
 имена, 126
 именованные фасеты сравнения, 1032
 имя, 62, 113, 126
 инвариант, 877
 инварианты, 877
 инициализатор, 125
 инициализаторы в операторах `for`, 958
 — членов класса, 311
 инициализация, 129

инициализация массивов, 135
 — ссылок, 145
 инициализирующие выражение, 128
 инкапсуляция, 883
 инкремент, 175, 360-361
 интегральное продвижение, 968
 интегральные типы, 114
 интеллектуальные указатели, 358
 интернационализация, 1008
 интерфейс, 75-76
 интерфейсные классы, 909, 911, 913
 — функции, 71
 интерфейсы, 895
 — и реализации, 888
 информация о типе объекта на этапе
 выполнения программы, 494
 исключение, 69
 — `bad_alloc`, 180-181, 685
 — `bad_typeid`, 501
 — `length_error`, 709
 — `out_of_range`, 695, 697, 708
 — `std::bad_alloc`, 675
 исключения, 219-252, 735, 1077-1114
 — `bad_alloc`, 469
 — в деструкторах, 455
 — в конструкторах, 452
 исключительные ситуации, 68
 исходный файл, 253-278, 930
 итератор `istreambuf_iterator`, 666
 — `ostreambuf_iterator`, 666
 — ввода, 612
 — вывода, 612
 — произвольного доступа, 612
 итераторы, 82-83, 535, 557, 577, 655-686,
 694
 — для вставок, 662
 — и последовательности, 656-667
 — произвольного доступа, 612
 — типа `const_iterator`, 96
 — `iterator`, 96

К

каркас приложения, 918
 категории итераторов, 660
 квалификатор, 66
 квалифицированные имена, 223
 класс `allocator`, 680
 — `basic_fstream`, 752
 — `_ios`, 765
 — `_istream`, 726, 762
 — `_istreamstream`, 755

— `_ostream`, 726, 752, 762
 — `_streambuf`, 760, 762
 — `_string`, 702
 — `complex`, 107
 — `date`, 1049
 — `facet`, 766, 1008, 1020
 — `gslice`, 796
 — `ifstream`, 752
 — `istrstream`, 756
 — `locale`, 766, 1008
 — `locale`, 764-765, 1011-1019
 — `ostream`, 722
 — `ostrstream`, 756
 — `reverse_iterator`, 664
 — `string`, 89, 362
 — `valarray`, 780
 — представления, 363
 — `complex`, 72
 классификация символов, 712, 1063
 классовая иерархия, 79, 378, 473
 классовой шаблон `auto_ptr`, 447
 класс-представление, 352
 классы, 114, 284-295, 860-883, 940-941
 — для математических вычислений,
 775-806
 — функциональных объектов, 607-652
 ключ, 576
 ключевое слово `catch`, 242
 — `const`, 141, 290
 — `explicit`, 353
 — `export`, 429
 — `extern`, 255, 266
 — `mutable`, 294
 — `operator`, 329
 — `protected`, 376
 — `static`, 257
 — `struct`, 151
 — `template`, 995
 — `this`, 292
 — `throw`, 242
 — `try`, 243
 — `typedef`, 130
 — `typename`, 993
 — `void`, 62, 200-201
 ключевые слова, 125, 151, 926
 — языка C++, 126
 код, ориентированный на выражения,
 176
 кодировка EBCDIC, 116
 — Unicode, 117, 690, 1067
 кодировки ASCII, 116

- комитет Х3J16, 46
- комментарии, 62, 78, 190-191
- комплексные числа, 107
- композиционные исключения, 440
- композиция пространств имен, 235
- компоненты, 884-885, 887, 889, 891
- компоновка, 254-263, 930
- компоновщик, 254
- конкатенация, 88, 704
- конкретизация, 995
 - шаблона, 406
 - шаблона функции, 1003
- конкретные типы, 73, 74, 301, 303, 895
- конкретный тип, 78
 - узловой класс, 905
- константные выражения, 967
 - ссылки, 319
 - функции-члены, 290
- константный объект, 141, 294
 - указатель, 144
- константы, 141, 143
- конструирование новых объектов
 - локализации, 1015
- конструктор, 72-74
- конструкторы, 182, 287, 538, 581, 590, 695
 - и деструкторы, 376
 - и инварианты, 1093
 - и преобразования типов, 339
 - класса `valarray`, 780
 - по умолчанию, 306
- контейнер, 890
 - `deque`, 570
 - `hash_map`, 594
 - `list`, 95
 - `map`, 96, 556
 - `vector`, 93
 - типа `vector`, 533-549
- контейнерные классы, 82, 379
- контейнерный адаптер, 571
- контейнеры, 59, 81, 82, 92-93, 95, 97
 - `bitset`, 743
 - STL, 531
 - с общим базовым классом, 527
- контекст `classic()`, 1018
 - `global()`, 1018
- контроль границ массива, 141
- конфликт имен, 231
- концепции делегирования, 360
 - последовательностей, 83
 - сокрытия информации, 66
- копирование объектов, 308
 - по фактической записи, 365
- копирующие алгоритмы, 632

- копирующий конструктор, 290, 309, 339, 343, 540, 697
- короткие синонимы, 130
- косвенное обращение, 134, 360
- критерии разрешения перегрузки, 202
- куча, 74, 178, 648, 979
- кэширование, 293

Л

- леводопустимые выражения, 130
- лексические соглашения, 927, 929
- ленивые вычисления, 295
- листовой класс, 905
- литералы с плавающей запятой, 119
- логическая операция `!`, 175
 - `&&`, 175
 - `||`, 175
- логические операции, 186
- логический тип, 115
- логическое «И», 173
 - «ИЛИ», 173
 - `false`, 185
 - постоянство, 292
- локализация, 764-765, 1007-1076
- локальные объекты, 307
 - переменные, 197
 - статические объекты, 315

М

- макроконстанта `NULL`, 134
- макроопределения, 944
- макрос `__cplusplus`, 263
 - `arg_two`, 216
 - `assert()`, 879
 - `DBL_MIN_EXP`, 778
 - `FLT_RADIX`, 778
 - `LDBL_MAX`, 778
 - `va_arg()`, 209
 - `__start`, 209
- макросы, 213, 215, 778
 - с аргументами, 214
- манипуляторы, 741, 743, 745
 - без аргументов, 748
 - с аргументами, 748
- , определяемые пользователем, 749
- , принимающие аргументы, 746
- маски, 797
- массив, 64
 - `mask_array`, 797
 - `slice_array`, 789
 - указателей, 200
- массивы, 64, 114, 135, 137, 973

массивы, valarray, 589
— массивов, 135
— символов, 90
математические функции, 108
машинозависимые аспекты
 фундаментальных типов, 121
менеджеры-распределители памяти, 534
метка public, 286
метки, 189
— case-ветвей, 142
механизм RTTI, 473, 493-503
— исключений языка C++, 436
— наследования, 79
многократное использование конкретных
 типов, 899
многомерные массивы, 135
многострочные комментарии, 78
многоточие, 202, 207
множественное наследование, 392,
 473-485
— — и контроль доступа, 492
множество, 97, 588
модификатор const, 182
— const, 134, 198
— explicit, 353
— extern, 262
— friend, 346
— inline, 196
— signed, 118
— static, 197, 346
— volatile, 182
модифицирующие алгоритмы, 632-641
модуль, 65
— stack, 71
модульное программирование, 65, 67, 69
модульность, 59

Н

наследование и шаблоны, 422-423, 425
— интерфейса, 871
— классов, 80, 371-400
— реализации, 871
небуферизованный выходной поток, 165
неименованные пространства имен, 257
— пространства имен, 232
неконстантная ссылка, 319
немодифицирующие алгоритмы, 626-631
неперехваченные исключения, 462-463
неявное преобразование типов, 121, 967,
 969
нулевой указатель, 134
— элемент, 656

О

область видимости, 127, 128
обобщенное программирование, 59, 61,
 81, 83, 401
обобщенные алгоритмы, 82, 521
— аллокаторы, 680
обобщенный код, 83
— срез, 796
обработка исключений, 59, 68, 433-470,
 943
— ошибок, 68, 164
обработчик исключений, 68, 243
обратные итераторы, 663
объединение, 50, 321, 977
объект loc, 763
объектно-ориентированное
 программирование, 59, 60, 77, 79
объекты, 77, 304-321
— locale, 1010
— типа locale, 1007
— функции, 356, 616
объявление, 62, 70
— класса, 285
— переменной внутри условий, 187
— функции, 195
объявления, 123-930, 935-939
— using, 224
— как операторы, 184
— функций, 195
одиночное наследование, 474
оператор break, 91, 187-190, 245
— continue, 190
— do, 188
— for, 188
— goto, 188, 189
— if, 185
— return, 68, 187-188, 200
— statement, 62
— switch, 64, 91, 142, 158, 162, 185, 186
— throw, 68, 188, 247, 437
— typedef, 117, 692
— while, 188
— цикла for, 65
— — while, 64
операторы, 934
— return, 201
— ветвления, 63
— выбора, 185
— цикла, 188
— языка C++, 183, 185, 187, 189
операции, 62, 895
— sizeof, 120, 329

- операции `static_cast` и `dynamic_cast`, 499
- `typeid`, 329
- битового сдвига влево и вправо, 158
- и пространства имен, 332
- препроцессора `##`, 215
- приведения `const_cast`, 294
- типов, 343
- присваивания, 63
- сравнения, 63, 185
- языка C++, 169-181
- операционная система UNIX, 42
- операция следования, 135
- `[]`, 697
- `&`, 174-175
- `&&`, 173
- `*`, 83
- `::`, 128
- `|`, 175
- `||`, 173
- `|=`, 175
- `++`, 83, 175
- `+=`, 89
- `->`, 149
- `>>`, 63, 164
- `delete`, 43, 177, 179, 309, 314, 684, 981
- `delete []`, 179, 314-315
- `dynamic_cast`, 43, 494-495
- `flush()`, 758
- `new`, 43, 50, 74, 177, 309, 684
- `- []`, 180
- `reinterpret_cast`, 182
- `sizeof`, 172
- `static_cast`, 181
- `throw`, 43
- `try`-блок, 43
- `typeid`, 43, 501
- добавления новых символов в конец строки, 703
- запятая, 173
- инкремента, 83
- инкремента `++`, 65
- преобразования `const_cast`, 182
- `dynamic_cast`, 182
- приведения `reinterpret_cast`, 320
- присваивания, 63, 290, 309, 343
- разрешения области видимости, 128
- разыменования, 83
- `->`, 358
- указателей на структуры, 149
- сравнения `>`, 63
- `!=`, 63
- `=`, 63

- `==`, 63
- `>=`, 63
- `>>`, 63
- на равенство, 63
- операция точка, 149
- определение интерфейсов, 830
- класса, 285
- функции, 196
- шаблона, 404
- определения, 124
- функций, 196
- организация стандартной библиотеки, 520
- отложенное выделение ресурсов, 1097
- отношение «дружбы», 989
- включения, 860, 867
- использования, 860, 874
- типа "иметь", 869
- отношения внутри класса, 860, 877
- наследования, 860
- отображение, 576
- исключений, 460
- отображенное значение, 576
- отрицатели, 625
- очередь, 97, 572
- с приоритетом, 97, 574
- очистка памяти, 73
- ошибки выделения памяти, 956
- на этапе выполнения, 244

П

- парадигма программирования, 59, 60, 65
- параметр типа, 81
- шаблона, 81
- параметризация и наследование, 424
- параметрический полиморфизм, 424
- параметры шаблонов, 406
- по умолчанию, 415
- паттерн специализации, 418
- паттерны, 833
- перегруженная операция присваивания, 540
- перегруженные операции, 303
- перегрузка, 942
- имен функций, 201, 203, 205
- операций, 303, 327-370
- функциональных шаблонов, 411
- передача аргументов, 197, 199
- перекрестное приведение, 495
- перестановки, 650
- перехват исключений, 441, 443

- перехват любых исключений, 442
- перечисления, 122
- побитовая логическая операция «И», 158
 - — — «ИЛИ», 158
 - — — «ИСКЛЮЧАЮЩЕЕ ИЛИ», 158
- побитовое копирование, 308
- побитовые логические операции, 174
 - операции, 591
- повторная генерация исключений, 441
- повторное использование кода, 838
 - — проектов, 833
- повышающее приведение, 495
- поддержка парадигмы программирования, 60
- подкласс, 373
- подстроки, 707
- подтипизация, 871
- поисковые алгоритмы, 631
- поисковый алгоритм `find_end()`, 631
 - алгоритм `search()`, 631
 - — `search_n()`, 631
- поле типа, 78
- полиморфизм, 383
 - времени выполнения, 424
 - — компиляции, 424
- полиморфные процедуры, 211
- полиморфный тип, 75, 383
- полное выражение, 318
- пользовательские предикаты, 618
 - специализации, 417
- пользовательский аллокатор, 678
 - тип, 42
- понижающее приведение, 494
- порядок вычисления аргументов вызова функций, 173
 - — подвыражений, 172
 - записи обработчиков, 443
 - специализаций, 420
- последовательность элементов, 82, 98
- последовательные контейнеры, 555, 565-569
- поток сегг, 722
 - `cin`, 162, 726
 - `clog`, 722
 - `cout`, 87, 722
 - `istream`, 729, 757
 - `stderr`, 722
 - `stdin`, 726
 - `stdout`, 722
 - ввода `cin`, 726
 - — `wcin`, 726
- потoki, 717-772, 1109
 - типа `ostream`, 174
- потокoвые итераторы, 664
- почтенное копирование, 308
- правила ассоциативности, 173
- правило «неявного `int`», 126
 - одного определения, 260
- предварительное объявление, 151
- предикатные функции, 115
- предикаты, 103, 115, 617, 1107
- предкомпиляция заголовочных файлов, 258
- представления, 914
- предусловия и постусловия, 882
- преобразования указателей и ссылок, 969
 - — на члены классов, 970
 - чисел с плавающей запятой, 969
 - шаблонов, 426
- препроцессор, 944
- препроцессорные директивы `#include`, 236
- префикс `L`, 138
 - `std::`, 166
 - `templateclass C`, 403
- приведение типов, 181
 - — в функциональном стиле, 183
- прикладные среды разработки, 895, 918-919
- принцип подстановки Liskov, 871
 - раздельной компиляции, 67
 - сокрытия данных, 65
- приоритет, 171
 - операций, 173
- приращения, 803
- пробельные символ, 119ы, 164
- проверка диапазона индексов, 94
- программируемые отношения, 860, 875
- продвижения, 968
 - типов вверх, 202
- проектирование, 849-894
 - иерархий классов, 386-395
- производные исключения, 438
 - классы, 372-383, 941
- пространства имен, 66, 219-252, 955, 983
 - имен `std`, 87, 98, 166, 238, 609, 691, 746-747, 798
- прототипирование, 835
- процедурное программирование, 59, 61, 63
- прямой итератор, 612
- псевдонимы, 234
- пунктуация чисел, 1033
- пустая строка, 137

Р

раздельная компиляция, 59, 67, 68, 253
 размещающее new, 320
 разновидности классов, 895
 разрешение перегрузки функций, 203
 разыменованное, 134
 раскрутка стека, 436, 446
 распределители памяти, 674-685
 расширенная информация о типе, 502
 расширенные символьные наборы, 965
 реализация интерфейса, 76
 — модуля, 70
 редуцированная схема вычислений, 173
 рекурсивный спуск, 156
 ромбовидное наследование, 484

С

свободная память, 178, 509, 511
 связывание имен, 996
 связывающие адаптеры, 621, 625
 семантика владения, 448
 — возврата значения из функции, 200
 — деструктивного копирования, 448
 — передачи аргументов, 195, 198
 символ, 86, 117
 — «перевода строки», 86
 — п, 63
 символы, 690
 символьная константа, 88
 символьные литералы, 117
 — наборы, 963, 965
 — типы, 116-117
 синтаксический анализатор, 156
 скалярное произведение, 803
 словарь, 97, 576
 служебный символ end-of-file, 166
 событие copyfmt_event, 766
 — erase_event, 766
 — imbue_event, 766
 совместимость С и C++, 948-949, 951
 — с языком С, 42
 соглашение компоновки, 262
 — о компоновке, 263
 сокрытие имен, 128
 сопровождение и поддержка программ, 837
 сортировка последовательностей, 643, 645, 647
 составные (комбинированные) операции присваивания, 158
 состояние, 729
 — смещения, 1067

специализации, 406
 —, определяемыми пользователем, 417
 специализация numeric_limits, 777
 — vectorbool, 550
 — шаблонов функций, 420
 спецификатор доступа private, 295
 спецификаторы, 125
 спецификации языка C++, 46
 спецификация исключений, 457, 459, 461
 списки, 45
 список, 95, 565
 — инициализации членов, 310
 средство вывода, 87
 срез, 786
 ссылки, 144-145, 147
 ссылочные типы, 114
 стандарт ANSI C, 49
 — C++, 947
 стандартная библиотека, 85-108, 517-523, 955
 — — языка C++, 82
 — библиотечная функция atexit(), 277
 — — — isalpha(), 162
 — — — isspace(), 164
 — функция set_new_handler(), 180
 стандартные алгоритмы, 607
 — арифметические преобразования, 171
 — исключения, 467
 — контейнеры, 97, 555-604
 — макросы, 208
 — манипуляторы ввода/вывода, 747
 — преобразования типов, 202
 — фасеты, 1026-1073
 стандартный алгоритм find(), 99
 — аллокатор, 674
 — библиотечный класс string, 689
 — заголовочный файл, 87
 — класс фасетов collate_bynamechar, 1020
 — поток ввода, 90
 — — вывода, 63, 88
 — — — std::cout, 86
 — — ошибок cerr, 95
 — строковый шаблон basic_string, 692-709
 — тип string, 305
 — шаблон allocator, 674
 статическая функция-член, 289
 статические переменные, 197
 — члены, 288-289, 990
 — —-константы, 312
 статический объект, 129
 стек, 65, 97, 571
 стековая память, 979
 стековые операции, 541

стиль программирования, 40
строгое слабое упорядочение, 562
строки, 88-89, 689-714, 1108
– типа `string`, 589
– языка C, 90
строковые потоки, 755
строковый литерал, 86, 136
структуры, 149, 151
– данных, 114
– и классы, 295
суперкласс, 373
схема, форма, 418
счетчик ссылок, 362
сырая память, 181, 501, 676

Т

таблица виртуальных функций, 77
тело определения класса, 285
– функции, 62
терминальный ноль, 90
тестирование, 836
тип, 62, 113
– `allocator_type`, 534
– `bool`, 115
– `char`, 62, 115-116
– `—*`, 207
– `double`, 72, 91, 115
– `ifstream`, 103
– `indirect_array`, 798
– `int`, 62, 115
– `istream`, 90
– `istream_iterator`, 103
– `long`, 171
– `mask`, 1063
– `—_array`, 797
– `ofstream`, 103
– `ostream`, 90
– `—_iterator`, 103
– `out_of_range`, 94
– `pair`, 104
– `rebind`, 677
– `size_t`, 510
– `—_type`, 697
– `streamsize`, 721
– `string`, 88, 90, 92
– `time_t`, 1048
– `vector`, 50, 94
– `void`, 121
– `—*`, 148
– `wchar_t`, 117, 692
– аллокаторов, 534
– итератора, 96

– стандартной библиотеки, 42
–, определяемый пользователем, 284
типы, 113
– `vaarg` и `complex`, 1110
– итераторов, 101
– с плавающей запятой, 114, 119
– стандартной библиотеки, 87
– целых литералов, 967
–, определяемые пользователем, 71, 114
–-значения, 303
точка входа в программу, 86
– конкретизации шаблона, 409
– объявления, 128
точное совпадение типов, 202

У

удобства функциональной декомпозиции, 43
узловой класс иерархии, 903
узловые классы, 903, 905, 907
узлы, 895
указатели, 64, 133
– на массивы, 133, 138-139
– на функции, 133, 148, 209, 211
– – члены классов, 148, 473, 505, 507, 989
указатель, 65
– `_unexpected_handler`, 462
– `this`, 346
– на член класса, 507
указательные типы, 114
унарная операция, 330
– – `&`, 65
унарный предикат, 612, 617
ундаментальные типы, 114
управление памятью, 979, 981
– режимом доступа, 285
управляющая строка, 207
уровни приоритетов операций, 173
условная компиляция, 215
условные выражения, 186
– операторы, 185
утверждения, 879
утечка памяти, 310

Ф

фабрика, 396
файл `cstdarg`, 208
– `cstdio`, 238
– `exception`, 462
– `functional`, 617, 619, 622
– `ios`, 721

файл `iostream`, 86, 721-722, 726

- `limits`, 121
- `stack`, 571
- `stdexcept`, 94
- `stdio.h`, 238
- `typeinfo`, 501
- `vector`, 533

фасет `_byname()`, 1021

- `codecvt`, 1068
- `collate_byname`, 1032
- `money_get`, 1040, 1041, 1044
- `_put`, 1040-1041, 1043
- `money_punct`, 1040
- `num_punct`, 1033
- `time_get`, 1052
- `_put`, 1049

фасеты, 1007, 1020-1025

фигурные скобки, 62

флаг `uppercase`, 743

флаги состояния потока, 175

формальная грамматика, 156

форматирующий флаг `boolalpha`, 724

функторы, 356, 616

функции, 195-218

- `exit()`, 277
- `insert()`, 96
- `get()`, 709
- `operator new()`, 674
- `printf()`, 207
- `seekp()`, 757
- `strcmp()`, 692
- доступа, 531
- обратного вызова для потоков, 766
- поддержки, 302, 341
- операции, 329, 331, 333

функции-члены, 284, 285, 300

- `operator*()`, 328
- `++()`, 328

функциональные объекты, 607, 616

функциональный шаблон `use_facet`, 1021

функция `abort()`, 277

- `apply()`, 783
- `at()`, 94, 695
- `atexit()`, 277
- `begin()`, 96, 99
- `bsearch()`, 650
- `c_str()`, 90, 700
- `clear()`, 735
- `clock()`, 1047
- `close()`, 753
- `complete_name()`, 704
- `data()`, 700
- `egptr()`, 761

- `end()`, 96, 99
- `eof()`, 734
- `eq()`, 600
- `error()`, 208
- `fill()`, 744
- `flags()`, 740
- `frac_digits()` фасета `money_punct`, 1040
- `get_allocator()`, 550
- `_time()`, 1053
- `getline()`, 709, 731, 732
- `hash()`, 600
- `ignore()`, 733
- `imbue()`, 759, 765
- `imbue()`, 765
- `in_avail()`, 759
- `itoa()`, 208
- `isword()`, 766
- `main()`, 86, 165, 275
- `narrow()`, 760
- `operator delete [] ()`, 511
- `delete()`, 321
- `new [] ()`, 511
- `--()`, 180-181, 320
- `!()`, 600
- `==()`, 303
- `precision()`, 743
- `put()`, 722
- `putback()`, 758
- `pword()`, 766
- `qsort()`, 650
- `readsome()`, 759
- `register_callback()`, 766
- `replace()`, 706
- `reserve()`, 548
- `setf()`, 740, 744
- `setg()`, 761
- `setp()`, 761
- `showmanyc()`, 763
- `strchr()`, 711
- `strtod()`, 712
- `substr()`, 707
- `swap()`, 709
- `swap()`, 1105
- `tie()`, 737
- `tolower()`, 1065
- `toupper()`, 1065
- `uflow()`, 763
- `underflow()`, 763
- `unget()`, 758
- `widen()`, 760
- `width()`, 729
- `width()`, 743
- `write()`, 722

функция `xalloc()`, 766

– выделения памяти оператор `new()`, 320

– обратного вызова, 388

– операции оператор `()()`, 356

– операция оператор `[]`, 355

функция-член `assign()`, 540

– оператор `delete()`, 509

– – `new()`, 509

Х–Ц–Ч

хэширование, 600

хэш-таблица, 598

целые литералы, 118

– типы, 118

цикл `do`, 189

– `while`, 69, 189

циклы, 63–64

часовой, 738

частичная специализация, 418

числовые пределы, 776–777

Ш–Э–Я

шаблон, 81

– `Assert()`, 880

– `auto_ptr`, 520

– `basic_ostream`, 720–721

– `char_traits`, 690

– `complex`, 798

– `iterator_traits`, 634, 658–659

– `mem_fun()`, 106

– `numeric_limits`, 776

шаблонные члены шаблонов, 424

шаблоны, 45, 50, 401–432, 886, 942, 956, 990–1003

– и пространства имен, 1001

– языка C++, 61

шаг, 787

эквивалентность типов, 152

элемент-часовой, 83

явная конкретизация, 1004

явное управление памятью, 1087

явный вызов деструктора, 321

язык C, 42, 49

Научно-техническое издание

Бьерн Страуструп

Язык программирования C++. Специальное издание

Подписано в печать 13.09.2010. Формат 70×100/16. Усл. печ. л. 92,3.

Гарнитура Таймс. Бумага газетная. Печать офсетная.

Тираж 3000 экз. Заказ 23355.

Издательство Бином, 2011 г.

103473, Москва, ул. Краснопролетарская, 16

Отпечатано по технологии StP

в ОАО “Печатный двор” им. А. М. Горького

197110, Санкт-Петербург, Чкаловский пр., 15.